MẠNG NƠ-RON NHÂN TẠO

Định hướng: xây dựng và huấn luyện mô hình phân loại phi tuyến từ đầu

1. Từ mô hình tuyến tính đến mạng nơ-ron

Ba bài trước đã tạo thành một chuỗi liền mạch, trong đó mỗi bài chỉ thay đổi một thành phần của khung "mô hình, hàm mất mát, thuật toán tối ưu". **Perceptron** biến điểm số tuyến tính $s = \tilde{\mathbf{x}}\mathbf{w}$ thành nhãn cứng bằng hàm dấu. **Logistic Regression** làm "mềm" hàm dấu thành hàm sigmoid để có xác suất và một hàm mất mát lồi. **Softmax Regression** mở rộng từ một điểm số lên K điểm số để phân loại nhiều lớp, với mô hình gọn gàng

$$\mathbf{S} = \tilde{\mathbf{X}}\mathbf{W}, \qquad \mathbf{P} = \text{softmax}(\mathbf{S}), \qquad \hat{y}_i = \underset{k}{\text{argmax}}\, p_{ik}$$

Qua ba bài, ta đã giải quyết rất nhiều vấn đề: nhiều lớp, xác suất, hàm mất mát lồi, gradient gọn dạng $\frac{1}{n}\tilde{\mathbf{X}}^T(\mathbf{P} - \mathbf{Y})$. Nhưng có một thứ ta **chưa hề đụng đến**: cả ba mô hình đều bắt đầu bằng **điểm số tuyến tính** $\tilde{\mathbf{x}}\mathbf{w}_k$. Cuối bài Softmax Regression (mục 4.5), ta đã gặp hậu quả của điều đó: trên dữ liệu ba vòng tròn đồng tâm, mô hình chỉ đạt 35,0%, ngang mức đoán mò, dù thuật toán huấn luyện đã chạy đúng và tìm được cực tiểu toàn cục. Lúc đó ta tạm "chữa cháy" bằng cách tự thêm đặc trưng bậc hai. Bài này đi theo hướng ngược lại: **để chính mô hình tự học ra các đặc trưng cần thiết**. Đó là ý tưởng của **mạng nơ-ron nhân tạo** (artificial neural network).

Trong mục 1, ta trả lời ba câu hỏi:

1. Giới hạn của các mô hình tuyến tính nằm ở đâu, và vì sao không thể khắc phục bằng cách huấn luyện tốt hơn hay thu thập thêm dữ liệu?
2. Cách "tự thêm đặc trưng" có cái giá gì, và làm thế nào để mô hình **tự học đặc trưng**?
3. Nơ-ron nhân tạo là gì, nó có nguồn gốc ra sao, và xếp chồng các nơ-ron thành mạng nhiều lớp như thế nào?

Quy ước ký hiệu

Ta giữ nguyên quy ước của các bài trước: chữ thường in nghiêng là đại lượng vô hướng, **chữ thường in đậm là vector**, **chữ hoa in đậm là ma trận**, vector đặc trưng là **vector hàng**. Có một thay đổi nhỏ về cách viết: ở bài Softmax Regression, ma trận đặc trưng $\mathbf{X}$ đã có sẵn cột 1 của hệ số chệch. Với mạng nhiều lớp, ta cần thêm cột 1 ở **mỗi lớp**, nên từ bài này ta tách hai khái niệm: $\mathbf{X}$ là ma trận đặc trưng **gốc** (chưa có cột 1) và $\tilde{\mathbf{X}} = [\mathbf{1} \quad \mathbf{X}]$ là ma trận sau khi thêm cột 1. Dấu ngã $\sim$ luôn có nghĩa là "đã thêm cột (hoặc phần tử) 1".

Đối tượng	Ký hiệu	Kích thước	Ý nghĩa
Số mẫu, số đặc trưng, số lớp, số nơ-ron ẩn	n, d, K, m	vô hướng	n quan sát, d đặc trưng, K lớp, m nơ-ron ở lớp ẩn
Ma trận đặc trưng gốc	$\mathbf{X}$	$n \times d$	Hàng thứ i là vector hàng $\mathbf{x}_i$
Ma trận đặc trưng có cột 1	$\tilde{\mathbf{X}}$	$n \times (d+1)$	$\tilde{\mathbf{X}} = [\mathbf{1} \quad \mathbf{X}]$; chính là $\mathbf{X}$ của bài Softmax
Ma trận nhãn one-hot	$\mathbf{Y}$	$n \times K$	$y_{ik} = 1$ nếu mẫu i thuộc lớp k
Hàm kích hoạt	$\varphi(\cdot)$	hàm vô hướng	Áp dụng lên từng phần tử (mục 1.3, chi tiết ở mục 2.3)
Ánh xạ đặc trưng	$\boldsymbol{\phi}(\mathbf{x})$	vector hàng	Đặc trưng mới tạo từ $\mathbf{x}$, ví dụ đa thức
Tham số lớp ẩn	$\mathbf{W}^{(1)}$	$(d+1) \times m$	Mỗi cột là vector tham số $\mathbf{w}_j^{(1)}$ của một nơ-ron ẩn
Ma trận kích hoạt lớp ẩn	$\mathbf{H}$	$n \times m$	$h_{ij} = \varphi\left(\tilde{\mathbf{x}}_i \mathbf{w}_j^{(1)}\right)$
Tham số lớp ra	$\mathbf{W}^{(2)}$	$(m+1) \times K$	Chính là ma trận $\mathbf{W}$ của Softmax, nhưng chạy trên $\mathbf{H}$
Ma trận điểm số, xác suất	$\mathbf{S}, \mathbf{P}$	$n \times K$	Như ở bài Softmax Regression

1.1. Hạn chế của Softmax Regression: đường biên luôn là siêu phẳng

Nhắc lại: mô hình làm được gì

Ở bài trước (mục 2.5), ta đã chứng minh mô hình dự đoán lớp k khi điểm số $s_k = \tilde{\mathbf{x}}\mathbf{w}_k$ lớn nhất, nên đường biên giữa hai lớp k và l là tập các điểm thỏa

$$\tilde{\mathbf{x}}(\mathbf{w}_k - \mathbf{w}_l) = 0$$

Đó là một **siêu phẳng** (đường thẳng khi $d = 2$), và vùng của lớp k là giao của $K - 1$ nửa không gian. Hệ quả là **mọi vùng quyết định đều lồi**. Khi các lớp xếp thành những cụm tách biệt, kiểu "mỗi lớp một dải" như bài toán rớt, đạt, giỏi, đó là một mô hình rất tốt. Nhưng dữ liệu thật không phải lúc nào cũng thân thiện như vậy. Có hai cách thấy rõ giới hạn này: một ví dụ nhỏ có thể chứng minh được bằng tay (XOR) và một ví dụ có hình học "không lồi" (các vòng tròn).

Hạn chế thứ nhất: bài toán XOR

XOR (phép "hoặc loại trừ") có bốn điểm trong mặt phẳng với hai nhãn:

x_1	x_2	Nhãn y
0	0	0
1	1	0
0	1	1
1	0	1

Hai điểm cùng nhãn nằm ở hai đỉnh **chéo nhau** của hình vuông (Hình 1a). Ta chứng minh được rằng **không có Softmax Regression nào (cũng như Logistic Regression hay Perceptron) phân loại đúng cả bốn điểm**. Với $K = 2$, softmax trở về sigmoid của hiệu hai điểm số (bài Softmax, mục 2.3), nên mô hình dự đoán lớp 1 khi điểm số $s = \tilde{\mathbf{x}}\mathbf{w} = w_0 + w_1x_1 + w_2x_2$ dương, với $\mathbf{w} = \mathbf{w}_1 - \mathbf{w}_0$. Để phân loại đúng cả bốn điểm, ta cần

$$w_0 < 0, \qquad w_0 + w_1 + w_2 < 0, \qquad w_0 + w_2 > 0, \qquad w_0 + w_1 > 0$$

Cộng hai bất đẳng thức đầu: $2w_0 + w_1 + w_2 < 0$. Cộng hai bất đẳng thức sau: $2w_0 + w_1 + w_2 > 0$. Hai kết luận mâu thuẫn nhau, nên **không tồn tại $\mathbf{w}$** thỏa mãn.

Có thể nhìn lý do này bằng hình học. Trung điểm của hai điểm lớp 0 và trung điểm của hai điểm lớp 1 **trùng nhau** tại $(1/2, 1/2)$:

$$\frac{(0,0) + (1,1)}{2} = \frac{(0,1) + (1,0)}{2} = \left(\frac{1}{2}, \frac{1}{2}\right)$$

Vì điểm số s là hàm tuyến tính (affine) theo $\mathbf{x}$ nên s tại trung điểm bằng trung bình của s tại hai đầu mút. Nếu $s > 0$ ở cả hai điểm lớp 1 thì $s > 0$ ở trung điểm, và nếu $s < 0$ ở cả hai điểm lớp 0 thì $s < 0$ cũng tại đúng điểm đó. Vô lý. Tổng quát: **hai lớp chỉ tách được bằng một siêu phẳng nếu bao lồi của chúng không giao nhau**.

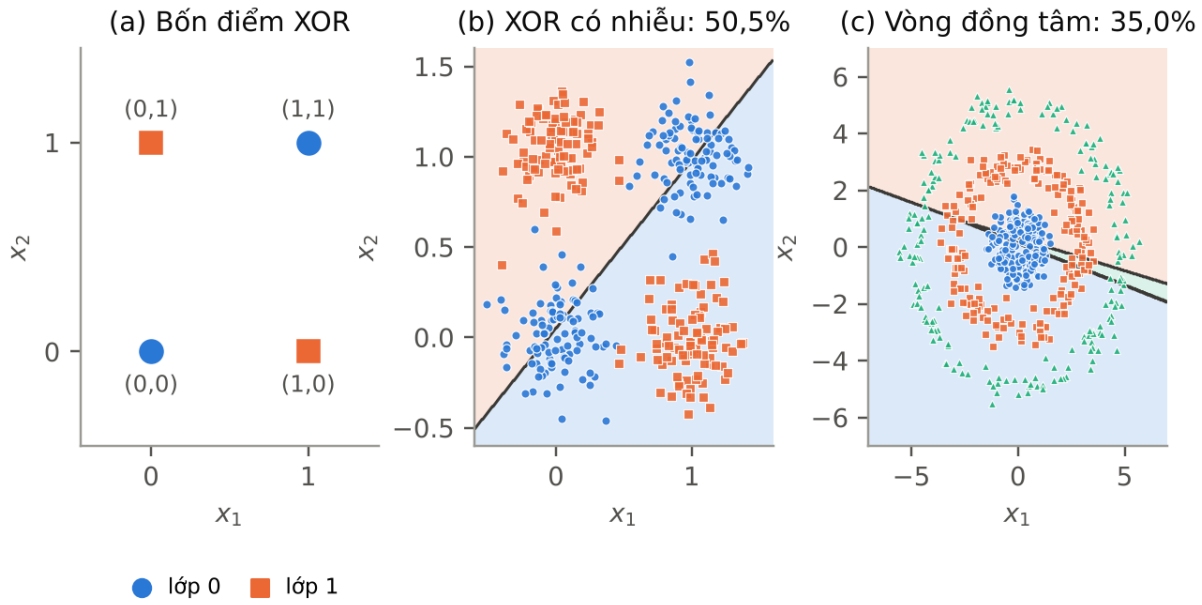
Ta kiểm tra bằng số trên bộ dữ liệu XOR có nhiễu: bốn cụm Gauss tâm $(0,0)$, $(1,1)$, $(0,1)$, $(1,0)$, độ lệch chuẩn $0{,}18$, mỗi cụm 100 điểm. Softmax Regression chỉ đạt $50{,}5\%$ (Hình 1b), tức đoán ngẫu nhiên: đường biên cắt ngang hai cụm mà nó không thể tách. Dữ liệu này đối xứng, nên cực tiểu của J rơi vào vùng trọng số gần $\mathbf{0}$ và xác suất dự đoán gần $1/2$ ở mọi điểm: sau khi hội tụ, $J \approx 0{,}6931 = \ln 2$, đúng bằng mốc $J_0 = \ln K$ của mô hình "chưa biết gì" (bài Softmax, mục 3.1).

Hạn chế thứ hai: lớp có hình dạng không lồi

XOR cho thấy một lớp có thể gồm **hai nhóm rời nhau**. Bài Softmax (mục 4.5) cho một ví dụ khác: ba vòng tròn đồng tâm bán kính 1, 3, 5, mỗi lớp 200 điểm. Lớp 0 là hình tròn ở giữa, lớp 1 là một chiếc nhẫn **bao quanh** lớp 0, lớp 2 là chiếc nhẫn ngoài cùng. Kết quả trên tập kiểm tra 180 điểm:

4. Độ chính xác chỉ $35{,}0\%$, so với mức đoán mò $33{,}3\%$.
5. Hàm mất mát $J = 1{,}0964$ chỉ thấp hơn mốc $J_0 = \ln 3 = 1{,}0986$ chút ít: mô hình gần như **không học được gì** từ hai đặc trưng.

Hình 1c cho thấy các vùng quyết định chỉ là những dải thẳng cắt ngang các vòng. Không một hình lồi nào có thể "ôm" lớp 0 mà không chứa luôn các điểm của lớp 1.



Hình 1. Giới hạn của Softmax Regression. (a) Bốn điểm XOR: hai điểm cùng lớp nằm ở hai đỉnh chéo nhau. (b) XOR có nhiễu (400 điểm): đường biên tuyến tính không tách được, độ chính xác 50,5%. (c) Ba vòng tròn đồng tâm (600 điểm): vùng quyết định là các dải thẳng, độ chính xác kiểm tra 35,0%.

Tổng hợp lại, các dạng dữ liệu sau nằm ngoài tầm của mọi mô hình có vùng quyết định lồi:

Hình dạng của lớp	**Ví dụ**	**Vì sao mô hình tuyến tính thất bại**
Gồm hai nhóm rời nhau	XOR	Bao lồi của hai lớp giao nhau
Bị lớp khác bao quanh	Vòng tròn đồng tâm	Lớp nằm trong không thể là một nửa không gian hay giao các nửa không gian
Hình cong, hình chữ "C"	Hình trăng khuyết	Vùng của lớp không lồi

Vì sao huấn luyện tốt hơn cũng không cứu được

Có một điểm cần phân biệt rõ để định hướng cho cả chương này. Khi mô hình cho kết quả tệ, ta có hai nhóm nguyên nhân rất khác nhau:

Nguyên nhân	**Biểu hiện**	**Có sửa được bằng huấn luyện kỹ hơn không?**
Thuật toán tối ưu chưa tốt	J giảm chậm hoặc dao động (learning rate chưa hợp lý, chưa chuẩn hóa đặc trưng)	**Có**: chỉnh η, chuẩn hóa, chạy thêm epoch
Nhiễu hoặc ít dữ liệu	Khoảng cách lớn giữa độ chính xác huấn luyện và kiểm tra	**Một phần**: thêm dữ liệu, điều chuẩn hóa
Mô hình không đủ khả năng biểu diễn	J đã hội tụ về cực tiểu nhưng vẫn cao; độ chính xác thấp ngay cả trên tập huấn luyện	**Không**: phải đổi mô hình

Trường hợp XOR và vòng tròn thuộc nhóm thứ ba. Hàm mất mát của Softmax Regression là lồi (bài Softmax, mục 4.3), thuật toán của ta **đã tìm đúng cực tiểu toàn cục**, và cực tiểu đó vẫn chỉ là $J \approx \ln 3$. Chạy thêm epoch, thêm dữ liệu hay điều chỉnh η đều vô ích, vì **trần chất lượng nằm ở lớp hàm mà mô hình có thể biểu diễn**, không nằm ở thuật toán. Muốn vượt qua, ta cần một mô hình có vùng quyết định **cong**, tức là một mô hình **phi tuyến**.

1.2. Ý tưởng: để mô hình tự học đặc trưng thay vì ta tự đoán đặc trưng đa thức

Cách chữa quen thuộc: thêm đặc trưng đa thức

Ở bài Softmax (mục 4.5), ta đã dùng một mẹo: mô hình chỉ cần **tuyến tính theo tham số**, không nhất thiết tuyến tính theo dữ liệu gốc. Ta biến đổi $\mathbf{x}$ thành vector đặc trưng mới $\boldsymbol{\phi}(\mathbf{x})$ rồi chạy Softmax Regression trên đó. Với hai đặc trưng gốc và bậc 2:

$$\boldsymbol{\phi}(\mathbf{x}) = [x_1 \quad x_2 \quad x_1^2 \quad x_1x_2 \quad x_2^2]$$

Điểm số của lớp k trở thành đa thức bậc hai theo (x_1, x_2), và đường biên là một đường conic. Với XOR, chỉ cần thêm **một** đặc trưng x_1x_2. Ví dụ điểm số

$$s = -\frac{1}{2} + x_1 + x_2 - 2x_1x_2$$

cho $-0{,}5$ tại $(0,0)$ và $(1,1)$ (lớp 0), cho $+0{,}5$ tại $(0,1)$ và $(1,0)$ (lớp 1): phân loại đúng cả bốn điểm. Trên dữ liệu XOR có nhiễu ở trên, Softmax Regression với ba đặc trưng x_1, x_2, x_1x_2 đạt $99{,}2\%$ trên tập kiểm tra 120 điểm, thay vì $50{,}5\%$. Với ba vòng tròn, đặc trưng bậc hai đưa độ chính xác từ $35{,}0\%$ lên $99{,}4\%$.

Cái giá của việc tự thêm đặc trưng

Cách chữa này hiệu quả, nhưng nó đặt ra ba vấn đề, và chúng nghiêm trọng dần khi bài toán lớn lên:

Thứ nhất, số đặc trưng bùng nổ. Với d đặc trưng gốc và bậc tối đa k, số đặc trưng đa thức (không tính cột 1) và số tham số của Softmax Regression là

$$\binom{d+k}{k} - 1, \qquad K\binom{d+k}{k}$$

Bài toán	d	Bậc k	K	Số đặc trưng mới	Số tham số
Vòng tròn đồng tâm	2	2	3	5	18
Dữ liệu bảng cỡ vừa	10	2	10	65	660
Ảnh chữ số 28×28	784	2	10	308 504	3 085 050
Ảnh chữ số 28×28	784	3	10	80 931 144	809 311 450

Chỉ với ảnh xám 28×28 (mỗi điểm ảnh là một đặc trưng), đặc trưng bậc hai đã có hơn ba trăm nghìn cột, và bậc ba hơn tám mươi triệu cột. Số tham số vượt xa số mẫu, kéo theo nguy cơ khớp quá mức và phân tách hoàn hảo (bài Softmax, mục 4.4).

Thứ hai, ta phải biết trước dạng đặc trưng. Ở ví dụ vòng tròn, ta "biết" biên là đường tròn nên đặc trưng bậc hai là hợp lý. Với ảnh, tiếng nói hay văn bản, không ai biết trước nên dùng tổ hợp nào của các điểm ảnh. Đặc trưng đa thức là một lựa chọn **chung chung** và tốn kém, không dựa trên cấu trúc của bài toán.

Thứ ba, đặc trưng và bộ phân loại bị tách rời. Ta chọn $\boldsymbol{\phi}$ trước, rồi mới huấn luyện Softmax Regression. Nếu $\boldsymbol{\phi}$ chọn chưa tốt, bộ phân loại phía sau không thể sửa. Cách làm thủ công truyền thống (gọi là *kỹ nghệ đặc trưng*, feature engineering) vì thế đòi hỏi nhiều thử nghiệm và hiểu biết chuyên môn.

Ý tưởng: tham số hóa đặc trưng rồi học chúng

Câu hỏi tự nhiên là: nếu đặc trưng quan trọng như vậy, **tại sao không để dữ liệu quyết định đặc trưng**, giống cách ta để dữ liệu quyết định $\mathbf{W}$? Ta cho ánh xạ đặc trưng **chứa tham số** có thể học, rồi huấn luyện chung với bộ phân loại bằng cùng hàm mất mát cross-entropy. Cụ thể, ta chọn đặc trưng mới là các điểm số tuyến tính **được "uốn cong" bởi một hàm phi tuyến** φ:

$$\mathbf{H} = \varphi\left(\tilde{\mathbf{X}}\mathbf{W}^{(1)}\right), \qquad \mathbf{S} = \tilde{\mathbf{H}}\mathbf{W}^{(2)}, \qquad \mathbf{P} = \operatorname{softmax}(\mathbf{S})$$

Trong đó φ áp dụng lên từng phần tử, $\tilde{\mathbf{H}} = [\mathbf{1} \quad \mathbf{H}]$, và $\mathbf{W}^{(1)}$ có kích thước $(d+1) \times m$. Mỗi cột $\mathbf{w}_j^{(1)}$ định nghĩa **một đặc trưng** $h_j = \varphi\left(\tilde{\mathbf{x}}\mathbf{w}_j^{(1)}\right)$. So với đặc trưng đa thức cố định, bây giờ các đặc trưng có dạng $\varphi\big($tổ hợp tuyến tính của $\mathbf{x}\big)$ và **hệ số của tổ hợp là tham số cần học**. Dòng thứ hai và thứ ba của công thức chính là Softmax Regression ở bài trước, chỉ khác là chạy trên đặc trưng $\mathbf{H}$ thay cho $\mathbf{X}$.

Vì sao φ bắt buộc phải phi tuyến? Nếu φ là hàm đồng nhất thì $\mathbf{H} = \tilde{\mathbf{X}}\mathbf{W}^{(1)}$. Viết $\mathbf{W}^{(2)}$ thành hàng chệch $\mathbf{b}$ (vector hàng $1 \times K$) và phần còn lại $\mathbf{V}$ (kích thước $m \times K$), ta có

$$\mathbf{S} = \mathbf{1}\mathbf{b} + \tilde{\mathbf{X}}\mathbf{W}^{(1)}\mathbf{V}$$

Đây vẫn chỉ là một hàm tuyến tính (affine) của $\mathbf{x}$, với ma trận tham số $\mathbf{W}^{(1)}\mathbf{V}$, nghĩa là **mạng sâu bao nhiêu cũng chỉ bằng Softmax Regression**. Chính φ phi tuyến mới làm cho việc xếp chồng có ý nghĩa (mục 2.3 sẽ phân tích kỹ).

Ví dụ: hai "bộ dò" làm XOR trở nên tách được

Ta thử thiết kế tay hai đặc trưng cho XOR theo công thức trên, dùng hàm bước (bằng 1 khi đối số không âm, ngược lại bằng 0) làm φ:

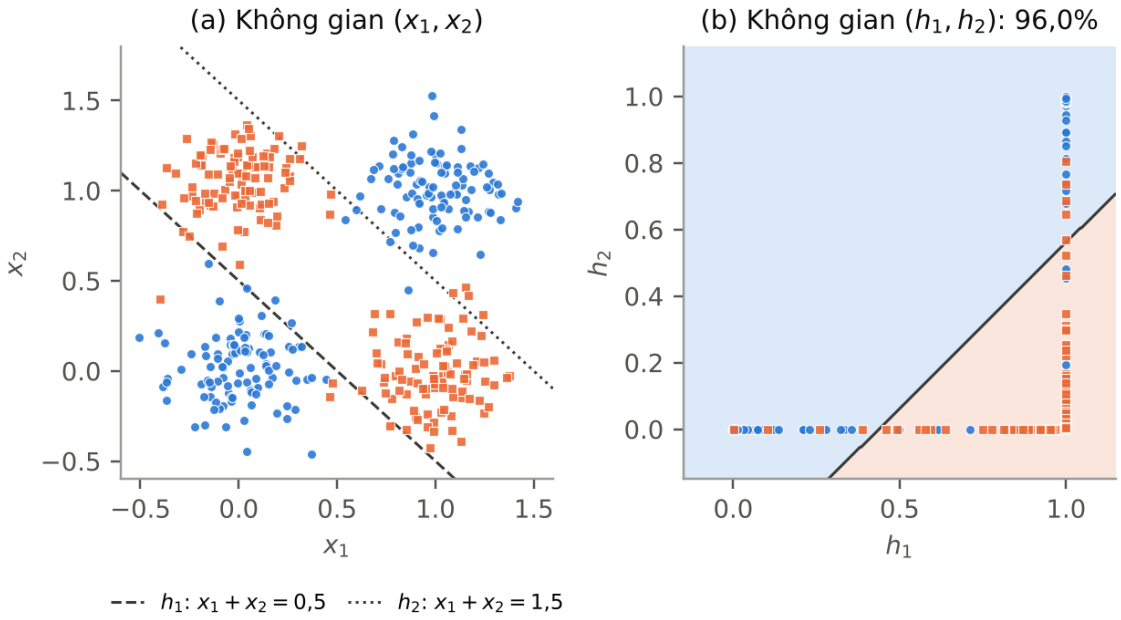
$$h_1 = \varphi(x_1 + x_2 - 0{,}5), \qquad h_2 = \varphi(x_1 + x_2 - 1{,}5)$$

Đặc trưng h_1 "bật" khi **ít nhất một** đầu vào bằng 1 (phép OR), còn h_2 "bật" khi **cả hai** đầu vào bằng 1 (phép AND). Mỗi đặc trưng là một bộ phân loại tuyến tính nhỏ, đúng nghĩa là một Perceptron. Bốn điểm XOR được biến thành

(x_1, x_2)	h_1	h_2	**Nhãn** y
(0,0)	0	0	0
(1,1)	1	1	0
(0,1)	1	0	1
(1,0)	1	0	1

Trong không gian (h_1, h_2), hai điểm lớp 1 trùng nhau tại (1,0), còn hai điểm lớp 0 nằm ở (0,0) và (1,1) ở hai phía. Bây giờ **một đường thẳng là đủ**: điểm số $s = -0{,}5 + h_1 - h_2$ cho $-0{,}5$, $-0{,}5$, $+0{,}5$, $+0{,}5$ ở bốn dòng, phân loại đúng cả bốn. Nói cách khác, bài toán không tách được trong không gian (x_1, x_2) **trở thành tách được trong không gian đặc trưng** (h_1, h_2).

Hình 2 làm lại thí nghiệm này trên dữ liệu XOR có nhiễu, với hàm bước được thay bằng sigmoid (một hàm "mềm", khả vi) với hai ngưỡng chọn tay như trên. Softmax Regression chạy trên (h_1, h_2) đạt 96,0% độ chính xác trên chính dữ liệu huấn luyện, so với 50,5% khi chạy trên (x_1, x_2). Chưa đạt 100% vì hai ngưỡng ta chọn tay chưa tối ưu và các cụm nhiễu nằm sát ngưỡng. Mạng nơ-ron sẽ **tự học các ngưỡng này** cùng với bộ phân loại.



Hình 2. Học đặc trưng biến bài toán không tách được thành tách được. (a) Dữ liệu XOR có nhiễu trong không gian gốc, cùng hai đường $x_1 + x_2 = 0{,}5$ và $x_1 + x_2 = 1{,}5$ định nghĩa hai đặc trưng. (b) Trong không gian đặc trưng (h_1, h_2) ba nhóm điểm nằm ở các vị trí khác nhau và một đường thẳng đã phân loại được với độ chính xác 96,0%.

Ba cách có được đặc trưng

	Thiết kế tay	**Đặc trưng đa thức**	**Đặc trưng học được**
Ai chọn đặc trưng?	Con người, dựa trên hiểu biết bài toán	Ta chọn bậc, công thức cố định	Dữ liệu, qua huấn luyện

	Thiết kế tay	Đặc trưng đa thức	Đặc trưng học được
Số tham số	Ít	Bùng nổ theo d và k	Do ta chọn số nơ-ron m
Phụ thuộc kiến thức chuyên môn	Rất nhiều	Một phần	Ít
Phù hợp với dữ liệu phức tạp (ảnh, tiếng nói, văn bản)	Khó	Khó	Có
Ví dụ	Đếm số góc trong ảnh chữ số	Thêm x_1x_2, x_1^2	Mạng nơ-ron

1.3. Nơ-ron nhân tạo: từ Perceptron năm 1958 đến mạng nhiều lớp

Nơ-ron nhân tạo

Các đặc trưng $h_j = \varphi\left(\tilde{\mathbf{x}}\mathbf{w}_j^{(1)}\right)$ ở mục 1.2 có cùng một hình dạng, và đó chính là **nơ-ron nhân tạo**. Một nơ-ron nhận vector đầu vào $\mathbf{x} = [x_1 \quad \cdots \quad x_d]$, tính tổng có trọng số cộng hệ số chệch, rồi cho kết quả đi qua hàm kích hoạt φ:

$$s = w_0 + \sum_{j=1}^{d} w_j x_j = \tilde{\mathbf{x}}\mathbf{w}, \qquad a = \varphi(s)$$

Cách đặt tên mượn từ sinh học: nơ-ron thần kinh nhận tín hiệu từ nhiều nơ-ron khác qua các khớp thần kinh (có "độ mạnh" khác nhau), cộng dồn tín hiệu ở thân tế bào và phát xung đi tiếp khi tín hiệu vượt một ngưỡng. Tương ứng: x_j là tín hiệu vào, w_j là độ mạnh của kết nối, $-w_0$ đóng vai trò ngưỡng, và φ quyết định nơ-ron "phát" như thế nào. Phép so sánh này chỉ mang tính gợi hứng: mạng nơ-ron nhân tạo là một họ hàm toán học huấn luyện được bằng gradient, **không** phải mô phỏng chính xác bộ não.

Perceptron, Logistic Regression và Softmax đều là nơ-ron

Điều thú vị là ba mô hình ta đã học chỉ khác nhau ở hàm kích hoạt và hàm mất mát:

Mô hình	Hàm kích hoạt φ	Số nơ-ron ở đầu ra	Hàm mất mát
Hồi quy tuyến tính	Hàm đồng nhất $\varphi(s) = s$	1	Bình phương sai số (MSE)
Perceptron	Hàm dấu $\varphi(s) = \text{sign}(s)$	1	Hàm mất mát Perceptron
Logistic Regression	Sigmoid $\varphi(s) = \sigma(s)$	1	Cross-entropy nhị phân
Softmax Regression	Softmax (chung cho cả K nơ-ron)	K	Cross-entropy nhiều lớp

Nghĩa là **mỗi mô hình ta đã học là một lớp nơ-ron duy nhất**, nối thẳng từ đầu vào ra đầu ra. Mục 2.1 sẽ phát biểu chính xác điều này. Mạng nơ-ron chỉ làm thêm một việc: **xếp chồng** các lớp nơ-ron, để đầu ra của lớp này trở thành đầu vào của lớp kế tiếp.

Một chút lịch sử

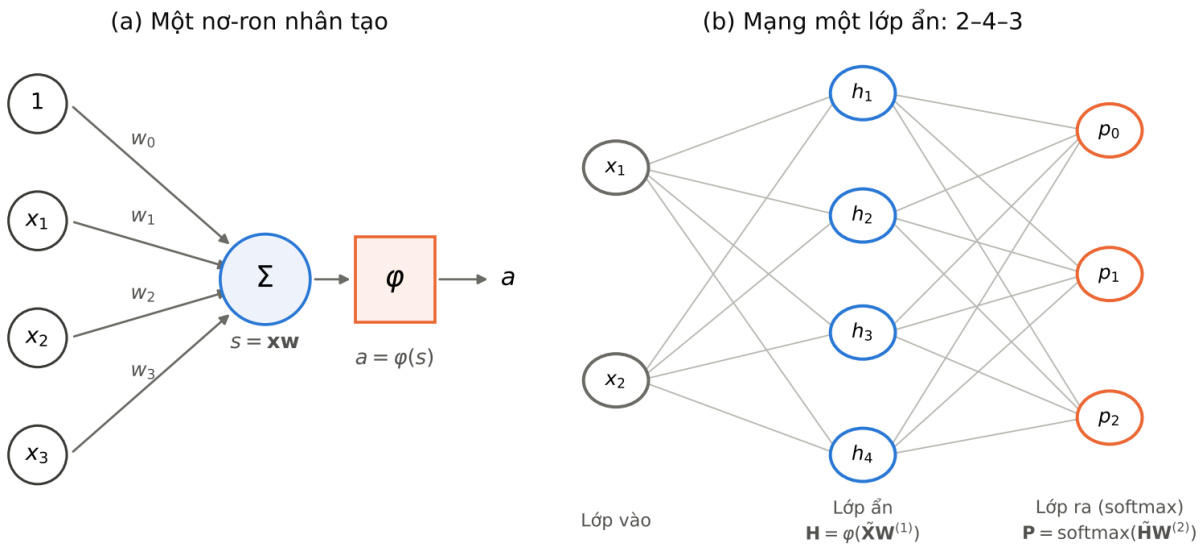
Câu chuyện bắt đầu từ trước khi có máy tính hiện đại, và có nhiều lần lên xuống:

Năm	Sự kiện	Ý nghĩa
1943	McCulloch và Pitts đề xuất mô hình nơ-ron dạng ngưỡng	Chứng tỏ các nơ-ron ngưỡng đơn giản có thể thực hiện các phép logic
1958	Rosenblatt công bố **Perceptron**	Thuật toán học tự động từ dữ liệu, sau đó được cài thành phần cứng (Mark I Perceptron)
1960	Widrow và Hoff đề xuất ADALINE	Quy tắc học dựa trên gradient của sai số bình phương
1969	Minsky và Papert xuất bản cuốn *Perceptrons*	Chỉ ra giới hạn của Perceptron một lớp (như XOR); sự quan tâm đến mạng nơ-ron giảm mạnh
1974 đến 1986	Werbos, rồi Rumelhart, Hinton và Williams phổ biến **lan truyền ngược**	Cách tính gradient hiệu quả cho mạng nhiều lớp, mở ra khả năng huấn luyện nơ-ron ẩn
1989	Cybenko, Hornik và cộng sự chứng minh định lý xấp xỉ phổ quát; LeCun áp dụng mạng tích chập cho nhận dạng chữ số	Mạng một lớp ẩn đủ rộng xấp xỉ được mọi hàm liên tục (mục 2.5); mạng cho kết quả thực tế
2012	AlexNet thắng cuộc thi nhận dạng ảnh ImageNet	Mở đầu làn sóng học sâu hiện đại với dữ liệu lớn và GPU
2017	Kiến trúc Transformer ra đời	Nền tảng của các mô hình ngôn ngữ lớn

Hai mốc đáng chú ý nhất với bài này. Mốc 1958 là **điểm xuất phát**: đúng mô hình Perceptron ta đã học, một nơ-ron với hàm dấu. Mốc 1969 là **bài học**: chính bài toán XOR ở mục 1.1 là ví dụ kinh điển cho thấy một nơ-ron đơn lẻ không đủ. Lối thoát (xếp nhiều lớp) đã được biết từ sớm, nhưng phải đến khi có lan truyền ngược (mục 3) thì việc **huấn luyện** nó mới thực tế.

Xếp chồng các nơ-ron: mạng nhiều lớp

Gộp m nơ-ron chạy song song thành một **lớp**, ta tính được tất cả bằng một phép nhân ma trận: cột thứ j của $\mathbf{W}^{(1)}$ là vector tham số của nơ-ron thứ j, và $\mathbf{H} = \varphi\left(\tilde{\mathbf{X}}\mathbf{W}^{(1)}\right)$ chứa đầu ra của cả m nơ-ron cho cả n mẫu. Đưa $\mathbf{H}$ vào một lớp Softmax, ta được **mạng nơ-ron một lớp ẩn** ở mục 1.2. Hình 3 minh họa một nơ-ron và một mạng như vậy.



Hình 3. (a) Một nơ-ron nhân tạo: tổng có trọng số của đầu vào (kèm hệ số chệch w_0) rồi qua hàm kích hoạt φ. (b) Mạng 2–4–3: hai đầu vào, bốn nơ-ron ẩn và ba nơ-ron ra (softmax). Mỗi đường nối là một trọng số; các nơ-ron ở một lớp nối với mọi nơ-ron ở lớp kế tiếp.

Các thuật ngữ cần nhớ:

6. **Lớp vào** (input layer): các đặc trưng $x_1, \ldots, x_d$. Lớp này không có tham số.
7. **Lớp ẩn** (hidden layer): các nơ-ron $h_1, \ldots, h_m$. Gọi là "ẩn" vì với chúng **ta không có nhãn trực tiếp**: dữ liệu chỉ cho biết lớp đúng của mẫu, không cho biết giá trị h_j "đúng" là bao nhiêu. Đây là điểm khó của việc huấn luyện, và là lý do ta cần lan truyền ngược (mục 3.1).
8. **Lớp ra** (output layer): K nơ-ron cho xác suất $p_0, \ldots, p_{K-1}$. Lớp này chính là Softmax Regression.
9. **Truyền thẳng** (feedforward): thông tin đi một chiều từ vào đến ra, không có vòng lặp. Mạng truyền thẳng nhiều lớp thường gọi là **perceptron nhiều lớp** (multi-layer perceptron, MLP).

Ta ghi kiến trúc bằng số nơ-ron mỗi lớp, ví dụ mạng 2–4–3. Mạng một lớp ẩn có m nơ-ron có số tham số

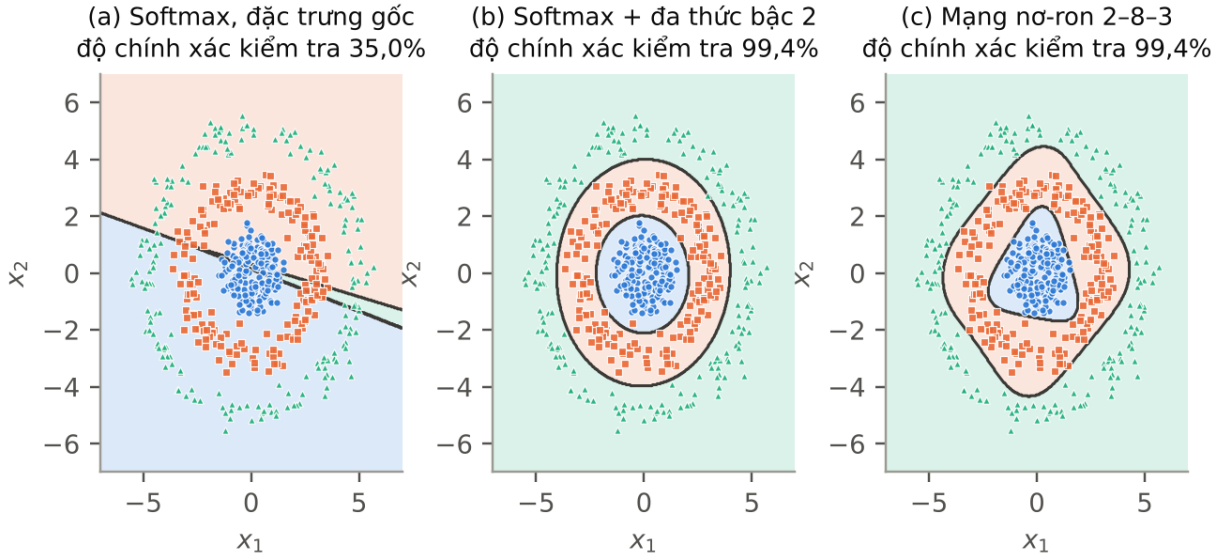
$$(d + 1)\, m + (m + 1)\, K$$

Với mạng 2–4–3 là $3 \times 4 + 5 \times 3 = 27$ tham số. Để so sánh, Softmax Regression (mạng không có lớp ẩn, kiến trúc 2–3) chỉ có $3 \times 3 = 9$ tham số. Khi bỏ lớp ẩn (coi $\mathbf{H} = \mathbf{X}$), mô hình trở về đúng Softmax Regression: **Softmax Regression là mạng nơ-ron không có lớp ẩn**.

Kết quả sơ bộ trên ba vòng tròn đồng tâm

Ta chạy mạng một lớp ẩn (hàm kích hoạt tanh, đầu ra softmax, hàm mất mát cross-entropy, huấn luyện bằng lan truyền ngược ở mục 3) trên cùng bộ dữ liệu vòng tròn, với cùng cách chia 70/30 và chuẩn hóa như bài Softmax. Mỗi cỡ lớp ẩn m được chạy với 5 khởi tạo ngẫu nhiên khác nhau:

Mô hình	Đặc trưng đưa vào	Số tham số	Độ chính xác kiểm tra
Softmax Regression	x_1, x_2	9	35,0%
Softmax Regression + đa thức bậc 2 (ta tự chọn đặc trưng)	5 đặc trưng	18	99,4%
Mạng 2–2–3	x_1, x_2	15	62,8% đến 70,0%
Mạng 2–4–3	x_1, x_2	27	96,1% đến 98,9%
Mạng 2–8–3	x_1, x_2	51	96,1% đến 99,4%



Hình 4. Ba mô hình trên dữ liệu ba vòng tròn đồng tâm. (a) Softmax Regression với đặc trưng gốc: vùng quyết định thẳng. (b) Softmax Regression với đặc trưng đa thức bậc hai do ta chọn: các biên là đường conic. (c) Mạng nơ-ron 2–8–3 chỉ nhận x_1, x_2: các biên cong do mạng tự học.

Có thể rút ra ba nhận xét, và chúng là "bản đồ" cho các mục sau:

1. **Mạng tự học được biên cong mà không cần ta biết trước dạng của chúng.** Mạng 2–8–3 chỉ nhận hai đặc trưng gốc, không được báo trước "biên là hình tròn", nhưng vẫn đạt tới 99,4%, ngang với mô hình đa thức mà ta phải tự chọn đặc trưng (Hình 4c). Cái giá là nhiều tham số hơn (51 so với 18).

2. **Kích thước lớp ẩn quan trọng.** Với $m = 2$, mạng quá nhỏ để tạo ra các vùng lồng nhau cho ba lớp, và độ chính xác chỉ khoảng 63% đến 70%. Mục 4.5 sẽ phân tích mối quan hệ giữa số nơ-ron, số lớp và khả năng biểu diễn.

3. **Kết quả phụ thuộc vào khởi tạo.** Với cùng m, các lần chạy khác nhau cho độ chính xác khác nhau. Đây là khác biệt cơ bản so với Softmax Regression, nơi hàm mất mát lồi và mọi khởi tạo cho cùng một nghiệm. Với mạng nơ-ron, hàm mất mát **không còn lồi** (mục 4.2).

Bản đồ của chương

Mỗi mục tiếp theo giải quyết một câu hỏi nảy sinh từ mục này:

Mục	Câu hỏi chính
2	Một nơ-ron, một lớp và nhiều lớp được viết bằng ma trận ra sao? Hàm kích hoạt nào, và vì sao bắt buộc phi tuyến? Mạng biểu diễn được những gì?
3	Làm sao huấn luyện khi nơ-ron ẩn không có nhãn? Gradient của từng trọng số tính thế nào (**lan truyền ngược**)?
4	Mạng học ra sao khi một mẫu bị sai? Vì sao hàm mất mát không còn lồi, và điều gì xảy ra với gradient khi mạng sâu hơn?
5	Đánh giá, chọn siêu tham số và chẩn đoán lỗi như thế nào?
6	Ưu điểm, hạn chế, và con đường đến học sâu (CNN, RNN, Transformer)

Mối liên hệ với các bài trước

Chuỗi bài giảng nay có thêm một mắt xích. Mỗi bước chỉ thay đổi **một** ý tưởng:

Perceptron $\xrightarrow{\text{làm mềm hàm dấu}}$ Logistic Regression $\xrightarrow{\text{từ một điểm số sang } K \text{ điểm số}}$ Softmax Regression $\xrightarrow[\text{xếp chồng các lớp nơ-ron, có phi tuyến}]{}$ Mạng nơ-ron

Mọi thứ ta đã học vẫn dùng được: lớp ra là Softmax Regression, hàm mất mát là cross-entropy, và gradient ở lớp ra vẫn có dạng $\mathbf{P} - \mathbf{Y}$ (bài Softmax, mục 3.3). Điều **mới** chỉ là các lớp ẩn phía trước, nơi mô hình học ra đặc trưng.

Tóm tắt mục 1

1. Softmax Regression (cũng như Logistic Regression và Perceptron) có **vùng quyết định lồi, biên là siêu phẳng**. Nó không thể phân loại XOR (hai lớp có bao lồi giao nhau) hay các lớp bị bao quanh như vòng tròn đồng tâm: trên XOR có nhiễu chỉ đạt 50,5%, trên ba vòng tròn 35,0%.

2. Với dữ liệu như vậy, hàm mất mát lồi đã hội tụ về cực tiểu toàn cục mà J vẫn cao ($\approx \ln 3$): giới hạn nằm ở **khả năng biểu diễn của mô hình**, không phải ở thuật toán hay lượng dữ liệu. Cần mô hình **phi tuyến**.

3. Đặc trưng đa thức $\boldsymbol{\phi}(\mathbf{x})$ chữa được (XOR chỉ cần thêm x_1x_2), nhưng số đặc trưng bùng nổ như $\binom{d+k}{k}$, ta phải biết trước dạng đặc trưng, và đặc trưng tách rời bộ phân loại.
4. **Ý tưởng chính:** tham số hóa đặc trưng và học chúng cùng bộ phân loại: $\mathbf{H} = \varphi(\tilde{\mathbf{X}}\mathbf{W}^{(1)})$, $\mathbf{P} = \text{softmax}(\tilde{\mathbf{H}}\mathbf{W}^{(2)})$. Hàm φ bắt buộc phi tuyến, nếu không mạng suy biến về một hàm tuyến tính.
5. **Nơ-ron nhân tạo** là $a = \varphi(\tilde{\mathbf{x}}\mathbf{w})$. Hồi quy tuyến tính, Perceptron, Logistic Regression và Softmax là các mạng **không có lớp ẩn**. Mạng nhiều lớp xếp chồng các lớp nơ-ron: lớp vào, lớp ẩn (không có nhãn trực tiếp), lớp ra; mạng một lớp ẩn có $(d+1)m + (m+1)K$ tham số.
6. Về lịch sử: Perceptron (1958) là điểm xuất phát, giới hạn XOR (1969) là bài học, lan truyền ngược (1974 đến 1986) là chìa khóa để huấn luyện mạng nhiều lớp.
7. Trên ba vòng tròn, mạng 2–8–3 tự học biên cong và đạt tới 99,4% chỉ từ x_1, x_2; đổi lại hàm mất mát không còn lồi nên kết quả phụ thuộc khởi tạo.

Câu hỏi kiểm tra nhanh

1. Dữ liệu OR có bốn điểm (0,0), (0,1), (1,0), (1,1) với nhãn 0, 1, 1, 1. Hãy tìm một vector $\mathbf{w} = [w_0 \quad w_1 \quad w_2]^T$ phân loại đúng cả bốn điểm bằng điểm số $s = \tilde{\mathbf{x}}\mathbf{w}$ (lớp 1 khi $s > 0$). Vì sao việc này làm được với OR nhưng không làm được với XOR?
2. Chứng minh rằng nếu điểm số $s(\mathbf{x}) = \tilde{\mathbf{x}}\mathbf{w}$ thì với mọi $\mathbf{x}_a$, $\mathbf{x}_b$ ta có $s\left(\frac{\mathbf{x}_a+\mathbf{x}_b}{2}\right) = \frac{s(\mathbf{x}_a)+s(\mathbf{x}_b)}{2}$. Dùng kết quả này để giải thích vì sao XOR không tách tuyến tính được.
3. Softmax Regression đã hội tụ về cực tiểu của J trên dữ liệu ba vòng tròn nhưng chỉ đạt khoảng 35%. Một bạn đề nghị "chạy thêm 10 lần số epoch". Đề nghị đó có giúp được không? Vì sao?
4. Một bộ dữ liệu có $d = 20$ đặc trưng và $K = 5$ lớp. Nếu thêm toàn bộ đặc trưng đa thức bậc 2 thì có bao nhiêu đặc trưng mới và bao nhiêu tham số của Softmax Regression?
5. Mạng một lớp ẩn có $d = 4$ đặc trưng, $m = 6$ nơ-ron ẩn và $K = 3$ lớp có bao nhiêu tham số? Kích thước của $\mathbf{W}^{(1)}$ và $\mathbf{W}^{(2)}$ là bao nhiêu?
6. Nếu hàm kích hoạt φ là hàm đồng nhất, mạng $\mathbf{S} = \tilde{\mathbf{H}}\mathbf{W}^{(2)}$ với $\mathbf{H} = \tilde{\mathbf{X}}\mathbf{W}^{(1)}$ tương đương với mô hình nào? Cho biết ma trận tham số tương đương.

7. Mạng hai lớp với $h_1 = \varphi(x_1 + x_2 - 0{,}5)$, $h_2 = \varphi(x_1 + x_2 - 1{,}5)$ (φ là hàm bước) và điểm số $s = -0{,}5 + h_1 - h_2$. Tính s tại $(1,1)$ và $(0,1)$ và cho biết lớp dự đoán.

8. Vì sao lớp ẩn được gọi là "ẩn"? Điều đó gây khó khăn gì khi huấn luyện so với lớp ra?

Gợi ý đáp án

1. Chọn $\mathbf{w} = [-0{,}5 \quad 1 \quad 1]^T$ thì $s = -0{,}5,\ 0{,}5,\ 0{,}5,\ 1{,}5$ ở bốn điểm: đúng cả bốn. Với OR, ba điểm lớp 1 và một điểm lớp 0 có bao lồi không giao nhau, còn với XOR hai đoạn thẳng nối các cặp điểm cùng lớp cắt nhau tại $(1/2\,, 1/2)$.

2. Vì s là hàm affine: $s\left(\frac{\mathbf{x}_a+\mathbf{x}_b}{2}\right) = w_0 + \frac{w_1(x_{a1}+x_{b1})+w_2(x_{a2}+x_{b2})}{2} = \frac{s(\mathbf{x}_a)+s(\mathbf{x}_b)}{2}$. Với XOR, trung điểm của hai điểm lớp 0 và của hai điểm lớp 1 trùng nhau; nếu $s < 0$ ở hai điểm lớp 0 thì $s < 0$ ở trung điểm, nếu $s > 0$ ở hai điểm lớp 1 thì $s > 0$ ở cùng điểm đó. Mâu thuẫn.

3. **Không.** J lồi và đã ở cực tiểu toàn cục ($\approx 1{,}0964 \approx \ln 3$), chạy thêm epoch không làm J giảm nữa. Giới hạn nằm ở khả năng biểu diễn của mô hình (vùng quyết định lồi), cần đổi mô hình hoặc đặc trưng.

4. Số đặc trưng mới là $\binom{22}{2} - 1 = 231 - 1 = 230$. Số tham số của Softmax Regression là $K\binom{22}{2} = 5 \times 231 = 1155$.

5. Số tham số là $(d+1)m + (m+1)K = 5 \times 6 + 7 \times 3 = 30 + 21 = 51$. $\mathbf{W}^{(1)}$ có kích thước 5×6 và $\mathbf{W}^{(2)}$ có kích thước 7×3.

6. Viết $\mathbf{W}^{(2)} = \begin{bmatrix}\mathbf{b} \\ \mathbf{V}\end{bmatrix}$ ($\mathbf{b}$ là hàng chệch, $\mathbf{V}$ có kích thước 6×3 nếu $m = 6$), ta có $\mathbf{S} = \mathbf{1b} + \tilde{\mathbf{X}}\mathbf{W}^{(1)}\mathbf{V}$. Đó là Softmax Regression với ma trận tham số $\mathbf{W}^{(1)}\mathbf{V}$ (cùng hệ số chệch $\mathbf{b}$): mạng không có thêm khả năng biểu diễn nào so với Softmax Regression.

7. Tại $(1,1)$: $h_1 = 1$, $h_2 = 1$ nên $s = -0{,}5 + 1 - 1 = -0{,}5 < 0$, dự đoán lớp 0. Tại $(0,1)$: $h_1 = 1$, $h_2 = 0$ nên $s = -0{,}5 + 1 - 0 = 0{,}5 > 0$, dự đoán lớp 1. Cả hai đều đúng.

8. Dữ liệu chỉ cho nhãn lớp ở đầu ra, không cho giá trị "đúng" của các nơ-ron ẩn. Vì vậy ta không thể tính sai số trực tiếp cho nơ-ron ẩn (như quy tắc Perceptron cần), mà phải truyền tín hiệu lỗi từ lớp ra ngược về các lớp ẩn bằng quy tắc đạo hàm hàm hợp (lan truyền ngược, mục 3).

2. Mô hình mạng nơ-ron truyền thẳng (MLP)

Ở mục 1, ta đã thấy Softmax Regression bị giới hạn bởi vùng quyết định lồi, và ta đã "phác" ý tưởng thoát khỏi giới hạn đó: để mô hình tự học đặc trưng bằng cách xếp chồng các lớp nơ-ron có hàm kích hoạt phi tuyến. Tuy nhiên, ở mục 1 ta mới viết mạng một lớp ẩn ở dạng "ví dụ", chưa nói rõ từng thành phần hoạt động ra sao. Mục này xây dựng mô hình đó **một cách có hệ thống**, từ nhỏ đến lớn:

một nơ-ron → một lớp nơ-ron → nhiều lớp nơ-ron (MLP)

Trong mục 2, ta trả lời năm câu hỏi:

1. Một nơ-ron làm gì, và Perceptron, Logistic Regression, Softmax Regression là những trường hợp riêng nào của nó?
2. Gộp nhiều nơ-ron thành một lớp thì viết bằng ma trận như thế nào, và các kích thước khớp nhau ra sao?
3. Có những hàm kích hoạt nào (sigmoid, tanh, ReLU), đạo hàm của chúng ra sao, và vì sao **bắt buộc** phải phi tuyến?
4. Lan truyền thuận qua mạng nhiều lớp được tính như thế nào, và nên chọn lớp ra và hàm mất mát nào cho từng loại bài toán?
5. Mạng có thể biểu diễn được những gì: giải XOR, tạo biên cong, và định lý xấp xỉ phổ quát nói lên điều gì?

Quy ước ký hiệu cho mạng nhiều lớp

Ta giữ quy ước của mục 1 ($\mathbf{X}$ là ma trận đặc trưng gốc kích thước $n \times d$, dấu ngã $\sim$ nghĩa là đã thêm cột 1) và bổ sung các ký hiệu theo lớp. Chỉ số trên trong ngoặc tròn, như $^{(l)}$, chỉ **lớp thứ** l, không phải lũy thừa.

Đối tượng	Ký hiệu	Kích thước	Ý nghĩa
Số lớp có tham số	L	vô hướng	Số lớp ẩn cộng một lớp ra; mạng một lớp ẩn có $L = 2$
Số nơ-ron ở lớp l	m_l	vô hướng	$m_0 = d$ (lớp vào), $m_L = K$ (lớp ra); mục 1 viết $m = m_1$
Ma trận trọng số lớp l	$\mathbf{W}^{(l)}$	$(m_{l-1} + 1) \times m_l$	Cột thứ j là vector tham số $\mathbf{w}_j^{(l)}$ của nơ-ron j ở lớp l; hàng đầu là hệ số chệch
Ma trận điểm số (trước kích hoạt)	$\mathbf{Z}^{(l)}$	$n \times m_l$	$\mathbf{Z}^{(l)} = \tilde{\mathbf{A}}^{(l-1)}\mathbf{W}^{(l)}$
Ma trận kích hoạt (đầu ra lớp l)	$\mathbf{A}^{(l)}$	$n \times m_l$	$\mathbf{A}^{(l)} = \varphi_l(\mathbf{Z}^{(l)})$

Đối tượng	Ký hiệu	Kích thước	Ý nghĩa
Đầu vào và đầu ra của mạng	$\mathbf{A}^{(0)}, \mathbf{A}^{(L)}$	$n \times d,\ n \times K$	$\mathbf{A}^{(0)} = \mathbf{X}$ và $\mathbf{A}^{(L)} = \mathbf{P}$ (với bài toán phân loại)
Phiên bản một mẫu	$\mathbf{z}_i^{(l)}, \mathbf{a}_i^{(l)}$	$1 \times m_l$	Hàng thứ i của $\mathbf{Z}^{(l)}$ và $\mathbf{A}^{(l)}$ (vector hàng)
Hàm kích hoạt của lớp l	φ_l	hàm vô hướng	Áp dụng lên **từng phần tử**; lớp ra có thể dùng hàm khác (softmax)

2.1. Một nơ-ron: Perceptron, Logistic Regression và Softmax là các trường hợp riêng

Một nơ-ron là một hàm hai bước

Một nơ-ron nhận vector đặc trưng (hàng) $\mathbf{x} = [x_1 \quad \cdots \quad x_d]$ và thực hiện **hai bước nối tiếp** mà ta đã thấy ở cả ba bài trước:

$$s = w_0 + w_1 x_1 + \cdots + w_d x_d = \tilde{\mathbf{x}}\mathbf{w}, \qquad a = \varphi(s)$$

Bước thứ nhất là **tổ hợp tuyến tính** (phần tuyến tính), bước thứ hai là **kích hoạt** (phần phi tuyến, hoặc một phép biến đổi "làm mềm"). Với n mẫu, ta tính cùng lúc bằng một phép nhân ma trận: $\mathbf{s} = \tilde{\mathbf{X}}\mathbf{w}$ và $\mathbf{a} = \varphi(\mathbf{s})$ (áp dụng φ lên từng phần tử).

Ba mô hình quen thuộc là ba cách chọn φ

Nhìn lại các bài trước dưới ánh mắt này, ta thấy mỗi mô hình chỉ khác nhau ở hàm kích hoạt và hàm mất mát:

Mô hình	Hàm kích hoạt $\varphi(s)$	Đầu ra a	Hàm mất mát
Hồi quy tuyến tính	s (hàm đồng nhất)	số thực	Bình phương sai số (MSE)
Perceptron	$\text{sign}(s)$	nhãn ± 1	Hàm mất mát Perceptron
Logistic Regression	$\sigma(s) = \dfrac{1}{1+e^{-s}}$	xác suất $p \in (0,1)$	Cross-entropy nhị phân
Softmax Regression	$\text{softmax}(\mathbf{s})$ trên K điểm số	K xác suất	Cross-entropy nhiều lớp

Hai điểm cần lưu ý:

Softmax Regression là một lớp gồm K nơ-ron có chung bước chuẩn hóa. Mỗi nơ-ron k có vector tham số $\mathbf{w}_k$ và tính điểm số $s_k = \tilde{\mathbf{x}}\mathbf{w}_k$. Khác với sigmoid (mỗi nơ-ron tự kích hoạt riêng), softmax cần **cả K điểm số** để chuẩn hóa: $p_k = e^{s_k} / \sum_j e^{s_j}$. Gộp K vector tham số thành các cột của $\mathbf{W}$, ta được đúng $\mathbf{S} = \tilde{\mathbf{X}}\mathbf{W}$ và $\mathbf{P} = \text{softmax}(\mathbf{S})$ ở bài trước.

Với $K = 2$, Softmax trở về Logistic Regression. Bài Softmax (mục 2.3) đã chứng minh $p_1 = \sigma\big(\tilde{\mathbf{x}}(\mathbf{w}_1 - \mathbf{w}_0)\big)$. Nghĩa là một nơ-ron sigmoid với $\mathbf{w} = \mathbf{w}_1 - \mathbf{w}_0$ làm đúng công việc của hai nơ-ron softmax. Vì vậy, trong mạng nơ-ron, bài toán nhị phân thường dùng **một** nơ-ron ra với sigmoid, còn bài toán $K > 2$ lớp dùng K nơ-ron ra với softmax.

Như vậy, **mỗi mô hình ta đã học là mạng nơ-ron không có lớp ẩn**: đầu vào nối thẳng tới lớp nơ-ron ra.

Hình học: một nơ-ron cắt không gian bằng một siêu phẳng

Với hàm kích hoạt dạng ngưỡng (hàm dấu hoặc hàm bước), nơ-ron chia không gian thành hai nửa theo dấu của s. Đường biên là siêu phẳng

$$w_0 + w_1 x_1 + \cdots + w_d x_d = 0$$

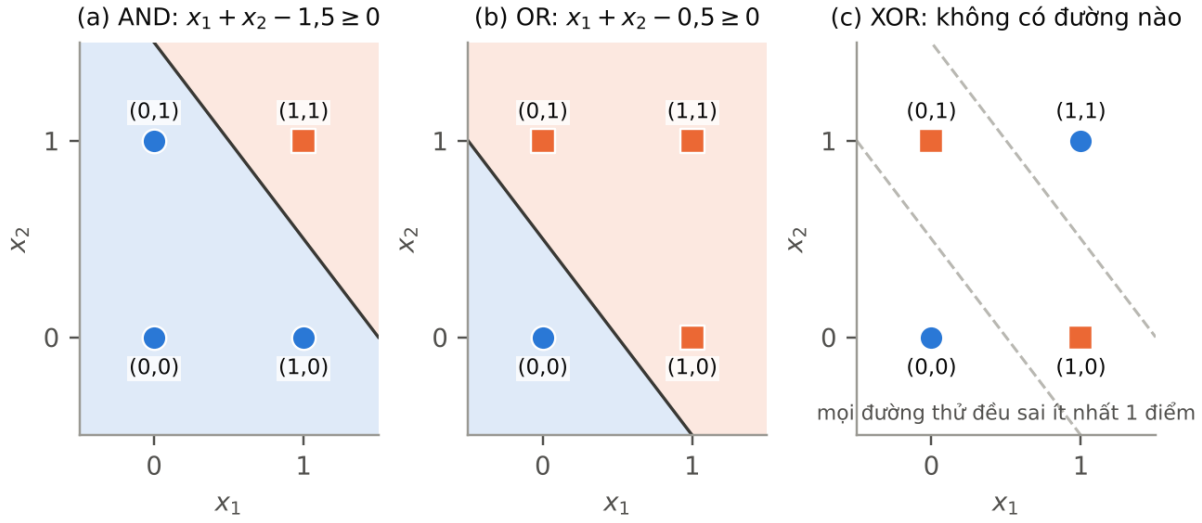
có vector pháp tuyến $[w_1 \quad \cdots \quad w_d]$; hệ số chệch w_0 quyết định siêu phẳng **dịch** đi đâu. Với sigmoid, biên $p = 0{,}5$ vẫn chính là siêu phẳng đó, nhưng xác suất đổi **mượt** khi ta rời xa nó. Chính vì mỗi nơ-ron chỉ vẽ được *một* siêu phẳng nên bộ ba mô hình trên có vùng quyết định lồi (mục 1.1).

Ví dụ: nơ-ron làm cổng logic

Một nơ-ron ngưỡng với trọng số chọn khéo có thể thực hiện các phép logic cơ bản. Với đầu vào $x_1, x_2 \in \{0,1\}$ và hàm bước $\varphi(s) = 1$ nếu $s \geq 0$, ngược lại bằng 0:

Phép logic	w_0	w_1	w_2	**Giải thích**
AND	$-1{,}5$	1	1	$s \geq 0$ chỉ khi $x_1 + x_2 \geq 1{,}5$, tức cả hai bằng 1
OR	$-0{,}5$	1	1	$s \geq 0$ khi $x_1 + x_2 \geq 0{,}5$, tức ít nhất một bằng 1
NOT x_1	0,5	-1	0	$s = 0{,}5 - x_1$, nên $s \geq 0$ khi $x_1 = 0$

Ví dụ với AND: $(1,1) \mapsto -1{,}5 + 1 + 1 = 0{,}5 \geq 0$ cho kết quả 1, còn $(1,0) \mapsto -1{,}5 + 1 = -0{,}5 < 0$ cho kết quả 0. Hình 5 vẽ đường biên của AND và OR: mỗi phép chỉ cần một đường thẳng. Với XOR (Hình 5c), như đã chứng minh ở mục 1.1, **không có đường thẳng nào** phân loại đúng cả bốn điểm.



Hình 5. Một nơ-ron ngưỡng thực hiện được AND (a) và OR (b) vì mỗi phép chỉ cần một đường thẳng, nhưng không thực hiện được XOR (c).

Lưu ý rằng XOR lại có thể viết được từ chính các phép đơn giản ấy: $x_1 \text{ XOR } x_2 = (x_1 \text{ OR } x_2) \text{ AND NOT } (x_1 \text{ AND } x_2)$. Công thức này chính là hai "bộ dò" h_1 (OR) và h_2 (AND) ở mục 1.2, được kết hợp bởi một nơ-ron thứ ba: **XOR không làm được bằng một nơ-ron, nhưng làm được bằng một mạng nhiều nơ-ron**. Để viết gọn việc kết hợp ấy cho hàng trăm nơ-ron, ta cần ký hiệu ma trận ở mục sau.

2.2. Một lớp nơ-ron: ma trận trọng số $\mathbf{W}^{(l)}$ và phép tính $\mathbf{A}^{(l)} = \boldsymbol{\varphi}\left(\widetilde{\mathbf{A}}^{(l-1)}\mathbf{W}^{(l)}\right)$

Từ một nơ-ron đến một lớp

Một **lớp** là m_l nơ-ron chạy song song, **cùng nhận một đầu vào** (đầu ra của lớp trước) và mỗi nơ-ron có vector tham số riêng. Gọi $\mathbf{a}^{(l-1)}$ là đầu ra của lớp trước (vector hàng $1 \times m_{l-1}$). Nơ-ron thứ j của lớp l tính

$$z_j^{(l)} = \tilde{\mathbf{a}}^{(l-1)}\mathbf{w}_j^{(l)}, \qquad a_j^{(l)} = \varphi_l\left(z_j^{(l)}\right), \qquad j = 1, \dots, m_l$$

Xếp m_l vector cột $\mathbf{w}_j^{(l)}$ cạnh nhau thành **ma trận trọng số** $\mathbf{W}^{(l)}$ kích thước $(m_{l-1}+1) \times m_l$:

$$\mathbf{W}^{(l)} = \begin{bmatrix} \mathbf{w}_1^{(l)} & \mathbf{w}_2^{(l)} & \cdots & \mathbf{w}_{m_l}^{(l)} \end{bmatrix} = \begin{bmatrix} b_1^{(l)} & b_2^{(l)} & \cdots & b_{m_l}^{(l)} \\ w_{11}^{(l)} & w_{12}^{(l)} & \cdots & w_{1m_l}^{(l)} \\ \vdots & \vdots & & \vdots \\ w_{m_{l-1}1}^{(l)} & w_{m_{l-1}2}^{(l)} & \cdots & w_{m_{l-1}m_l}^{(l)} \end{bmatrix}$$

Hàng đầu $\begin{bmatrix} b_1^{(l)} & \cdots & b_{m_l}^{(l)} \end{bmatrix}$ là các **hệ số chệch**, và $w_{ij}^{(l)}$ là trọng số nối từ nơ-ron i của lớp $l-1$ tới nơ-ron j của lớp l. Đây **đúng là cách xếp** của ma trận $\mathbf{W}$ ở bài Softmax Regression (mỗi cột một "giám khảo"), chỉ khác là các cột bây giờ là các nơ-ron ẩn thay vì các lớp.

Công thức một lớp

Với một mẫu, nhân vector hàng $\tilde{\mathbf{a}}^{(l-1)}$ với $\mathbf{W}^{(l)}$ cho cả m_l điểm số cùng lúc; với cả n mẫu, một phép nhân ma trận là đủ:

$$\mathbf{Z}^{(l)} = \widetilde{\mathbf{A}}^{(l-1)}\mathbf{W}^{(l)}, \qquad \mathbf{A}^{(l)} = \varphi_l\left(\mathbf{Z}^{(l)}\right), \qquad \widetilde{\mathbf{A}}^{(l-1)} = \begin{bmatrix} \mathbf{1} & \mathbf{A}^{(l-1)} \end{bmatrix}$$

Kiểm tra kích thước, một thói quen cần có: $\widetilde{\mathbf{A}}^{(l-1)}$ là $n \times (m_{l-1}+1)$, $\mathbf{W}^{(l)}$ là $(m_{l-1}+1) \times m_l$, nên $\mathbf{Z}^{(l)}$ và $\mathbf{A}^{(l)}$ đều là $n \times m_l$. Số tham số của lớp là $(m_{l-1}+1)\, m_l$. Dạng này gọi là **lớp kết nối đầy đủ** (fully connected hay dense layer), vì mỗi nơ-ron nhận tín hiệu từ **mọi** nơ-ron của lớp trước.

Có thể đọc ma trận $\mathbf{W}^{(l)}$ theo hai cách bổ sung cho nhau: **theo cột**, cột j là "bộ lọc" của nơ-ron j (nó tìm một mẫu hình nào đó trong đầu vào); **theo hàng**, hàng i cho biết đặc trưng i của lớp trước đóng góp vào tất cả các nơ-ron của lớp này ra sao.

Ví dụ: một lớp ba nơ-ron ReLU

Xét hai mẫu $\mathbf{x}_1 = [1 \quad 2]$ và $\mathbf{x}_2 = [3 \quad -1]$, và một lớp ba nơ-ron với $\varphi(z) = \max(0, z)$ (ReLU, mục 2.3):

$$\mathbf{X} = \begin{bmatrix} 1 & 2 \\ 3 & -1 \end{bmatrix}, \qquad \mathbf{W}^{(1)} = \begin{bmatrix} 0 & 1 & -2 \\ 1 & -1 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

Ma trận này có kích thước 3×3, nghĩa là $m_0 + 1 = 3$ và $m_1 = 3$. Hàng đầu $[0 \quad 1 \quad -2]$ là ba hệ số chệch. Ta tính

$$\mathbf{Z}^{(1)} = \tilde{\mathbf{X}}\mathbf{W}^{(1)} = \begin{bmatrix} 1 & 1 & 2 \\ 1 & 3 & -1 \end{bmatrix} \begin{bmatrix} 0 & 1 & -2 \\ 1 & -1 & 1 \\ -1 & 0 & 1 \end{bmatrix} = \begin{bmatrix} -1 & 0 & 1 \\ 4 & -2 & 0 \end{bmatrix}, \qquad \mathbf{A}^{(1)} = \max\left(0, \mathbf{Z}^{(1)}\right) = \begin{bmatrix} 0 & 0 & 1 \\ 4 & 0 & 0 \end{bmatrix}$$

Ví dụ phần tử đầu của $\mathbf{Z}^{(1)}$: $1 \cdot 0 + 1 \cdot 1 + 2 \cdot (-1) = -1$. Các phần tử âm bị ReLU đưa về 0: với mẫu thứ nhất chỉ nơ-ron thứ ba "bật", với mẫu thứ hai chỉ nơ-ron thứ nhất "bật". Đây là điều ta muốn: **mỗi nơ-ron chuyên phản ứng với một kiểu đầu vào**.

Mã Python tương ứng chỉ có ba dòng:

```
def them_bias(A):
    return np.hstack([np.ones((A.shape[0], 1)), A])

def lop(A_truoc, W, phi):
    """Một lớp nơ-ron: A = phi(A~ W)."""
    return phi(them_bias(A_truoc) @ W)

relu = lambda Z: np.maximum(0, Z)
X  = np.array([[1, 2], [3, -1.]])
W1 = np.array([[0, 1, -2], [1, -1, 1], [-1, 0, 1.]])
print(lop(X, W1, relu))
```

```
[[0. 0. 1.]
 [4. 0. 0.]]
```

Kết quả khớp với phép tính tay ở trên.

2.3. Hàm kích hoạt: sigmoid, tanh, ReLU và đạo hàm của chúng

Ba hàm kích hoạt phổ biến

Hàm	Công thức $\varphi(z)$	Đạo hàm $\varphi'(z)$	Miền giá trị
Sigmoid	$\sigma(z) = \dfrac{1}{1 + e^{-z}}$	$\sigma(z)\big(1 - \sigma(z)\big)$	$(0,1)$
Tanh	$\tanh(z) = \dfrac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$	$1 - \tanh^2(z)$	$(-1,1)$

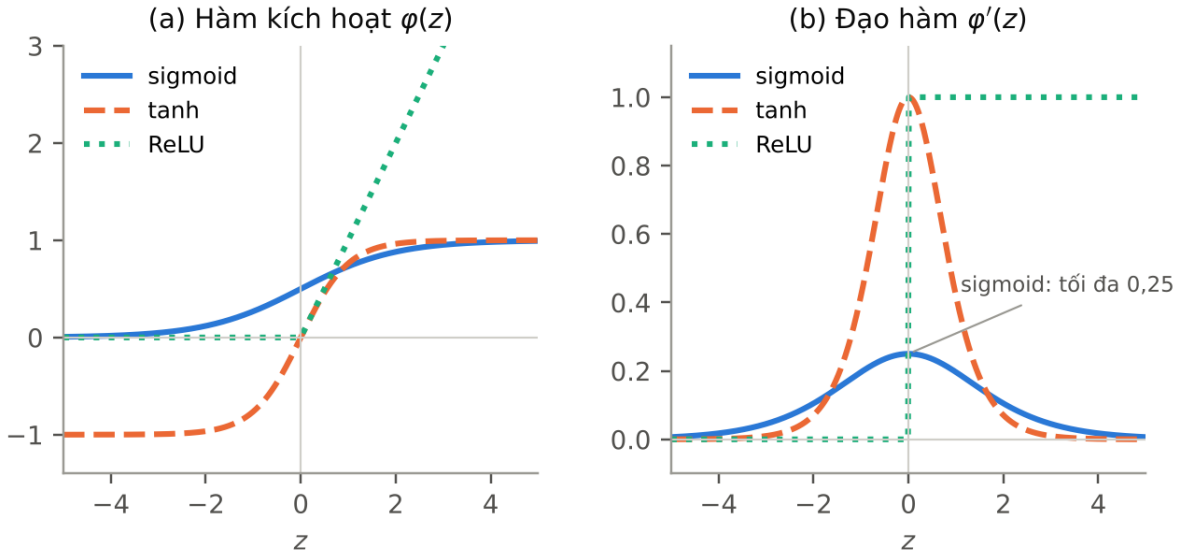
Hàm	Công thức $\varphi(z)$	Đạo hàm $\varphi'(z)$	Miền giá trị
ReLU	$\max(0, z)$	$\begin{cases} 1 & z > 0 \\ 0 & z < 0 \end{cases}$	$[0, \infty)$

Đạo hàm của sigmoid đã chứng minh ở bài Logistic Regression. Với tanh, ta có $\tanh(z) = 2\sigma(2z) - 1$ nên đạo hàm là $4\sigma'(2z) = 1 - \tanh^2(z)$. ReLU không khả vi tại $z = 0$; trong thực hành ta quy ước $\varphi'(0) = 0$ (hoặc 1), vì xác suất rơi đúng vào điểm đó gần như bằng 0 và lựa chọn này không ảnh hưởng đến huấn luyện. Điều đáng chú ý là **cả sigmoid lẫn tanh đều có đạo hàm viết qua chính đầu ra** $a = \varphi(z)$:

$$\sigma'(z) = a\,(1 - a), \qquad \tanh'(z) = 1 - a^2$$

nên khi đã tính $\mathbf{A}^{(l)}$ ở lan truyền thuận, ta tính đạo hàm gần như không tốn thêm. Tính chất này sẽ được dùng ở mục 3. Bảng sau cho vài giá trị cụ thể:

z	$\sigma(z)$	$\sigma'(z)$	$\tanh(z)$	$\tanh'(z)$	ReLU	ReLU′
-4	0,0180	0,0177	$-0{,}9993$	0,0013	0	0
-2	0,1192	0,1050	$-0{,}9640$	0,0707	0	0
0	0,5000	0,2500	0,0000	1,0000	0	0 (quy ước)
2	0,8808	0,1050	0,9640	0,0707	2	1
4	0,9820	0,0177	0,9993	0,0013	4	1



Hình 6. (a) Ba hàm kích hoạt phổ biến. (b) Đạo hàm của chúng. Đạo hàm của sigmoid không bao giờ vượt quá 0,25 và cả sigmoid lẫn tanh có đạo hàm rất nhỏ khi $|z|$ lớn ("bão hòa"); đạo hàm của ReLU bằng 1 trên toàn nửa dương.

So sánh và cách chọn

6. **Sigmoid.** Đầu ra nằm trong $(0,1)$ nên hợp để diễn giải là xác suất, vì vậy nó thường dùng ở **lớp ra** của bài toán nhị phân. Ở lớp ẩn, nó có hai nhược điểm: đầu

ra luôn dương (không tập trung quanh 0), và đạo hàm $\leq 0{,}25$ nên khi nhân qua nhiều lớp, tín hiệu gradient co lại rất nhanh (mục 4.3).

7. **Tanh.** Là phiên bản sigmoid "căn giữa" về 0 và co giãn: $\tanh(z) = 2\sigma(2z) - 1$. Đầu ra cân xứng quanh 0 thường giúp việc học dễ hơn sigmoid ở lớp ẩn, nhưng vẫn bão hòa khi $|z|$ lớn (Hình 6b).

8. **ReLU** (rectified linear unit). Tính nhanh, đạo hàm bằng 1 khi $z > 0$ nên không bão hòa ở phía dương, và cho các nơ-ron "thưa" (nhiều nơ-ron bằng 0, như ở ví dụ trên). Nhược điểm: nơ-ron có $z < 0$ với mọi mẫu sẽ có gradient bằng 0 và có thể "chết". Biến thể **Leaky ReLU**, $\varphi(z) = \max(\alpha z, z)$ với α nhỏ (ví dụ 0,01), tránh được điều này. ReLU và các biến thể là lựa chọn mặc định cho lớp ẩn trong học sâu hiện đại.

Trong bài này ta dùng tanh ở lớp ẩn cho các ví dụ nhỏ (như ở mục 1.3) vì đạo hàm gọn và dễ kiểm tra bằng tay; ReLU sẽ được dùng khi mạng sâu hơn.

Vì sao hàm kích hoạt bắt buộc phải phi tuyến?

Ở mục 1.2 ta đã thấy mạng hai lớp với kích hoạt đồng nhất suy biến về một mô hình tuyến tính. Điều đó đúng cho **mọi số lớp**. Giả sử tất cả các lớp ẩn dùng $\varphi(z) = z$. Khi đó, viết $\mathbf{W}^{(l)}$ thành hàng chệch $\mathbf{b}^{(l)}$ và phần còn lại $\mathbf{V}^{(l)}$ (kích thước $m_{l-1} \times m_l$), ta có

$$\mathbf{A}^{(l)} = \widetilde{\mathbf{A}}^{(l-1)}\mathbf{W}^{(l)} = \mathbf{1}\mathbf{b}^{(l)} + \mathbf{A}^{(l-1)}\mathbf{V}^{(l)}$$

Thay lần lượt $\mathbf{A}^{(1)}$ vào $\mathbf{A}^{(2)}$, rồi vào $\mathbf{A}^{(3)}$,… ta được

$$\mathbf{A}^{(L)} = \mathbf{1}\mathbf{b}_{\text{eff}} + \mathbf{X}\underbrace{\mathbf{V}^{(1)}\mathbf{V}^{(2)}\cdots\mathbf{V}^{(L)}}_{\widetilde{\mathbf{V}}_{\text{eff}}}$$

với $\mathbf{b}_{\text{eff}}$ là một vector hàng nào đó. Đó là một hàm **affine** của $\mathbf{x}$, tức mạng sâu bao nhiêu cũng chỉ **tương đương một lớp duy nhất** với ma trận tham số $\begin{bmatrix}\mathbf{b}_{\text{eff}} \\ \mathbf{V}_{\text{eff}}\end{bmatrix}$. Nếu lớp ra là softmax thì ta chỉ có lại Softmax Regression.

Phi tuyến là thứ biến việc xếp chồng thành khả năng biểu diễn mới. Hàm φ phải phi tuyến, và (để học bằng gradient) nên khả vi gần như khắp nơi. Ngoài ra, nó không nên là hàm đa thức, vì khi đó mạng chỉ biểu diễn được đa thức bậc hữu hạn (mục 2.5 nói thêm).

Hàm kích hoạt ở lớp ra

Lớp ra thường dùng hàm kích hoạt **khác** lớp ẩn, vì nó phải cho đầu ra đúng dạng mà bài toán cần:

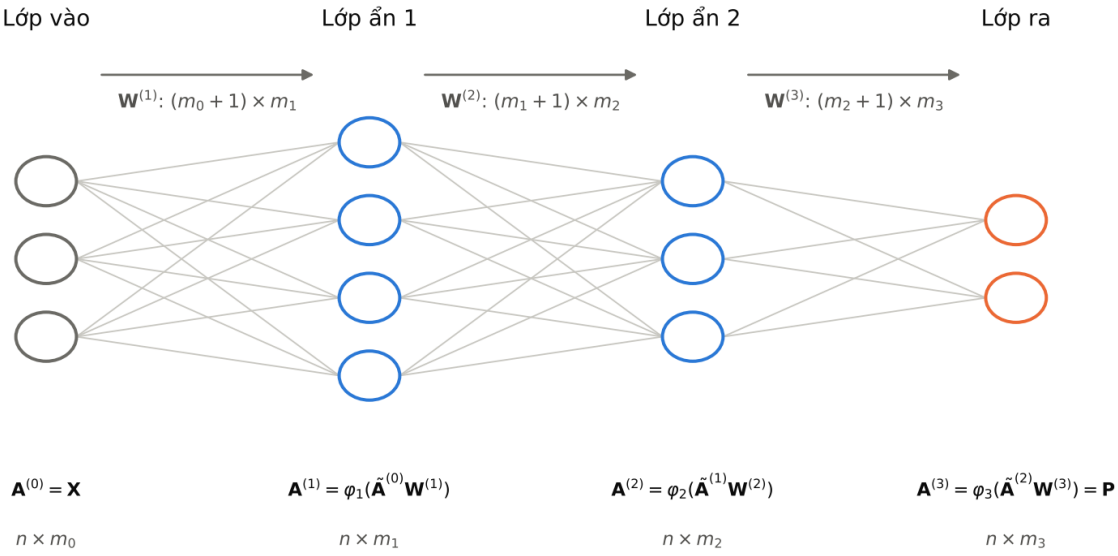
Bài toán	Số nơ-ron ra	Kích hoạt lớp ra	Ý nghĩa đầu ra
Hồi quy (dự đoán số thực)	1	Đồng nhất	Giá trị dự đoán

Bài toán	Số nơ-ron ra	Kích hoạt lớp ra	Ý nghĩa đầu ra
Phân loại nhị phân	1	Sigmoid	$p = P(y = 1 \mid \mathbf{x})$
Phân loại K lớp (mỗi mẫu một lớp)	K	Softmax	p_k, cộng lại bằng 1
Phân loại nhiều nhãn (mỗi mẫu nhiều nhãn cùng lúc)	K	K sigmoid độc lập	K xác suất không ràng buộc nhau

2.4. Mạng nhiều lớp: lan truyền thuận dạng ma trận, chọn lớp ra và hàm mất mát

Mạng truyền thẳng L lớp

Mạng truyền thẳng nhiều lớp (**MLP**, multi-layer perceptron) là phép hợp của L lớp ở mục 2.2: đầu ra của lớp này là đầu vào của lớp kế tiếp, không có vòng lặp ngược. Hình 7 minh họa mạng 3–4–3–2 (hai lớp ẩn, $L = 3$).



Hình 7. Mạng truyền thẳng 3–4–3–2 với $L = 3$ lớp có tham số. Dưới mỗi lớp là công thức và kích thước ma trận kích hoạt; phía trên các mũi tên là kích thước ma trận trọng số tương ứng. Hệ số chệch không được vẽ.

Lan truyền thuận (forward propagation)

Lan truyền thuận là quá trình tính đầu ra của mạng từ đầu vào, lớp này đến lớp kia:

9. **Khởi tạo:** $\mathbf{A}^{(0)} = \mathbf{X}$.
10. **Với** $l = 1,2,\dots,L$**:** $\mathbf{Z}^{(l)} = \tilde{\mathbf{A}}^{(l-1)}\mathbf{W}^{(l)}, \quad \mathbf{A}^{(l)} = \varphi_l(\mathbf{Z}^{(l)})$.
11. **Đầu ra:** $\hat{\mathbf{Y}} = \mathbf{A}^{(L)}$ (với bài toán K lớp, $\hat{\mathbf{Y}} = \mathbf{P}$).

Viết lồng nhau, mạng một lớp ẩn ở mục 1.2 là $\mathbf{P} = \text{softmax}\big(\widetilde{\mathbf{H}}\mathbf{W}^{(2)}\big)$ với $\mathbf{H} = \varphi\big(\widetilde{\mathbf{X}}\mathbf{W}^{(1)}\big)$. Tổng quát, mạng là một hàm hợp

$$\mathbf{P} = f_L\big(f_{L-1}(\cdots f_1(\mathbf{X})\cdots)\big), \qquad f_l(\mathbf{A}) = \varphi_l\big(\widetilde{\mathbf{A}}\mathbf{W}^{(l)}\big)$$

và chính cấu trúc hợp này khiến việc tính gradient phải dùng quy tắc đạo hàm hàm hợp (lan truyền ngược, mục 3). Ta cần **lưu lại** các ma trận trung gian $\mathbf{A}^{(l)}$ của lan truyền thuận, vì lan truyền ngược sẽ dùng lại chúng.

Bảng kích thước, với mạng m_0–m_1–⋯–m_L:

Lớp l	$\widetilde{\mathbf{A}}^{(l-1)}$	$\mathbf{W}^{(l)}$	$\mathbf{Z}^{(l)}, \mathbf{A}^{(l)}$
1	$n \times (m_0 + 1)$	$(m_0 + 1) \times m_1$	$n \times m_1$
2	$n \times (m_1 + 1)$	$(m_1 + 1) \times m_2$	$n \times m_2$
⋮	⋮	⋮	⋮
L	$n \times (m_{L-1} + 1)$	$(m_{L-1} + 1) \times m_L$	$n \times m_L$

Tổng số tham số của mạng là

$$\sum_{l=1}^{L} (m_{l-1} + 1)\, m_l$$

Ví dụ mạng 784–128–64–10 (ảnh chữ số 28×28, hai lớp ẩn, 10 lớp) có $785 \cdot 128 + 129 \cdot 64 + 65 \cdot 10 = 100\,480 + 8\,256 + 650 = 109\,386$ tham số. So với 3 085 050 tham số của Softmax Regression trên đặc trưng đa thức bậc hai ở mục 1.2, mạng này nhỏ hơn khoảng 28 lần mà vẫn học được đặc trưng phi tuyến.

Ví dụ: lan truyền thuận qua mạng 2–3–3

Ta dùng lại hai mẫu và lớp ẩn ở mục 2.2 nhưng đổi hàm kích hoạt thành tanh, rồi thêm lớp ra softmax với ba lớp. Nhãn thật của hai mẫu là $y_1 = 2$ và $y_2 = 0$ (tức $\mathbf{Y} = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix}$). Ma trận trọng số lớp ra có kích thước $(3 + 1) \times 3$:

$$\mathbf{W}^{(2)} = \begin{bmatrix} 0 & 0{,}5 & -0{,}5 \\ 1 & 0 & -1 \\ -1 & 1 & 0 \\ 0 & -1 & 1 \end{bmatrix}$$

Lớp ẩn. $\mathbf{Z}^{(1)}$ giữ nguyên như mục 2.2, rồi áp dụng tanh:

$$\mathbf{A}^{(1)} = \tanh\begin{bmatrix} -1 & 0 & 1 \\ 4 & -2 & 0 \end{bmatrix} \approx \begin{bmatrix} -0{,}7616 & 0 & 0{,}7616 \\ 0{,}9993 & -0{,}9640 & 0 \end{bmatrix}$$

Lớp ra. Thêm cột 1 rồi nhân với $\mathbf{W}^{(2)}$ để được điểm số, sau đó qua softmax từng hàng:

$$\mathbf{S} = \tilde{\mathbf{A}}^{(1)}\mathbf{W}^{(2)} \approx \begin{bmatrix} -0{,}7616 & -0{,}2616 & 1{,}0232 \\ 1{,}9634 & -0{,}4640 & -1{,}4993 \end{bmatrix}, \qquad \mathbf{P} = \text{softmax}(\mathbf{S})$$
$$\approx \begin{bmatrix} 0{,}1162 & 0{,}1916 & 0{,}6923 \\ 0{,}8932 & 0{,}0788 & 0{,}0280 \end{bmatrix}$$

Ví dụ phần tử s_{11}: $0 + 1 \cdot (-0{,}7616) + (-1) \cdot 0 + 0 \cdot 0{,}7616 = -0{,}7616$. Mỗi hàng của **P** cộng lại bằng 1 và đều là một phân phối xác suất, đúng như ở bài Softmax.

Hàm mất mát. Xác suất mạng gán cho lớp thật là $q_1 = p_{1,2} = 0{,}6923$ và $q_2 = p_{2,0} = 0{,}8932$, nên

$$l_1 = -\ln 0{,}6923 \approx 0{,}3678, \qquad l_2 = -\ln 0{,}8932 \approx 0{,}1130, \qquad J = \frac{0{,}3678 + 0{,}1130}{2}$$
$$\approx 0{,}2404$$

Mạng dự đoán đúng cả hai mẫu (argmax của hai hàng lần lượt là lớp 2 và lớp 0), mặc dù các trọng số chỉ được chọn tay. Hàm mất mát cho thấy mẫu thứ nhất được dự đoán kém tự tin hơn. Mục 3 sẽ hướng dẫn thuật toán để máy tìm các ma trận $\mathbf{W}^{(l)}$ giúp J nhỏ dần.

Mã Python cho toàn bộ phép tính trên (dùng lại *lop* và *softmax* của bài Softmax):

```
def lan_truyen_thuan(X, Ws, phis):
    """Trả về danh sách [A0, A1, ..., AL]."""
    A, cache = X, [X]
    for W, phi in zip(Ws, phis):
        A = lop(A, W, phi)
        cache.append(A)
    return cache

W2 = np.array([[0, .5, -.5], [1, 0, -1], [-1, 1, 0], [0, -1, 1.]])
cache = lan_truyen_thuan(X, [W1, W2], [np.tanh, softmax])
P = cache[-1]
y = np.array([2, 0]); Y = np.eye(3)[y]
q = (Y * P).sum(axis=1)
print(P.round(4)); print("J =", -np.log(q).mean().round(4))
```

```
[[0.1162 0.1916 0.6923]
 [0.8932 0.0788 0.028 ]]
J = 0.2404
```

Chọn lớp ra và hàm mất mát theo bài toán

Lớp ra và hàm mất mát phải **đi đôi** với nhau, theo nguyên tắc ta đã dùng suốt bài Logistic và Softmax: hàm mất mát là âm log-hợp lý của mô hình xác suất tương ứng. Khi đó gradient theo điểm số ở lớp ra luôn có dạng "dự đoán trừ thực tế":

Bài toán	Lớp ra	Hàm mất mát cho một mẫu	Gradient theo điểm số $\mathbf{z}_i^{(L)}$
Hồi quy	Đồng nhất, $\hat{y}$	$1/2\,(\hat{y} - y)^2$	$\hat{y} - y$
Nhị phân	Sigmoid, p	$-y\ln p - (1-y)\ln(1-p)$	$p - y$

Bài toán	Lớp ra	Hàm mất mát cho một mẫu	Gradient theo điểm số $\mathbf{z}_i^{(L)}$
K lớp	Softmax, $\mathbf{p}$	$-\sum_k y_k \ln p_k$	$\mathbf{p}-\mathbf{y}$

Với hồi quy, ta đã ghi hệ số $1/2$ để gradient gọn; các hằng số này không ảnh hưởng đến vị trí cực tiểu. Với bài toán phân loại K lớp, mạng nơ-ron dùng đúng cross-entropy nhiều lớp ở bài Softmax:

$$J = -\frac{1}{n}\sum_{i=1}^{n}\sum_{k=0}^{K-1} y_{ik}\ln p_{ik} = -\frac{1}{n}\mathbf{1}_n^T(\mathbf{Y}\odot \ln\mathbf{P})\mathbf{1}_K$$

Ta cũng thấy lại lý do ở bài Softmax (mục 3.1) khiến cross-entropy tốt hơn MSE cho phân loại: kết hợp softmax với cross-entropy làm gradient theo điểm số **chỉ còn $\mathbf{p}-\mathbf{y}$**, không bị thừa số $p(1-p)$ làm gradient tắt khi mô hình sai chắc chắn. Kết quả này là "bước đầu tiên" của lan truyền ngược (mục 3.3).

Các siêu tham số của kiến trúc

Khác với Softmax Regression (chỉ cần chọn η và số epoch), mạng nơ-ron có thêm các lựa chọn **kiến trúc** mà dữ liệu không tự quyết định, gọi là **siêu tham số**: số lớp ẩn $L-1$, số nơ-ron mỗi lớp $m_1, \dots, m_{L-1}$, hàm kích hoạt φ_l. Cùng với các siêu tham số của thuật toán tối ưu (tốc độ học, kích thước mini-batch, hệ số điều chuẩn hóa), chúng được chọn bằng tập kiểm định (mục 5.2). Mục 4.5 sẽ thảo luận ảnh hưởng của số nơ-ron và số lớp.

2.5. Khả năng biểu diễn: mạng giải XOR, đường biên cong, định lý xấp xỉ phổ quát

Mạng 2–2–1 giải XOR

Ở mục 1.2 ta mô tả ý tưởng hai "bộ dò" OR và AND. Giờ ta viết nó đúng bằng ngôn ngữ ma trận của mục 2.2. Với hàm bước ở cả hai lớp:

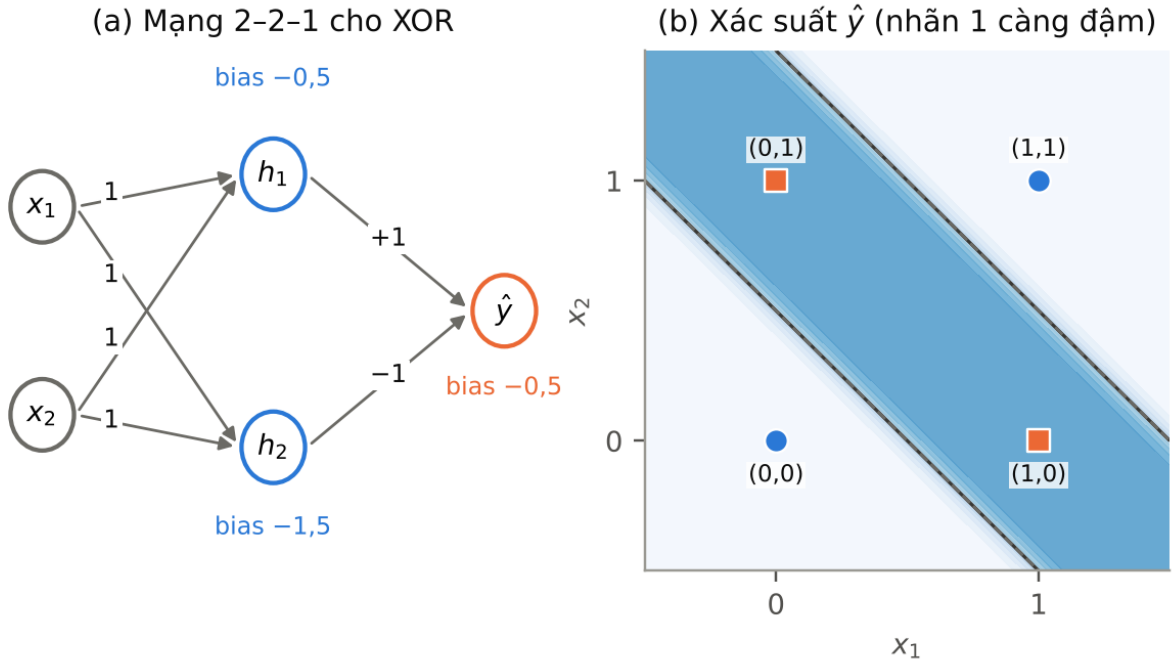
$$\mathbf{W}^{(1)} = \begin{bmatrix} -0{,}5 & -1{,}5 \\ 1 & 1 \\ 1 & 1 \end{bmatrix}, \qquad \mathbf{W}^{(2)} = \begin{bmatrix} -0{,}5 \\ 1 \\ -1 \end{bmatrix}$$

Cột thứ nhất của $\mathbf{W}^{(1)}$ là nơ-ron OR, cột thứ hai là nơ-ron AND, và $\mathbf{W}^{(2)}$ kết hợp chúng thành "OR nhưng không AND". Với bốn mẫu $\mathbf{X} = \begin{bmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{bmatrix}$:

(x_1, x_2)	$z_1^{(1)}$	$z_2^{(1)}$	h_1	h_2	$z^{(2)} = -0{,}5 + h_1 - h_2$	$\hat{y}$	Nhãn
$(0,0)$	$-0{,}5$	$-1{,}5$	0	0	$-0{,}5$	0	0
$(0,1)$	$0{,}5$	$-0{,}5$	1	0	$0{,}5$	1	1

(x_1, x_2)	$z_1^{(1)}$	$z_2^{(1)}$	h_1	h_2	$z^{(2)} = -0{,}5 + h_1 - h_2$	$\hat{y}$	**Nhãn**
$(1,0)$	$0{,}5$	$-0{,}5$	1	0	$0{,}5$	1	1
$(1,1)$	$1{,}5$	$0{,}5$	1	1	$-0{,}5$	0	0

Cả bốn mẫu được phân loại đúng. Điều quan trọng là **nguyên nhân**: lớp ẩn là phép biến đổi **phi tuyến** biến bốn điểm XOR thành ba điểm $(0,0)$, $(1,0)$, $(1,1)$ trong không gian (h_1, h_2) (hai điểm lớp 1 chập thành một), nơi một đường thẳng đã đủ tách. Nếu thay hàm bước bằng sigmoid $\sigma(10z)$ ở cả hai lớp (mềm, khả vi, dùng được cho lan truyền ngược), xác suất ra cho bốn mẫu là $0{,}007$; $0{,}992$; $0{,}992$; $0{,}007$: vẫn đúng và còn tự tin. Hình 8 vẽ mạng và vùng quyết định của nó: biên không còn là một đường thẳng mà là **một dải nằm giữa hai đường song song**, hai đường chính là biên của hai nơ-ron ẩn. Vùng của lớp 1 ($\hat{y} \approx 1$) là giao của hai nửa mặt phẳng ($x_1 + x_2 \geq 0{,}5$ và $x_1 + x_2 < 1{,}5$), còn vùng của lớp 0 là **hợp của hai** nửa mặt phẳng nằm ở hai phía của dải đó, tức là **không lồi**: lớp 0 gồm hai nhóm rời nhau $(0,0)$ và $(1,1)$. Đây đúng là điều Softmax Regression không làm được.



Hình 8. (a) Mạng 2–2–1 giải XOR; số trên các cạnh là trọng số, số cạnh các nơ-ron là hệ số chệch. (b) Xác suất lớp 1 của mạng với sigmoid $\sigma(10z)$: vùng sẫm (lớp 1) là dải giữa hai đường biên của hai nơ-ron ẩn (nét đứt).

Đường biên cong và vùng không lồi

Một nơ-ron ẩn tạo một "đường thẳng"; một lớp ẩn có m nơ-ron tạo m đường thẳng, và lớp ra **tổ hợp** các nửa mặt phẳng đó (giao, hợp, trừ) để dựng nên vùng quyết định phức tạp. Khi dùng hàm kích hoạt mềm như tanh, các biên gấp khúc trở thành các đường **cong trơn**. Đó là cách mạng 2–8–3 ở Hình 4c (mục 1.3) tạo ra các vòng khép kín ôm lấy lớp ở giữa, thứ mà Softmax Regression không tạo được vì mọi vùng của nó đều lồi.

Có thể khái quát thành một nguyên lý: **số nơ-ron ẩn càng nhiều thì vùng quyết định càng có thể phức tạp**. Với $m = 2$ nơ-ron ẩn, ta chỉ vẽ được hai đường biên, chưa đủ để lồng hai vòng tròn vào nhau (độ chính xác 63% đến 70% ở mục 1.3). Với $m = 4$ hoặc 8, mạng có đủ "nguyên liệu" và đạt trên 96%.

Định lý xấp xỉ phổ quát (mức trực giác)

Câu hỏi tự nhiên tiếp theo là: mạng có giới hạn biểu diễn nào không? Có một kết quả toán học đáng ngạc nhiên:

> **Định lý xấp xỉ phổ quát** (Cybenko 1989, Hornik và cộng sự 1989, 1991). Cho một hàm liên tục g trên một tập compact (ví dụ một hình hộp) và một sai số $\varepsilon > 0$ bất kỳ. Với hàm kích hoạt phi tuyến, bị chặn và tăng (ví dụ sigmoid), tồn tại một mạng **một lớp ẩn** với số nơ-ron ẩn m đủ lớn sao cho hàm do mạng tính được khác g không quá ε tại mọi điểm của tập đó.

Nói cách khác, một lớp ẩn đủ rộng **xấp xỉ được mọi hàm liên tục** với độ chính xác tùy ý. Kết quả này cũng đúng với ReLU và với mọi hàm kích hoạt liên tục, không phải đa thức (Leshno và cộng sự 1993), đây là lý do mục 2.3 nói hàm kích hoạt không nên là đa thức. Với bài toán phân loại, hệ quả là vùng quyết định (miền mà một hàm liên tục nào đó vượt ngưỡng) cũng xấp xỉ được tùy ý.

Trực giác bằng một chiều. Ta không chứng minh định lý, nhưng có thể thấy cơ chế bằng một dựng hình đơn giản. Một nơ-ron sigmoid $\sigma\big(k(x-t)\big)$ với k lớn là một **bậc thang mượt**: gần bằng 0 khi $x < t$ và gần bằng 1 khi $x > t$. Hiệu của hai bậc thang cho một "đỉnh" (Hình 9a):

$$\sigma\big(k(x-a)\big) - \sigma\big(k(x-b)\big) \approx \begin{cases} 1 & a < x < b \\ 0 & \text{ngoài khoảng đó} \end{cases}$$

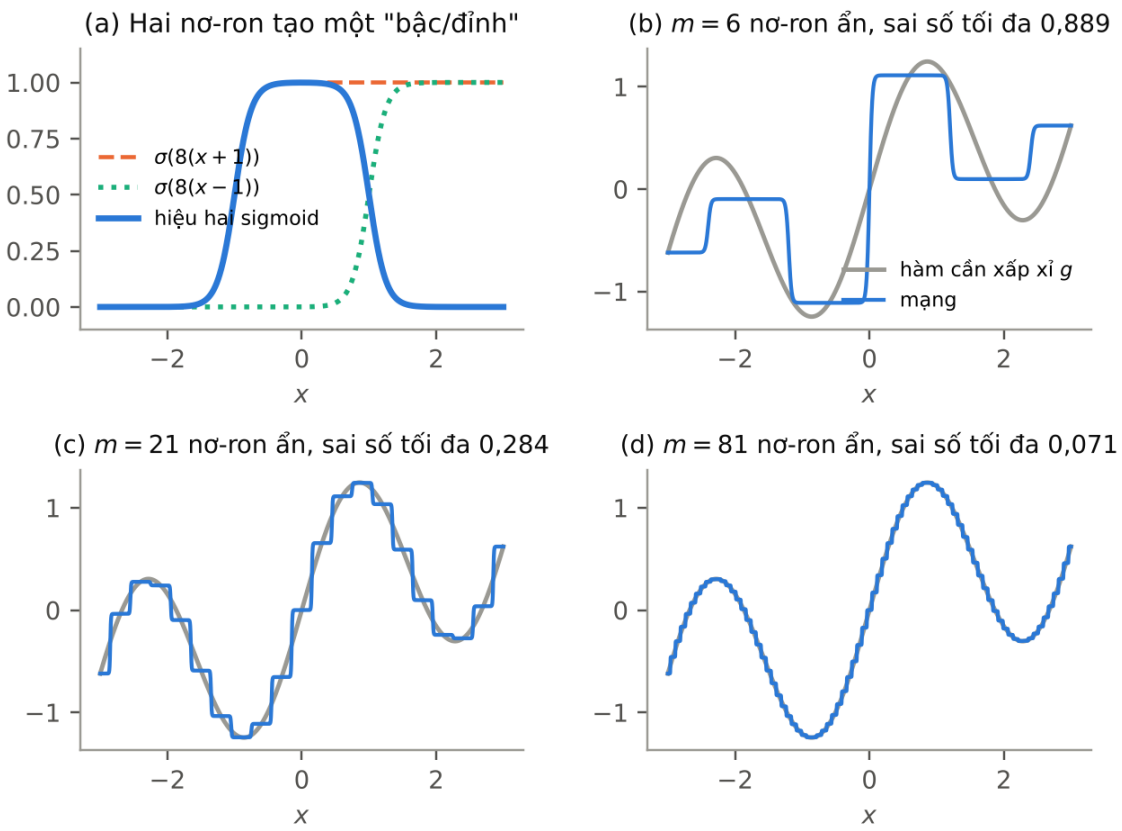
Để xấp xỉ g trên $[-3,3]$, ta chia thành m đoạn nhỏ bằng nhau, tại mỗi vị trí đặt một bậc thang có độ cao bằng **phần tăng thêm** của g ở đó:

$$f(x) = \sum_{j=1}^{m} c_j \, \sigma\Big(k\big(x - t_j\big)\Big), \qquad c_j = g\big(x_j\big) - g\big(x_{j-1}\big),\ g(x_0) := 0$$

Biểu thức này đúng là một **mạng một lớp ẩn** với m nơ-ron ẩn: trọng số vào k, hệ số chệch $-k\,t_j$, trọng số ra c_j. Chia càng nhiều đoạn thì f càng sát g. Ta thử với $g(x) = \sin(2x) + 0{,}3x$ trên $[-3,3]$:

Số nơ-ron ẩn m	**Số tham số** $3m + 1$	**Sai số lớn nhất** $\max_x \lvert f(x) - g(x) \rvert$
6	19	0,889
21	64	0,284
81	244	0,071

Số nơ-ron tăng khoảng 4 lần (từ 21 lên 81) thì sai số giảm khoảng 4 lần: sai số tỉ lệ với **độ rộng mỗi đoạn**, vốn tỉ lệ với $1/m$ (Hình 9b đến d).



Hình 9. Trực giác của định lý xấp xỉ phổ quát. (a) Hiệu của hai sigmoid dốc tạo một "đỉnh". (b), (c), (d) Tổng m bậc thang sigmoid xấp xỉ $g(x) = \sin(2x) + 0{,}3x$ (nét xám); m càng lớn, sai số càng nhỏ.

Định lý nói gì, và không nói gì

Định lý rất đẹp, nhưng cần đọc đúng, vì nó dễ bị hiểu thành một lời hứa nhiều hơn những gì nó đảm bảo:

Định lý nói	**Định lý không nói**
Tồn tại một mạng một lớp ẩn đủ rộng xấp xỉ được hàm đích	Cần **bao nhiêu** nơ-ron: có hàm đòi số nơ-ron rất lớn (tăng nhanh theo số chiều d)
Mạng không bị giới hạn về *khả năng biểu diễn* như mô hình tuyến tính	Thuật toán huấn luyện có **tìm được** các trọng số đó hay không (hàm mất mát không còn lồi, mục 4.2)
Một lớp ẩn là đủ về mặt lý thuyết	Mạng sẽ **khái quát hóa** tốt trên dữ liệu mới (có thể khớp quá mức nhiễu, mục 4.4)

Cũng vì lý do thứ nhất ở bảng trên, trong thực hành người ta thường dùng **mạng sâu hơn** thay vì rộng hơn: một số hàm mà mạng một lớp ẩn cần số nơ-ron rất lớn thì mạng nhiều lớp biểu diễn gọn hơn nhiều, nhờ mỗi lớp tái sử dụng đặc trưng của lớp trước. Ta sẽ so sánh rộng và sâu ở mục 4.5.

Mối liên hệ với mục trước và mục sau

Mục 1 đặt vấn đề và phác ý tưởng; mục 2 hoàn thiện **mô hình**. Ta có thể đối chiếu với khung "mô hình, hàm mất mát, thuật toán tối ưu":

Thành phần	Softmax Regression	Mạng nơ-ron (MLP)
Mô hình	$\mathbf{P} = \text{softmax}(\tilde{\mathbf{X}}\mathbf{W})$	$\mathbf{A}^{(l)} = \varphi_l(\tilde{\mathbf{A}}^{(l-1)}\mathbf{W}^{(l)})$, $\mathbf{P} = \mathbf{A}^{(L)}$
Hàm mất mát	Cross-entropy, **lồi** theo $\mathbf{W}$	Cùng cross-entropy, **không lồi** theo $\{\mathbf{W}^{(l)}\}$
Gradient	$\frac{1}{n}\tilde{\mathbf{X}}^T(\mathbf{P} - \mathbf{Y})$	Lan truyền ngược (mục 3)

Điểm khác biệt duy nhất ở bảng, ở dòng gradient, chính là chủ đề của mục 3: mạng có hàng trăm ma trận trọng số lồng vào nhau, nên ta cần một cách có hệ thống để tính gradient theo từng ma trận $\mathbf{W}^{(l)}$, trong khi các nơ-ron ẩn không có nhãn "đúng" để so sánh.

Tóm tắt mục 2

12. **Một nơ-ron** là $a = \varphi(\tilde{\mathbf{x}}\mathbf{w})$: tổ hợp tuyến tính rồi kích hoạt. Hồi quy tuyến tính (φ = đồng nhất), Perceptron (φ = hàm dấu), Logistic Regression (φ = sigmoid) và Softmax Regression (K nơ-ron dùng chung softmax) là các mạng **không có lớp ẩn**. Một nơ-ron vẽ một siêu phẳng; nó làm được AND, OR, NOT nhưng không làm được XOR.

13. **Một lớp** gồm m_l nơ-ron có ma trận trọng số $\mathbf{W}^{(l)}$ kích thước $(m_{l-1} + 1) \times m_l$ (mỗi cột một nơ-ron, hàng đầu là hệ số chệch), và tính $\mathbf{A}^{(l)} = \varphi_l(\tilde{\mathbf{A}}^{(l-1)}\mathbf{W}^{(l)})$ cho cả n mẫu bằng một phép nhân ma trận. Số tham số của lớp là $(m_{l-1} + 1)\, m_l$.

14. **Hàm kích hoạt:** sigmoid $\sigma' = a(1 - a)$, tanh $\tanh' = 1 - a^2$, ReLU $\max(0, z)$ với đạo hàm 0 hoặc 1. Sigmoid và tanh bão hòa (đạo hàm rất nhỏ khi $|z|$ lớn), ReLU không bão hòa ở phía dương và là lựa chọn mặc định cho lớp ẩn. **Phi tuyến là bắt buộc**: nếu tất cả lớp ẩn là đồng nhất, mạng L lớp chỉ là một hàm affine với ma trận $\mathbf{V}^{(1)} \cdots \mathbf{V}^{(L)}$.

15. **Lan truyền thuận** qua mạng L lớp: $\mathbf{A}^{(0)} = \mathbf{X}$, rồi $\mathbf{Z}^{(l)} = \tilde{\mathbf{A}}^{(l-1)}\mathbf{W}^{(l)}$, $\mathbf{A}^{(l)} = \varphi_l(\mathbf{Z}^{(l)})$ với $l = 1, \ldots, L$. Tổng số tham số là $\sum_l (m_{l-1} + 1)\, m_l$. Lớp ra và hàm mất mát đi đôi: hồi quy (đồng nhất, MSE), nhị phân (sigmoid, cross-entropy nhị phân), K lớp (softmax, cross-entropy nhiều lớp); gradient theo điểm số ở lớp ra luôn là "dự đoán trừ thực tế".

16. **Khả năng biểu diễn:** mạng 2–2–1 giải được XOR bằng hai nơ-ron ẩn OR và AND; nhiều nơ-ron ẩn tạo biên cong và vùng không lồi. Định lý xấp xỉ phổ quát nói một lớp ẩn đủ rộng xấp xỉ được mọi hàm liên tục (trực giác: tổng các bậc

thang sigmoid; sai số tỉ lệ với $1/m$), nhưng **không** đảm bảo số nơ-ron vừa phải, cũng không đảm bảo huấn luyện tìm được nghiệm hay mạng khái quát hóa tốt.

Câu hỏi kiểm tra nhanh

17. Một nơ-ron có $\mathbf{w} = [-1 \quad 2 \quad -3]^T$ nhận $\mathbf{x} = [1 \quad 2]$. Tính s, rồi đầu ra a nếu hàm kích hoạt là sigmoid, tanh, ReLU.

18. Một mô hình Softmax hai lớp có $\mathbf{w}_0 = [1 \quad 0 \quad 2]^T$ và $\mathbf{w}_1 = [0 \quad 1 \quad 1]^T$. Tìm vector tham số $\mathbf{w}$ của nơ-ron sigmoid tương đương. Tính xác suất lớp 1 tại $\mathbf{x} = [2 \quad 1]$.

19. Mạng 5–8–4–3 có kích thước của $\mathbf{W}^{(1)}$, $\mathbf{W}^{(2)}$, $\mathbf{W}^{(3)}$ là bao nhiêu? Có bao nhiêu tham số?

20. Tính $\sigma'(0)$, $\tanh'(1)$ (biết $\tanh 1 \approx 0{,}7616$) và $\mathrm{ReLU}'(-3)$.

21. Chứng minh rằng mạng hai lớp ($L = 2$) với kích hoạt đồng nhất ở lớp ẩn và softmax ở lớp ra tương đương Softmax Regression. Ma trận tham số tương đương là gì?

22. Dùng mạng XOR ở mục 2.5, tính $\mathbf{z}^{(1)}$, $\mathbf{h}$ và $\hat{y}$ tại $(1,0)$ (hàm bước).

23. Một nơ-ron ra sigmoid có $\mathbf{w} = [0{,}5 \quad 1 \quad -1 \quad 2]^T$ nhận đầu vào là kích hoạt lớp ẩn $\mathbf{a}^{(1)} = [0 \quad 0 \quad 1]$ (mẫu thứ nhất ở mục 2.2). Tính xác suất đầu ra.

24. Nên chọn lớp ra và hàm mất mát cho: (a) dự đoán giá nhà; (b) nhận diện ảnh có chứa người hay không; (c) nhận diện chữ số viết tay; (d) gắn thẻ cho bài báo (một bài có thể thuộc nhiều chủ đề cùng lúc).

25. Trong thí nghiệm xấp xỉ $g(x) = \sin(2x) + 0{,}3x$, sai số giảm từ $0{,}284$ ($m = 21$) xuống $0{,}071$ ($m = 81$). Tỉ lệ giảm là bao nhiêu, và nó nói điều gì về quan hệ giữa sai số và m?

Gợi ý đáp án

26. $s = -1 + 2 \cdot 1 - 3 \cdot 2 = -5$. Sigmoid: $\sigma(-5) \approx 0{,}0067$. Tanh: $\tanh(-5) \approx -0{,}9999$. ReLU: 0.

27. $\mathbf{w} = \mathbf{w}_1 - \mathbf{w}_0 = [-1 \quad 1 \quad -1]^T$. Tại $\mathbf{x} = [2 \quad 1]$: $s = -1 + 2 - 1 = 0$ nên $p_1 = \sigma(0) = 0{,}5$.

28. $\mathbf{W}^{(1)}$ có kích thước 6×8, $\mathbf{W}^{(2)}$ là 9×4, $\mathbf{W}^{(3)}$ là 5×3. Số tham số: $48 + 36 + 15 = 99$.

29. $\sigma'(0) = 0{,}25$. $\tanh'(1) = 1 - 0{,}7616^2 \approx 1 - 0{,}5800 = 0{,}4200$. $\mathrm{ReLU}'(-3) = 0$.

30. Với $\mathbf{A}^{(1)} = \tilde{\mathbf{X}}\mathbf{W}^{(1)}$ và $\mathbf{W}^{(2)} = \begin{bmatrix}\mathbf{b}\\ \mathbf{V}\end{bmatrix}$ thì $\mathbf{S} = \mathbf{1b} + \tilde{\mathbf{X}}\mathbf{W}^{(1)}\mathbf{V}$ ($\mathbf{W}^{(1)}\mathbf{V}$ là ma trận $(d+1) \times K$). Ta có $\mathbf{S} = \tilde{\mathbf{X}}\mathbf{W}_{\text{eff}}$ với $\mathbf{W}_{\text{eff}} = \mathbf{W}^{(1)}\mathbf{V} + \mathbf{e}_1\mathbf{b}$, trong đó $\mathbf{e}_1 = [1 \quad 0 \quad \cdots \quad 0]^T$ chỉ cộng $\mathbf{b}$ vào hàng đầu (hàng hệ số chệch). Đó là Softmax Regression với ma trận $\mathbf{W}_{\text{eff}}$.

31. Tại $(1,0)$: $\mathbf{z}^{(1)} = [-0{,}5 + 1 \quad -1{,}5 + 1] = [0{,}5 \quad -0{,}5]$, nên $\mathbf{h} = [1 \quad 0]$. Điểm số ra: $-0{,}5 + 1 - 0 = 0{,}5 \geq 0$ nên $\hat{y} = 1$, đúng với nhãn XOR.

32. $s = 0{,}5 + 1 \cdot 0 + (-1) \cdot 0 + 2 \cdot 1 = 2{,}5$, nên $p = \sigma(2{,}5) \approx 0{,}9241$.

33.
    (a) Hồi quy: một nơ-ron ra đồng nhất, MSE. (b) Nhị phân: một nơ-ron ra sigmoid, cross-entropy nhị phân. (c) 10 lớp loại trừ nhau: 10 nơ-ron ra softmax, cross-entropy nhiều lớp. (d) Nhiều nhãn cùng lúc: mỗi chủ đề một nơ-ron sigmoid độc lập, tổng các cross-entropy nhị phân (không dùng softmax vì các xác suất không bị ràng buộc cộng bằng 1).

34. $0{,}284/0{,}071 \approx 4{,}0$, đúng bằng tỉ lệ $81/21 \approx 3{,}9$ của số nơ-ron. Sai số tỉ lệ nghịch với m (tỉ lệ với độ rộng đoạn chia $6/m$), với hàm trơn như g.

3. Huấn luyện mạng nơ-ron: lan truyền ngược

Ở mục 2, ta đã hoàn thiện **mô hình**: mạng nhiều lớp tính $\mathbf{A}^{(l)} = \varphi_l\big(\tilde{\mathbf{A}}^{(l-1)}\mathbf{W}^{(l)}\big)$ và cho ra $\mathbf{P} = \mathbf{A}^{(L)}$. Ta cũng biết hàm mất mát phù hợp là cross-entropy. Nhưng các ma trận $\mathbf{W}^{(1)}, \dots, \mathbf{W}^{(L)}$ vẫn do ta chọn tay (như ở các ví dụ 2–3–3 và 2–2–1). Mục này để máy tự tìm chúng từ dữ liệu. Ta tiếp tục bám theo khung đã dùng từ đầu chuỗi bài:

$$\text{Mô hình} + \text{Hàm mất mát} + \text{Thuật toán tối ưu} = \text{Huấn luyện}$$

Hai thành phần đầu đã có. Thuật toán tối ưu vẫn là Gradient Descent quen thuộc, áp dụng cho từng ma trận trọng số:

$$\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \eta\, \nabla_{\mathbf{W}^{(l)}} J, \qquad l = 1, \dots, L$$

Vì vậy toàn bộ khó khăn quy về **một việc**: tính $\nabla_{\mathbf{W}^{(l)}} J$ cho **mọi** lớp l. Với Softmax Regression, việc này chỉ cần một công thức $\frac{1}{n}\tilde{\mathbf{X}}^T(\mathbf{P} - \mathbf{Y})$. Với mạng nhiều lớp, các ma trận lồng vào nhau nên ta cần một phương pháp có hệ thống: **thuật toán lan truyền ngược** (backpropagation).

Trong mục 3, ta trả lời sáu câu hỏi:

35. Vì sao các quy tắc học đã biết (Perceptron, Logistic, Softmax) không dùng được trực tiếp cho nơ-ron ẩn, và vì sao không thể chỉ dùng đạo hàm số?
36. Quy tắc đạo hàm hàm hợp và đồ thị tính toán cho ta công cụ gì?
37. Gradient của từng ma trận trọng số trong mạng một lớp ẩn được suy ra như thế nào?
38. Thuật toán tổng quát cho mạng L lớp ra sao, và làm sao biết cài đặt của mình đúng?
39. Cần chọn cách khởi tạo, tốc độ học và thuật toán tối ưu (SGD, momentum, Adam) thế nào?
40. Toàn bộ quy trình trông như thế nào khi tính bằng tay và khi lập trình từ đầu?

Ký hiệu bổ sung cho mục 3

Đối tượng	Ký hiệu	Kích thước	Ý nghĩa
Sai số tại lớp l	$\mathbf{D}^{(l)} = \dfrac{\partial J}{\partial \mathbf{Z}^{(l)}}$	$n \times m_l$	Đạo hàm của J theo điểm số (trước kích hoạt) của lớp l; gọi là "tín hiệu lỗi"
Trọng số không kể chệch	$\mathbf{V}^{(l)}$	$m_{l-1} \times m_l$	$\mathbf{W}^{(l)}$ bỏ hàng đầu (hàng hệ số chệch)
Đạo hàm hàm kích hoạt	$\varphi_l'\big(\mathbf{Z}^{(l)}\big)$	$n \times m_l$	Đạo hàm áp dụng lên từng phần tử của $\mathbf{Z}^{(l)}$

Đối tượng	Ký hiệu	Kích thước	Ý nghĩa
Phép nhân từng phần tử	$\odot$		$(\mathbf{A} \odot \mathbf{B})_{ij} = a_{ij} b_{ij}$
Gradient theo trọng số	$\nabla_{\mathbf{W}^{(l)}} J$	$(m_{l-1} + 1) \times m_l$	Cùng kích thước với $\mathbf{W}^{(l)}$

3.1. Vì sao cần lan truyền ngược?

Điều ta đã biết cách làm: học khi có "đáp án" tại nơ-ron

Nhìn lại cách ba mô hình trước học. Quy tắc **Perceptron** (nhãn ± 1) khi gặp mẫu bị phân loại sai là kéo vector tham số về phía mẫu đó: $\mathbf{w} \leftarrow \mathbf{w} + \eta\, y_i \tilde{\mathbf{x}}_i^T$. **Logistic Regression** và **Softmax Regression** tính gradient

$$\nabla_{\mathbf{W}} J = \frac{1}{n} \tilde{\mathbf{X}}^T (\mathbf{P} - \mathbf{Y})$$

Cả hai có một điểm chung then chốt: **nơ-ron nào cũng có "đáp án" trực tiếp** từ dữ liệu. Nơ-ron ra thứ k của Softmax được so với cột thứ k của $\mathbf{Y}$, nên sai số $p_{ik} - y_{ik}$ tính được ngay. Ta chỉ việc nhân sai số đó với đầu vào $\tilde{\mathbf{x}}_i^T$ của nơ-ron.

Điều khó: nơ-ron ẩn không có đáp án

Xét mạng 2–2–1 cho XOR. Nhãn y_i chỉ cho biết đầu ra cuối cùng $\hat{y}_i$ nên được là 0 hay 1. Với hai nơ-ron ẩn h_1, h_2, **không ai cho ta biết** h_1 "đúng" phải bằng bao nhiêu với mẫu $(0,1)$. Nếu dự đoán sai, lỗi nằm ở nơ-ron ra, hay ở h_1, hay ở h_2, hay ở cả ba? Đây gọi là **bài toán gán trách nhiệm** (credit assignment): với hàng chục hay hàng triệu trọng số, ta phải xác định mỗi trọng số **góp bao nhiêu** vào sai số cuối cùng.

Quy tắc Perceptron không trả lời được câu hỏi này vì nó cần sai số của *chính nơ-ron đang cập nhật*, mà ta không có sai số đó cho nơ-ron ẩn. Đây là lý do mô hình nhiều lớp từng bị bỏ quên sau năm 1969 (mục 1.3): mô hình biểu diễn được, nhưng chưa có cách huấn luyện hiệu quả.

Phương án thử: đạo hàm số

Có một cách "ngây thơ" luôn dùng được: tính đạo hàm theo từng tham số bằng định nghĩa. Với mỗi trọng số w, ta thay w bằng $w \pm \varepsilon$ và chạy lại lan truyền thuận:

$$\frac{\partial J}{\partial w} \approx \frac{J(w + \varepsilon) - J(w - \varepsilon)}{2\varepsilon}$$

Cách này đúng nhưng đắt: mỗi tham số cần **hai** lần lan truyền thuận, nên một lần tính gradient cho P tham số tốn $2P$ lần lan truyền thuận.

Mạng	Số tham số P	Số lần lan truyền thuận để có một gradient (đạo hàm số)
2–4–3	27	54
2–8–3	51	102

Mạng	Số tham số P	Số lần lan truyền thuận để có một gradient (đạo hàm số)
784–128–64–10	109 386	218 772
Mạng cỡ 10^8 tham số	10^8	2×10^8

Ta đo thử trên mạng 2–8–3 với 420 mẫu của dữ liệu vòng tròn: một lần lan truyền thuận mất khoảng 0,063 ms, một lần lan truyền thuận cộng ngược mất 0,098 ms (chỉ gấp 1,56 lần), còn đạo hàm số mất 9,6 ms, tức **gấp khoảng** 98 **lần**, đúng bằng cỡ $2P$. Với mạng lớn, chênh lệch này là không thể chấp nhận (huấn luyện cần hàng triệu lần cập nhật).

Lan truyền ngược: dùng đạo hàm hàm hợp và tái sử dụng kết quả

Lan truyền ngược trả lời cả hai câu hỏi trên bằng một ý tưởng: mạng là một **hàm hợp** của nhiều phép tính đơn giản (mục 2.4), nên gradient tính được bằng **quy tắc đạo hàm hàm hợp**. Quan trọng hơn, các đạo hàm riêng của các lớp sau được **dùng lại** khi tính cho các lớp trước, nhờ vậy cả P đạo hàm có giá cỡ **vài lần** một lượt lan truyền thuận, không phải $2P$ lần. Ý tưởng "đi ngược từ đầu ra, tái sử dụng kết quả" này do nhiều tác giả độc lập đề xuất từ những năm 1960 đến 1970 (như Werbos 1974), và được Rumelhart, Hinton và Williams phổ biến năm 1986.

Có thể tóm gọn trước nội dung của mục 3 bằng một ví dụ quen thuộc. Ở lớp ra của Softmax Regression, sai số tại điểm số là $\mathbf{P} - \mathbf{Y}$ và gradient là $\tilde{\mathbf{X}}^T(\mathbf{P} - \mathbf{Y})/n$. Lan truyền ngược cho thấy **mỗi lớp** của mạng đều có dạng đó:

$$\underbrace{\nabla_{\mathbf{W}^{(l)}} J}_{\text{gradient}} = \underbrace{\tilde{\mathbf{A}}^{(l-1)T}}_{\text{đầu vào của lớp}} \underbrace{\mathbf{D}^{(l)}}_{\text{tín hiệu lỗi tại lớp}}$$

Điều mới chỉ là: tín hiệu lỗi $\mathbf{D}^{(l)}$ của các lớp ẩn **không đọc từ nhãn mà được truyền ngược** từ lớp sau.

3.2. Quy tắc đạo hàm hàm hợp và đồ thị tính toán

Quy tắc đạo hàm hàm hợp

Với hàm hợp một biến $y = f\big(g(x)\big)$, đặt $u = g(x)$ thì

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx} = f'(u)\, g'(x)$$

Trong mạng nơ-ron, một đại lượng thường ảnh hưởng đến J qua **nhiều đường**. Ví dụ một nơ-ron ẩn nối đến nhiều nơ-ron ra. Khi đó ta **cộng** đóng góp của mọi đường:

$$\frac{\partial J}{\partial x} = \sum_k \frac{\partial J}{\partial u_k} \cdot \frac{\partial u_k}{\partial x}$$

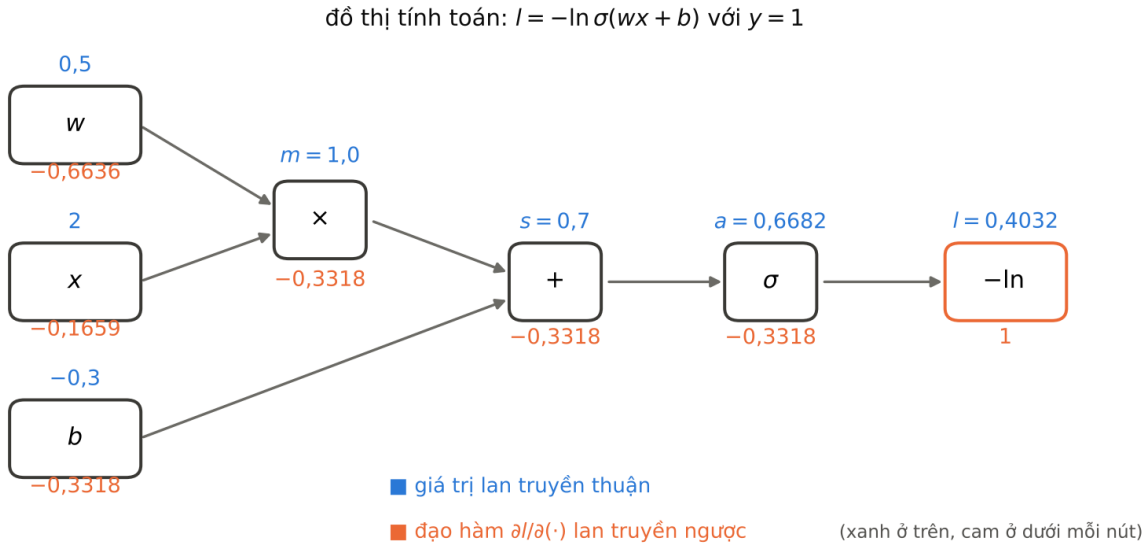
Công thức cộng theo đường này chính là lý do ở mục 3.3 xuất hiện phép nhân ma trận $\mathbf{D}^{(2)}\mathbf{V}^{(2)T}$: nó là tổng $\sum_c$ trên mọi nơ-ron ra mà nơ-ron ẩn đó nối tới.

Đồ thị tính toán

Mọi phép tính của mạng có thể vẽ thành một **đồ thị tính toán**: mỗi nút là một phép tính đơn giản (nhân, cộng, hàm kích hoạt, logarit,…), mỗi cạnh chuyển một giá trị từ nút này sang nút kia. Quy trình luôn gồm hai chiều:

41. **Chiều thuận:** tính giá trị tại mỗi nút từ trái sang phải, đồng thời lưu các giá trị trung gian.
42. **Chiều ngược:** bắt đầu từ $\partial J/\partial J = 1$, đi từ phải sang trái; tại mỗi nút, lấy đạo hàm đã nhận từ phía sau **nhân với đạo hàm riêng cục bộ** của nút đó, rồi chuyển cho các nút phía trước (cộng lại nếu có nhiều đường).

Ví dụ. Xét một nơ-ron sigmoid duy nhất với mất mát $l = -\ln\sigma(wx + b)$ (cross-entropy khi nhãn $y = 1$), với $w = 0{,}5$, $x = 2$, $b = -0{,}3$ (Hình 10).



Hình 10. Đồ thị tính toán của $l = -\ln\sigma(wx + b)$. Giá trị lan truyền thuận ghi ở trên mỗi nút (xanh), đạo hàm $\partial l/\partial(\cdot)$ lan truyền ngược ghi ở dưới (cam).

Chiều thuận. $m = wx = 1{,}0$; $s = m + b = 0{,}7$; $a = \sigma(0{,}7) \approx 0{,}6682$; $l = -\ln 0{,}6682 \approx 0{,}4032$.

Chiều ngược. Đi từ phải sang trái và nhân các đạo hàm cục bộ:

$$\frac{\partial l}{\partial a} = -\frac{1}{a} \approx -1{,}4966, \qquad \frac{\partial a}{\partial s} = a(1-a) \approx 0{,}2217, \qquad \frac{\partial l}{\partial s} = -\frac{1}{a}\cdot a(1-a) = a - 1 \approx -0{,}3318$$

Kết quả $\partial l/\partial s = a - 1 = p - y$ là một lần nữa **sai số "dự đoán trừ thực tế"** mà ta đã gặp ở bài Logistic Regression: thừa số a của sigmoid triệt tiêu mẫu số của logarit. Tiếp tục đi ngược qua phép cộng ($\partial s/\partial m = \partial s/\partial b = 1$) và phép nhân ($\partial m/\partial w = x$, $\partial m/\partial x = w$):

$$\frac{\partial l}{\partial b} = -0{,}3318, \qquad \frac{\partial l}{\partial w} = \frac{\partial l}{\partial s} \cdot x = -0{,}3318 \times 2 = -0{,}6636, \qquad \frac{\partial l}{\partial x} = \frac{\partial l}{\partial s} \cdot w = -0{,}1659$$

Kiểm tra bằng đạo hàm số ($\varepsilon = 10^{-6}$) cho $-0{,}66362446$ và $-0{,}33181223$ cho $\partial l / \partial w$ và $\partial l / \partial b$, khớp đến 10^{-10}. Một bước Gradient Descent với $\eta = 0{,}5$ cho $w \leftarrow 0{,}5 + 0{,}5 \times 0{,}6636 = 0{,}8318$ và $b \leftarrow -0{,}3 + 0{,}5 \times 0{,}3318 = -0{,}1341$. Khi đó $a \approx 0{,}8219$ và $l \approx 0{,}1961$, giảm từ $0{,}4032$.

Dạng ma trận của các quy tắc cho một lớp

Mạng nơ-ron tính cho cả n mẫu bằng ma trận, nên ta cần các quy tắc đạo hàm dạng ma trận. Chỉ cần ba quy tắc, tương ứng ba phép tính trong một lớp: phép nhân ma trận $\mathbf{Z} = \tilde{\mathbf{A}}\mathbf{W}$, việc bỏ cột chệch, và hàm kích hoạt từng phần tử. Giả sử đã biết $\mathbf{D} = \partial J / \partial \mathbf{Z}$ (kích thước $n \times m$).

Quy tắc 1: gradient theo trọng số. Viết theo phần tử, $z_{ij} = \sum_k \tilde{a}_{ik} w_{kj}$. Vì w_{kj} chỉ ảnh hưởng đến cột j của $\mathbf{Z}$, nhưng ảnh hưởng đến cả n mẫu, ta cộng theo i:

$$\frac{\partial J}{\partial w_{kj}} = \sum_{i=1}^{n} \frac{\partial J}{\partial z_{ij}} \cdot \frac{\partial z_{ij}}{\partial w_{kj}} = \sum_{i=1}^{n} d_{ij}\ \tilde{a}_{ik} = \left(\tilde{\mathbf{A}}^T \mathbf{D}\right)_{kj} \quad \Rightarrow \quad \boxed{\nabla_{\mathbf{W}} J = \tilde{\mathbf{A}}^T \mathbf{D}}$$

Quy tắc 2: gradient theo đầu vào. Đầu vào $\tilde{a}_{ik}$ ảnh hưởng đến mọi cột j của hàng i trong $\mathbf{Z}$, ta cộng theo j:

$$\frac{\partial J}{\partial \tilde{a}_{ik}} = \sum_{j=1}^{m} d_{ij}\ w_{kj} = (\mathbf{D}\mathbf{W}^T)_{ik}$$

Cột đầu của $\tilde{\mathbf{A}}$ là hằng số 1 nên không truyền ngược tiếp; ta bỏ cột đó, tức bỏ hàng đầu của $\mathbf{W}$:

$$\boxed{\frac{\partial J}{\partial \mathbf{A}} = \mathbf{D}\mathbf{V}^T} \qquad (\mathbf{V} \text{ là } \mathbf{W} \text{ bỏ hàng đầu})$$

Quy tắc 3: hàm kích hoạt từng phần tử. Với $\mathbf{A} = \varphi(\mathbf{Z})$ (a_{ij} chỉ phụ thuộc z_{ij}), đạo hàm cũng chỉ ghép từng phần tử một:

$$\boxed{\frac{\partial J}{\partial \mathbf{Z}} = \frac{\partial J}{\partial \mathbf{A}} \odot \varphi'(\mathbf{Z})}$$

Có thể nhớ nhanh nhờ **kiểm tra kích thước**: gradient của một đại lượng luôn có cùng kích thước với chính đại lượng đó. Ví dụ $\tilde{\mathbf{A}}^T$ là $(m_{l-1} + 1) \times n$ và $\mathbf{D}$ là $n \times m_l$, nên tích là $(m_{l-1} + 1) \times m_l$, đúng kích thước của $\mathbf{W}^{(l)}$. Tương tự $\mathbf{D}\mathbf{V}^T$ là $(n \times m_l)(m_l \times m_{l-1}) = n \times m_{l-1}$, đúng kích thước $\mathbf{A}^{(l-1)}$.

Hệ số chệch cũng nằm trong các công thức này: hàng đầu của $\tilde{\mathbf{A}}^T \mathbf{D}$ là $\mathbf{1}^T \mathbf{D}$, tức tổng các cột của $\mathbf{D}$ theo mẫu.

3.3. Lan truyền ngược cho mạng một lớp ẩn: suy ra từng công thức gradient

Mô hình và các bước

Xét mạng một lớp ẩn, ở dạng mục 1.2 và 2.4, với m nơ-ron ẩn:

$$\mathbf{Z}^{(1)} = \tilde{\mathbf{X}}\mathbf{W}^{(1)}, \quad \mathbf{H} = \varphi\left(\mathbf{Z}^{(1)}\right), \quad \mathbf{Z}^{(2)} = \tilde{\mathbf{H}}\mathbf{W}^{(2)}, \quad \mathbf{P} = \text{softmax}\left(\mathbf{Z}^{(2)}\right), \quad J = -\frac{1}{n}\sum_i \sum_k y_{ik} \ln p_{ik}$$

Ta đi **ngược** theo sơ đồ trên, từ J về $\mathbf{W}^{(1)}$, và dùng các quy tắc ở mục 3.2.

Bước 1: sai số ở lớp ra. Đây chính là kết quả ở bài Softmax (mục 3.3), thu được qua chuỗi $J \to \mathbf{P} \to \mathbf{Z}^{(2)}$:

$$\mathbf{D}^{(2)} = \frac{\partial J}{\partial \mathbf{Z}^{(2)}} = \frac{1}{n}(\mathbf{P} - \mathbf{Y}) \qquad (n \times K)$$

Bước 2: gradient của lớp ra. Áp dụng Quy tắc 1 cho $\mathbf{Z}^{(2)} = \tilde{\mathbf{H}}\mathbf{W}^{(2)}$:

$$\nabla_{\mathbf{W}^{(2)}} J = \tilde{\mathbf{H}}^T \mathbf{D}^{(2)} \qquad \left((m+1) \times K\right)$$

Đây **đúng là công thức của Softmax Regression**, chỉ khác là đầu vào $\tilde{\mathbf{X}}$ được thay bằng đặc trưng $\tilde{\mathbf{H}}$ mà mạng tự học. Vậy ta không phải làm lại phần này.

Bước 3: truyền sai số về lớp ẩn. Áp dụng Quy tắc 2 cho $\mathbf{Z}^{(2)} = \tilde{\mathbf{H}}\mathbf{W}^{(2)}$:

$$\frac{\partial J}{\partial \mathbf{H}} = \mathbf{D}^{(2)}\mathbf{V}^{(2)T} \qquad (n \times m)$$

Có thể đọc phần tử (i, j) của tích này: $\left[\mathbf{D}^{(2)}\mathbf{V}^{(2)T}\right]_{ij} = \sum_k d_{ik}^{(2)} v_{jk}^{(2)}$. Sai số của nơ-ron ẩn j là **tổng sai số của các nơ-ron ra, mỗi sai số được nhân với trọng số của kết nối từ j đến nơ-ron ra đó**. Nghĩa là nơ-ron ẩn nào nối mạnh tới nơ-ron ra đang sai nhiều thì "chịu trách nhiệm" nhiều. Đây là câu trả lời cho bài toán gán trách nhiệm ở mục 3.1: mạng dùng **chính các trọng số $\mathbf{V}^{(2)}$ nhưng theo chiều ngược** để phân bổ lỗi.

Bước 4: qua hàm kích hoạt, rồi gradient của lớp ẩn. Áp dụng Quy tắc 3 rồi Quy tắc 1 cho $\mathbf{Z}^{(1)} = \tilde{\mathbf{X}}\mathbf{W}^{(1)}$:

$$\mathbf{D}^{(1)} = \left(\mathbf{D}^{(2)}\mathbf{V}^{(2)T}\right) \odot \varphi'\left(\mathbf{Z}^{(1)}\right), \qquad \nabla_{\mathbf{W}^{(1)}} J = \tilde{\mathbf{X}}^T \mathbf{D}^{(1)} \qquad \left((d+1) \times m\right)$$

Với các hàm kích hoạt ở mục 2.3, $\varphi'\left(\mathbf{Z}^{(1)}\right)$ tính ngay từ $\mathbf{H}$ đã lưu ở lan truyền thuận:

φ	$\varphi'\left(\mathbf{Z}^{(1)}\right)$
tanh	$\mathbf{1} - \mathbf{H} \odot \mathbf{H}$
sigmoid	$\mathbf{H} \odot (\mathbf{1} - \mathbf{H})$

φ	$\varphi'(\mathbf{Z}^{(1)})$
ReLU	$\mathbb{1}[\mathbf{Z}^{(1)} > 0]$ (bằng 1 ở phần tử dương, 0 ở phần tử còn lại)

Tổng hợp: bốn công thức gọn trong một khối, kèm kích thước:

$$\begin{aligned} \mathbf{D}^{(2)} &= 1/n\,(\mathbf{P}-\mathbf{Y}) && (n\times K)\\ \nabla_{\mathbf{W}^{(2)}} J &= \tilde{\mathbf{H}}^{T}\mathbf{D}^{(2)} && \big((m+1)\times K\big)\\ \mathbf{D}^{(1)} &= \big(\mathbf{D}^{(2)}\mathbf{V}^{(2)T}\big)\odot \varphi'(\mathbf{Z}^{(1)}) && (n\times m)\\ \nabla_{\mathbf{W}^{(1)}} J &= \tilde{\mathbf{X}}^{T}\mathbf{D}^{(1)} && \big((d+1)\times m\big) \end{aligned}$$

Công thức này khớp với công thức đã giới thiệu ở mục 6.3 của bài Softmax Regression, giờ ta đã hiểu **vì sao** nó đúng. Hai nhận xét quan trọng:

43. **Thừa số φ' là "cái cổng".** Nếu một nơ-ron ẩn bão hòa ($\varphi' \approx 0$ với sigmoid hoặc tanh khi $|z|$ lớn) hoặc đang tắt (ReLU với $z<0$) thì sai số truyền về nó bị chặn, và các trọng số đi vào nơ-ron đó gần như không được cập nhật. Đây là mầm mống của hiện tượng gradient biến mất ở mục 4.3.

44. **Nếu bỏ lớp ẩn ($\mathbf{H}=\mathbf{X}$)** thì chỉ còn Bước 1 và 2, đúng công thức Softmax Regression.

Ví dụ: gradient của mạng 2–3–3 ở mục 2.4

Ta dùng lại đúng mạng, dữ liệu và trọng số ở ví dụ lan truyền thuận mục 2.4 (hàm kích hoạt tanh, hai mẫu, $n=2$, nhãn $y=(2,0)$). Đã biết ở đó

$$\mathbf{H}=\tanh(\mathbf{Z}^{(1)})\approx\begin{bmatrix}-0{,}7616 & 0 & 0{,}7616\\ 0{,}9993 & -0{,}9640 & 0\end{bmatrix},\qquad \mathbf{P}\approx\begin{bmatrix}0{,}1162 & 0{,}1916 & 0{,}6923\\ 0{,}8932 & 0{,}0788 & 0{,}0280\end{bmatrix},\qquad J\approx 0{,}2404$$

Bước 1 và 2. Với $\mathbf{Y}=\begin{bmatrix}0 & 0 & 1\\ 1 & 0 & 0\end{bmatrix}$, ta tính

$$\mathbf{D}^{(2)}=\frac{1}{2}(\mathbf{P}-\mathbf{Y})\approx\begin{bmatrix}0{,}0581 & 0{,}0958 & -0{,}1539\\ -0{,}0534 & 0{,}0394 & 0{,}0140\end{bmatrix},\qquad \nabla_{\mathbf{W}^{(2)}}J=\tilde{\mathbf{H}}^{T}\mathbf{D}^{(2)}\approx\begin{bmatrix}0{,}0047 & 0{,}1352 & -0{,}1399\\ -0{,}0976 & -0{,}0336 & 0{,}1312\\ 0{,}0515 & -0{,}0380 & -0{,}0135\\ 0{,}0442 & 0{,}0729 & -0{,}1172\end{bmatrix}$$

Bước 3. Bỏ hàng đầu của $\mathbf{W}^{(2)}$ ta được $\mathbf{V}^{(2)}=\begin{bmatrix}1 & 0 & -1\\ -1 & 1 & 0\\ 0 & -1 & 1\end{bmatrix}$, nên

$$\mathbf{D}^{(2)}\mathbf{V}^{(2)T}\approx\begin{bmatrix}0{,}2120 & 0{,}0377 & -0{,}2496\\ -0{,}0674 & 0{,}0928 & -0{,}0254\end{bmatrix}$$

Bước 4. Đạo hàm của tanh tại các nơ-ron ẩn là $\mathbf{1} - \mathbf{H} \odot \mathbf{H} \approx \begin{bmatrix} 0{,}4200 & 1 & 0{,}4200 \\ 0{,}0013 & 0{,}0707 & 1 \end{bmatrix}$. Do đó

$$\mathbf{D}^{(1)} \approx \begin{bmatrix} 0{,}0890 & 0{,}0377 & -0{,}1048 \\ -0{,}0001 & 0{,}0066 & -0{,}0254 \end{bmatrix}, \qquad \nabla_{\mathbf{W}^{(1)}} J = \widetilde{\mathbf{X}}^T \mathbf{D}^{(1)}$$
$$\approx \begin{bmatrix} 0{,}0889 & 0{,}0442 & -0{,}1303 \\ 0{,}0887 & 0{,}0574 & -0{,}1811 \\ 0{,}1781 & 0{,}0688 & -0{,}1843 \end{bmatrix}$$

Có hai điều đáng chú ý. Thứ nhất, với mẫu thứ hai, nơ-ron ẩn đầu tiên có $\tanh = 0{,}9993$ (gần bão hòa) nên $\varphi' = 0{,}0013$: sai số truyền về nó là $-0{,}0674$ nhưng sau khi qua cổng chỉ còn $-0{,}0001$. Đây là minh họa cụ thể của "cái cổng". Thứ hai, ta kiểm tra bằng đạo hàm số: với phần tử $w_{31}^{(2)}$ (hàng 3, cột 1 của $\mathbf{W}^{(2)}$), đạo hàm số cho $0{,}0514955164$, còn lan truyền ngược cho $0{,}0514955165$. Sai số tương đối lớn nhất trên cả 21 trọng số của mạng này là $1{,}6 \times 10^{-9}$.

Cuối cùng, một bước Gradient Descent với $\eta = 0{,}5$ đưa J từ $0{,}2404$ xuống $0{,}1562$. Nghĩa là gradient tính bằng lan truyền ngược thật sự chỉ về hướng làm giảm mất mát.

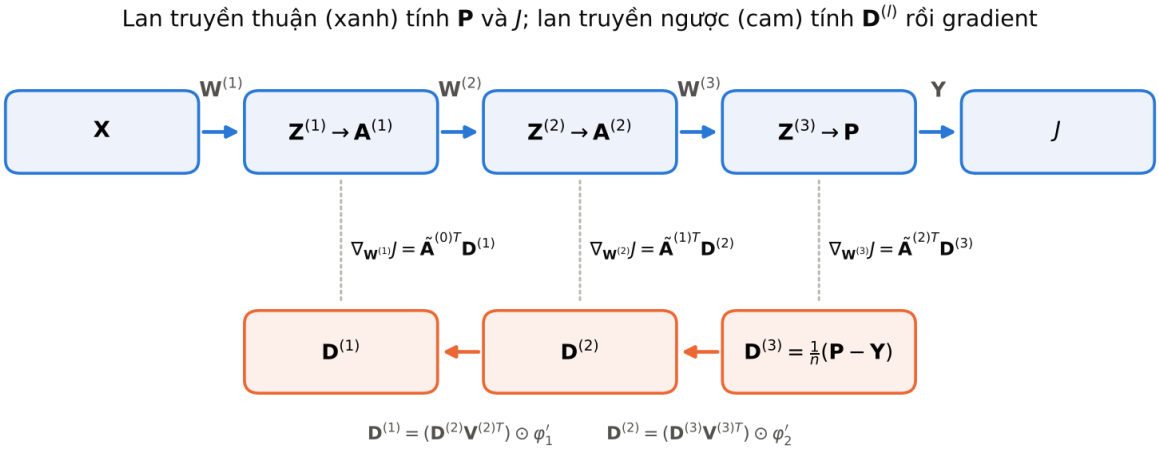
3.4. Thuật toán lan truyền ngược cho mạng L lớp và kiểm tra gradient bằng đạo hàm số

Từ hai lớp lên L lớp: một công thức lặp

Quan sát công thức ở mục 3.3: Bước 3 và 4 truyền sai số **từ lớp** 2 **về lớp** 1 theo một mẫu hình không phụ thuộc vào việc ta đang ở lớp nào. Do đó với mạng L lớp, ta chỉ việc **lặp lại** mẫu hình đó. Định nghĩa $\mathbf{D}^{(l)} = \partial J / \partial \mathbf{Z}^{(l)}$ cho mọi lớp:

$$\begin{aligned} \mathbf{D}^{(L)} &= 1/n\,(\mathbf{P} - \mathbf{Y}) && \text{(lớp ra: softmax + cross-entropy)} \\ \nabla_{\mathbf{W}^{(l)}} J &= \widetilde{\mathbf{A}}^{(l-1)T} \mathbf{D}^{(l)} && l = L, L-1, \dots, 1 \\ \mathbf{D}^{(l-1)} &= \left(\mathbf{D}^{(l)} \mathbf{V}^{(l)T}\right) \odot \varphi'_{l-1}\left(\mathbf{Z}^{(l-1)}\right) && l = L, L-1, \dots, 2 \end{aligned}$$

Dòng đầu cũng đúng khi lớp ra là sigmoid với cross-entropy nhị phân (khi đó $\mathbf{P}$ chỉ có một cột). Hình 11 minh họa luồng tính cho mạng ba lớp.



Hình 11. Lan truyền thuận (xanh) tính các ma trận $\mathbf{Z}^{(l)}$, $\mathbf{A}^{(l)}$ và J từ trái sang phải. Lan truyền ngược (cam) bắt đầu từ $\mathbf{D}^{(L)} = \frac{1}{n}(\mathbf{P} - \mathbf{Y})$ rồi truyền về trái; ở mỗi lớp, gradient theo trọng số là $\nabla_{\mathbf{W}^{(l)}} J = \tilde{\mathbf{A}}^{(l-1)T}\mathbf{D}^{(l)}$.

Thuật toán

Đầu vào: ma trận đặc trưng $\mathbf{X}$ $(n \times d)$, ma trận nhãn $\mathbf{Y}$ $(n \times K)$, các ma trận $\mathbf{W}^{(1)}, \dots, \mathbf{W}^{(L)}$, các hàm kích hoạt $\varphi_1, \dots, \varphi_{L-1}$.

Đầu ra: J và các gradient $\nabla_{\mathbf{W}^{(l)}} J$, $l = 1, \dots, L$.

45. **Lan truyền thuận.** Đặt $\mathbf{A}^{(0)} = \mathbf{X}$. Với $l = 1, \dots, L$: tính $\mathbf{Z}^{(l)} = \tilde{\mathbf{A}}^{(l-1)}\mathbf{W}^{(l)}$ và $\mathbf{A}^{(l)} = \varphi_l\left(\mathbf{Z}^{(l)}\right)$ (lớp ra dùng softmax hoặc sigmoid). **Lưu lại** mọi $\mathbf{A}^{(l)}$ và $\mathbf{Z}^{(l)}$. Tính J từ $\mathbf{P} = \mathbf{A}^{(L)}$.

46. **Khởi tạo sai số ra.** $\mathbf{D}^{(L)} = \frac{1}{n}(\mathbf{P} - \mathbf{Y})$.

47. **Lan truyền ngược.** Với $l = L, L-1, \dots, 1$:
    - Gradient của lớp: $\nabla_{\mathbf{W}^{(l)}} J = \tilde{\mathbf{A}}^{(l-1)T}\mathbf{D}^{(l)}$.
    - Nếu $l > 1$: truyền sai số về lớp trước, $\mathbf{D}^{(l-1)} = \left(\mathbf{D}^{(l)}\mathbf{V}^{(l)T}\right) \odot \varphi'_{l-1}\left(\mathbf{Z}^{(l-1)}\right)$.

48. **Cập nhật** (sau khi đã tính xong **mọi** gradient): $\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \eta\, \nabla_{\mathbf{W}^{(l)}} J$.

Hai lưu ý thực hành. Thứ nhất, bước cập nhật phải đặt **sau** vòng ngược: công thức $\mathbf{D}^{(l-1)}$ dùng $\mathbf{V}^{(l)}$ **cũ**, nếu ta cập nhật $\mathbf{W}^{(l)}$ ngay rồi mới truyền ngược thì sai số truyền đi là hỗn hợp, không còn là gradient thật. Thứ hai, ta phải **lưu** các ma trận trung gian của lan truyền thuận, nên bộ nhớ tăng theo số lớp và theo kích thước mini-batch.

Chi phí

Mỗi lớp có hai phép nhân ma trận ở chiều ngược ($\widetilde{\mathbf{A}}^{(l-1)T}\mathbf{D}^{(l)}$ và $\mathbf{D}^{(l)}\mathbf{V}^{(l)T}$) so với **một** phép nhân ở chiều thuận, nên chi phí lan truyền ngược cỡ **gấp đôi** lan truyền thuận, tổng một lần tính gradient cỡ **ba lần** một lượt lan truyền thuận, **bất kể số tham số**. Đo thực nghiệm ở mục 3.1 cho tỉ lệ 1,56 vì mạng rất nhỏ nên chi phí gọi hàm chiếm phần lớn. So với $2P$ lượt của đạo hàm số, đó là lý do lan truyền ngược là nền tảng của mọi thư viện học sâu.

Kiểm tra gradient bằng đạo hàm số

Lan truyền ngược viết rất dễ sai (nhầm chuyển vị, quên chia n,…) mà chương trình vẫn chạy và J vẫn có thể giảm chậm. Vì vậy, sau khi cài đặt cần **kiểm tra gradient** (gradient checking): so sánh gradient giải tích với đạo hàm số. Ta dùng **sai phân trung tâm**, chính xác hơn sai phân tiến ($\mathcal{O}(\varepsilon^2)$ thay vì $\mathcal{O}(\varepsilon)$):

$$g_{\text{số}} = \frac{J(w+\varepsilon) - J(w-\varepsilon)}{2\varepsilon}, \qquad \text{sai số tương đối} = \frac{\left|g_{\text{số}} - g_{\text{ngược}}\right|}{\max\left(10^{-8},\ \left|g_{\text{số}}\right| + \left|g_{\text{ngược}}\right|\right)}$$

Với $\varepsilon \approx 10^{-6}$ và số thực 64 bit, quy ước thường dùng:

Sai số tương đối	Nhận định
$< 10^{-6}$	Gần như chắc chắn đúng
10^{-6} đến 10^{-4}	Đáng ngờ, nên xem lại (có thể do hàm gần điểm gãy như ReLU)
$> 10^{-3}$	Gần như chắc chắn có lỗi

Lưu ý khi kiểm tra: (i) dùng mạng nhỏ và vài mẫu, vì cần $2P$ lượt thuận; (ii) tắt mọi yếu tố ngẫu nhiên (như dropout ở mục 4.4); (iii) dùng số thực 64 bit; (iv) tránh kiểm tra ReLU ngay tại $z = 0$; (v) chỉ kiểm tra khi cài đặt hoặc sửa mã, **không** bật trong lúc huấn luyện thật vì quá chậm.

Kết quả với cài đặt ở mục 3.7 trên mạng 3–5–4–3 (59 tham số, hệ số chệch khác 0, 6 mẫu ngẫu nhiên): sai số tương đối lớn nhất là $1{,}9 \times 10^{-7}$ với tanh và $3{,}2 \times 10^{-7}$ với sigmoid, đều thuộc nhóm "gần như chắc chắn đúng". Các lỗi thường gặp mà phép kiểm tra này phát hiện:

Lỗi	Biểu hiện khi kiểm tra
Quên chia cho n ở $\mathbf{D}^{(L)}$	Gradient lớn gấp n lần đạo hàm số
Nhầm $\mathbf{V}$ với $\mathbf{W}$ (quên bỏ hàng chệch)	Lỗi kích thước, hoặc gradient lớp trước sai
Quên nhân với φ'	Gradient các lớp ẩn sai, lớp ra vẫn đúng
Cập nhật $\mathbf{W}$ trước khi truyền ngược	Gradient các lớp trước sai
Quên tính gradient của hệ số chệch	Hàng đầu của gradient sai

3.5. Khởi tạo trọng số, learning rate và thuật toán tối ưu (SGD, momentum, Adam)

Với Softmax Regression, hàm mất mát lồi nên khởi tạo $\mathbf{W} = \mathbf{0}$, tốc độ học và thuật toán nào hợp lý cũng đều dẫn đến cùng một nghiệm. Với mạng nơ-ron, ba lựa chọn này **quan trọng hơn nhiều**. Hàm mất mát không còn lồi (mục 4.2), nên chúng quyết định cả việc mạng có học được hay không.

Khởi tạo: vì sao không thể là $\mathbf{0}$?

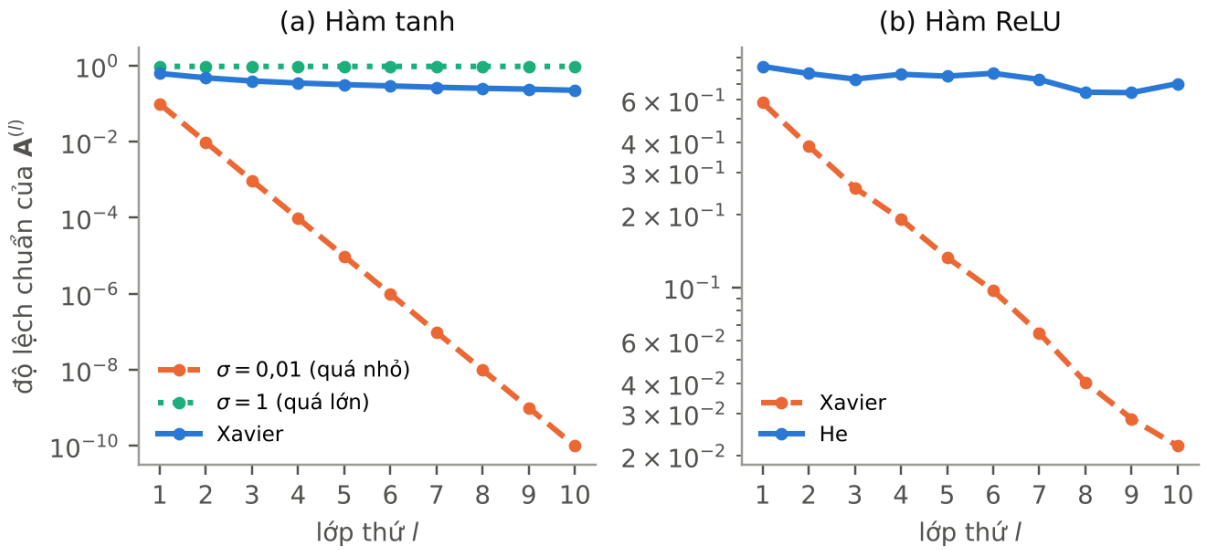
Giả sử tất cả trọng số bằng nhau (kể cả bằng $\mathbf{0}$). Khi đó mọi nơ-ron trong một lớp ẩn nhận cùng đầu vào, tính cùng đầu ra, rồi nhận **cùng tín hiệu lỗi** nên có **cùng gradient** và được cập nhật giống hệt nhau. Chúng sẽ **mãi mãi giống nhau**: một lớp ẩn m nơ-ron chỉ có sức biểu diễn của đúng một nơ-ron. Hiện tượng này gọi là **đối xứng** (symmetry). (Với $\mathbf{W} = \mathbf{0}$ còn tệ hơn: do $\mathbf{V}^{(2)} = \mathbf{0}$ nên $\mathbf{D}^{(1)} = \mathbf{0}$ và lớp ẩn không nhận được gradient nào.)

Ta kiểm chứng trên XOR bằng mạng 2–3–1 với mọi trọng số bằng 0,3. Sau 3000 epoch, J dừng ở 0,4832 (không tiến về 0), và ba cột của $\mathbf{W}^{(1)}$ vẫn **giống hệt nhau**: mỗi cột là $[-0{,}5901 \quad 6{,}5197 \quad 6{,}5197]^T$. Ba nơ-ron ẩn làm việc như một, và một nơ-ron ẩn thì không giải được XOR. Cách phá đối xứng đơn giản: **khởi tạo ngẫu nhiên**. (Hệ số chệch có thể khởi tạo bằng 0 vì các trọng số ngẫu nhiên đã phá đối xứng.)

Khởi tạo ngẫu nhiên: độ lớn bao nhiêu là vừa?

Khởi tạo ngẫu nhiên còn một câu hỏi: lấy phân phối với độ lệch chuẩn bao nhiêu? Thử với mạng 10 lớp, mỗi lớp 100 nơ-ron, hàm kích hoạt $\tanh$, đầu vào $\mathcal{N}(0,1)$, và đo độ lệch chuẩn của đầu ra từng lớp (Hình 12a):

Cách khởi tạo	**Lớp** 1	**Lớp** 5	**Lớp** 10	**Hậu quả**
$\mathcal{N}(0; 0,01^2)$ (quá nhỏ)	0,099	$\approx 10^{-5}$	$\approx 10^{-10}$	Tín hiệu **co về** 0 qua các lớp, gradient cũng co theo
$\mathcal{N}(0; 1)$ (quá lớn)	0,959	0,958	0,957	Đầu ra gần ± 1 ở mọi lớp: **bão hòa**, $\tanh' \approx 0$
Xavier (bên dưới)	0,625	0,319	0,226	Tín hiệu giữ ở cỡ hợp lý



Hình 12. Độ lệch chuẩn của đầu ra qua 10 lớp, mỗi lớp 100 nơ-ron (thang logarit). (a) Hàm tanh với ba cách khởi tạo. (b) Hàm ReLU: khởi tạo Xavier làm tín hiệu co dần, khởi tạo He giữ được độ lớn.

Khởi tạo Xavier (Glorot) chọn độ lệch chuẩn sao cho phương sai của tín hiệu không đổi khi đi qua một lớp. Lập luận: với đầu vào độc lập, kỳ vọng 0, mỗi điểm số $z_j = \sum_{i=1}^{m_{l-1}} a_i\, w_{ij}$ có phương sai $m_{l-1}\,\mathrm{Var}(w)\,\mathrm{Var}(a)$. Muốn giữ phương sai thì cần $\mathrm{Var}(w) = 1/m_{l-1}$. Chiều ngược (gradient) lại gợi ý $1/m_l$. Xavier lấy một thỏa hiệp:

$$w_{ij}^{(l)} \sim \mathcal{N}\left(0,\ \frac{2}{m_{l-1} + m_l}\right)$$

Khởi tạo He dành cho ReLU: ReLU đưa khoảng một nửa số điểm số về 0, làm phương sai đầu ra chỉ còn khoảng một nửa, nên cần nhân đôi phương sai trọng số:

$$w_{ij}^{(l)} \sim \mathcal{N}\left(0,\ \frac{2}{m_{l-1}}\right)$$

Hình 12b cho thấy với ReLU, Xavier làm độ lệch chuẩn co từ 0,585 xuống 0,022 sau 10 lớp, còn He giữ ở khoảng 0,6 đến 1,0. Cách chọn gọn:

Hàm kích hoạt lớp ẩn	Khởi tạo thường dùng
tanh, sigmoid	Xavier
ReLU và biến thể	He

Tốc độ học

Cũng như ở Softmax Regression, η quá nhỏ thì học chậm, quá lớn thì dao động hoặc phân kỳ. Khác biệt là với mạng nơ-ron, η **tốt nhất phụ thuộc vào thuật toán tối ưu**. Ta chạy mạng 2–8–3 (hàm tanh, mini-batch 32) trên 420 mẫu vòng tròn, đo J sau 100 epoch (trung bình 3 khởi tạo):

η	SGD	Momentum ($\beta = 0{,}9$)	Adam
0,001			0,863
0,003			0,412
0,01	0,998	0,293	0,148
0,03	0,665	0,118	0,070
0,1	0,292	0,062	0,030
0,3	0,173	0,140	
1	0,478		

Mỗi thuật toán có một "vùng η tốt" khác nhau, và ở hai phía đều tệ hơn: SGD tốt nhất quanh 0,3, Momentum quanh 0,1, Adam quanh 0,1 (ô trống là các giá trị ta không thử). Thực hành: thử η trên lưới theo thang logarit ($10^{-3}, 3\times10^{-3}, 10^{-2}, \ldots$) và theo dõi đường cong J như ở mục 5.1. Giá trị mặc định thường dùng của Adam là $0{,}001$, nhưng với mạng rất nhỏ và chỉ 100 epoch như ở đây thì nó quá nhỏ. Điều này nhắc ta rằng "mặc định" chỉ là điểm xuất phát.

Mini-batch

Gradient $\nabla_{\mathbf{W}^{(l)}} J$ là trung bình của n gradient từng mẫu, nên ta có thể ước lượng nó bằng một tập con (**mini-batch**) $\mathcal{B}$ gồm b mẫu: thay $\mathbf{X}$, $\mathbf{Y}$ và n bằng $\mathbf{X}_{\mathcal{B}}$, $\mathbf{Y}_{\mathcal{B}}$ và b trong thuật toán ở mục 3.4. Một **epoch** là một lượt duyệt qua toàn bộ n mẫu, gồm $\lceil n/b \rceil$ lần cập nhật. Đây là biến thể thông dụng nhất (như đã so sánh ở bài Softmax, mục 3.5). Ngoài việc nhanh hơn, nhiễu của mini-batch còn giúp mạng thoát khỏi một số vùng phẳng của hàm mất mát.

Ba thuật toán cập nhật

Gọi $\mathbf{G}$ là gradient (của mini-batch) theo một ma trận trọng số $\mathbf{W}$ bất kỳ.

SGD (Gradient Descent thuần): đi ngược gradient một đoạn η:

$$\mathbf{W} \leftarrow \mathbf{W} - \eta\,\mathbf{G}$$

Momentum giữ lại "vận tốc" $\mathbf{M}$ là tổng tích lũy có suy giảm của các gradient trước đó, với $\beta \in [0,1)$ (thường 0,9):

$$\mathbf{M} \leftarrow \beta\,\mathbf{M} + \mathbf{G}, \qquad \mathbf{W} \leftarrow \mathbf{W} - \eta\,\mathbf{M}$$

Hướng nào gradient liên tục chỉ về cùng phía thì vận tốc tích lũy, đi nhanh hơn: với gradient hằng $\mathbf{G}$, vận tốc tiến tới $\mathbf{G}/(1-\beta)$, tức bước đi hiệu dụng lớn gấp $1/(1-\beta) = 10$ lần với $\beta = 0{,}9$. Hướng nào gradient đổi dấu liên tục (dao động qua lại ở thung lũng hẹp) thì các đóng góp triệt tiêu nhau.

Adam (adaptive moment estimation) duy trì hai trung bình trượt: $\mathbf{M}$ của gradient và $\mathbf{V}$ của **bình phương** gradient (từng phần tử), rồi chia để chuẩn hóa độ lớn bước *riêng cho từng tham số*:

$$\mathbf{M} \leftarrow \beta_1 \mathbf{M} + (1-\beta_1)\mathbf{G}, \qquad \mathbf{V} \leftarrow \beta_2 \mathbf{V} + (1-\beta_2)\,\mathbf{G} \odot \mathbf{G}$$

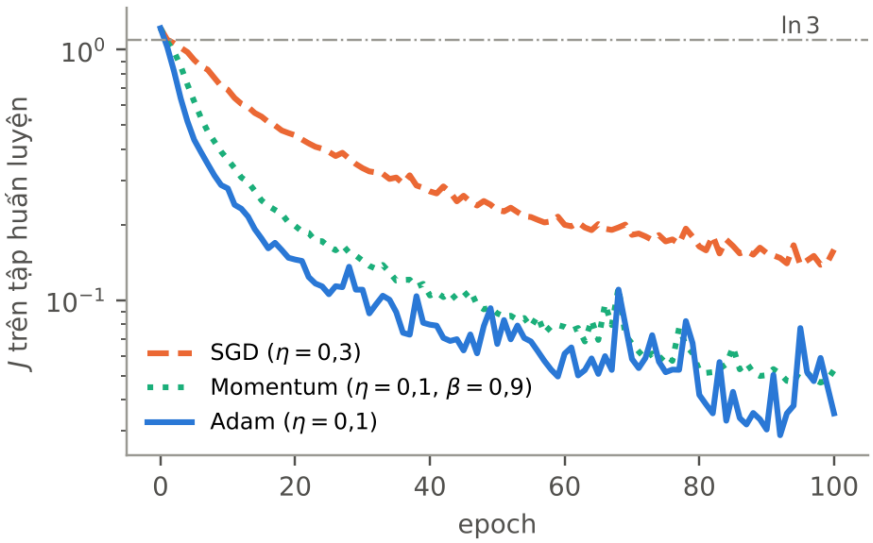
$$\widehat{\mathbf{M}} = \frac{\mathbf{M}}{1-\beta_1^t}, \qquad \widehat{\mathbf{V}} = \frac{\mathbf{V}}{1-\beta_2^t}$$

$$\mathbf{W} \leftarrow \mathbf{W} - \eta\, \frac{\widehat{\mathbf{M}}}{\sqrt{\widehat{\mathbf{V}}} + \epsilon}$$

với t là số bước đã thực hiện, $\beta_1 = 0{,}9$, $\beta_2 = 0{,}999$, $\epsilon = 10^{-8}$ (phép chia, căn, bình phương đều theo từng phần tử). Các thừa số $1/(1-\beta^t)$ **hiệu chỉnh độ chệch**, vì $\mathbf{M}$ và $\mathbf{V}$ khởi tạo bằng $\mathbf{0}$ nên ở các bước đầu bị kéo về 0. Ở bước đầu tiên, $\widehat{\mathbf{M}} = \mathbf{G}$ và $\sqrt{\widehat{\mathbf{V}}} = |\mathbf{G}|$ nên mỗi tham số đi một đoạn xấp xỉ η (theo dấu của gradient). Tham số có gradient luôn nhỏ vẫn được bước đi cỡ η, một điểm khác hẳn SGD.

So sánh thực nghiệm. Ta huấn luyện mạng 2–8–3 (tanh, softmax, mini-batch 32, 100 epoch), mỗi thuật toán dùng η tốt nhất trong lưới ở trên và chạy 5 khởi tạo khác nhau (Hình 13):

Thuật toán	η	J huấn luyện cuối (trung bình $\pm$ độ lệch chuẩn)	Độ chính xác kiểm tra trung bình
SGD	0,3	$0{,}159 \pm 0{,}046$	94,8%
Momentum	0,1	$0{,}053 \pm 0{,}035$	96,2%
Adam	0,1	$0{,}035 \pm 0{,}010$	97,2%



Hình 13. Hàm mất mát huấn luyện theo epoch (thang logarit, trung bình 5 khởi tạo) của ba thuật toán trên mạng 2–8–3, dữ liệu ba vòng tròn. Momentum và Adam giảm J nhanh hơn SGD; hai đường này dao động nhiều hơn ở cuối vì η lớn.

Với cùng số epoch, Momentum và Adam đạt J thấp hơn rõ rệt so với SGD. Về độ chính xác kiểm tra, chênh lệch 1% đến 2% chỉ tương đương 2 đến 4 trong 180 mẫu, nên chưa kết luận chắc chắn được. Không thuật toán nào "luôn thắng": Adam dễ dùng vì ít nhạy với thang đo gradient, còn SGD cộng momentum vẫn rất phổ biến khi cần tinh chỉnh cho khả năng tổng quát hóa tốt.

3.6. Ví dụ tính toán thủ công một vòng huấn luyện trên mạng **2–2–1** cho XOR

Ta tính tay trọn một lần cập nhật cho bài toán XOR, để thấy mọi công thức ở mục 3.3 hoạt động cùng nhau. Mạng có hai nơ-ron ẩn dùng sigmoid và một nơ-ron ra dùng sigmoid (bài toán nhị phân), hàm mất mát cross-entropy nhị phân, $n = 4$ mẫu, tốc độ học $\eta = 0{,}5$. Các số được làm tròn đến 4 chữ số thập phân; với bài toán nhị phân thì $\mathbf{P}$ và $\mathbf{Y}$ chỉ có một cột, và công thức $\mathbf{D}^{(2)} = \frac{1}{n}(\mathbf{P} - \mathbf{Y})$ vẫn đúng.

Dữ liệu và trọng số khởi tạo.

$$\tilde{\mathbf{X}} = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}, \quad \mathbf{W}^{(1)} = \begin{bmatrix} 0{,}1 & -0{,}1 \\ 0{,}5 & 0{,}3 \\ -0{,}4 & 0{,}6 \end{bmatrix}, \quad \mathbf{W}^{(2)} = \begin{bmatrix} 0{,}2 \\ 0{,}7 \\ -0{,}6 \end{bmatrix}$$

Bước 1: lan truyền thuận

Lớp ẩn: $\mathbf{Z}^{(1)} = \tilde{\mathbf{X}}\mathbf{W}^{(1)}$ rồi qua sigmoid:

$$\mathbf{Z}^{(1)} = \begin{bmatrix} 0{,}1 & -0{,}1 \\ -0{,}3 & 0{,}5 \\ 0{,}6 & 0{,}2 \\ 0{,}2 & 0{,}8 \end{bmatrix}, \qquad \mathbf{H} = \sigma\left(\mathbf{Z}^{(1)}\right) \approx \begin{bmatrix} 0{,}5250 & 0{,}4750 \\ 0{,}4256 & 0{,}6225 \\ 0{,}6457 & 0{,}5498 \\ 0{,}5498 & 0{,}6900 \end{bmatrix}$$

Ví dụ phần tử đầu của $\mathbf{Z}^{(1)}$ ở hàng 2: $0{,}1 \cdot 1 + 0{,}5 \cdot 0 + (-0{,}4) \cdot 1 = -0{,}3$, và $\sigma(-0{,}3) = 0{,}4256$.

Lớp ra: $\mathbf{z}^{(2)} = \tilde{\mathbf{H}}\mathbf{W}^{(2)}$ rồi qua sigmoid. Ví dụ mẫu 1: $0{,}2 + 0{,}7 \times 0{,}5250 - 0{,}6 \times 0{,}4750 = 0{,}2825$.

$$\mathbf{z}^{(2)} \approx \begin{bmatrix} 0{,}2825 \\ 0{,}1244 \\ 0{,}3221 \\ 0{,}1709 \end{bmatrix}, \qquad \mathbf{p} = \sigma\left(\mathbf{z}^{(2)}\right) \approx \begin{bmatrix} 0{,}5702 \\ 0{,}5311 \\ 0{,}5798 \\ 0{,}5426 \end{bmatrix}$$

Bước 2: hàm mất mát

Mất mát từng mẫu là $l_i = -\ln q_i$ với q_i là xác suất gán cho nhãn thật: $q_1 = 1 - 0{,}5702$, $q_2 = 0{,}5311$, $q_3 = 0{,}5798$, $q_4 = 1 - 0{,}5426$.

Mẫu	(x_1, x_2)	y	p	$l = -\ln q$
1	(0,0)	0	0,5702	0,8443
2	(0,1)	1	0,5311	0,6329
3	(1,0)	1	0,5798	0,5450
4	(1,1)	0	0,5426	0,7822

$$J = \frac{0{,}8443 + 0{,}6329 + 0{,}5450 + 0{,}7822}{4} \approx 0{,}7011$$

Đây chỉ cao hơn mốc $\ln 2 = 0{,}6931$ của mô hình "chưa biết gì": mạng khởi tạo gần như đoán 0,5 cho mọi mẫu.

Bước 3: lan truyền ngược

Lớp ra. $\mathbf{D}^{(2)} = \frac{1}{4}(\mathbf{p} - \mathbf{y}) \approx [0{,}1425 \quad -0{,}1172 \quad -0{,}1050 \quad 0{,}1357]^T$ và

$$\nabla_{\mathbf{W}^{(2)}} J = \tilde{\mathbf{H}}^T \mathbf{D}^{(2)} \approx \begin{bmatrix} 0{,}0559 \\ 0{,}0317 \\ 0{,}0306 \end{bmatrix}$$

Hàng đầu là tổng của $\mathbf{D}^{(2)}$: $0{,}1425 - 0{,}1172 - 0{,}1050 + 0{,}1357 = 0{,}0560$ (khớp 0,0559 do làm tròn).

Truyền về lớp ẩn. $\mathbf{V}^{(2)} = \begin{bmatrix} 0{,}7 \\ -0{,}6 \end{bmatrix}$, nên $\mathbf{D}^{(2)}\mathbf{V}^{(2)T}$ là tích ngoài của $\mathbf{D}^{(2)}$ với $[0{,}7 \quad -0{,}6]$:

$$\mathbf{D}^{(2)}\mathbf{V}^{(2)T} \approx \begin{bmatrix} 0{,}0998 & -0{,}0855 \\ -0{,}0821 & 0{,}0703 \\ -0{,}0735 & 0{,}0630 \\ 0{,}0950 & -0{,}0814 \end{bmatrix}$$

Đạo hàm sigmoid $\mathbf{H} \odot (\mathbf{1} - \mathbf{H})$ rồi nhân từng phần tử:

$$\mathbf{H} \odot (\mathbf{1} - \mathbf{H}) \approx \begin{bmatrix} 0{,}2494 & 0{,}2494 \\ 0{,}2445 & 0{,}2350 \\ 0{,}2288 & 0{,}2475 \\ 0{,}2475 & 0{,}2139 \end{bmatrix}, \qquad \mathbf{D}^{(1)} \approx \begin{bmatrix} 0{,}0249 & -0{,}0213 \\ -0{,}0201 & 0{,}0165 \\ -0{,}0168 & 0{,}0156 \\ 0{,}0235 & -0{,}0174 \end{bmatrix}$$

$$\nabla_{\mathbf{W}^{(1)}} J = \tilde{\mathbf{X}}^T \mathbf{D}^{(1)} \approx \begin{bmatrix} 0{,}0115 & -0{,}0066 \\ 0{,}0067 & -0{,}0018 \\ 0{,}0034 & -0{,}0009 \end{bmatrix}$$

Cách kiểm tra nhanh: hàng đầu là tổng các cột của $\mathbf{D}^{(1)}$ ($0{,}0249 - 0{,}0201 - 0{,}0168 + 0{,}0235 = 0{,}0115$). Hàng x_1 chỉ cộng các mẫu có $x_1 = 1$ (mẫu 3,4): $-0{,}0168 + 0{,}0235 = 0{,}0067$. Hàng x_2 cộng các mẫu có $x_2 = 1$ (mẫu 2,4): $-0{,}0201 + 0{,}0235 = 0{,}0034$. Gradient này cũng được xác nhận bằng đạo hàm số (sai số tương đối lớn nhất $3{,}4 \times 10^{-8}$ trên 9 trọng số).

Bước 4: cập nhật

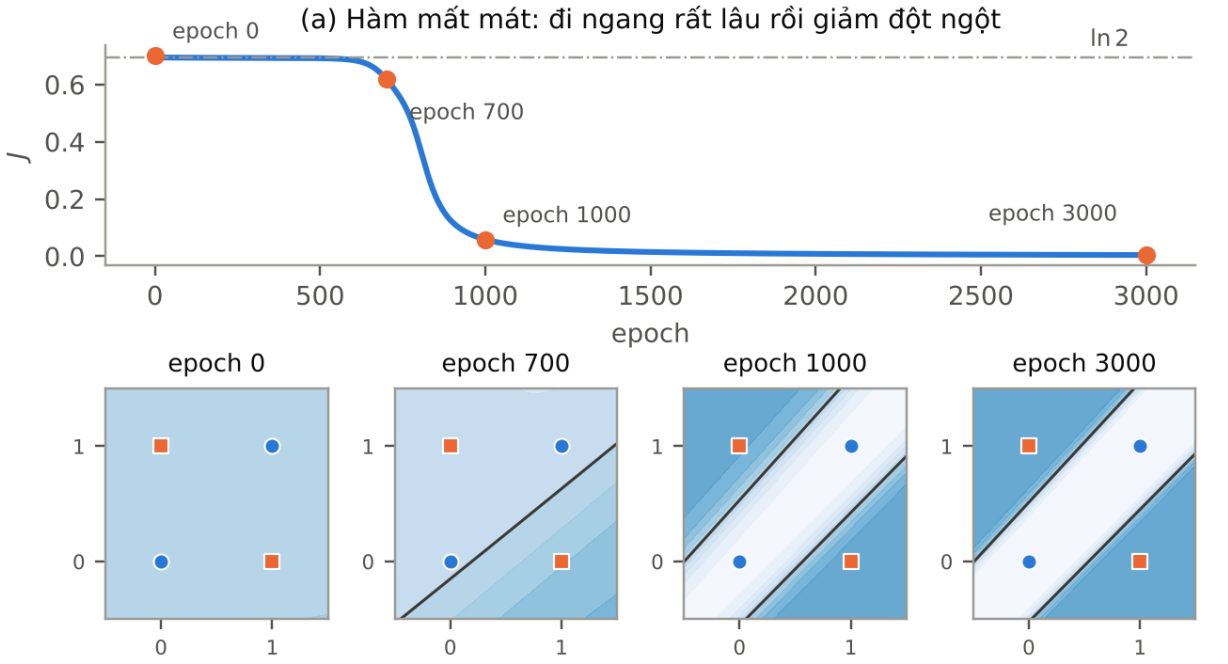
$$\mathbf{W}^{(1)} \leftarrow \mathbf{W}^{(1)} - 0{,}5\,\nabla_{\mathbf{W}^{(1)}} J \approx \begin{bmatrix} 0{,}0942 & -0{,}0967 \\ 0{,}4967 & 0{,}3009 \\ -0{,}4017 & 0{,}6004 \end{bmatrix}, \qquad \mathbf{W}^{(2)} \leftarrow \mathbf{W}^{(2)} - 0{,}5\,\nabla_{\mathbf{W}^{(2)}} J$$
$$\approx \begin{bmatrix} 0{,}1720 \\ 0{,}6841 \\ -0{,}6153 \end{bmatrix}$$

Chạy lại lan truyền thuận với trọng số mới: $\mathbf{p} \approx [0{,}5591 \quad 0{,}5196 \quad 0{,}5679 \quad 0{,}5303]^T$ và $J \approx 0{,}6987$, **giảm** từ 0,7011. Sau một bước, **p** của mẫu 1 và 4 (nhãn 0) giảm, còn **p** của mẫu 2 và 3 (nhãn 1) cũng giảm dù nhãn là 1. Điều này không mâu thuẫn: bước đầu chủ yếu làm cho đầu ra của cả bốn mẫu đều dịch cùng chiều (do hệ số chệch của lớp ra giảm), vì gradient quanh điểm khởi tạo rất nhỏ (cỡ 10^{-2}) và chưa đủ để phân biệt các mẫu. Việc phân hóa chỉ xảy ra sau nhiều bước.

Các bước tiếp theo: một cao nguyên dài rồi giảm đột ngột

Một bước chỉ giảm J rất ít. Ta lặp lại thuật toán từ đúng điểm khởi tạo này. Với $\eta = 0{,}5$, J vẫn còn 0,6917 sau 1000 epoch, và chỉ xuống 0,0584 sau 2000 epoch. Với $\eta = 1$ (Hình 14, cùng điểm khởi tạo), J gần như **đi ngang quanh** ln2 **trong khoảng** 600 **epoch đầu**, rồi giảm rất nhanh trong vài trăm epoch, và còn 0,0023 sau 5000 epoch:

Epoch	0	100	500	700	800	1000	5000
J ($\eta = 1$)	0,7011	0,6935	0,6917	0,6194	0,3886	0,0587	0,0023



Hình 14. Huấn luyện mạng 2–2–1 cho XOR từ trọng số khởi tạo ở mục 3.6 ($\eta = 1$). (a) J đi ngang gần ln2 rồi giảm đột ngột. (b) Xác suất đầu ra tại epoch 0, 700, 1000 và 3000: vùng sẫm là xác suất lớp 1 cao; ban đầu mạng cho $\approx 0{,}5$ khắp nơi, sau đó hình thành một dải ở giữa nhận lớp 1, đúng dạng của hình 8b.

Sau 5000 epoch, xác suất đầu ra là $[0{,}0022 \quad 0{,}9970 \quad 0{,}9980 \quad 0{,}0019]^T$: phân loại đúng cả bốn mẫu và rất tự tin. Thú vị là trọng số học được **không** phải $\mathbf{W}^{(1)}$ gồm OR và AND như ta thiết kế ở mục 2.5, mà là

$$\mathbf{W}^{(1)} \approx \begin{bmatrix} -3{,}79 & 3{,}47 \\ 7{,}14 & 7{,}27 \\ -7{,}33 & -6{,}98 \end{bmatrix}, \qquad \mathbf{W}^{(2)} \approx \begin{bmatrix} 6{,}19 \\ 13{,}48 \\ -13{,}00 \end{bmatrix}$$

Cột thứ nhất xấp xỉ "x_1 và không x_2", cột thứ hai xấp xỉ "x_1 hoặc không x_2", và lớp ra kết hợp chúng thành XOR. Có nhiều bộ trọng số cho cùng hàm, và thuật toán tìm ra một bộ trong số đó.

Hiện tượng cao nguyên (J đứng yên ở gần $\ln 2$ rất lâu) là biểu hiện của hàm mất mát không lồi: điểm khởi tạo nằm gần một vùng phẳng, nơi gradient nhỏ (ta thấy cỡ 10^{-2} ở bước đầu). Ta sẽ phân tích kỹ ở mục 4.2 và 4.3. Cũng vì thế, với khởi tạo ngẫu nhiên kiểu Xavier và $\eta = 1$, mạng 2–2–1 giải được XOR trong 18 trên 20 lần chạy khi dùng sigmoid ở lớp ẩn và 14 trên 20 lần khi dùng tanh (5000 epoch): chỉ hai nơ-ron ẩn là rất sát giới hạn, nên một số khởi tạo bị kẹt.

3.7. Mã hóa mạng nơ-ron từ đầu bằng Python

Ta cài đặt một mạng MLP tổng quát (số lớp tùy ý, hàm kích hoạt tanh, sigmoid hoặc ReLU, lớp ra softmax hoặc sigmoid) chỉ bằng NumPy, đúng theo thuật toán ở mục 3.4. Trong mã, `Ws` là danh sách các ma trận $[\mathbf{W}^{(1)}, \dots, \mathbf{W}^{(L)}]$ (đánh số từ 0 trong Python) và `As`, `Zs` là các ma trận trung gian lưu lại.

Các hàm cơ bản

```
import numpy as np

def them_bias(A):
    """Thêm cột 1 vào đầu ma trận: (n, m) -> (n, m+1)."""
    return np.hstack([np.ones((A.shape[0], 1)), A])

def softmax(S):
    S = S - S.max(axis=1, keepdims=True)
    E = np.exp(S)
    return E / E.sum(axis=1, keepdims=True)

def sigmoid(Z):
    return 0.5 * (1 + np.tanh(Z / 2))          # dạng ổn định số học

# Mỗi hàm kích hoạt đi kèm đạo hàm, viết qua đầu ra A (và Z nếu cần)
KICH_HOAT = {
    "tanh":    (np.tanh,                    lambda A, Z: 1 - A ** 2),
    "sigmoid": (sigmoid,                    lambda A, Z: A * (1 - A)),
    "relu":    (lambda Z: np.maximum(0, Z), lambda A, Z: (Z > 0).astype(float
)),
}
```

Khởi tạo, lan truyền thuận và hàm mất mát

```
def khoi_tao(kich_thuoc, kich_hoat="tanh", seed=0):
    """Xavier cho tanh/sigmoid, He cho ReLU; hệ số chệch khởi tạo bằng 0."""
    rng = np.random.default_rng(seed)
    Ws = []
```

```
    for l in range(1, len(kich_thuoc)):
        vao, ra = kich_thuoc[l - 1], kich_thuoc[l]
        la_an = l < len(kich_thuoc) - 1
        std = np.sqrt(2 / vao) if (la_an and kich_hoat == "relu") else np.sqr
t(2 / (vao + ra))
        W = np.zeros((vao + 1, ra))
        W[1:] = rng.normal(0, std, (vao, ra))
        Ws.append(W)
    return Ws

def lan_truyen_thuan(X, Ws, kich_hoat="tanh", lop_ra="softmax"):
    """Trả về As = [A0, ..., AL] và Zs = [Z1, ..., ZL] (lưu lại cho lan truyề
n ngược)."""
    phi = KICH_HOAT[kich_hoat][0]
    A, As, Zs = X, [X], []
    for l, W in enumerate(Ws):
        Z = them_bias(A) @ W
        Zs.append(Z)
        if l == len(Ws) - 1:
            A = softmax(Z) if lop_ra == "softmax" else sigmoid(Z)
        else:
            A = phi(Z)
        As.append(A)
    return As, Zs

def mat_mat(P, Y):
    """Cross-entropy trung bình (nhiều lớp, hoặc nhị phân khi Y, P có một cột
)."""
    P = np.clip(P, 1e-12, 1 - 1e-12)
    if Y.shape[1] == 1:
        return -np.mean(Y * np.log(P) + (1 - Y) * np.log(1 - P))
    return -np.mean(np.sum(Y * np.log(P), axis=1))
```

Lan truyền ngược và kiểm tra gradient

Hàm *lan_truyen_nguoc* là cài đặt trực tiếp của ba công thức ở mục 3.4, chỉ khoảng mười dòng:

```
def lan_truyen_nguoc(As, Zs, Y, Ws, kich_hoat="tanh"):
    """Gradient của J theo từng W^(l): D^(L) = (P - Y)/n, D^(l-1) = (D^(l) V^
T) * phi'(Z^(l-1))."""
    dphi = KICH_HOAT[kich_hoat][1]
    n, L = len(Y), len(Ws)
    grads = [None] * L
    D = (As[-1] - Y) / n                          # sai số ở lớp ra
    for l in range(L - 1, -1, -1):
        grads[l] = them_bias(As[l]).T @ D         # gradient theo W của l
ớp thứ l (đánh số từ 0)
        if l > 0:
            V = Ws[l][1:]                         # bỏ hàng hệ số chệch
            D = (D @ V.T) * dphi(As[l], Zs[l - 1])    # truyền ngược qua lớp
```

```
ẩn trước đó
    return grads

def kiem_tra_gradient(X, Y, Ws, kich_hoat="tanh", lop_ra="softmax", eps=1e-6)
:
    """Sai số tương đối lớn nhất giữa gradient giải tích và đạo hàm số (sai p
hân trung tâm)."""
    As, Zs = lan_truyen_thuan(X, Ws, kich_hoat, lop_ra)
    grads = lan_truyen_nguoc(As, Zs, Y, Ws, kich_hoat)
    sai_max = 0.0
    for W, g in zip(Ws, grads):
        for idx in np.ndindex(W.shape):
            goc = W[idx]
            W[idx] = goc + eps
            Jp = mat_mat(lan_truyen_thuan(X, Ws, kich_hoat, lop_ra)[0][-1], Y
)
            W[idx] = goc - eps
            Jm = mat_mat(lan_truyen_thuan(X, Ws, kich_hoat, lop_ra)[0][-1], Y
)
            W[idx] = goc
            so = (Jp - Jm) / (2 * eps)
            sai = abs(so - g[idx]) / max(1e-8, abs(so) + abs(g[idx]))
            sai_max = max(sai_max, sai)
    return sai_max
```

Chạy kiểm tra trên một mạng nhỏ ngẫu nhiên (đặt hệ số chệch khác 0 để kiểm tra cả hàng đầu của gradient):

```
rng = np.random.default_rng(0)
X = rng.normal(size=(6, 3)); Y = np.eye(3)[rng.integers(0, 3, 6)]
for kh in ("tanh", "sigmoid"):
    Ws = khoi_tao([3, 5, 4, 3], kh, seed=1)
    for W in Ws: W[0] = rng.normal(0, 0.5, W.shape[1])
    print(kh, "sai so tuong doi lon nhat:", kiem_tra_gradient(X, Y, Ws, kh, "
softmax"))
```

```
tanh sai so tuong doi lon nhat: 1.92e-07
sigmoid sai so tuong doi lon nhat: 3.21e-07
```

Cập nhật tham số và vòng huấn luyện

```
def tao_trang_thai(Ws):
    return {"m": [np.zeros_like(W) for W in Ws], "v": [np.zeros_like(W) for W
in Ws], "t": 0}

def cap_nhat(Ws, grads, tt, thuat_toan="sgd", eta=0.1, beta=0.9, beta2=0.999,
eps=1e-8):
    """Cập nhật tại chỗ: SGD, momentum hoặc Adam."""
    tt["t"] += 1
    for l, (W, g) in enumerate(zip(Ws, grads)):
        if thuat_toan == "sgd":
```

```
            W -= eta * g
        elif thuat_toan == "momentum":
            tt["m"][l] = beta * tt["m"][l] + g
            W -= eta * tt["m"][l]
        elif thuat_toan == "adam":
            tt["m"][l] = beta * tt["m"][l] + (1 - beta) * g
            tt["v"][l] = beta2 * tt["v"][l] + (1 - beta2) * g * g
            m_mu = tt["m"][l] / (1 - beta ** tt["t"])
            v_mu = tt["v"][l] / (1 - beta2 ** tt["t"])
            W -= eta * m_mu / (np.sqrt(v_mu) + eps)

def huan_luyen(X, Y, kich_thuoc, kich_hoat="tanh", lop_ra="softmax", thuat_to
an="sgd",
               eta=0.5, so_epoch=1000, b=None, seed=0, Ws=None):
    """Huấn luyện MLP bằng lan truyền ngược; b = None là batch, b = 1 là SGD,
còn lại là mini-batch."""
    n = len(X)
    b = n if b is None else b
    rng = np.random.default_rng(seed)
    Ws = khoi_tao(kich_thuoc, kich_hoat, seed) if Ws is None else Ws
    tt = tao_trang_thai(Ws)
    lich_su = [mat_mat(lan_truyen_thuan(X, Ws, kich_hoat, lop_ra)[0][-1], Y)]
    for epoch in range(so_epoch):
        idx = rng.permutation(n) if b < n else np.arange(n)
        for k in range(0, n, b):
            lo = idx[k:k + b]
            As, Zs = lan_truyen_thuan(X[lo], Ws, kich_hoat, lop_ra)
            grads = lan_truyen_nguoc(As, Zs, Y[lo], Ws, kich_hoat)
            cap_nhat(Ws, grads, tt, thuat_toan, eta)
        lich_su.append(mat_mat(lan_truyen_thuan(X, Ws, kich_hoat, lop_ra)[0][
-1], Y))
    return Ws, lich_su

def du_doan(X, Ws, kich_hoat="tanh", lop_ra="softmax"):
    P = lan_truyen_thuan(X, Ws, kich_hoat, lop_ra)[0][-1]
    return P.argmax(axis=1) if P.shape[1] > 1 else (P[:, 0] >= 0.5).astype(in
t)
```

Lưu ý hàm *cap_nhat* sửa *W* **tại chỗ** (*W -= ...*) và chỉ được gọi **sau** khi *lan_truyen_nguoc* đã trả về mọi gradient, đúng lưu ý ở mục 3.4.

Thử trên XOR

```
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1.]]); y = np.array([[0], [1], [1],
[0.]])
Ws, ls = huan_luyen(X, y, [2, 2, 1], "sigmoid", "sigmoid", "sgd", eta=1.0, so
_epoch=5000, seed=1)
print("J cuoi =", round(ls[-1], 4))
print("p      =", lan_truyen_thuan(X, Ws, "sigmoid", "sigmoid")[0][-1].ravel(
```

```
).round(3))
print("nhan   =", du_doan(X, Ws, "sigmoid", "sigmoid"))

J cuoi = 0.0021
p      = [0.002 0.998 0.998 0.002]
nhan   = [0 1 1 0]
```

Thử trên ba vòng tròn đồng tâm

Ta dùng lại dữ liệu và cách chia 70/30, chuẩn hóa ở mục 1 (hàm *sinh_vong*, *chia_train_test*, *chuan_hoa* của bài Softmax), rồi huấn luyện mạng 2–8–3 bằng Adam:

```
Xv, yv = sinh_vong(); Xa, Xb, ya, yb = chia_train_test(Xv, yv)
Za, Zb, _, _ = chuan_hoa(Xa, Xb); Ya = np.eye(3)[ya]
Ws, ls = huan_luyen(Za, Ya, [2, 8, 3], "tanh", "softmax", "adam", eta=0.01, s
o_epoch=300, b=32, seed=0)
print("J theo epoch 0,1,10,50,100,300:", [round(ls[i], 4) for i in (0, 1, 10,
50, 100, 300)])
print("acc huan luyen:", np.mean(du_doan(Za, Ws) == ya).round(3), "| acc kiem
tra:", np.mean(du_doan(Zb, Ws) == yb).round(3))

J theo epoch 0,1,10,50,100,300: [1.122, 1.0863, 0.8923, 0.2962, 0.1596, 0.058
2]
acc huan luyen: 1.0 | acc kiem tra: 0.967
```

J khởi đầu cỡ $\ln 3 \approx 1{,}0986$ (mô hình chưa biết gì) rồi giảm đều. Chạy với 5 khởi tạo ngẫu nhiên khác nhau (seed $= 0, \ldots, 4$) cho độ chính xác kiểm tra 96,7%, 96,1%, 97,8%, 98,9%, 97,8% và J huấn luyện cuối từ 0,036 đến 0,070. Kết quả này cùng cỡ với bảng ở mục 1.3 (96,1% đến 99,4% cho mạng 2–8–3), khác với Softmax Regression chỉ đạt 35,0% trên cùng dữ liệu. Mạng tìm được biên cong **tự học**, không cần ta tự thêm đặc trưng đa thức. Sự khác nhau giữa các lần chạy (do khởi tạo ngẫu nhiên và thứ tự mini-batch) là điều không có ở Softmax Regression, và sẽ được phân tích ở mục 4.2.

Mối liên hệ với mục trước và mục sau

Mục 2 xây dựng mô hình, mục 3 làm nốt hai thành phần còn lại của khung huấn luyện. Đối chiếu với Softmax Regression:

Thành phần	Softmax Regression	Mạng nơ-ron (MLP)
Gradient	$\frac{1}{n}\tilde{\mathbf{X}}^T(\mathbf{P}-\mathbf{Y})$	$\nabla_{\mathbf{W}^{(l)}} J = \tilde{\mathbf{A}}^{(l-1)T}\mathbf{D}^{(l)}$, với $\mathbf{D}^{(l)}$ truyền ngược
Khởi tạo	$\mathbf{W} = \mathbf{0}$ được	Ngẫu nhiên (Xavier hoặc He), **không** được là $\mathbf{0}$
Hội tụ	Về cùng một J^* từ mọi điểm xuất phát	Phụ thuộc điểm xuất phát; có cao nguyên
Tối ưu	Gradient Descent là đủ	Mini-batch, thường kèm momentum hoặc Adam

Công thức $\mathbf{P} - \mathbf{Y}$ vẫn nằm đúng ở đầu mọi chuỗi lan truyền ngược. Mục 4 sẽ đặt câu hỏi "vì sao hoạt động": điều gì xảy ra khi một mẫu bị phân loại sai, vì sao hiện tượng cao nguyên ở Hình 14 xuất hiện, và điều gì làm gradient biến mất hay bùng nổ khi mạng sâu hơn.

Tóm tắt mục 3

49. **Vì sao cần lan truyền ngược.** Nơ-ron ẩn không có "đáp án" trực tiếp nên quy tắc Perceptron không dùng được (bài toán gán trách nhiệm). Đạo hàm số tốn $2P$ lượt lan truyền thuận cho một gradient (gấp khoảng 98 lần lan truyền ngược trên mạng 2–8–3). Lan truyền ngược dùng đạo hàm hàm hợp và tái sử dụng kết quả, nên chi phí chỉ cỡ ba lượt lan truyền thuận bất kể P.

50. **Công cụ.** Quy tắc cộng theo mọi đường, và ba quy tắc ma trận cho một lớp với $\mathbf{D} = \partial J/\partial \mathbf{Z}$: $\nabla_{\mathbf{W}} J = \tilde{\mathbf{A}}^T \mathbf{D}$; $\partial J/\partial \mathbf{A} = \mathbf{D}\mathbf{V}^T$; $\partial J/\partial \mathbf{Z} = \partial J/\partial \mathbf{A} \odot \varphi'(\mathbf{Z})$. Gradient luôn có cùng kích thước với đại lượng của nó.

51. **Mạng một lớp ẩn.** $\mathbf{D}^{(2)} = \frac{1}{n}(\mathbf{P} - \mathbf{Y})$; $\nabla_{\mathbf{W}^{(2)}} J = \tilde{\mathbf{H}}^T \mathbf{D}^{(2)}$; $\mathbf{D}^{(1)} = \left(\mathbf{D}^{(2)} \mathbf{V}^{(2)T}\right) \odot \varphi'\left(\mathbf{Z}^{(1)}\right)$; $\nabla_{\mathbf{W}^{(1)}} J = \tilde{\mathbf{X}}^T \mathbf{D}^{(1)}$. Sai số của nơ-ron ẩn là tổng sai số các nơ-ron ra nhân trọng số nối; φ' là cái cổng, nơ-ron bão hòa hoặc tắt nhận gần như không có gradient.

52. **Mạng L lớp.** Lặp công thức trên từ lớp ra về lớp vào. Phải lưu các ma trận trung gian và cập nhật **sau** khi tính xong mọi gradient. Kiểm tra gradient bằng sai phân trung tâm với sai số tương đối $< 10^{-6}$ là dấu hiệu đúng; đây là bước bắt buộc khi tự cài đặt.

53. **Khởi tạo.** Không được là $\mathbf{0}$ (đối xứng làm các nơ-ron ẩn giống hệt nhau mãi mãi); phải ngẫu nhiên với độ lớn hợp lý để tín hiệu không co lại hay bão hòa qua các lớp: Xavier $\mathrm{Var}(w) = \frac{2}{m_{l-1}+m_l}$ cho tanh, sigmoid; He $\mathrm{Var}(w) = \frac{2}{m_{l-1}}$ cho ReLU.

54. **Tối ưu.** η tốt nhất phụ thuộc thuật toán. Momentum $\mathbf{M} \leftarrow \beta\mathbf{M} + \mathbf{G}$ tích lũy hướng nhất quán; Adam chuẩn hóa bước theo từng tham số bằng căn trung bình bình phương gradient, có hiệu chỉnh độ chệch. Trên dữ liệu vòng tròn, Momentum và Adam giảm J nhanh hơn SGD.

55. **Ví dụ XOR 2–2–1.** Một bước tính tay giảm J từ 0,7011 xuống 0,6987. Huấn luyện đầy đủ cho thấy một cao nguyên dài ở gần $\ln 2$ rồi giảm đột ngột (đặc trưng của hàm mất mát không lồi), và kết quả 2–8–3 trên vòng tròn đạt 96% đến 99% nhờ mã lan truyền ngược viết từ đầu chỉ vài chục dòng.

Câu hỏi kiểm tra nhanh

56. Với $l = -\ln\sigma(wx + b)$, $w = 1$, $x = 1$, $b = 0$, $y = 1$: tính $\partial l/\partial s$, $\partial l/\partial w$ và $\partial l/\partial b$ bằng đồ thị tính toán.

57. Mạng 4–6–3 (hàm kích hoạt tanh, softmax ở lớp ra) huấn luyện với mini-batch $n = 10$. Cho biết kích thước của $\mathbf{Z}^{(1)}$, $\mathbf{H}$, $\mathbf{D}^{(2)}$, $\mathbf{D}^{(1)}$, $\nabla_{\mathbf{W}^{(1)}} J$ và $\nabla_{\mathbf{W}^{(2)}} J$.

58. Chứng minh công thức $\nabla_{\mathbf{W}} J = \tilde{\mathbf{A}}^T \mathbf{D}$ cho $\mathbf{Z} = \tilde{\mathbf{A}}\mathbf{W}$ bằng cách viết theo từng phần tử w_{kj}.

59. Vì sao phải tính xong gradient của **tất cả** các lớp rồi mới cập nhật trọng số? Điều gì sai nếu cập nhật $\mathbf{W}^{(2)}$ ngay rồi mới truyền ngược về lớp 1?

60. Một mạng có $n = 1$, $K = 2$, $\mathbf{D}^{(2)} = [0{,}2 \quad -0{,}2]$, $\mathbf{V}^{(2)} = \begin{bmatrix} 1 & 2 \\ -1 & 0 \end{bmatrix}$, và đầu ra lớp ẩn (tanh) $\mathbf{H} = [0{,}5 \quad -0{,}5]$. Tính $\mathbf{D}^{(1)}$.

61. Đạo hàm số cho $g_{\text{số}} = 0{,}05149552$ và lan truyền ngược cho $g_{\text{ngược}} = 0{,}05149551$. Tính sai số tương đối. Nếu lan truyền ngược cho $0{,}0258$ thì kết luận gì?

62. Tính độ lệch chuẩn khởi tạo Xavier cho lớp có 100 đầu vào và 50 đầu ra, và độ lệch chuẩn khởi tạo He cho lớp ReLU có 100 đầu vào.

63. Với Momentum $\beta = 0{,}9$, vận tốc ban đầu $\mathbf{M}_0 = 0$ và gradient hằng $g = 1$, tính M_1, M_2, M_3 và giá trị giới hạn. Nói gì về bước đi hiệu dụng?

64. Với Adam ở bước đầu tiên ($t = 1$) và gradient của một tham số $g = 0{,}5$, tính m, v, $\hat{m}$, $\hat{v}$ và độ dời của tham số (lấy $\epsilon \approx 0$).

65. Vì sao khởi tạo $\mathbf{W} = \mathbf{0}$ dùng được cho Softmax Regression nhưng không dùng được cho mạng nơ-ron nhiều lớp?

66. Ở Hình 14, vì sao J đi ngang quanh $\ln 2$ rất lâu ở đầu quá trình huấn luyện?

Gợi ý đáp án

67. $a = \sigma(1) \approx 0{,}7311$ nên $\partial l / \partial s = a - 1 \approx -0{,}2689$. Do $s = wx + b$ với $x = 1$: $\partial l / \partial w = -0{,}2689 \times 1 = -0{,}2689$ và $\partial l / \partial b = -0{,}2689$.

68. $\mathbf{Z}^{(1)}$ và $\mathbf{H}$: 10×6. $\mathbf{D}^{(2)}$: 10×3. $\mathbf{D}^{(1)}$: 10×6. $\nabla_{\mathbf{W}^{(1)}} J$: 5×6. $\nabla_{\mathbf{W}^{(2)}} J$: 7×3.

69. $z_{ij} = \sum_k \tilde{a}_{ik} w_{kj}$. Trọng số w_{kj} chỉ xuất hiện trong cột j của $\mathbf{Z}$, nhưng xuất hiện ở cả n hàng, nên $\partial J / \partial w_{kj} = \sum_{i=1}^{n} \frac{\partial J}{\partial z_{ij}} \frac{\partial z_{ij}}{\partial w_{kj}} = \sum_i d_{ij} \tilde{a}_{ik}$, chính là phần tử (k, j) của $\tilde{\mathbf{A}}^T \mathbf{D}$.

70. Công thức $\mathbf{D}^{(1)} = \left(\mathbf{D}^{(2)} \mathbf{V}^{(2)T}\right) \odot \varphi'$ dùng $\mathbf{V}^{(2)}$ ở **điểm đang xét**. Nếu $\mathbf{W}^{(2)}$ đã bị cập nhật thì $\mathbf{V}^{(2)}$ đã khác, sai số truyền về lớp 1 sẽ ứng với một mạng "nửa cũ, nửa mới" chứ không phải gradient của J tại điểm hiện tại.

71. $\mathbf{D}^{(2)}\mathbf{V}^{(2)T} = [0{,}2 \cdot 1 + (-0{,}2) \cdot 2 \quad 0{,}2 \cdot (-1) + (-0{,}2) \cdot 0] = [-0{,}2 \quad -0{,}2]$. Đạo hàm tanh: $1 - \mathbf{H} \odot \mathbf{H} = [0{,}75 \quad 0{,}75]$. Vậy $\mathbf{D}^{(1)} = [-0{,}15 \quad -0{,}15]$.

72. Sai số tương đối $= \frac{|0{,}05149552 - 0{,}05149551|}{0{,}05149552 + 0{,}05149551} \approx \frac{10^{-8}}{0{,}103} \approx 10^{-7}$: gần như chắc chắn đúng. Với $0{,}0258$: $\frac{|0{,}0515 - 0{,}0258|}{0{,}0515 + 0{,}0258} \approx 0{,}33$, rất lớn, gần như chắc chắn có lỗi (con số này xấp xỉ một nửa nên có thể nghi ngờ đã chia thừa một hệ số 2, ví dụ chia cho n hai lần).

73. Xavier: $\sqrt{2/(100 + 50)} = \sqrt{0{,}01333} \approx 0{,}1155$. He: $\sqrt{2/100} = \sqrt{0{,}02} \approx 0{,}1414$.

74. $M_1 = 1$; $M_2 = 0{,}9 \cdot 1 + 1 = 1{,}9$; $M_3 = 0{,}9 \cdot 1{,}9 + 1 = 2{,}71$; giới hạn là $1/(1 - 0{,}9) = 10$. Bước đi hiệu dụng ở trạng thái ổn định lớn gấp 10 lần so với SGD cùng η.

75. $m = (1 - 0{,}9) \cdot 0{,}5 = 0{,}05$; $v = (1 - 0{,}999) \cdot 0{,}25 = 0{,}00025$. $\hat{m} = 0{,}05/(1 - 0{,}9) = 0{,}5$ và $\hat{v} = 0{,}00025/(1 - 0{,}999) = 0{,}25$. Độ dời $= \eta\, \hat{m}/\sqrt{\hat{v}} = \eta \cdot 0{,}5/0{,}5 = \eta$: đúng một bước cỡ η theo dấu của gradient.

76. Softmax Regression chỉ có một lớp có tham số, hàm mất mát lồi, và cột k của $\mathbf{W}$ nhận gradient riêng nhờ cột k của $\mathbf{Y}$, nên các cột phân hóa dù khởi tạo bằng $\mathbf{0}$. Ở mạng nhiều lớp, các nơ-ron ẩn cùng lớp có cùng đầu vào, nếu có cùng trọng số thì có cùng đầu ra, cùng gradient và mãi giống nhau (đối xứng). Với $\mathbf{W} = \mathbf{0}$ còn có $\mathbf{V}^{(2)} = \mathbf{0}$ nên gradient của lớp ẩn bằng 0 ngay từ đầu.

77. Khởi tạo có trọng số nhỏ nên mọi nơ-ron ẩn gần như không phân biệt được các mẫu và đầu ra $p \approx 0{,}5$ cho mọi mẫu. Điểm này nằm trên một vùng phẳng của hàm mất mát (không lồi): gradient rất nhỏ nên trọng số thay đổi chậm. Khi các trọng số lớp ẩn lệch dần ra khỏi sự gần-đối-xứng, gradient tăng lên và J giảm đột ngột (xem thêm mục 4.2 và 4.3).

4. Phân tích cơ chế học của mạng nơ-ron

Ở mục 3, ta đã có đủ ba thành phần của khung huấn luyện: mô hình (mục 2), hàm mất mát cross-entropy và thuật toán tối ưu dựa trên lan truyền ngược. Chương trình ở mục 3.7 chạy được, kết quả tốt trên XOR và dữ liệu ba vòng tròn. Nhưng "chạy được" chưa có nghĩa là hiểu. Ở mục 3 cũng đã để lại nhiều câu hỏi mở:

78. Khi mạng phân loại sai một mẫu, tín hiệu lỗi đi qua các lớp như thế nào, và lớp nào "chịu trách nhiệm"?
79. Vì sao đường cong J ở Hình 14 đi ngang rất lâu rồi mới giảm đột ngột? Vì sao có khởi tạo giải được XOR, có khởi tạo thì không?
80. Vì sao mạng 2–8–3 học tốt, còn mạng sâu hơn lại có thể không học được gì?
81. Khi mạng đủ lớn để khớp hoàn hảo tập huấn luyện, điều gì xảy ra với dữ liệu mới?
82. Nên thêm nơ-ron vào mỗi lớp hay thêm lớp mới?

Mục 4 trả lời lần lượt các câu hỏi đó. Điểm xuất phát vẫn là bốn công thức lan truyền ngược ở mục 3.3 và 3.4:

$$\mathbf{D}^{(L)} = 1/n\,(\mathbf{P}-\mathbf{Y}), \qquad \nabla_{\mathbf{W}^{(l)}} J = \tilde{\mathbf{A}}^{(l-1)T}\mathbf{D}^{(l)}, \qquad \mathbf{D}^{(l-1)} = \left(\mathbf{D}^{(l)}\mathbf{V}^{(l)T}\right) \odot \varphi'_{l-1}\left(\mathbf{Z}^{(l-1)}\right)$$

Mọi hiện tượng trong mục này, từ cao nguyên, bão hòa đến gradient biến mất, đều có thể đọc ra từ ba công thức trên.

Ký hiệu bổ sung cho mục 4

Đối tượng	Ký hiệu	Ý nghĩa
Tín hiệu lỗi riêng mẫu	$\mathbf{d}_i^{(l)}$	Hàng i của $n\,\mathbf{D}^{(l)}$ (vector hàng, $1 \times m_l$); không chia n
Ma trận Hessian	$\mathbf{H}_J$	Ma trận các đạo hàm riêng cấp hai của J theo mọi tham số
Hệ số điều chuẩn hóa	λ	Hệ số của số hạng phạt trọng số (L2)
Xác suất bỏ nơ-ron	p	Xác suất một nơ-ron ẩn bị tắt trong dropout
Mặt nạ dropout	$\mathbf{M}^{(l)}$	Ma trận cùng kích thước với $\mathbf{A}^{(l)}$, phần tử bằng 0 hoặc $1/(1-p)$

4.1. Điều gì xảy ra khi một mẫu bị phân loại sai?

Gradient là trung bình của các "lời phàn nàn" từng mẫu

Hàm mất mát là trung bình của mất mát từng mẫu, $J = \frac{1}{n}\sum_i l_i$, nên gradient cũng là trung bình của gradient từng mẫu. Gọi $\mathbf{d}_i^{(l)}$ là hàng i của $n\,\mathbf{D}^{(l)}$, tức tín hiệu lỗi của riêng mẫu i (chưa chia n). Từ $\nabla_{\mathbf{W}^{(l)}} J = \tilde{\mathbf{A}}^{(l-1)T}\mathbf{D}^{(l)}$, khai triển tích ma trận theo hàng ta được

$$\nabla_{\mathbf{W}^{(l)}} J = \frac{1}{n}\sum_{i=1}^{n} \tilde{\mathbf{a}}_i^{(l-1)T}\ \mathbf{d}_i^{(l)}, \qquad \mathbf{d}_i^{(L)} = \mathbf{p}_i - \mathbf{y}_i, \qquad \mathbf{d}_i^{(l-1)} = \left(\mathbf{d}_i^{(l)}\mathbf{V}^{(l)T}\right) \odot \varphi'_{l-1}\left(\mathbf{z}_i^{(l-1)}\right)$$

Mỗi mẫu đóng góp một ma trận hạng một $\tilde{\mathbf{a}}_i^T \mathbf{d}_i$: cột đầu vào của lớp ($\tilde{\mathbf{a}}_i^T$) nhân với "lời phàn nàn" ($\mathbf{d}_i$). Cập nhật $\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \eta\, \nabla_{\mathbf{W}^{(l)}} J$ nghĩa là mỗi mẫu kéo trọng số về phía làm giảm mất mát của riêng nó, và kết quả là trung bình của các lực kéo đó. Bây giờ ta xem lực kéo của một mẫu sai khác mẫu đúng như thế nào.

Lớp ra: mẫu sai kéo mạnh, mẫu đúng gần như không kéo

Ở lớp ra, $\mathbf{d}_i^{(L)} = \mathbf{p}_i - \mathbf{y}_i$. Hai trường hợp:

83. Mẫu thuộc lớp c được dự đoán đúng và tự tin, $\mathbf{p}_i = (0{,}98;\ 0{,}01;\ 0{,}01)$ và $\mathbf{y}_i = (1{,}0{,}0)$: $\mathbf{d}_i = (-0{,}02;\ 0{,}01;\ 0{,}01)$, độ dài $\| \mathbf{d}_i \| \approx 0{,}024$.

84. Mẫu cùng lớp bị dự đoán sai, $\mathbf{p}_i = (0{,}70;\ 0{,}20;\ 0{,}10)$ với $\mathbf{y}_i = (0{,}1{,}0)$: $\mathbf{d}_i = (0{,}70;\ -0{,}80;\ 0{,}10)$, độ dài $\| \mathbf{d}_i \| \approx 1{,}068$, gấp khoảng 44 lần.

Dấu của $\mathbf{d}_i$ nói rõ cần làm gì. Phần tử dương $(0{,}70)$ ở lớp bị đoán nhầm: điểm số lớp này phải giảm. Phần tử âm $(-0{,}80)$ ở lớp thật: điểm số lớp này phải tăng. Đây chính là công thức "dự đoán trừ thực tế" quen thuộc từ Softmax Regression.

Ví dụ: lan ngược qua mạng 2–3–3

Ta dùng lại mạng, dữ liệu và trọng số ở ví dụ mục 2.4 và 3.3, chỉ đổi nhãn của **mẫu thứ hai**: mạng đoán lớp 0 với xác suất 0,8932, nhưng giả sử nhãn thật là lớp 1 ($\mathbf{y}_2 = (0{,}1{,}0)$). Đã biết $\mathbf{p}_2 = (0{,}8932;\ 0{,}0788;\ 0{,}0280)$ và đầu ra lớp ẩn $\mathbf{a}_2^{(1)} = (0{,}9993;\ -0{,}9640;\ 0)$. Ta xét riêng mẫu này (không chia n).

Lớp ra:

$$\mathbf{d}_2^{(2)} = \mathbf{p}_2 - \mathbf{y}_2 = [0{,}8932 \quad -0{,}9212 \quad 0{,}0280]$$

Về lớp ẩn: với $\mathbf{V}^{(2)} = \begin{bmatrix} 1 & 0 & -1 \\ -1 & 1 & 0 \\ 0 & -1 & 1 \end{bmatrix}$ (mục 3.3),

$$\mathbf{d}_2^{(2)}\mathbf{V}^{(2)T} = [0{,}8652 \quad -1{,}8143 \quad 0{,}9492]$$

Qua cổng: đạo hàm của tanh tại ba nơ-ron ẩn là $\varphi' = 1 - \mathbf{a} \odot \mathbf{a} = (0{,}0013;\ 0{,}0707;\ 1{,}0000)$ nên

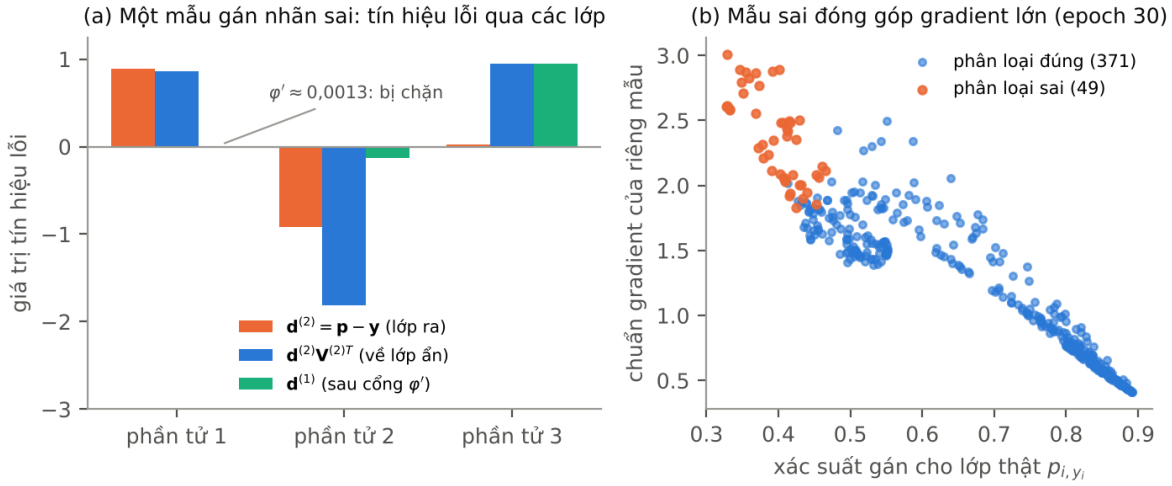
$$\mathbf{d}_2^{(1)} = [0{,}8652 \quad -1{,}8143 \quad 0{,}9492] \odot [0{,}0013 \quad 0{,}0707 \quad 1{,}0000]$$
$$= [0{,}0012 \quad -0{,}1282 \quad 0{,}9492]$$

Hình 15a vẽ ba bước này. Có ba điều đáng nhận xét:

85. **Lỗi được phân bổ theo trọng số nối.** Nơ-ron ẩn thứ hai nối đến lớp ra bằng các trọng số $(-1,1,0)$: nối âm đến lớp 0 (cần giảm) và nối dương đến lớp 1 (cần tăng). Hai đóng góp $0{,}8932 \times (-1)$ và $-0{,}9212 \times 1$ cùng dấu nên cộng dồn thành $-1{,}8143$, tín hiệu mạnh nhất. Nơ-ron ẩn thứ ba nối đến lớp 1 bằng -1 và đến lớp 2 bằng 1, nhận $0{,}9212 + 0{,}0280 = 0{,}9492$.

86. **Cổng φ' chặn nơ-ron bão hòa.** Nơ-ron ẩn thứ nhất có $\tanh = 0{,}9993$ (bão hòa), nhận $0{,}8652$ nhưng chỉ còn $0{,}0012$ sau cổng. Nơ-ron thứ hai ($\tanh = -0{,}9640$, gần bão hòa) từ $-1{,}8143$ chỉ còn $-0{,}1282$. Chỉ nơ-ron thứ ba (đang ở vùng nhạy, $\tanh = 0$) nhận trọn vẹn tín hiệu.

87. **Dấu có thể đổi qua mỗi lớp** (ví dụ phần tử thứ hai của $\mathbf{d}^{(2)}$ âm, nhưng phần tử đầu dương). Dấu quyết định mỗi nơ-ron ẩn phải tăng hay giảm đầu ra, tùy nó nối đến lớp ra bằng trọng số dương hay âm.



Hình 15. (a) Tín hiệu lỗi của mẫu thứ hai (nhãn giả sử là lớp 1) ở mạng 2–3–3: tại lớp ra, sau khi truyền về lớp ẩn, và sau cổng φ'. Nơ-ron ẩn bão hòa gần như không nhận được tín hiệu. (b) Mạng 2–8–3 trên dữ liệu ba vòng tròn, sau 30 epoch Adam: mỗi chấm là một mẫu huấn luyện, trục tung là chuẩn gradient riêng của mẫu đó. Mẫu càng bị gán xác suất thấp cho lớp thật thì gradient càng lớn.

Cập nhật. Gradient riêng của mẫu này theo $\mathbf{W}^{(2)}$ là $\tilde{\mathbf{a}}_2^{(1)T}\mathbf{d}_2^{(2)}$, với $\tilde{\mathbf{a}}_2^{(1)} = (1;\ 0{,}9993;\ -0{,}9640;\ 0)$:

$$\tilde{\mathbf{a}}_2^{(1)T}\mathbf{d}_2^{(2)} \approx \begin{bmatrix} 0{,}8932 & -0{,}9212 & 0{,}0280 \\ 0{,}8926 & -0{,}9205 & 0{,}0280 \\ -0{,}8610 & 0{,}8880 & -0{,}0270 \\ 0 & 0 & 0 \end{bmatrix}$$

Cột thứ nhất (lớp 0, đang đoán quá cao) có gradient dương cho các nơ-ron ẩn đang dương, nên bước $\mathbf{W} \leftarrow \mathbf{W} - \eta\,\nabla$ hạ điểm số lớp 0; cột thứ hai (lớp thật) bị đẩy ngược lại. Dòng cuối bằng 0 vì nơ-ron ẩn thứ ba có đầu ra bằng 0, nên nó không đóng góp vào điểm số nào ở lớp ra. Chuẩn gradient của mẫu này là 2,196 ở lớp 2 và 3,177 ở lớp 1; với mẫu thứ nhất (đoán đúng lớp 2 với

0,6923) chỉ là 0,559 và 0,699. Nếu chỉ dùng mẫu này để cập nhật một bước với $\eta = 0{,}5$ thì xác suất gán cho lớp thật tăng từ 0,0788 lên 0,7863, và mất mát của mẫu giảm từ 2,540 xuống 0,240.

Trên dữ liệu thật: mẫu sai chi phối gradient, nhất là về sau

Hình 15b đo điều này trên mạng 2–8–3 ở dữ liệu ba vòng tròn (mục 3.7). Sau 30 epoch Adam, 49 trong 420 mẫu huấn luyện (11,7%) bị phân loại sai; chuẩn gradient riêng trung bình của chúng là 2,36, gấp đôi 1,15 của các mẫu đúng. Tổng chuẩn gradient riêng của nhóm sai chiếm 21% tổng chung, gần gấp đôi tỉ lệ số mẫu. Càng về sau, mạng càng đúng nhiều nên số mẫu sai giảm; nhưng gradient của mỗi mẫu sai vẫn lớn, nên nhóm nhỏ này kéo trọng số mạnh. Ở đầu huấn luyện (epoch 2), 238 trong 420 mẫu sai và chiếm 63% tổng chuẩn gradient.

Hai hệ quả cần nhớ cho các mục sau:

88. **Mẫu nhiễu (nhãn sai thật sự) kéo rất mạnh.** Mạng không biết nhãn sai hay đúng, nên nó cũng điều chỉnh trọng số để khớp mẫu đó. Mạng càng lớn càng có khả năng làm được điều này, và đó là gốc rễ của quá khớp ở mục 4.4.

89. **Cross-entropy là lựa chọn có lý do.** Hãy so sánh với hàm mất mát bình phương $l = \frac{1}{2}(a - y)^2$ cho nơ-ron sigmoid ở lớp ra. Đạo hàm theo điểm số s là $(a - y)\, a(1 - a)$, thêm thừa số $a(1 - a)$ so với cross-entropy (mục 3.2, nơi thừa số này triệt tiêu). Với $y = 1$ và mạng đoán sai nặng ($a = 0{,}01$):

Hàm mất mát	$\partial l / \partial s$	**Nhận xét**
Cross-entropy	$a - y = -0{,}99$	Lỗi nặng, kéo mạnh
Bình phương	$(a - y)\, a(1 - a) = -0{,}0098$	Nhỏ hơn 101 lần: nơ-ron bão hòa ở sai chỗ nên gần như "đứng yên"

Bình phương sai số nhận phạt đúng nhưng sửa chậm chính khi lỗi lớn nhất. Đó là một trong các lý do bài toán phân loại dùng cross-entropy (mục 2.4).

4.2. Hàm mất mát không còn lồi: cực tiểu địa phương, điểm yên ngựa và vai trò của khởi tạo

Từ lồi sang không lồi

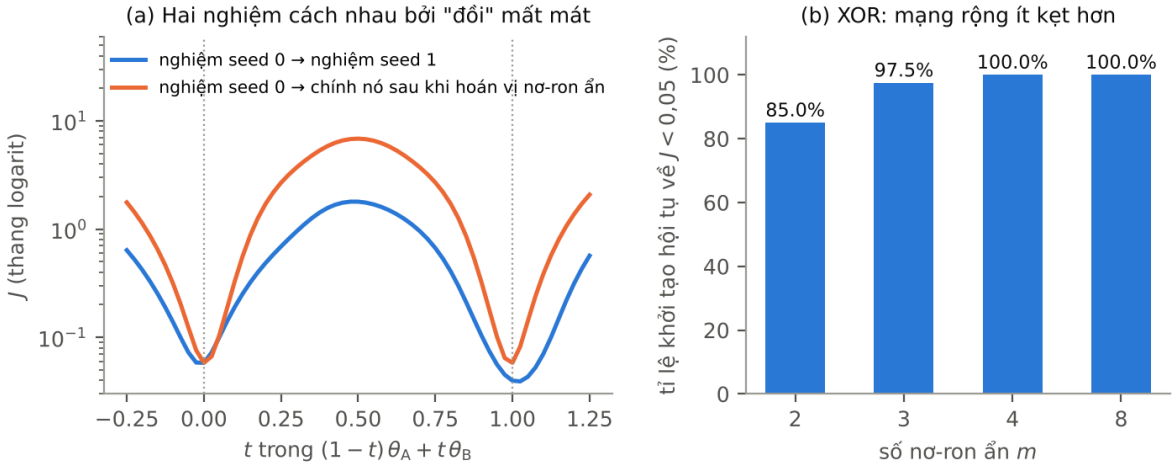
Với Softmax Regression, J là hàm lồi theo $\mathbf{W}$: mọi điểm dừng của Gradient Descent đều là cực tiểu toàn cục, và kết quả không phụ thuộc điểm xuất phát. Với mạng nơ-ron điều này không còn đúng. Có một chứng minh đơn giản. Một hàm lồi thỏa mãn

$$J(1/2\,\boldsymbol{\theta}_A + 1/2\,\boldsymbol{\theta}_B) \le 1/2\,J(\boldsymbol{\theta}_A) + 1/2\,J(\boldsymbol{\theta}_B)$$

với mọi cặp tham số $\boldsymbol{\theta}_A$, $\boldsymbol{\theta}_B$ (ở đây $\boldsymbol{\theta}$ gộp mọi ma trận trọng số thành một vector). Ta tạo ra hai bộ tham số có **cùng** mất mát nhờ tính **đối xứng hoán vị**: nếu đổi chỗ hai nơ-ron ẩn trong cùng một lớp (đổi hai cột của $\mathbf{W}^{(1)}$ và hai hàng tương ứng của $\mathbf{V}^{(2)}$) thì tổng $\sum_j \varphi\left(z_j\right) v_j$ không đổi, nên hàm do mạng biểu diễn y hệt và J cũng y hệt.

Ta kiểm chứng trên mạng 2–8–3 đã huấn luyện (seed 0, dữ liệu vòng tròn): hoán vị ngẫu nhiên 8 nơ-ron ẩn cho đầu ra giống nhau đến sai số làm tròn (8×10^{-16}) và $J = 0{,}0582$ ở cả hai bộ tham số. Nhưng ở giữa đoạn nối hai bộ tham số đó, $J = 6{,}85$ (Hình 16a, đường cam), lớn hơn nhiều lần 0,0582. Hàm lồi không thể như vậy, nên J **của mạng nơ-ron không lồi**.

Số bộ tham số tương đương như vậy rất nhiều: với một lớp ẩn m nơ-ron có $m!$ cách hoán vị (và với tanh còn nhân thêm 2^m cách đổi dấu cả trọng số vào lẫn ra của một nơ-ron, vì tanh là hàm lẻ). Với $m = 8$ đã là $8! = 40\,320$ hoặc $2^8 \cdot 8! = 10\,321\,920$ bộ tham số cho cùng một hàm. Đây không phải điều đáng lo: các cực tiểu "giống nhau" này cho cùng chất lượng. Điều đáng lo là các cực tiểu **thật sự khác nhau**.



Hình 16. (a) J dọc theo đoạn thẳng nối hai nghiệm của mạng 2–8–3 (dữ liệu vòng tròn): giữa hai nghiệm từ hai khởi tạo khác nhau (xanh) và giữa một nghiệm với chính nó sau khi hoán vị nơ-ron ẩn (cam), J đều tăng lên rất cao ở giữa. (b) Tỉ lệ 40 khởi tạo ngẫu nhiên hội tụ được về $J < 0{,}05$ trên XOR theo số nơ-ron ẩn ($\eta = 1$, 5000 epoch).

Cực tiểu, cực đại và điểm yên ngựa

Gradient Descent dừng ở nơi $\nabla J \approx \mathbf{0}$ ("điểm dừng"). Loại điểm dừng được xác định bởi ma trận Hessian $\mathbf{H}_J$ (đạo hàm cấp hai). Khai triển Taylor bậc hai quanh điểm dừng:

$$J(\boldsymbol{\theta} + \boldsymbol{\delta}) \approx J(\boldsymbol{\theta}) + 1/2\, \boldsymbol{\delta}^T \mathbf{H}_J\, \boldsymbol{\delta}$$

Gọi $\mu_1, \ldots, \mu_P$ là các giá trị riêng của $\mathbf{H}_J$ (đối xứng nên đều thực), thì $\boldsymbol{\delta}^T \mathbf{H}_J \boldsymbol{\delta}$ theo hướng riêng thứ k bằng $\mu_k \parallel \boldsymbol{\delta} \parallel^2$. Phân loại:

Giá trị riêng của $\mathbf{H}_J$	Loại điểm dừng	Hành vi của Gradient Descent
Tất cả > 0	Cực tiểu địa phương	Bị giữ lại

Giá trị riêng của $\mathbf{H}_J$	Loại điểm dừng	Hành vi của Gradient Descent
Cả dương lẫn âm	**Điểm yên ngựa**	Đi ra theo hướng $\mu_k < 0$, nhưng chậm nếu $\| \mu_k \|$ nhỏ
Tất cả < 0	Cực đại địa phương	Rời đi ngay
Có giá trị ≈ 0	Vùng phẳng	Di chuyển rất chậm

Khi số tham số P rất lớn, để một điểm dừng là cực tiểu cần **cả** P giá trị riêng đều dương, nên các điểm dừng "ngẫu nhiên" hầu hết có cả dấu âm lẫn dương, tức là điểm yên ngựa. Điều này phù hợp với phân tích lý thuyết và thực nghiệm ở các mạng lớn (chẳng hạn công trình của Dauphin và cộng sự năm 2014), nơi cực tiểu địa phương "xấu" hiếm hơn nhiều so với điểm yên ngựa. Thực tế khó hơn lý thuyết một chút: dưới đây ta đo trực tiếp trên XOR.

Quan sát trên XOR 2–2–1

Ta chạy 40 khởi tạo ngẫu nhiên (mục 3.5, Xavier, hàm sigmoid, $\eta = 1$, 5000 epoch) trên XOR với số nơ-ron ẩn $m = 2,3,4,8$ (Hình 16b):

Số nơ-ron ẩn m	2	3	4	8
Số lần hội tụ về $J < 0{,}05$ (trên 40)	34	39	40	40

Hai điều thú vị. Thứ nhất, mạng càng rộng thì càng ít bị kẹt; ta quay lại điều này ở mục 4.5. Thứ hai, ở $m = 2$ có 6 lần kẹt, cả sáu đều ở $J \approx 0{,}348$. Phân tích một lần kẹt (seed 6): xác suất đầu ra là $\mathbf{p} = (0{,}002;\ 0{,}499;\ 0{,}999;\ 0{,}500)$, nghĩa là mạng đã phân loại đúng mẫu 1 và 3 nhưng gán $\approx 0{,}5$ cho mẫu 2 và 4 ($J \approx \frac{1}{4}(0{,}002 + 0{,}695 + 0{,}001 + 0{,}693) \approx 0{,}348$). Gradient ở đó chỉ còn $\| \nabla J \| = 5{,}7 \times 10^{-4}$. Hessian ($9 \times 9$, tính bằng sai phân của gradient) có 8 giá trị riêng trong khoảng $[0;\ 0{,}003]$ và một giá trị riêng lớn ($0{,}25$), **không có giá trị riêng âm đáng kể**. Đây là một thung lũng gần như phẳng theo hầu hết các hướng: gần với một cực tiểu địa phương "xấu" mà Gradient Descent hầu như không thoát ra được (các nơ-ron sigmoid ở đó đã bão hòa nên cả hai đạo hàm đều rất nhỏ). Đó là ví dụ cụ thể cho thấy **kết quả huấn luyện phụ thuộc điểm xuất phát**.

Cao nguyên ở Hình 14 là vùng điểm yên ngựa

Giờ ta quay lại Hình 14. Từ đúng điểm khởi tạo ở mục 3.6 ($J = 0{,}7011$), ta tính Hessian (9×9) tại ba thời điểm:

Epoch	J	$\| \nabla J \|$	**Giá trị riêng nhỏ nhất**	**Số giá trị riêng** $< -10^{-4}$
0	0,7011	0,0728	$-0{,}0203$	4
300	0,6930	0,0015	$-0{,}0051$	3
600	0,6850	0,0133	$-0{,}0128$	3

Trong suốt cao nguyên, $\| \nabla J \|$ nhỏ (tại epoch 300 chỉ còn $0{,}0015$) nhưng Hessian luôn có giá trị riêng âm: đây là vùng **điểm yên ngựa**, có hướng thoát nhưng độ cong theo hướng đó rất nhỏ ($-0{,}005$ đến $-0{,}02$), nên Gradient Descent đi rất chậm trước khi tăng tốc rồi giảm đột ngột.

Cũng là lời giải thích cho việc ở mục 3.6, $\eta = 0{,}5$ cần khoảng gấp đôi số epoch của $\eta = 1$ để rơi khỏi cao nguyên.

Hai nghiệm tốt vẫn khác nhau: vai trò của khởi tạo

Với mạng 2–8–3 ở dữ liệu vòng tròn, hai khởi tạo seed $= 0$ và seed $= 1$ cho $J = 0{,}0582$ và $0{,}0398$ (sau 300 epoch Adam), độ chính xác kiểm tra cùng cỡ 96–98% (mục 3.7). Nhưng giữa hai nghiệm này, J nhô lên tới $1{,}79$ tại điểm giữa (Hình 16a, đường xanh). Hai nghiệm nằm ở hai "thung lũng" khác nhau, ngăn cách bởi một "sườn đồi" mất mát. Ở các mạng lớn, các thung lũng khác nhau thường cho chất lượng gần như nhau, nên điều ta cần là:

90. **Khởi tạo ngẫu nhiên có kiểm soát** (Xavier hoặc He, mục 3.5) để bắt đầu ở vùng nhạy, không bão hòa.
91. **Chạy nhiều khởi tạo khác nhau** và báo cáo trung bình cùng độ lệch chuẩn, như ta đã làm ở mục 3.5 và 3.7.
92. **Dùng mạng đủ rộng** (Hình 16b) và các thuật toán có quán tính (momentum, Adam) và nhiễu của mini-batch để vượt điểm yên ngựa và vùng phẳng.

4.3. Phá đối xứng, bão hòa và gradient biến mất hoặc bùng nổ

Mục 3.5 đã nói ngắn gọn về hai hiện tượng: đối xứng và bão hòa. Bây giờ ta phân tích chúng bằng các công thức lan truyền ngược, rồi mở rộng sang gradient biến mất hoặc bùng nổ.

Phá đối xứng

Giả sử hai nơ-ron ẩn j và k cùng lớp có trọng số đi vào giống nhau (hai cột j, k của $\mathbf{W}^{(1)}$ bằng nhau) và trọng số đi ra giống nhau (hai hàng j, k của $\mathbf{V}^{(2)}$ bằng nhau). Khi đó đầu ra $h_j = h_k$ với mọi mẫu, và tín hiệu lỗi nhận được cũng bằng nhau: $d_j^{(1)} = d_k^{(1)}$ (vì $d_j^{(1)} = \left(\mathbf{d}^{(2)}\mathbf{v}_j\right)\varphi'\left(z_j\right)$ với $\mathbf{v}_j$ là hàng thứ j của $\mathbf{V}^{(2)}$). Do đó hai cột của $\nabla_{\mathbf{W}^{(1)}} J = \tilde{\mathbf{X}}^T\mathbf{D}^{(1)}$ cũng bằng nhau, và sau cập nhật chúng **vẫn bằng nhau**. Trạng thái đối xứng này là một điểm dừng được bảo toàn mãi mãi. Nó là một điểm yên ngựa: Hessian tại điểm mà mục 3.5 bị kẹt ($J = 0{,}4832$, mạng 2–3–1, mọi trọng số bằng 0,3) có hai giá trị riêng âm ($\approx -2 \times 10^{-4}$), ứng với hướng làm hai nơ-ron lệch nhau.

Nếu có một nhiễu nhỏ ε thì nhiễu đó khuếch đại dần theo hướng âm của yên ngựa. Ta đo bằng thực nghiệm: mạng 2–3–1 XOR, mọi trọng số bằng 0,3 cộng nhiễu Gauss độ lệch chuẩn ε, $\eta = 1$; số epoch để $J < 0{,}1$ (trung vị của 5 lần chạy):

ε	0	10^{-1}	10^{-2}	10^{-3}	10^{-4}	10^{-6}
Số epoch để $J < 0{,}1$	không thoát	780	1020	1180	3040 đến 4290 (thoát 3/5 lần)	không thoát sau 40 000

Nhiễu càng nhỏ, thoát khỏi yên ngựa càng chậm; đến mức rất nhỏ, về mặt thực hành là không thoát. Vì vậy ta chọn khởi tạo ngẫu nhiên với độ lệch chuẩn cỡ Xavier hoặc He thay vì ε rất nhỏ.

Bão hòa: hàm kích hoạt "tắt"

Công thức $\mathbf{D}^{(l-1)} = \left(\mathbf{D}^{(l)}\mathbf{V}^{(l)T}\right) \odot \varphi'\left(\mathbf{Z}^{(l-1)}\right)$ có thừa số φ' ở mỗi lớp ẩn. Khi $|z|$ lớn, sigmoid và tanh **bão hòa** và đạo hàm tiến về 0:

z	0	2	5	10
$\sigma'(z)$	0,25	0,105	0,0066	$4{,}5 \times 10^{-5}$
$\tanh'(z)$	1	0,0707	$1{,}8 \times 10^{-4}$	$8{,}2 \times 10^{-9}$

Nơ-ron bão hòa không nhận tín hiệu lỗi (ví dụ ở mục 4.1: 0,8652 thành 0,0012) và các trọng số đi vào nó gần như không học nữa. Ở mục 3.5 ta cũng đã thấy khởi tạo quá lớn ($\sigma = 1$) làm đầu ra tanh bám sát ± 1 ở mọi lớp.

Với ReLU, $\varphi' \in \{0,1\}$, nên không có bão hòa ở phía dương, nhưng có một vấn đề khác: nếu một nơ-ron có $z < 0$ với **mọi** mẫu thì $\varphi' = 0$ với mọi mẫu, gradient của nó luôn bằng 0 và nơ-ron đó "chết", không bao giờ hồi sinh. Tốc độ học quá lớn hoặc khởi tạo xấu làm điều này xảy ra nhiều hơn. Biến thể Leaky ReLU, $\varphi(z) = \max(\alpha z, z)$ với α nhỏ như 0,01, giữ lại một chút gradient ở phía âm để tránh nơ-ron chết.

Gradient biến mất và bùng nổ

Mở rộng công thức lan truyền ngược qua nhiều lớp. Với một mẫu, viết $\mathbf{d}^{(l-1)} = \left(\mathbf{d}^{(l)}\mathbf{V}^{(l)T}\right) \odot \varphi'\left(\mathbf{z}^{(l-1)}\right)$. Phép $\odot$ với một vector không làm độ dài tăng quá $\gamma = \max|\varphi'|$ lần, còn phép nhân với $\mathbf{V}^{(l)T}$ không làm độ dài tăng quá $\| \mathbf{V}^{(l)} \|_2$ lần ($\|\cdot\|_2$ là chuẩn phổ, tức giá trị kỳ dị lớn nhất). Do đó

$$\|\mathbf{d}^{(l-1)}\| \le \gamma \|\mathbf{V}^{(l)}\|_2 \|\mathbf{d}^{(l)}\| \quad \Rightarrow \quad \|\mathbf{d}^{(1)}\| \le \left(\gamma \prod_{l=2}^{L} \|\mathbf{V}^{(l)}\|_2\right) \|\mathbf{d}^{(L)}\|$$

(với γ^{L-1} nếu mọi lớp dùng cùng một hàm kích hoạt). Đây là một **cận trên**, nhưng nó cho thấy cơ chế: tín hiệu lỗi bị nhân lên qua $L-1$ lớp, mỗi lớp một thừa số cỡ $\gamma \| \mathbf{V} \|_2$. Nếu thừa số này < 1 thì gradient ở các lớp đầu co lại theo cấp số nhân (**biến mất**); nếu thừa số > 1 thì gradient có thể lớn lên theo cấp số nhân (**bùng nổ**). Sigmoid có $\gamma = 0{,}25$, nên chỉ cần $\| \mathbf{V} \|_2$ cỡ 2 ($\gamma \| \mathbf{V} \|_2 = 0{,}5$) thì qua 10 lớp gradient đã nhỏ đi cỡ $0{,}5^{10} \approx 10^{-3}$.

Cụ thể hơn, với khởi tạo Xavier và $m_{l-1} = m_l = m$ thì $\mathrm{Var}(w) = 1/m$ và mỗi lớp nhân phương sai của tín hiệu lỗi với $\mathbb{E}[\varphi'^2]$. Với tanh ở gần $z = 0$, $\varphi' \approx 1$ nên hệ số ≈ 1 và gradient được giữ nguyên độ lớn. Với sigmoid, $\varphi' \le 0{,}25$ nên hệ số phương sai $\le 1/16$, tức độ lệch chuẩn co lại ≥ 4 lần mỗi lớp.

Đo bằng thực nghiệm

Ta xây mạng 10 lớp ẩn, mỗi lớp 50 nơ-ron, đầu vào là dữ liệu ba vòng tròn (đã chuẩn hóa) và đo chuẩn gradient của từng lớp, $\| \nabla_{\mathbf{V}^{(l)}} J \|_F$, ngay tại khởi tạo. Lớp 1 gần đầu vào nhất, lớp 11 là lớp ra. Hàm đo và kết quả (một khởi tạo, `seed = 0`):

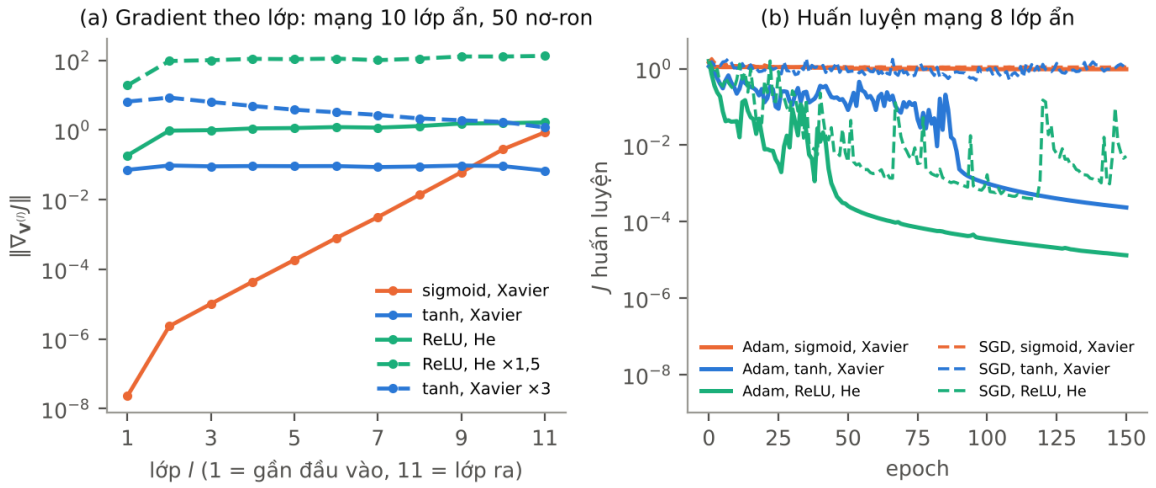
```python
def chuan_gradient_theo_lop(X, Y, Ws, kich_hoat):
    """Chuẩn Frobenius của gradient theo V^(l) (bỏ hàng chệch) cho từng lớp."
""
    As, Zs = lan_truyen_thuan(X, Ws, kich_hoat)
    grads = lan_truyen_nguoc(As, Zs, Y, Ws, kich_hoat)
    return [np.linalg.norm(g[1:]) for g in grads]

kich_thuoc = [2] + [50] * 10 + [3]
for kh in ("sigmoid", "tanh", "relu"):
    Ws = khoi_tao(kich_thuoc, kh, seed=0)
    print(kh, chuan_gradient_theo_lop(Za, Ya, Ws, kh))
```

```
sigmoid  2.3e-08 2.2e-06 8.9e-06 3.9e-05 1.6e-04 6.7e-04 2.2e-03 9.3e-03 4.1e
-02 1.9e-01 6.1e-01
tanh     1.0e-01 1.5e-01 1.4e-01 1.1e-01 1.1e-01 1.2e-01 1.1e-01 1.2e-01 1.3e
-01 1.1e-01 9.3e-02
relu     1.8e-01 8.2e-01 8.4e-01 9.2e-01 1.3e+00 1.3e+00 1.1e+00 1.3e+00 1.4e
+00 1.2e+00 1.4e+00
```

Với sigmoid, gradient giảm đều từ 0,61 ở lớp ra xuống $2{,}3 \times 10^{-8}$ ở lớp đầu tiên: nhỏ hơn khoảng $2{,}7 \times 10^{7}$ lần, tức mỗi lớp lùi lại gradient co khoảng 0,18 lần, cùng cỡ với ước lượng lý thuyết (0,25 hoặc nhỏ hơn). Với tanh và ReLU, gradient gần như đồng đều giữa các lớp: nhờ Xavier và He được chọn đúng để giữ độ lớn của tín hiệu. (Lớp 1 của ReLU và tanh có chuẩn nhỏ hơn đôi chút vì ma trận $\mathbf{V}^{(1)}$ chỉ có 2×50 phần tử.)

Hình 17a vẽ kết quả trung bình hình học của 5 khởi tạo, thêm hai trường hợp "khởi tạo quá lớn": với ReLU và độ lệch chuẩn gấp 1,5 lần He, gradient ở mọi lớp cỡ 100, lớn hơn khoảng 80 đến 100 lần so với He chuẩn. Tín hiệu thuận bị phóng đại qua các lớp: độ lớn trung bình của điểm số ở lớp ra tăng từ 0,63 lên 54,7 (khoảng 86 lần), hàm softmax bão hòa và mất mát khởi đầu là 15,3 thay vì 1,39. Đó là biểu hiện của **bùng nổ**.



Hình 17. (a) Chuẩn gradient theo lớp tại khởi tạo, mạng 10 lớp ẩn × 50 nơ-ron, trung bình hình học 5 khởi tạo (thang logarit). Sigmoid: gradient co theo cấp số nhân khi lùi về các lớp đầu. tanh với Xavier và ReLU với He: gần đều. Khởi tạo quá lớn (nét đứt): gradient lớn lên. (b) Mất mát huấn luyện của mạng 8 lớp ẩn × 30 nơ-ron trên dữ liệu vòng tròn, trung bình 3 khởi tạo, nét liền là Adam, nét đứt là SGD.

Hậu quả khi huấn luyện

Ta huấn luyện mạng 2–30–⋯–30–3 (8 lớp ẩn, mini-batch 32, 150 epoch, 3 khởi tạo) trên dữ liệu vòng tròn của mục 3.7 (Hình 17b):

Hàm kích hoạt, khởi tạo	**Thuật toán, η**	**J huấn luyện cuối (trung bình)**	**Độ chính xác kiểm tra (3 lần chạy)**
sigmoid, Xavier	Adam, 0,01	0,969	43,3%, 42,8%, 43,9%
sigmoid, Xavier	SGD, 0,5	1,107	33,3% ở cả ba lần (đoán mò)
tanh, Xavier	Adam, 0,01	2×10^{-4}	97,8%, 99,4%, 98,3%
tanh, Xavier	SGD, 0,5	0,950	66,1%, 43,9%, 33,3%
ReLU, He	Adam, 0,01	1×10^{-5}	99,4%, 100%, 99,4%
ReLU, He	SGD, 0,1	0,005	99,4%, 99,4%, 98,9%

Mạng sigmoid sâu gần như không học được: với SGD, J dừng ở 1,107, xấp xỉ $\ln 3 \approx 1{,}099$ của mô hình chưa biết gì, kể cả sau 150 epoch; Adam cho kết quả tốt hơn một chút ($J = 0{,}969$) nhưng vẫn chỉ khoảng 43%. Mạng tanh với SGD chỉ học được một phần ở một khởi tạo (66,1%) và thất bại ở hai khởi tạo còn lại; Adam giúp rất nhiều vì nó chuẩn hóa độ lớn bước theo từng tham số (mục 3.5), nên gradient nhỏ vẫn được bước đi cỡ η. ReLU với He học tốt ở cả hai thuật toán. Kết quả này chỉ đúng với mạng, dữ liệu và siêu tham số cụ thể ở đây; nó không có nghĩa sigmoid luôn tệ hơn ReLU, nhưng cho thấy vì sao ReLU gần như thành mặc định cho các lớp ẩn.

Cách khắc phục

Vấn đề	**Cách khắc phục thường dùng**
Đối xứng	Khởi tạo ngẫu nhiên; không bao giờ khởi tạo mọi trọng số bằng nhau
Bão hòa tanh, sigmoid	Khởi tạo Xavier; chuẩn hóa đầu vào (mục 3.7 đã chuẩn hóa dữ liệu); hạn chế dùng sigmoid ở lớp ẩn
Gradient biến mất	ReLU và biến thể (Leaky ReLU), khởi tạo He, thuật toán thích nghi (Adam); ngoài phạm vi chương này: chuẩn hóa theo lô (batch normalization) và kết nối tắt (residual connection)
Gradient bùng nổ	Khởi tạo đúng độ lớn, giảm η, **cắt gradient**
ReLU chết	Leaky ReLU, giảm η, khởi tạo He

Cắt gradient (gradient clipping) co toàn bộ gradient về độ dài tối đa c khi nó quá dài:

$$\mathbf{g} \leftarrow \mathbf{g} \cdot \min\left(1,\ \frac{c}{\|\ \mathbf{g}\ \|}\right)$$

Ví dụ $\mathbf{g} = (3{,}4)$ có $\|\ \mathbf{g}\ \| = 5$; với $c = 1$ ta được $\mathbf{g} \leftarrow (0{,}6;\ 0{,}8)$, giữ nguyên hướng nhưng độ dài chỉ còn 1. Cài đặt (với $grads$ là danh sách gradient của các lớp):

```python
def cat_gradient(grads, nguong):
    """Cắt gradient: nếu tổng chuẩn vượt ngưỡng thì co lại cho đúng bằng ngưỡng."""
    chuan = np.sqrt(sum((g ** 2).sum() for g in grads))
    he_so = min(1.0, nguong / (chuan + 1e-12))
    return [g * he_so for g in grads]
```

Thói quen tốt khi huấn luyện mạng sâu: in chuẩn gradient theo lớp (như hàm *chuan_gradient_theo_lop* ở trên) ở vài epoch đầu; nếu các lớp đầu nhỏ hơn lớp cuối vài bậc độ lớn thì có vấn đề biến mất, nếu lớn đột biến thì có vấn đề bùng nổ.

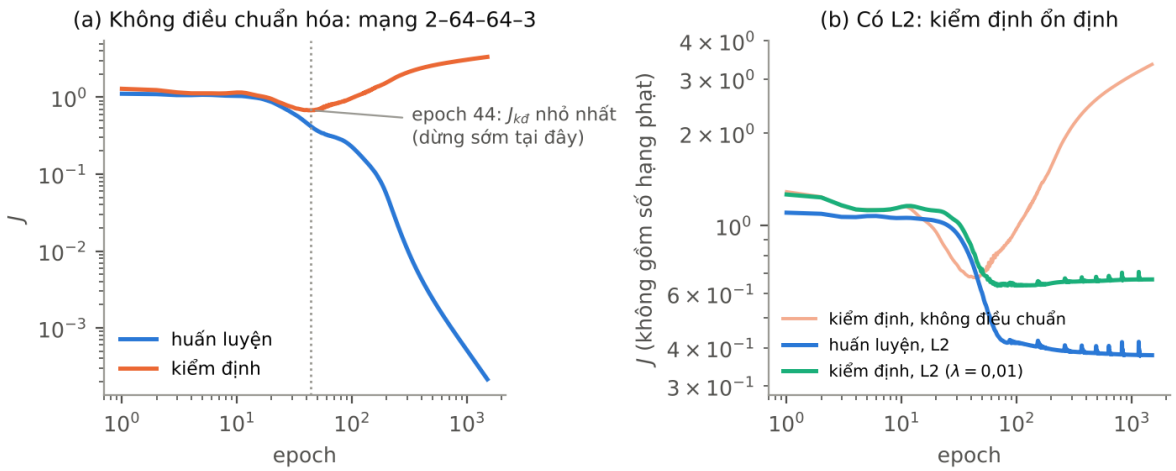
4.4. Quá khớp và các kỹ thuật điều chuẩn hóa (L2, dừng sớm, dropout)

Quá khớp: khi mạng học thuộc lòng

Ở bài Softmax (mục 4.4), ta đã thấy khi số tham số vượt số mẫu thì mô hình có thể khớp hoàn hảo tập huấn luyện mà không học được quy luật chung. Mạng nơ-ron rất dễ rơi vào tình huống đó vì có rất nhiều tham số. **Quá khớp** (overfitting) là khi mất mát trên tập huấn luyện tiếp tục giảm nhưng mất mát trên dữ liệu chưa từng thấy bắt đầu tăng.

Thiết lập thí nghiệm. Ta sinh dữ liệu ba vòng tròn đồng tâm bán kính 1, 2,5, 4 nhưng với độ nhòe lớn ($\sigma = 0{,}8$) để các lớp lấn lên nhau, nên không phân loại hoàn hảo được: lỗi tối thiểu là có thật. Tập huấn luyện chỉ có 60 mẫu (20 mỗi lớp), tập kiểm định 300 mẫu, tập kiểm tra 900 mẫu. Mạng 2–64–64–3 (tanh, 4547 tham số, gấp khoảng 76 lần số mẫu), Adam với $\eta = 0{,}01$, toàn bộ lô, 1500 epoch. Để biết mức tốt nhất có thể, ta huấn luyện cùng cấu trúc trên 6000 mẫu: độ chính xác kiểm tra 75,3%. Đó là mức tham chiếu.

Kết quả khi không điều chuẩn hóa (Hình 18a): J huấn luyện giảm đều đến 2×10^{-4} và độ chính xác huấn luyện đạt 100%. Nhưng J kiểm định giảm đến nhỏ nhất (0,677) tại epoch 44 rồi **tăng trở lại** đến 3,35 ở cuối. Độ chính xác kiểm tra chỉ còn 65,2%, kém 10 điểm phần trăm so với mức tham chiếu. Vùng quyết định ở Hình 19a cho thấy đường biên uốn lượn quanh từng điểm nhiễu để khớp các mẫu sai, thay vì giữ dạng ba vành tròn.



Hình 18. Đường cong học của mạng 2–64–64–3 trên 60 mẫu huấn luyện (thang logarit cả hai trục). (a) Không điều chuẩn hóa: J kiểm định nhỏ nhất ở epoch 44 rồi tăng. (b) Có L2: J kiểm định ổn định; đường mờ là kiểm định không điều chuẩn để so sánh.

L2: phạt trọng số lớn

Mạng khớp nhiễu bằng cách dùng trọng số rất lớn để tạo biên cong gắt (chuẩn của toàn bộ $\mathbf{V}$ lên đến 36,1 trong thí nghiệm trên). Điều chuẩn hóa L2 thêm vào hàm mất mát một số hạng phạt độ lớn trọng số (không phạt hệ số chệch):

$$J_\lambda = J + \frac{\lambda}{2}\sum_{l=1}^{L}\|\mathbf{V}^{(l)}\|_F^2$$

với $\lambda \geq 0$ là hệ số điều chuẩn hóa. Chỉ cần thêm $\lambda\mathbf{V}^{(l)}$ vào gradient của lớp l (hàng đầu, ứng với hệ số chệch, giữ nguyên):

$$\nabla_{\mathbf{V}^{(l)}} J_\lambda = \nabla_{\mathbf{V}^{(l)}} J + \lambda\mathbf{V}^{(l)}$$

Với SGD, bước cập nhật trở thành

$$\mathbf{V}^{(l)} \leftarrow \mathbf{V}^{(l)} - \eta\left(\nabla_{\mathbf{V}^{(l)}} J + \lambda\mathbf{V}^{(l)}\right) = (1-\eta\lambda)\,\mathbf{V}^{(l)} - \eta\,\nabla_{\mathbf{V}^{(l)}} J$$

Mỗi bước, trọng số bị co lại một chút trước khi cập nhật theo gradient, nên L2 còn được gọi là **suy giảm trọng số** (weight decay). (Với Adam, L2 và suy giảm trọng số không còn tương đương; phiên bản AdamW tách riêng hai việc này.) Ví dụ: $V = 2{,}0$, $\eta = 0{,}1$, $\lambda = 0{,}5$, $\nabla J = 0{,}3$ cho $V \leftarrow 0{,}95 \times 2{,}0 - 0{,}1 \times 0{,}3 = 1{,}87$.

Trực giác: trọng số nhỏ làm hàm số mà mạng biểu diễn "trơn" hơn. Theo cận ở mục 4.3, độ lớn của đạo hàm của đầu ra theo đầu vào bị chặn bởi tích $\gamma \parallel \mathbf{V}^{(l)} \parallel_2$ qua các lớp, nên giữ trọng số nhỏ thì đầu ra không thay đổi nhanh khi đầu vào thay đổi một chút. Từ góc độ xác suất, L2 tương ứng với giả định trọng số có phân phối Gauss quanh 0.

Chọn λ theo tập kiểm định (5 khởi tạo mỗi giá trị):

λ	10^{-4}	10^{-3}	3×10^{-3}	10^{-2}	3×10^{-2}	10^{-1}
Độ chính xác kiểm định	65,3%	67,2%	68,9%	72,4%	71,5%	33,3%
Độ chính xác kiểm tra	65,9%	65,0%	66,8%	72,1%	71,3%	33,3%

λ quá nhỏ gần như không có tác dụng; λ quá lớn (0,1) ép mọi trọng số về 0, mạng chỉ còn đoán mò (**thiếu khớp**, underfitting). Giá trị tốt nhất theo kiểm định là $\lambda = 0{,}01$. Bước chọn này làm trên tập kiểm định, không phải tập kiểm tra (mục 5.2). Với $\lambda = 0{,}01$, J kiểm định không còn tăng trở lại (Hình 18b) và đường biên ở Hình 19c trơn, gần đúng ba vành tròn.

Dừng sớm

Hình 18a gợi ý một cách đơn giản: **ngừng huấn luyện khi J kiểm định bắt đầu tăng**. Quy trình:

93. Sau mỗi epoch, tính J trên tập kiểm định.
94. Lưu lại bộ trọng số có $J_{\text{kiểm định}}$ nhỏ nhất từ trước đến giờ.
95. Nếu sau k epoch (gọi là "kiên nhẫn") mà $J_{\text{kiểm định}}$ không cải thiện, dừng và trả về bộ trọng số đã lưu.

Mạng khởi tạo với trọng số nhỏ và chỉ lớn dần theo huấn luyện, nên dừng sớm cũng giữ trọng số nhỏ, cùng tác dụng với L2. Trong thí nghiệm, với kiên nhẫn $k = 30$, mạng dừng với bộ trọng số tốt nhất ở epoch 46 (trung bình 10 khởi tạo), chuẩn $\| \mathbf{V} \|$ còn 12,5 và độ chính xác kiểm tra 69,7% (Hình 19b).

Dropout

Dropout ngẫu nhiên "tắt" các nơ-ron ẩn trong lúc huấn luyện, buộc mạng không dựa quá nhiều vào một vài nơ-ron. Với xác suất tắt p, mỗi lần tính cho một mini-batch ta sinh ma trận mặt nạ $\mathbf{M}^{(l)}$ cùng kích thước với $\mathbf{A}^{(l)}$, mỗi phần tử độc lập bằng 0 với xác suất p và bằng $1/(1-p)$ với xác suất $1-p$, rồi nhân từng phần tử:

$$\widehat{\mathbf{A}}^{(l)} = \varphi_l\left(\mathbf{Z}^{(l)}\right) \odot \mathbf{M}^{(l)}$$

Phép chia cho $1-p$ (dropout "đảo ngược") giữ kỳ vọng của đầu ra không đổi: $\mathbb{E}[m] = 0 \cdot p + \frac{1}{1-p}(1-p) = 1$. Nhờ vậy khi **dự đoán** ta dùng mạng bình thường, không cần mặt nạ hay điều chỉnh gì thêm. Ví dụ với $p = 0{,}5$ một đầu ra $a = 0{,}8$ hoặc thành 0, hoặc thành 1,6, và kỳ vọng vẫn là 0,8.

Lan truyền ngược chỉ cần thêm mặt nạ vào chỗ truyền sai số: nơ-ron bị tắt không đóng góp vào đầu ra nên không nhận lỗi, còn nơ-ron được giữ nhận lỗi nhân thêm $1/(1-p)$. Công thức mục 3.4 đổi thành

$$\mathbf{D}^{(l-1)} = \left(\mathbf{D}^{(l)}\mathbf{V}^{(l)T} \odot \mathbf{M}^{(l-1)}\right) \odot \varphi'_{l-1}\left(\mathbf{Z}^{(l-1)}\right)$$

trong đó φ' vẫn tính từ $\mathbf{Z}^{(l-1)}$ (không phải từ đầu ra đã nhân mặt nạ). Vì mặt nạ ngẫu nhiên, khi **kiểm tra gradient** ở mục 3.4 cần cố định mặt nạ (nếu không, hai lần tính $J(w \pm \varepsilon)$ dùng hai mặt nạ khác nhau và đạo hàm số vô nghĩa). Với mặt nạ cố định, cài đặt dưới đây cho sai số tương đối $1{,}6 \times 10^{-7}$ trên mạng 3–5–4–3 ($p = 0{,}3$, $\lambda = 0{,}1$).

Hiệu quả. Dropout huấn luyện đồng thời rất nhiều mạng con (mỗi mặt nạ là một mạng con) có chung trọng số, và dự đoán cuối cùng gần như là trung bình của chúng.

Cài đặt

```
def thuan_dropout(X, Ws, kich_hoat="tanh", p=0.0, rng=None, masks=None):
    """Lan truyền thuận có dropout đảo ngược trên các lớp ẩn (p = 0: không dr
opout)."""
    phi = KICH_HOAT[kich_hoat][0]
    A, As, Zs, Ms = X, [X], [], []
    for l, W in enumerate(Ws):
        Z = them_bias(A) @ W
        Zs.append(Z)
        if l == len(Ws) - 1:
            A = softmax(Z)
        else:
            A = phi(Z)
            if p > 0:
                M = masks[l] if masks is not None else (rng.random(A.shape) >
= p) / (1 - p)
                Ms.append(M)
                A = A * M
        As.append(A)
    return As, Zs, Ms

def nguoc_dropout(As, Zs, Ms, Y, Ws, kich_hoat="tanh", lam=0.0):
    """Lan truyền ngược có mặt nạ dropout và số hạng phạt L2 (không phạt hệ s
ố chệch)."""
    phi, dphi = KICH_HOAT[kich_hoat]
    n, L = len(Y), len(Ws)
    grads = [None] * L
    D = (As[-1] - Y) / n
    for l in range(L - 1, -1, -1):
        grads[l] = them_bias(As[l]).T @ D
        if lam > 0:
            grads[l][1:] += lam * Ws[l][1:]              # thêm lambda * V
        if l > 0:
            dA = D @ Ws[l][1:].T
            if Ms:
                dA = dA * Ms[l - 1]                      # mặt nạ dropout
            D = dA * dphi(phi(Zs[l - 1]), Zs[l - 1])     # phi' tính từ Z, k
hông từ A đã mặt nạ
    return grads

def huan_luyen_dc(X, Y, kich_thuoc, Xv=None, Yv=None, kich_hoat="tanh", thuat
```

```
_toan="adam", eta=0.01,
                  so_epoch=500, b=None, seed=0, lam=0.0, p=0.0, dung_som=None
):
    """MLP softmax; L2 (lam), dropout (p), dừng sớm theo mất mát kiểm định (k
iên nhẫn dung_som epoch)."""
    n = len(X); b = n if b is None else b
    rng = np.random.default_rng(seed)
    Ws = khoi_tao(kich_thuoc, kich_hoat, seed)
    tt = tao_trang_thai(Ws)
    def danh_gia(X_, Y_):
        return mat_mat(thuan_dropout(X_, Ws, kich_hoat)[0][-1], Y_)    # khôn
g dropout khi đánh giá
    ls_tr, ls_va = [danh_gia(X, Y)], [danh_gia(Xv, Yv) if Xv is not None else
np.nan]
    tot, Ws_tot, ep_tot = np.inf, None, 0
    for ep in range(so_epoch):
        idx = rng.permutation(n) if b < n else np.arange(n)
        for k in range(0, n, b):
            lo = idx[k:k + b]
            As, Zs, Ms = thuan_dropout(X[lo], Ws, kich_hoat, p, rng)
            grads = nguoc_dropout(As, Zs, Ms, Y[lo], Ws, kich_hoat, lam)
            cap_nhat(Ws, grads, tt, thuat_toan, eta)
        ls_tr.append(danh_gia(X, Y)); ls_va.append(danh_gia(Xv, Yv) if Xv is
not None else np.nan)
        if dung_som is not None:
            if ls_va[-1] < tot: tot, Ws_tot, ep_tot = ls_va[-1], [W.copy() fo
r W in Ws], ep + 1
            elif ep + 1 - ep_tot >= dung_som: break
    if dung_som is not None: Ws = Ws_tot
    return Ws, ls_tr, ls_va, ep_tot
```

Hàm dùng lại *them_bias*, *softmax*, *KICH_HOAT*, *khoi_tao*, *mat_mat*, *tao_trang_thai*, *cap_nhat* ở mục 3.7. Gọi ví dụ cho L2 với $\lambda = 0{,}01$ (kiểm định *Xv*, *Yv* chỉ để theo dõi):

```
Ws, ls_tr, ls_va, _ = huan_luyen_dc(Xt, Yt, [2, 64, 64, 3], Xv, Yv, lam=0.01,
so_epoch=1500, seed=0)
```

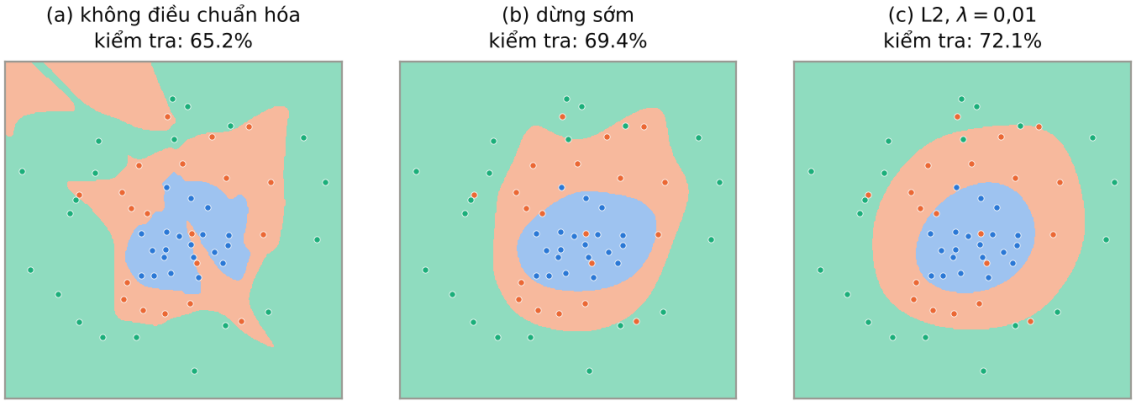
So sánh các cách điều chuẩn hóa

Ta chạy từng cách với 10 khởi tạo (cùng dữ liệu) và đo trên tập kiểm tra 900 mẫu:

Cách	Độ chính xác kiểm tra (TB $\pm$ độ lệch chuẩn)	J kiểm tra	$\|\| \mathbf{V} \|\|$ cuối
Không điều chuẩn hóa	$66{,}3\% \pm 0{,}8\%$	2,826	36,1
Dừng sớm ($k = 30$)	$69{,}7\% \pm 0{,}5\%$	0,645	12,5
L2 ($\lambda = 0{,}01$)	$72{,}1\% \pm 0{,}0\%$	0,654	5,7
Dropout ($p = 0{,}1$)	$66{,}8\% \pm 0{,}8\%$	1,372	41,4

Cách	Độ chính xác kiểm tra (TB $\pm$ độ lệch chuẩn)	J kiểm tra	$\|\| \mathbf{V} \|\|$ cuối
L2 + dropout ($\lambda = 0{,}01$, $p = 0{,}1$)	$71{,}6\% \pm 0{,}6\%$	0,619	6,0

(Mức tham chiếu với 6000 mẫu là 75,3%.) Hình 19 vẽ vùng quyết định.



Hình 19. Vùng quyết định của mạng 2–64–64–3 (khởi tạo 0) huấn luyện trên 60 mẫu (các chấm). (a) Không điều chuẩn hóa: biên chằng chịt, khớp cả mẫu nhiễu. (b) Dừng sớm. (c) L2 với $\lambda = 0{,}01$: biên trơn, gần ba vành tròn đồng tâm. Con số dưới mỗi hình là độ chính xác trên 900 mẫu kiểm tra.

Cần đọc bảng cẩn thận:

96. **L2 và dừng sớm cải thiện rõ**, L2 gần tới mức tham chiếu. Dừng sớm cho J kiểm tra tương đương L2 nhưng độ chính xác kém hơn.

97. **Dropout với $p = 0{,}1$ không giúp ở đây.** Ta thử thêm $p = 0{,}3$ và 0,5 (với 5 khởi tạo): độ chính xác kiểm tra 66,4% và 66,5%; với mạng rộng hơn (2–256–256–3) là 64,8% và 65,6%, trong khi không dropout là 66,5%. Dropout thiết kế cho mạng lớn trên tập dữ liệu lớn; với 60 mẫu, huấn luyện toàn lô thì gần như không có ích, đừng suy ra rằng dropout "luôn tốt". Kết hợp L2 và dropout (71,6%) không tốt hơn L2 riêng (72,1%).

98. Độ lệch chuẩn chỉ phản ánh sự khác nhau do khởi tạo, **không** phản ánh sự khác nhau do dữ liệu (ta dùng một tập huấn luyện duy nhất), và 1% trên 900 mẫu chỉ là 9 mẫu. Kết quả này chỉ có ý nghĩa như minh họa cho một bài toán nhỏ.

Ngoài ra còn các cách khác: thu thập thêm dữ liệu, tăng cường dữ liệu (data augmentation), giảm kích thước mạng, chuẩn hóa theo lô. Tuy nhiên, quy trình chọn cách nào và siêu tham số nào luôn giống nhau: **chọn trên tập kiểm định, báo cáo trên tập kiểm tra** (mục 5.2).

4.5. Rộng hay sâu: số nơ-ron, số lớp và khả năng biểu diễn so với khả năng học

Câu hỏi cuối của mục 4 là thiết kế: khi cần một mạng mạnh hơn, ta thêm nơ-ron vào mỗi lớp (**rộng**) hay thêm lớp (**sâu**)? Hai khái niệm cần tách bạch:

99. **Khả năng biểu diễn**: tập các hàm mà mạng *có thể* biểu diễn nếu chọn trọng số phù hợp.

100. **Khả năng học**: liệu thuật toán huấn luyện (mục 3) có *tìm được* các trọng số đó từ dữ liệu, trong thời gian chấp nhận được, hay không.

Khả năng biểu diễn: một lớp ẩn là đủ về lý thuyết

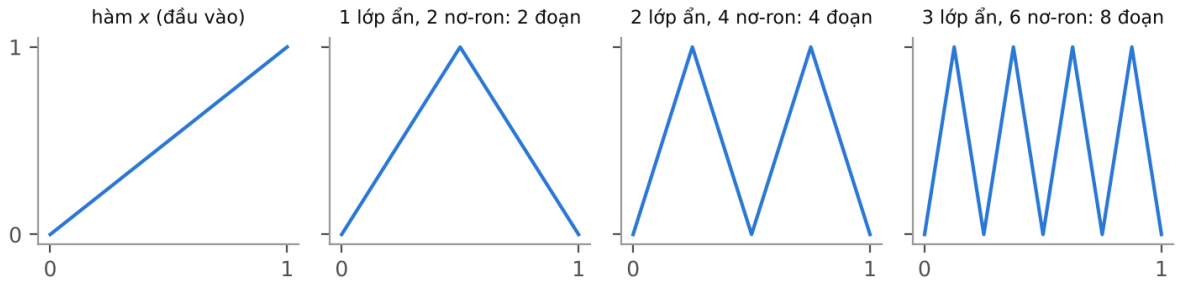
Định lý xấp xỉ phổ quát (mục 2.5) khẳng định mạng một lớp ẩn đủ rộng xấp xỉ được hàm liên tục bất kỳ. Nhưng "đủ rộng" có thể là rất rộng. Để thấy sự khác biệt giữa rộng và sâu, ta đếm số đoạn tuyến tính của hàm do mạng ReLU biểu diễn.

Mạng nông. Mạng ReLU một biến vào một lớp ẩn có m nơ-ron biểu diễn hàm tuyến tính từng khúc với **tối đa** $m+1$ **đoạn** (mỗi nơ-ron thêm tối đa một điểm gãy). Với đầu vào hai chiều, mỗi nơ-ron ẩn chia mặt phẳng bằng một đường thẳng, và m nơ-ron tạo tối đa $1 + m + \binom{m}{2}$ vùng.

Mạng sâu. Xét hàm "răng cưa" một biến, trên $[0,1]$:

$$h(x) = 2\,\mathrm{ReLU}(x) - 4\,\mathrm{ReLU}(x - 1/2) = \begin{cases} 2x & 0 \le x \le \frac{1}{2} \\ 2 - 2x & \frac{1}{2} \le x \le 1 \end{cases}$$

Đây là một lớp ẩn gồm 2 nơ-ron, vẽ một "tam giác" lên $[0,1]$. Nếu xếp k lớp như vậy nối tiếp, ta được hàm hợp $h \circ h \circ \cdots \circ h$ (k lần), mỗi lần hợp lại gấp đôi số răng. Hình 20 đếm: 1 lớp có 2 đoạn, 2 lớp có 4 đoạn, 3 lớp có 8 đoạn; tổng quát 2^k **đoạn chỉ với** $2k$ **nơ-ron**. Mạng nông muốn có 2^k đoạn thì cần ít nhất $2^k - 1$ nơ-ron. Với $k = 10$: mạng sâu cần 20 nơ-ron, mạng nông cần ít nhất 1023.

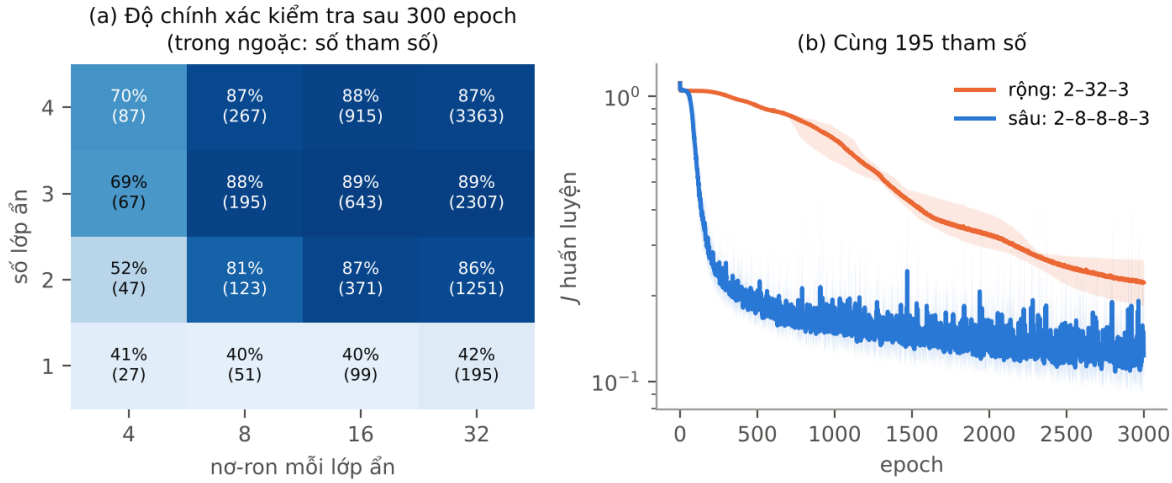


Hình 20. Hợp liên tiếp hàm $h(x)$ bằng các lớp ẩn 2 nơ-ron ReLU: số đoạn tuyến tính tăng gấp đôi sau mỗi lớp (2, 4, 8 đoạn với 2, 4, 6 nơ-ron), trong khi một lớp ẩn nông với m nơ-ron chỉ có tối đa $m+1$ đoạn.

Nghĩa là độ sâu cho phép biểu diễn *hiệu quả* các hàm có cấu trúc lặp (một đặc trưng được dùng lại và hợp thành đặc trưng phức tạp hơn), còn mạng nông phải "liệt kê" từng đoạn bằng một nơ-ron riêng.

Khả năng học: thực nghiệm trên dữ liệu xoắn ốc

Biểu diễn được chưa có nghĩa là học được. Ta dùng bài toán ba nhánh xoắn ốc (mỗi nhánh 200 điểm huấn luyện, 400 điểm kiểm tra, mỗi nhánh quay khoảng 1,4 vòng với một ít nhiễu), huấn luyện bằng Adam ($\eta = 0{,}01$, mini-batch 32, 300 epoch, tanh, trung bình 3 khởi tạo) với số lớp ẩn $d \in \{1,2,3,4\}$ và số nơ-ron mỗi lớp $m \in \{4,8,16,32\}$ (Hình 21a).



Hình 21. (a) Độ chính xác kiểm tra (và số tham số trong ngoặc) sau 300 epoch cho mạng có 1 đến 4 lớp ẩn và 4 đến 32 nơ-ron mỗi lớp, dữ liệu ba nhánh xoắn ốc. (b) Cùng 195 tham số: mạng rộng 2–32–3 và mạng sâu 2–8–8–8–3 (đường là trung bình, vùng mờ là khoảng min–max của 5 khởi tạo).

Kết quả cho thấy ba điều:

101. **Mạng một lớp ẩn gần như không học được trong 300 epoch**: dù có 4, 8, 16 hay 32 nơ-ron, độ chính xác chỉ 40 đến 42%, gần đoán mò. Mạng 2–256–3 (1539 tham số) và 2–1024–3 (6147 tham số) cũng chỉ đạt 41,7% và 42,7% với cùng 300 epoch: nhiều nơ-ron hơn không tự giải quyết vấn đề. Mạng 2–32–3 phải huấn luyện 2000 epoch mới đạt 82,7%.

102. **Từ hai lớp ẩn trở lên, mạng học nhanh**. Từ $m \geq 8$, độ chính xác 81% đến 89%.

103. **Thêm lớp không phải lúc nào cũng tốt hơn**: với $m \geq 8$, bốn lớp ẩn không cải thiện so với ba lớp (87 đến 88% so với 88 đến 89%). Mạng quá nhỏ ($m = 4$) thì thiếu sức biểu diễn dù sâu hay không.

So sánh công bằng. Mạng rộng 2–32–3 và mạng sâu 2–8–8–8–3 đều có đúng 195 tham số. Ta huấn luyện cả hai 3000 epoch với 5 khởi tạo (Hình 21b):

Mạng	J ở epoch 300	J ở epoch 1000	J ở epoch 3000	Độ chính xác kiểm tra cuối
Rộng 2–32–3	1,004	0,705	0,222	$85{,}9\% \pm 0{,}8\%$
Sâu 2–8–8–8–3	0,209	0,175	0,123	$87{,}1\% \pm 0{,}8\%$

Mạng sâu giảm mất mát nhanh hơn nhiều (ở epoch 300: 0,209 so với 1,004) và đạt chất lượng cuối cùng nhỉnh hơn một chút (1,2 điểm phần trăm, cỡ một đến hai độ lệch chuẩn, nên chưa kết luận chắc). Phần chênh lệch rõ nhất là tốc độ học, không phải khả năng biểu diễn cuối cùng: mạng rộng nông cũng biểu diễn được hàm này, nhưng thuật toán mất nhiều thời gian hơn để tìm ra.

Hướng dẫn thực hành

Từ các kết quả ở mục 4 có thể rút ra vài nguyên tắc, kèm lưu ý rằng chúng đến từ các bài toán nhỏ:

104. **Bắt đầu nhỏ**: một đến hai lớp ẩn, vài chục nơ-ron mỗi lớp, rồi tăng dần nếu mạng thiếu khớp (mất mát huấn luyện còn cao).
105. **Tăng độ rộng** giúp mạng ít bị kẹt hơn (Hình 16b) và tăng sức biểu diễn, nhưng tăng số tham số theo m^2 cho mỗi lớp ẩn và làm quá khớp dễ hơn (mục 4.4). Số tham số của mạng là $\sum_l (m_{l-1} + 1)\ m_l$.
106. **Tăng độ sâu** hiệu quả khi hàm cần học có cấu trúc phân cấp, và với cùng số tham số thường học nhanh hơn trong thí nghiệm trên, nhưng khó huấn luyện hơn (mục 4.3): cần ReLU, khởi tạo He, thuật toán thích nghi và theo dõi gradient từng lớp.
107. **Luôn đối chiếu** mất mát huấn luyện và mất mát kiểm định: huấn luyện cao thì thiếu khớp (cần mạng lớn hơn hoặc huấn luyện lâu hơn); huấn luyện thấp nhưng kiểm định cao thì quá khớp (cần điều chuẩn hóa hoặc mạng nhỏ hơn). Mục 5 sẽ làm việc này bài bản.

Mối liên hệ với mục trước và mục sau

Mục 3 cho ta thuật toán huấn luyện; mục 4 giải thích khi nào và vì sao nó hoạt động hoặc thất bại. Đối chiếu với Softmax Regression:

Khía cạnh	Softmax Regression	Mạng nơ-ron
Hình dạng J	Lồi: một cực tiểu toàn cục	Không lồi: nhiều cực tiểu, điểm yên ngựa, vùng phẳng
Phụ thuộc khởi tạo	Không	Có (kết quả khác nhau giữa các lần chạy)
Độ sâu	Một lớp	Gradient có thể biến mất hoặc bùng nổ khi sâu
Quá khớp	Điều chuẩn hóa L2	L2, dừng sớm, dropout
Cấu trúc	Chỉ chọn η, số epoch	Còn chọn số lớp, số nơ-ron, hàm kích hoạt, khởi tạo

Các câu hỏi như "mạng đã đủ tốt chưa", "lỗi nằm ở đâu" hay "chọn siêu tham số thế nào" cần các công cụ đánh giá có hệ thống: đường cong học, chia tập, các chỉ số cho bài toán nhiều lớp. Đó là nội dung của mục 5.

Tóm tắt mục 4

108. **Mẫu sai.** Gradient là trung bình của các lực kéo từng mẫu $\tilde{\mathbf{a}}_i^T \mathbf{d}_i$. Mẫu sai có $\| \mathbf{p}_i - \mathbf{y}_i \|$ lớn nên kéo mạnh; mẫu đúng và tự tin gần như không kéo. Lỗi được phân bổ về lớp ẩn theo trọng số nối và bị cổng φ' chặn ở nơ-ron bão hòa. Cross-entropy giữ gradient lớn khi mạng sai nặng, còn bình phương sai số thì không.

109. **Không lồi.** Hoán vị nơ-ron ẩn cho cùng hàm nhưng J ở điểm giữa hai bộ tham số tăng cao, nên J không lồi. Điểm dừng được phân loại qua giá trị riêng của Hessian. Trên XOR, có cực tiểu địa phương gần phẳng ($J \approx 0{,}348$) và cao nguyên là vùng điểm yên ngựa. Kết quả phụ thuộc khởi tạo; mạng rộng hơn ít kẹt hơn.

110. **Đối xứng, bão hòa, biến mất/bùng nổ.** Trạng thái đối xứng là điểm yên ngựa được bảo toàn; nhiễu càng nhỏ thì thoát càng chậm. Tín hiệu lỗi qua L lớp bị nhân với tích các thừa số cỡ $\gamma \| \mathbf{V} \|_2$, nên sigmoid ($\gamma = 0{,}25$) làm gradient co theo cấp số nhân. Cách khắc phục: ReLU, khởi tạo He hoặc Xavier, Adam, cắt gradient; theo dõi chuẩn gradient theo lớp.

111. **Quá khớp và điều chuẩn hóa.** Mạng nhiều tham số khớp cả nhiễu: J huấn luyện giảm, J kiểm định tăng trở lại. L2 thêm $\lambda \mathbf{V}$ vào gradient (suy giảm trọng số), dừng sớm giữ trọng số tốt nhất theo kiểm định, dropout tắt ngẫu nhiên nơ-ron ẩn và nhân $\frac{1}{1-p}$ để giữ kỳ vọng. Mọi siêu tham số chọn trên tập kiểm định. Trong thí nghiệm, L2 cho kết quả tốt nhất, dropout không giúp được với 60 mẫu.

112. **Rộng và sâu.** Một lớp ẩn đủ về lý thuyết, nhưng mạng sâu biểu diễn hiệu quả hơn các hàm có cấu trúc lặp (2^k đoạn với $2k$ nơ-ron so với tối thiểu $2^k - 1$ nơ-ron nếu nông). Biểu diễn được chưa có nghĩa là học được: trên dữ liệu xoắn ốc, mạng một lớp ẩn học chậm hơn nhiều so với mạng sâu cùng số tham số. Nhưng thêm lớp không phải lúc nào cũng tốt hơn.

Câu hỏi kiểm tra nhanh

113. Mẫu thứ nhất có $\mathbf{p} = (0{,}98;\ 0{,}01;\ 0{,}01)$, $\mathbf{y} = (1{,}0{,}0)$. Mẫu thứ hai có $\mathbf{p} = (0{,}7;\ 0{,}2;\ 0{,}1)$, $\mathbf{y} = (0{,}1{,}0)$. Tính $\| \mathbf{p} - \mathbf{y} \|$ của từng mẫu và tỉ số của hai độ dài.

114. Nơ-ron ra sigmoid, nhãn $y = 1$, đầu ra $a = 0{,}01$. Tính $\partial l / \partial s$ với cross-entropy và với hàm mất mát bình phương $l = \frac{1}{2}(a - y)^2$. Hàm nào sửa sai nhanh hơn?

115. Lớp ra có $\mathbf{d}^{(2)} = [0{,}5 \quad -0{,}5]$, $\mathbf{V}^{(2)} = \begin{bmatrix} 1 & 2 \\ -1 & 0{,}5 \end{bmatrix}$, đầu ra lớp ẩn (tanh) $\mathbf{a}^{(1)} = [0{,}9 \quad 0]$. Tính $\mathbf{d}^{(1)}$.

116. Một lớp ẩn tanh có $m = 5$ nơ-ron. Có bao nhiêu bộ trọng số cho cùng một hàm do hoán vị và đổi dấu?

117. Hai bộ tham số có $J(\boldsymbol{\theta}_A) = J(\boldsymbol{\theta}_B) = 0{,}1$, còn điểm giữa có $J = 2{,}0$. Vì sao điều này chứng tỏ J không lồi?

118. Hessian tại một điểm dừng có các giá trị riêng $\{2;\ 0{,}5;\ -0{,}1;\ 0\}$. Đó là loại điểm dừng nào? Gradient Descent đi tiếp như thế nào?

119. Mạng 10 lớp ẩn, mỗi lớp có $\| \mathbf{V}^{(l)} \|_2 = 2$. Tính cận trên của hệ số nhân gradient qua 10 lớp với sigmoid ($\gamma = 0{,}25$) và với tanh ($\gamma = 1$). Cận tanh có chứng tỏ gradient bùng nổ không?

120. Gradient $\mathbf{g}$ có độ dài 50; ngưỡng cắt $c = 5$. Gradient sau khi cắt có độ dài bao nhiêu và hệ số nhân là bao nhiêu?

121. Với SGD có L2: $V = 2{,}0$, $\eta = 0{,}1$, $\lambda = 0{,}5$, $\nabla J = 0{,}3$. Tính V mới. Phần suy giảm trọng số đóng góp bao nhiêu?

122. Dropout với $p = 0{,}5$, đầu ra $a = 0{,}8$. Các giá trị có thể có của $\hat{a}$ và kỳ vọng của nó là bao nhiêu?

123. Mạng ReLU nông một biến có 6 nơ-ron ẩn biểu diễn tối đa bao nhiêu đoạn tuyến tính? Mạng sâu gồm 3 lớp ẩn, mỗi lớp 2 nơ-ron (hàm răng cưa h) có bao nhiêu đoạn?

124. Mạng 2–64–64–3 có bao nhiêu tham số? Nếu chỉ có 60 mẫu huấn luyện thì vì sao dễ quá khớp? Vì sao chọn λ trên tập kiểm định mà không phải tập kiểm tra?

Gợi ý đáp án

125. Mẫu thứ nhất: $\mathbf{p} - \mathbf{y} = (-0{,}02;\ 0{,}01;\ 0{,}01)$, độ dài $\sqrt{0{,}0004 + 0{,}0001 + 0{,}0001} = 0{,}0245$. Mẫu thứ hai: $(0{,}7;\ -0{,}8;\ 0{,}1)$, độ dài $\sqrt{0{,}49 + 0{,}64 + 0{,}01} = 1{,}068$. Tỉ số $\approx 43{,}6$: mẫu sai kéo mạnh gấp khoảng 44 lần.

126. Cross-entropy: $a - y = -0{,}99$. Bình phương: $(a - y)\, a(1 - a) = -0{,}99 \times 0{,}01 \times 0{,}99 = -0{,}0098$. Cross-entropy sửa sai nhanh hơn cỡ 101 lần, vì bình phương bị thừa số $a(1 - a)$ làm nhỏ đi khi nơ-ron bão hòa ở sai chỗ.

127. $\mathbf{d}^{(2)}\mathbf{V}^{(2)T} = [0{,}5 \cdot 1 + (-0{,}5) \cdot 2 \quad 0{,}5 \cdot (-1) + (-0{,}5) \cdot 0{,}5] = [-0{,}5 \quad -0{,}75]$. Đạo hàm tanh: $1 - \mathbf{a} \odot \mathbf{a} = [0{,}19 \quad 1]$. Vậy $\mathbf{d}^{(1)} = [-0{,}095 \quad -0{,}75]$.

128. Hoán vị: $5! = 120$; đổi dấu: $2^5 = 32$; tổng $2^5 \cdot 5! = 3840$ bộ trọng số.

129. Nếu J lồi thì J ở điểm giữa phải $\leq \frac{1}{2}(0{,}1 + 0{,}1) = 0{,}1$. Thực tế là $2{,}0 > 0{,}1$, trái với tính lồi.

130. Có cả giá trị riêng dương và âm: điểm yên ngựa (giá trị riêng 0 cho biết thêm hướng phẳng). Gradient Descent đi ra theo hướng ứng với giá trị riêng $-0{,}1$, nhưng chậm vì độ cong theo hướng đó nhỏ.

131. Sigmoid: $(0{,}25 \times 2)^{10} = 0,5^{10} \approx 9{,}8 \times 10^{-4}$ (gradient có thể nhỏ đi khoảng ba bậc độ lớn). tanh: $(1 \times 2)^{10} = 1024$. Đó chỉ là cận trên, không chứng tỏ bùng nổ; gradient thực tế có thể nhỏ hơn nhiều, tùy hướng của $\mathbf{d}$ so với các hướng riêng của $\mathbf{V}$.

132. Hệ số nhân $\min(1{,}5/50) = 0{,}1$; độ dài sau khi cắt là 5, hướng giữ nguyên.

133. $V \leftarrow (1 - 0{,}1 \times 0{,}5) \times 2{,}0 - 0{,}1 \times 0{,}3 = 1{,}9 - 0{,}03 = 1{,}87$. Phần suy giảm trọng số làm V giảm 0,1 (từ 2,0 xuống 1,9), phần gradient làm giảm thêm 0,03.

134. $\hat{a} \in \{0;\ 1{,}6\}$ với xác suất mỗi giá trị 0,5, kỳ vọng $0{,}5 \times 0 + 0{,}5 \times 1{,}6 = 0{,}8 = a$.

135. Mạng nông: tối đa $6 + 1 = 7$ đoạn. Mạng sâu: $2^3 = 8$ đoạn, với cũng 6 nơ-ron, nhiều hơn mạng nông.

136. Số tham số: $(2 + 1) \times 64 + (64 + 1) \times 64 + (64 + 1) \times 3 = 192 + 4160 + 195 = 4547$. Có 4547 tham số cho 60 mẫu nên mạng có thể khớp từng mẫu, kể cả mẫu nhiễu. Nếu chọn λ trên tập kiểm tra thì độ chính xác báo cáo trên chính tập đó bị lạc quan (đã dùng nó để chọn), không còn là ước lượng khách quan cho dữ liệu mới.

5. Đánh giá và trực quan hóa mô hình

Ở mục 4, ta đã hiểu vì sao mạng nơ-ron có thể học tốt hoặc thất bại: hàm mất mát không lồi nên kết quả phụ thuộc khởi tạo, gradient có thể biến mất, mạng quá lớn sẽ quá khớp. Nhưng làm sao biết **mô hình của mình** đang rơi vào trường hợp nào? Tất cả các kết luận ở mục 4 đều dựa trên những phép đo: J trên tập huấn luyện và tập kiểm định, độ chính xác trên tập kiểm tra, đồ thị vùng quyết định. Mục 5 trình bày các phép đo đó một cách có hệ thống:

137. Nhìn vào quá trình học để biết mô hình đang làm gì: đường cong học, vùng quyết định, biểu diễn của lớp ẩn (mục 5.1).
138. Chia dữ liệu và chọn siêu tham số sao cho kết quả báo cáo là trung thực (mục 5.2).
139. Đo chất lượng bằng các chỉ số phù hợp với bài toán nhiều lớp, nhất là khi các lớp không cân bằng (mục 5.3).
140. Chẩn đoán khi mô hình chưa tốt: thiếu khớp hay quá khớp, lỗi nằm ở đâu (mục 5.4).
141. Đối chiếu với thư viện chuẩn `MLPClassifier` và với Softmax Regression để kiểm tra cài đặt của ta (mục 5.5).

Công thức nền tảng vẫn là cross-entropy J của mục 2.4, nhưng giờ ta dùng nó như một **thước đo**, không chỉ là hàm để cực tiểu hóa.

Ký hiệu bổ sung cho mục 5

Đối tượng	Ký hiệu	Ý nghĩa
Nhãn dự đoán	$\hat{y}_i = \text{argmax}_k p_{ik}$	Lớp có xác suất lớn nhất của mẫu i
Ma trận nhãn dự đoán	$\hat{\mathbf{Y}}$	One-hot của $\hat{y}_i$, kích thước $n \times K$
Ma trận nhầm lẫn	$\mathbf{C}$	$K \times K$; c_{kj} là số mẫu thật thuộc lớp k bị dự đoán là lớp j
Chỉ số theo lớp	Prec_k, Rec_k, F1_k	Precision, recall, F1 của lớp k
Kích thước các tập	$n_{\text{tr}}, n_{\text{kđ}}, n_{\text{kt}}$	Số mẫu huấn luyện, kiểm định, kiểm tra
Sai số chuẩn	SE	Độ lệch chuẩn của một số ước lượng (ví dụ độ chính xác)

5.1. Minh họa đường cong học, vùng quyết định và biểu diễn của lớp ẩn

Dữ liệu cho mục 5: ba vòng tròn nhòe, các lớp không cân bằng

Ở mục 3.7, ba vòng tròn gần như tách được (97% đến 99% đúng), nên rất khó thấy mô hình sai ở đâu. Để việc đánh giá có ý nghĩa, ta dùng một biến thể "khó" hơn:

142. Ba vòng tròn đồng tâm bán kính 1, 2,8, 4,6 với độ nhòe (độ lệch chuẩn của bán kính) 0,7 nên các vòng chồng lấn lên nhau.

143. Các lớp **không cân bằng**: 750, 500 và 250 mẫu (tổng 1500), giống tình huống thực tế thường gặp.

144. Chia theo tỉ lệ 60/20/20 (phân tầng theo lớp, xem mục 5.2): tập huấn luyện 900 mẫu (450/300/150), tập kiểm định 300 mẫu (150/100/50), tập kiểm tra 300 mẫu (150/100/50). Chuẩn hóa bằng trung bình và độ lệch chuẩn của riêng tập huấn luyện.

Hai mốc để so sánh. **Mốc tầm thường:** luôn đoán lớp 0 (lớp đông nhất) đã đúng 50%. **Mốc tham chiếu:** một mạng 2–16–16–3 huấn luyện trên 15 000 mẫu cùng phân phối đạt 86,9% và $J = 0{,}322$ trên 15 000 mẫu mới; vì vùng lấn giữa các vòng là nhiễu thật, lỗi khoảng 13,1% gần như không thể giảm thêm. Mục tiêu của ta là tiến gần mốc này với 900 mẫu.

Đường cong học theo epoch

Ta huấn luyện mạng 2–8–3 (tanh, Adam, $\eta = 0{,}01$, mini-batch 32, 300 epoch) và ghi lại J cùng độ chính xác trên tập huấn luyện và kiểm định sau **mỗi epoch**. Đây là hàm *huan_luyen* của mục 3.7 với phần ghi nhật ký:

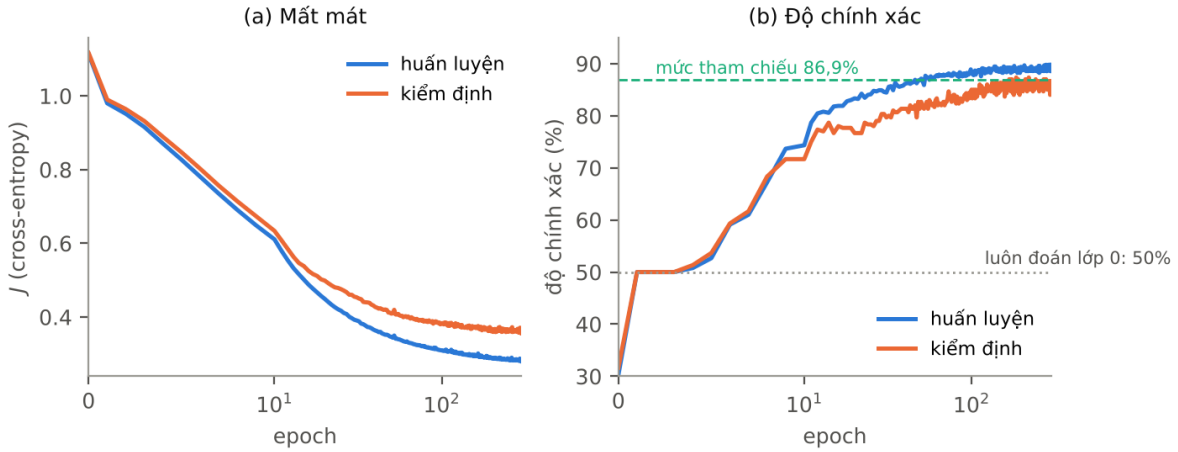
```
def huan_luyen_co_ghi(X, Y, Xv, Yv, kich_thuoc, kich_hoat="tanh", eta=0.01, so_epoch=300, b=32, seed=0):
    """Như huan_luyen (mục 3.7) nhưng ghi J và độ chính xác trên tập huấn luyện và kiểm định sau mỗi epoch."""
    rng = np.random.default_rng(seed)
    Ws = khoi_tao(kich_thuoc, kich_hoat, seed)
    tt = tao_trang_thai(Ws)
    n, nhat_ky = len(X), {"J_tr": [], "J_kd": [], "acc_tr": [], "acc_kd": []}
    def ghi():
        for ten, (A, B) in (("tr", (X, Y)), ("kd", (Xv, Yv))):
            P = lan_truyen_thuan(A, Ws, kich_hoat)[0][-1]
            nhat_ky["J_" + ten].append(mat_mat(P, B))
            nhat_ky["acc_" + ten].append(np.mean(P.argmax(1) == B.argmax(1)))
    ghi()
    for epoch in range(so_epoch):
        idx = rng.permutation(n)
        for k in range(0, n, b):
            lo = idx[k:k + b]
            As, Zs = lan_truyen_thuan(X[lo], Ws, kich_hoat)
            cap_nhat(Ws, lan_truyen_nguoc(As, Zs, Y[lo], Ws, kich_hoat), tt, "adam", eta)
        ghi()
    return Ws, nhat_ky
```

Hình 22 là kết quả. Cách đọc:

145. J **huấn luyện và** J **kiểm định cùng giảm** và khoảng cách giữa chúng nhỏ: cuối epoch 300, $J_{\text{tr}} = 0{,}281$ và $J_{\text{kđ}} = 0{,}358$. $J_{\text{kđ}}$ chưa tăng trở lại nên chưa có dấu hiệu quá khớp.

146. **Độ chính xác đứng yên ở** 50% **trong vài epoch đầu**, đúng bằng mốc "luôn đoán lớp 0": mạng bắt đầu bằng cách học tỉ lệ lớp (hệ số chệch của lớp ra) rồi mới học hình dạng. Đây là hiện tượng cùng bản chất với cao nguyên ở Hình 14: gần khởi tạo, gradient chỉ cho phép học được phần đơn giản nhất của bài toán.

147. **Độ chính xác cuối cùng** đạt 89,8% (huấn luyện) và 86,3% (kiểm định), tiến gần mốc tham chiếu 86,9%.



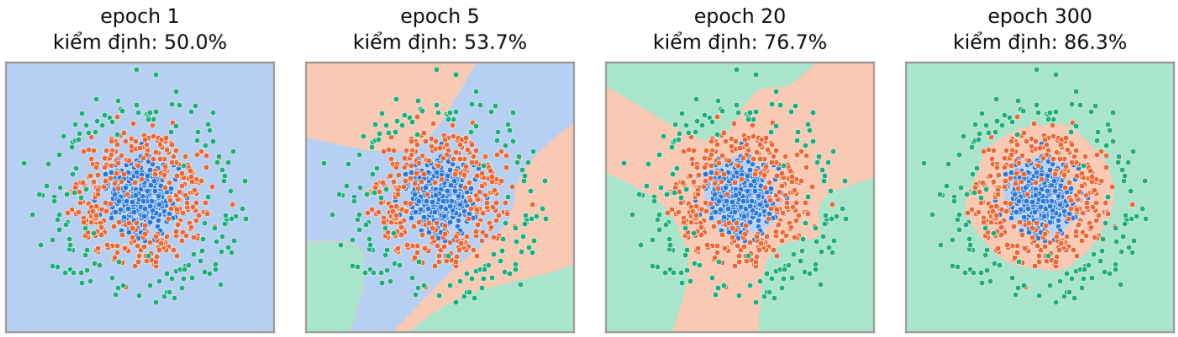
Hình 22. Đường cong học theo epoch của mạng 2–8–3 trên dữ liệu ba vòng tròn nhòe (trục hoành chia theo thang symlog). (a) J huấn luyện và kiểm định. (b) Độ chính xác; vài epoch đầu mạng chỉ đoán lớp đông nhất (50%).

Bốn dạng đường cong thường gặp và ý nghĩa của chúng:

Dạng đường cong	Chẩn đoán	Việc cần làm
Cả hai giảm và hội tụ gần nhau	Học tốt	Giữ nguyên; có thể huấn luyện thêm nếu còn giảm
J_{tr} tiếp tục giảm, $J_{\text{kđ}}$ tăng trở lại	Quá khớp (mục 4.4)	Điều chuẩn hóa, dừng sớm, mạng nhỏ hơn hoặc thêm dữ liệu
Cả hai cao và gần như phẳng	Thiếu khớp, hoặc η quá nhỏ, hoặc kẹt ở cao nguyên (mục 4.2, 4.3)	Mạng lớn hơn, tăng η, kiểm tra gradient từng lớp
J dao động mạnh hoặc thành *nan*	η quá lớn, gradient bùng nổ (mục 4.3)	Giảm η, cắt gradient, kiểm tra khởi tạo

Vùng quyết định theo epoch

Với dữ liệu hai chiều ta nhìn thẳng được cách mạng chia mặt phẳng. Hình 23 vẽ vùng quyết định (mỗi màu là vùng một lớp thắng) tại bốn thời điểm của cùng lần huấn luyện.



Hình 23. Vùng quyết định của mạng 2–8–3 tại epoch 1, 5, 20 và 300 (các chấm là mẫu huấn luyện, màu theo lớp thật). Độ chính xác kiểm định tăng từ 50,0% lên 86,3%.

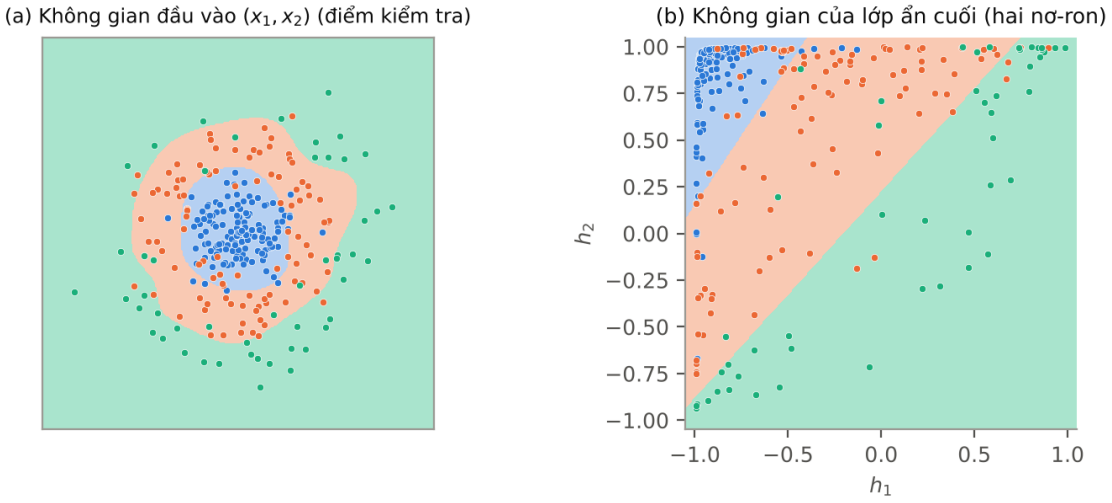
Ở epoch 1, toàn mặt phẳng thuộc lớp 0 (50,0%, đúng mốc tầm thường). Ở epoch 5 đã xuất hiện các dải thẳng, mạng chưa biết cấu trúc vòng (53,7%). Ở epoch 20 biên cong dần quanh tâm (76,7%), và ở epoch 300 ta thấy ba vành gần tròn đồng tâm (86,3%). Quá trình này là hình ảnh trực quan của việc các nơ-ron ẩn (mỗi nơ-ron là một "đường thẳng" ở lớp đầu) được kết hợp thành biên cong, như ta đã giải thích ở mục 2.5.

Biểu diễn của lớp ẩn

Mục 1.2 đặt ra ý tưởng: thay vì ta đoán đặc trưng đa thức, mạng **tự học đặc trưng** rồi đưa cho một mô hình Softmax. Giờ ta nhìn thấy điều đó. Ta xây một mạng "thắt cổ chai" 2–8–2–3 với lớp ẩn cuối chỉ có hai nơ-ron tanh, để đầu ra $\mathbf{h} = (h_1, h_2)$ của lớp đó vẽ được trên mặt phẳng. Điểm số của lớp ra là

$$\mathbf{s} = \tilde{\mathbf{h}}\,\mathbf{W}^{(L)}$$

tức là **tuyến tính theo h**. Vì vậy, trong không gian (h_1, h_2), vùng thắng của mỗi lớp là một đa giác lồi bị chặn bởi các đường thẳng: đó là Softmax Regression chạy trên các đặc trưng **h** do mạng học được. Hình 24 vẽ điểm kiểm tra trong cả hai không gian.



Hình 24. Mạng 2–8–2–3 (độ chính xác kiểm tra 85,3%). (a) Không gian đầu vào với vùng quyết định cong. (b) Cùng các điểm kiểm tra ở không gian của lớp ẩn cuối (h_1, h_2): ở đây biên giữa các lớp đã là **đường thẳng**.

Trong Hình 24a, ba lớp không tách được bằng đường thẳng. Sau khi đi qua các lớp ẩn, ở Hình 24b lớp 0 (xanh dương) dồn về góc trên bên trái, lớp 2 (xanh lá) về phía dưới bên phải, lớp 1 (cam) nằm giữa, và ba vùng được ngăn cách bằng hai đường thẳng. Các điểm còn lẫn là các mẫu nằm ở vùng lấn thật của dữ liệu, không phải lỗi của mô hình. Dù chỉ có hai nơ-ron ở lớp cuối, mạng này đạt 85,3% trên tập kiểm tra, cùng cỡ mạng 2–8–3 (84,7%).

Với dữ liệu nhiều chiều không vẽ trực tiếp được, ta thường chiếu biểu diễn của lớp ẩn xuống hai chiều (bằng PCA hoặc t-SNE) rồi tô màu theo lớp; nếu các lớp tách rõ trong biểu diễn đó thì các lớp đầu đã học được đặc trưng tốt. Phần này sẽ được nhắc lại ở mục 6.3.

5.2. Chia tập huấn luyện, kiểm định, kiểm tra và chọn siêu tham số

Ba tập dữ liệu, ba vai trò

Đánh giá trên chính dữ liệu đã dùng để huấn luyện luôn lạc quan: mạng càng lớn càng "thuộc" dữ liệu (mục 4.4). Vì vậy dữ liệu được chia thành ba phần với vai trò khác nhau:

Tập	Dùng để	Được nhìn bao nhiêu lần
Huấn luyện	Tính gradient và cập nhật trọng số $\mathbf{W}^{(l)}$	Mọi epoch
Kiểm định	Chọn siêu tham số (số nơ-ron, λ, η, số epoch dừng sớm,…)	Nhiều lần, mỗi cấu hình thử
Kiểm tra	Ước lượng chất lượng cuối cùng trên dữ liệu mới	**Một lần**, ở cuối cùng

Nguyên tắc cốt lõi: **mọi quyết định thiết kế đều dựa trên tập kiểm định; tập kiểm tra chỉ để báo cáo**. Nếu ta chọn mô hình theo tập kiểm tra, con số báo cáo trên chính tập đó không còn là ước lượng khách quan (nó đã "nhìn" thấy đáp án thông qua việc chọn).

Cách chia. Tỉ lệ 60/20/20 hoặc 70/15/15 là thông dụng; dữ liệu càng ít thì tập kiểm định và kiểm tra càng khó đủ lớn, khi đó nên dùng kiểm định chéo (xem bên dưới). Với bài toán phân

loại ta **chia phân tầng**: mỗi tập giữ nguyên tỉ lệ các lớp (ở đây 3: 2: 1), nếu không một tập có thể gần như thiếu hẳn lớp nhỏ.

```
def chia_3_tap(X, y, ti_le=(0.6, 0.2, 0.2), seed=0):
    """Chia phân tầng thành huấn luyện / kiểm định / kiểm tra; trả về ba mảng
chỉ số."""
    rng = np.random.default_rng(seed)
    tr, kd, kt = [], [], []
    for k in np.unique(y):
        idx = rng.permutation(np.where(y == k)[0])
        a, b = int(ti_le[0] * len(idx)), int((ti_le[0] + ti_le[1]) * len(idx)
)
        tr += list(idx[:a]); kd += list(idx[a:b]); kt += list(idx[b:])
    return [rng.permutation(i) for i in (tr, kd, kt)]

itr, ikd, ikt = chia_3_tap(X, y)
mu, sd = X[itr].mean(axis=0), X[itr].std(axis=0)                    # chỉ tín
h trên tập huấn luyện
Xtr, Xkd, Xkt = [(X[i] - mu) / sd for i in (itr, ikd, ikt)]        # rồi áp d
ụng cho cả ba tập
```

Lưu ý ở hai dòng cuối: trung bình $\boldsymbol{\mu}$ và độ lệch chuẩn $\boldsymbol{\sigma}$ chỉ được tính từ **tập huấn luyện**, rồi dùng chính hai giá trị đó chuẩn hóa tập kiểm định và kiểm tra. Nếu tính trên toàn bộ dữ liệu, thông tin về tập kiểm tra đã "rò" vào quá trình huấn luyện (**rò rỉ dữ liệu**, data leakage). Mọi bước tiền xử lý có "học" từ dữ liệu (chuẩn hóa, giảm chiều, chọn đặc trưng) đều phải theo quy tắc này.

Siêu tham số là gì và chọn thế nào

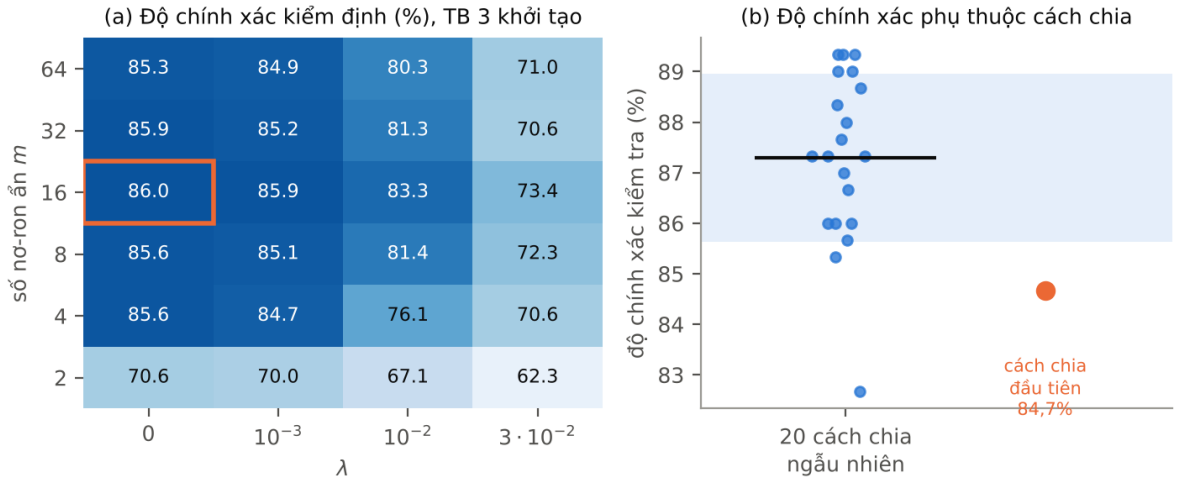
Tham số (các $\mathbf{W}^{(l)}$) được học bằng gradient. **Siêu tham số** là các lựa chọn ta đặt trước khi huấn luyện, gồm: số lớp ẩn và số nơ-ron mỗi lớp, hàm kích hoạt, tốc độ học η, kích thước mini-batch, số epoch (hoặc kiên nhẫn của dừng sớm), λ của L2, xác suất dropout p, cách khởi tạo. Không có công thức cho các giá trị này; ta thử và đo trên tập kiểm định.

Tìm kiếm lưới (grid search): chọn vài giá trị cho mỗi siêu tham số rồi thử mọi tổ hợp. Ta thử số nơ-ron ẩn $m \in \{2,4,8,16,32,64\}$ và $\lambda \in \{0;\ 10^{-3};\ 10^{-2};\ 3\times10^{-2}\}$ (24 cấu hình), mỗi cấu hình huấn luyện với 3 khởi tạo ngẫu nhiên (200 epoch, Adam $\eta = 0{,}01$, mini-batch 32):

```
ket_qua = {}
for m in (2, 4, 8, 16, 32, 64):
    for lam in (0.0, 1e-3, 1e-2, 3e-2):
        acc_kd = []
        for seed in range(3):
            Ws, _, _, _ = huan_luyen_dc(Xtr, Ytr, [2, m, 3], so_epoch=200, b=
32, seed=seed, lam=lam)
            acc_kd.append(np.mean(du_doan(Xkd, Ws) == ykd))
        ket_qua[(m, lam)] = np.mean(acc_kd)
m_tot, lam_tot = max(ket_qua, key=ket_qua.get)
```

(*huan_luyen_dc* là hàm huấn luyện có L2, dropout và dừng sớm ở mục 4.4.) Hình 25a là độ chính xác kiểm định:

m / λ	0	10^{-3}	10^{-2}	3×10^{-2}
2	70,6	70,0	67,1	62,3
4	85,6	84,7	76,1	70,6
8	85,6	85,1	81,4	72,3
16	**86,0**	85,9	83,3	73,4
32	85,9	85,2	81,3	70,6
64	85,3	84,9	80,3	71,0



Hình 25. (a) Độ chính xác kiểm định (%) theo số nơ-ron ẩn m và hệ số λ; ô viền cam là tốt nhất. (b) Độ chính xác kiểm tra của cùng một cấu hình ($m = 16$, $\lambda = 10^{-3}$) qua 20 cách chia ngẫu nhiên khác nhau: đường đen là trung bình, vùng mờ là ± 1 độ lệch chuẩn; chấm cam là cách chia dùng ở mục này.

Từ bảng có thể đọc ra ba điều:

148. **Thiếu khớp rõ ở $m = 2$** ($\approx 70\%$): hai nơ-ron ẩn không đủ để tạo biên cong cho ba vành. Từ $m = 4$ trở lên đạt $\approx 85\%$, và tăng thêm nơ-ron không cải thiện thêm.

149. **λ lớn làm thiếu khớp** (cột 3×10^{-2} chỉ còn 62% đến 73%), giống kết quả ở mục 4.4. Với dữ liệu lớn hơn (900 mẫu thay vì 60), $\lambda = 0$ đã tốt nên điều chuẩn hóa không cần thiết ở đây.

150. **Nhiều cấu hình ngang nhau.** Từ $m = 4$ đến 64 với $\lambda \leq 10^{-3}$, độ chính xác kiểm định đều nằm trong 84,7% đến 86,0%.

Chênh lệch bao nhiêu thì có nghĩa?

Độ chính xác đo trên n mẫu là một ước lượng có nhiễu. Nếu độ chính xác thật là p, sai số chuẩn của ước lượng là

$$\mathrm{SE} = \sqrt{\frac{p(1-p)}{n}}$$

và khoảng tin cậy xấp xỉ 95% là $\hat{p} \pm 1{,}96\,\mathrm{SE}$. Với tập kiểm định $n_{\text{kđ}} = 300$ và $p \approx 0{,}86$ ta có $\mathrm{SE} = 2{,}0$ điểm phần trăm. Vì thế các chênh lệch 0,4 hay 1,0 điểm trong bảng trên **nằm trong nhiễu**; chỉ chênh lệch lớn như $m = 2$ so với $m \geq 4$ mới đáng tin. Cùng lý do, độ chính xác trên tập kiểm tra 300 mẫu là 84,7% có khoảng tin cậy

$$0{,}847 \pm 1{,}96\sqrt{\frac{0{,}847 \times 0{,}153}{300}} = 0{,}847 \pm 0{,}041$$

tức khoảng 80,6% đến 88,8%: rộng hơn nhiều so với chênh lệch giữa các phương pháp mà ta hay so sánh.

Khi nhiều cấu hình ngang nhau, một quy tắc thực dụng là **quy tắc một sai số chuẩn**: chọn mô hình **đơn giản nhất** có độ chính xác kiểm định không kém cấu hình tốt nhất quá một SE. Ở đây tốt nhất là 86,0% với $\mathrm{SE} = 2{,}0$, nên mọi cấu hình $\geq 84{,}0\%$ được coi là ngang nhau, và mô hình đơn giản nhất trong số đó là $m = 4$, $\lambda = 0$ (85,6%). Mạng nhỏ hơn thì huấn luyện nhanh hơn, ít quá khớp hơn và dễ diễn giải hơn.

Kết quả phụ thuộc cách chia

Một lần chia ngẫu nhiên chỉ cho một ước lượng. Ta lặp lại việc chia 60/20/20 với 20 hạt giống khác nhau và huấn luyện cùng một cấu hình ($m = 16$, $\lambda = 10^{-3}$) (Hình 25b). Độ chính xác kiểm tra (huấn luyện trên tập huấn luyện của từng cách chia) là 87,3% trung bình, độ lệch chuẩn 1,7 điểm, nhỏ nhất 82,7% và lớn nhất 89,3%. Cách chia dùng ở mục này (84,7%) nằm ở phía thấp của khoảng đó. Điều này không có nghĩa là mô hình "tệ đi"; nó chỉ nhắc rằng **một con số duy nhất trên một tập kiểm tra nhỏ có sai số cỡ vài điểm phần trăm**.

Kiểm định chéo (cross-validation)

Khi dữ liệu ít, dành 20% cho kiểm định rất tốn. **Kiểm định chéo K lần** dùng mọi mẫu cho cả huấn luyện lẫn kiểm định: chia tập (huấn luyện + kiểm định) thành K phần phân tầng, lần lượt dùng phần thứ f để kiểm định và $K - 1$ phần còn lại để huấn luyện, rồi lấy trung bình:

$$\mathrm{CV}_K = \frac{1}{K}\sum_{f=1}^{K} \mathrm{acc}_f$$

Tập kiểm tra vẫn tách riêng và chỉ dùng một lần ở cuối. Với $K = 5$ trên 1200 mẫu (huấn luyện + kiểm định), mỗi lần huấn luyện dùng 960 mẫu và kiểm định trên 240 mẫu:

Cấu hình	CV_5 (trung bình $\pm$ độ lệch chuẩn qua 5 phần)
$m = 2, \lambda = 0$	$69{,}9\% \pm 1{,}9\%$
$m = 4, \lambda = 0$	$86{,}4\% \pm 2{,}6\%$
$m = 16, \lambda = 10^{-3}$	$87{,}2\% \pm 2{,}1\%$

Cấu hình	CV_5 (trung bình $\pm$ độ lệch chuẩn qua 5 phần)
$m = 16, \lambda = 3 \times 10^{-2}$	$73{,}2\% \pm 0{,}4\%$

Cùng kết luận như kiểm định đơn: $m = 2$ và λ quá lớn kém rõ rệt, còn $m = 4$ và $m = 16$ ngang nhau trong khoảng nhiễu. Cái giá là phải huấn luyện K lần; vì vậy với mạng lớn người ta thường dùng một tập kiểm định đơn.

Quy trình hoàn chỉnh

151. Chia phân tầng; tính thống kê tiền xử lý **chỉ trên tập huấn luyện**.
152. Thử các cấu hình, chọn theo tập kiểm định (hoặc kiểm định chéo), ưu tiên mô hình đơn giản khi chênh lệch nằm trong nhiễu.
153. **Huấn luyện lại** cấu hình đã chọn trên huấn luyện + kiểm định (nhiều dữ liệu hơn thì tốt hơn), giữ nguyên siêu tham số.
154. Đánh giá **một lần** trên tập kiểm tra và báo cáo kèm khoảng tin cậy.

Nếu chọn theo log-loss kiểm định thay vì accuracy thì cấu hình tốt nhất là $m = 16$, $\lambda = 10^{-3}$ ($J_{\text{kđ}} = 0{,}354$, accuracy kiểm định 85,9%); ta dùng cấu hình này. Huấn luyện lại trên 1200 mẫu, kết quả trên tập kiểm tra là độ chính xác 84,7% và $J = 0{,}338$. Ta phân tích kỹ kết quả này ở mục 5.3.

Để thấy chọn theo tập kiểm tra gây lạc quan thế nào: nếu ta chọn cấu hình theo accuracy **kiểm tra** thay vì kiểm định thì sẽ chọn $m = 4$, $\lambda = 0$ với 85,7% (trung bình 3 khởi tạo), cao hơn 1,0 điểm so với 84,7% của cấu hình chọn theo kiểm định ($m = 16$, $\lambda = 0$). Phần chênh đó là sự lạc quan do tập kiểm tra đã được dùng để chọn. Với tập chỉ 300 mẫu và 24 cấu hình thì phần lạc quan còn nhỏ; nó tăng khi thử nhiều cấu hình hơn.

5.3. Các chỉ số đánh giá cho bài toán nhiều lớp: ma trận nhầm lẫn, precision, recall, F1, log-loss

Vì sao độ chính xác chưa đủ

Độ chính xác (accuracy) là tỉ lệ dự đoán đúng, nhưng nó che giấu hai điều. Thứ nhất, với lớp không cân bằng, nó bị lớp đông chi phối: mô hình luôn đoán lớp 0 của ta đạt 50% mà chưa học được gì. Thứ hai, nó không cho biết mô hình **sai theo kiểu nào**, mà đó lại là thông tin cần để sửa. Các chỉ số dưới đây bổ sung hai điều đó.

Ma trận nhầm lẫn

Gọi $\mathbf{Y}$ là ma trận one-hot của nhãn thật ($n \times K$) và $\widehat{\mathbf{Y}}$ là ma trận one-hot của nhãn dự đoán $\hat{y}_i = \operatorname{argmax}_k p_{ik}$. **Ma trận nhầm lẫn** là

$$\mathbf{C} = \mathbf{Y}^T \widehat{\mathbf{Y}}, \qquad c_{kj} = \#\{i : y_i = k,\ \hat{y}_i = j\}$$

Hàng k là các mẫu thật thuộc lớp k; cột j là các mẫu được dự đoán là lớp j. Đường chéo là các dự đoán đúng. Tổng mọi phần tử bằng n. Từ $\mathbf{C}$ ta tính mọi chỉ số:

$$\text{Accuracy} = \frac{\operatorname{tr}(\mathbf{C})}{n}, \qquad \text{Prec}_k = \frac{c_{kk}}{\sum_i c_{ik}}, \qquad \text{Rec}_k = \frac{c_{kk}}{\sum_j c_{kj}}, \qquad \text{F1}_k = \frac{2\,\text{Prec}_k\,\text{Rec}_k}{\text{Prec}_k + \text{Rec}_k}$$

155. **Precision của lớp** k (độ chính xác của dự đoán): trong số mẫu mô hình **gán** vào lớp k (tổng cột k), bao nhiêu phần đúng thật sự thuộc lớp k. Precision thấp nghĩa là mô hình "báo động nhầm" lớp này.

156. **Recall của lớp** k (độ phủ): trong số mẫu **thật sự** thuộc lớp k (tổng hàng k), mô hình tìm lại được bao nhiêu phần. Recall thấp nghĩa là mô hình "bỏ sót" lớp này.

157. **F1** là trung bình điều hòa của hai số trên. Trung bình điều hòa nhỏ hơn trung bình cộng và bị kéo về số nhỏ hơn, nên F1 chỉ cao khi **cả** precision và recall cùng cao.

Muốn có **một** con số cho cả mô hình, ta lấy trung bình F1 của các lớp theo hai cách:

$$\text{F1 macro} = \frac{1}{K}\sum_{k=1}^{K} \text{F1}_k\,, \qquad \text{F1 có trọng số} = \sum_{k=1}^{K} \frac{n_k}{n}\,\text{F1}_k \quad \left(n_k = \sum_j c_{kj}\right)$$

F1 macro coi mọi lớp quan trọng như nhau, nên phạt nặng mô hình bỏ qua lớp nhỏ; F1 có trọng số theo kích thước lớp thì gần với accuracy hơn. (Trung bình "micro" gộp mọi dự đoán rồi tính một lần; với bài toán một nhãn cho mỗi mẫu nó đúng bằng accuracy.)

Ví dụ nhỏ tính tay. Với 10 mẫu, ba lớp:

$$\mathbf{C} = \begin{bmatrix} 3 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 0 & 2 \end{bmatrix}$$

Accuracy $= (3+2+2)/10 = 0{,}7$. Tổng cột là $(4,3,3)$ và tổng hàng là $(4,4,2)$, nên Prec $= (3/4,\ 2/3,\ 2/3)$ và Rec $= (3/4,\ 2/4,\ 2/2)$. Lớp 1 có $\text{F1}_1 = 2 \cdot \frac{2}{3} \cdot \frac{1}{2} / \left(\frac{2}{3} + \frac{1}{2}\right) = 0{,}571$, và F1 $= (0{,}75;\ 0{,}571;\ 0{,}8)$. F1 macro $= 0{,}707$; F1 có trọng số $= (4 \cdot 0{,}75 + 4 \cdot 0{,}571 + 2 \cdot 0{,}8)/10 = 0{,}689$. Lớp 2 có recall bằng 1 nhưng precision chỉ $2/3$: mô hình tìm đủ lớp 2 nhưng đã gán nhầm một mẫu lớp 1 vào đó.

Cài đặt gọn bằng NumPy (hàm này cho đúng các số trên):

```
def ma_tran_nham_lan(y_that, y_du_doan, K):
    """C = Y^T Y_hat: C[k, j] = số mẫu thật thuộc lớp k bị dự đoán là lớp j."
""
    return np.eye(K)[y_that].T @ np.eye(K)[y_du_doan]

def chi_so_theo_lop(C):
    precision = np.diag(C) / C.sum(axis=0)     # chia cho số mẫu được dự đoán
là lớp k (tổng cột)
    recall = np.diag(C) / C.sum(axis=1)        # chia cho số mẫu thật thuộc l
```

```
ớp k (tổng hàng)
    f1 = 2 * precision * recall / (precision + recall)
    return precision, recall, f1
```

(Nếu có lớp không bao giờ được dự đoán thì tổng cột bằng 0 và precision không xác định; thư viện thường quy ước bằng 0.)

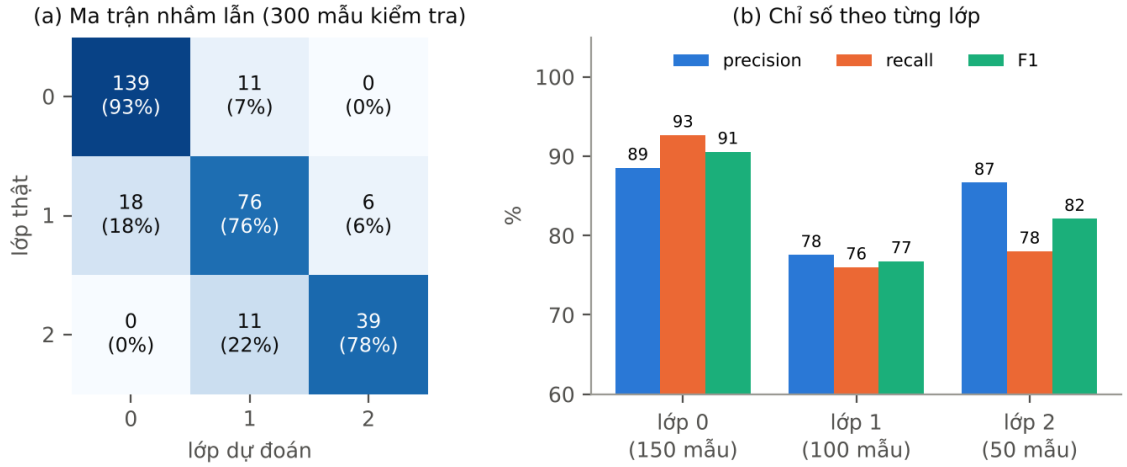
Kết quả trên tập kiểm tra

Với mô hình cuối cùng của mục 5.2 ($m = 16$, $\lambda = 10^{-3}$, huấn luyện trên 1200 mẫu), ma trận nhầm lẫn trên 300 mẫu kiểm tra (hàng là lớp thật, cột là lớp dự đoán):

$$\mathbf{C} = \begin{bmatrix} 139 & 11 & 0 \\ 18 & 76 & 6 \\ 0 & 11 & 39 \end{bmatrix}$$

Lớp	Số mẫu	Precision	Recall	F1
0	150	0,885	0,927	0,906
1	100	0,776	0,760	0,768
2	50	0,867	0,780	0,821

Accuracy $= (139 + 76 + 39)/300 = 0{,}847$; F1 macro $= 0{,}831$; F1 có trọng số $= 0{,}846$; J (log-loss, xem bên dưới) $= 0{,}338$. Hình 26 vẽ lại ma trận (chuẩn hóa theo hàng, tức là recall) và các chỉ số theo lớp.



Hình 26. (a) Ma trận nhầm lẫn của mô hình cuối cùng trên 300 mẫu kiểm tra; trong mỗi ô là số mẫu và tỉ lệ theo hàng. (b) Precision, recall và F1 theo từng lớp.

Ma trận cho thấy nhiều điều mà con số 84,7% không cho thấy:

158. **Mô hình chỉ nhầm giữa các vòng kề nhau**: lớp 0 với lớp 1 ($11 + 18$ mẫu) và lớp 1 với lớp 2 ($6 + 11$ mẫu). Số mẫu nhầm giữa lớp 0 và lớp 2 bằng 0 ở cả hai chiều.

Điều này đúng hình học của dữ liệu: hai vòng ngoài cùng và trong cùng cách nhau 3,6 đơn vị còn vòng giữa lấn lên cả hai.

159. **Lớp 1 (vòng giữa) khó nhất**, F1 $= 0{,}768$, vì nó bị kẹp giữa hai lớp kia.

160. **Lớp nhỏ nhất (lớp 2) có recall** 0,78: cứ 50 mẫu thật thì 11 mẫu bị gán sang lớp 1. Đây là hiện tượng thường gặp ở lớp ít mẫu.

161. **Mốc tầm thường**: mô hình luôn đoán lớp 0 có **C** chỉ có cột đầu khác 0, accuracy 0,5, $\text{F1}_0 = 2 \cdot 0{,}5 \cdot 1/1{,}5 = 0{,}667$ và $\text{F1}_1 = \text{F1}_2 = 0$, nên F1 macro $= 0{,}222$. Đúng bằng giá trị của Softmax Regression trên đặc trưng gốc ở mục 5.5: accuracy 50% nhưng F1 macro chỉ 0,222 lộ rõ rằng mô hình không học được gì.

Log-loss: đánh giá cả xác suất

Accuracy và F1 chỉ dùng nhãn $\hat{y}_i$, bỏ qua mô hình **tự tin đến mức nào**. **Log-loss** chính là cross-entropy mà ta đã dùng để huấn luyện, tính trên tập đánh giá:

$$\text{Log-loss} = -\frac{1}{n}\sum_{i=1}^{n} \ln p_{i,y_i}$$

Mô hình đoán sai nhưng tự tin bị phạt rất nặng: $p_{i,y_i} = 0{,}9$ chỉ tốn $-\ln 0{,}9 = 0{,}105$, còn $p_{i,y_i} = 0{,}01$ tốn 4,605 (gấp khoảng 44 lần). Mốc tầm thường là mô hình chưa biết gì (xác suất đều $1/K$): $\ln K = \ln 3 = 1{,}099$; một mô hình chỉ biết tỉ lệ lớp (0,5; 0,33; 0,17) có log-loss bằng entropy của tỉ lệ đó, 1,011.

Để thấy log-loss quan trọng thế nào, ta so hai mạng trên cùng tập huấn luyện 900 mẫu: mạng 2–64–64–3 huấn luyện 1500 epoch không điều chuẩn hóa và cùng mạng với L2 ($\lambda = 0{,}01$):

Mạng	Accuracy huấn luyện	Accuracy kiểm tra	Log-loss kiểm tra
2–64–64–3, không điều chuẩn hóa	98,6%	82,7%	1,007
2–64–64–3, L2 $\lambda = 0{,}01$	88,8%	85,3%	0,347

Chênh lệch accuracy chỉ 2,6 điểm, nhưng log-loss gấp khoảng 3 lần: mạng quá khớp đoán sai với xác suất rất cao (đúng cái giá ở mục 4.4). Log-loss nhạy với quá khớp hơn accuracy, và là chỉ số cần theo dõi khi ta dùng xác suất để ra quyết định (ví dụ chỉ chấp nhận dự đoán nếu $\max_k p_{ik} > 0{,}9$).

Chọn chỉ số nào?

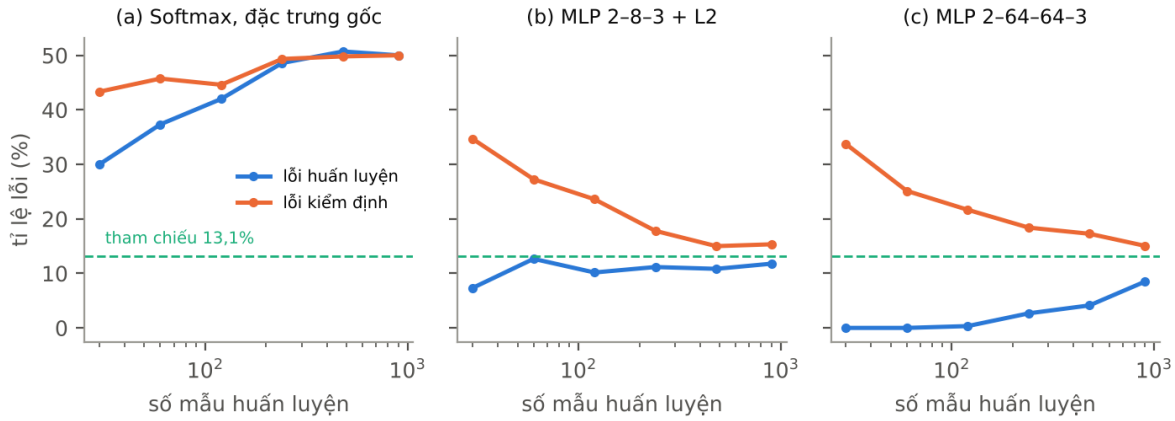
Tình huống	Chỉ số nên xem
Các lớp cân bằng, mọi lỗi tốn như nhau	Accuracy
Các lớp mất cân bằng	F1 macro, recall từng lớp, ma trận nhầm lẫn
Bỏ sót một lớp rất tốn kém (ví dụ phát hiện bệnh)	Recall của lớp đó

Tình huống	Chỉ số nên xem
Báo động nhầm rất tốn kém (ví dụ chặn nhầm giao dịch hợp lệ)	Precision của lớp đó
Dùng xác suất đầu ra	Log-loss

5.4. Chẩn đoán lỗi: đường cong học, thiên lệch và phương sai, xem các mẫu bị sai

Đường cong học theo số mẫu

Đường cong ở mục 5.1 vẽ theo epoch. Một công cụ khác trả lời câu hỏi **"có nên thu thập thêm dữ liệu hay đổi mô hình?"**: huấn luyện trên n_s mẫu đầu tiên với n_s tăng dần (30, 60, 120, 240, 480, 900) rồi vẽ lỗi huấn luyện và lỗi kiểm định (trên cùng 300 mẫu kiểm định) theo n_s; mỗi điểm lấy trung bình 5 lần chạy (mạng nhỏ 200 epoch, mạng lớn 600 epoch). Ta so ba mô hình (Hình 27): Softmax Regression trên đặc trưng gốc, mạng 2–8–3 có L2 nhẹ ($\lambda = 10^{-3}$), và mạng lớn 2–64–64–3 không điều chuẩn hóa.



Hình 27. Lỗi huấn luyện (xanh) và lỗi kiểm định (cam) theo số mẫu huấn luyện; đường xanh lá nét đứt là mức tham chiếu 13,1%. (a) Softmax Regression trên đặc trưng gốc. (b) Mạng 2–8–3 với L2. (c) Mạng 2–64–64–3 không điều chuẩn hóa.

Số liệu tại ba điểm (lỗi, tính bằng %):

Mô hình	$n_s = 30$ (huấn luyện / kiểm định)	$n_s = 240$	$n_s = 900$
Softmax, đặc trưng gốc	30,0 / 43,3	48,6 / 49,3	50,0 / 50,0
MLP 2–8–3 + L2	7,3 / 34,7	11,2 / 17,8	11,8 / 15,3
MLP 2–64–64–3	0,0 / 33,7	2,7 / 18,4	8,5 / 15,1

Thiên lệch và phương sai

Ta gọi **lỗi tham chiếu** là mức lỗi mà một mô hình tốt có thể đạt (ở đây 13,1%; trong thực tế có thể là sai số của chuyên gia người hoặc của hệ thống tốt nhất đã biết). Hai nguồn lỗi, theo thuật ngữ thống kê:

$$\underbrace{\text{lỗi}_{\text{tr}} - \text{lỗi}_{\text{tham chiếu}}}_{\text{thiên lệch (bias)}}, \qquad \underbrace{\text{lỗi}_{\text{kđ}} - \text{lỗi}_{\text{tr}}}_{\text{phương sai (variance)}}$$

Thiên lệch lớn: ngay cả trên dữ liệu huấn luyện mô hình cũng sai nhiều (mô hình quá đơn giản: thiếu khớp). Phương sai lớn: mô hình tốt trên dữ liệu đã thấy nhưng kém trên dữ liệu mới (quá nhạy với mẫu huấn luyện cụ thể: quá khớp). Đây là các ước lượng thực dụng của hai khái niệm đó, không phải định nghĩa hình thức. (Cần phân biệt "thiên lệch" ở đây với **hệ số chệch** b của nơ-ron.)

Các đường cong ở Hình 27 minh họa ba tình huống điển hình:

162. **Softmax đặc trưng gốc (a): thiên lệch cao.** Lỗi huấn luyện tăng lên và hội tụ về lỗi kiểm định ở 50,0%, rất xa mức 13,1%. Hai đường gặp nhau nhưng ở mức cao: mô hình không thể biểu diễn ba vòng tròn bằng biên thẳng. **Thêm dữ liệu không giúp**, cần mô hình biểu diễn tốt hơn (mạng nơ-ron, hoặc đặc trưng đa thức ở mục 1.2).

163. **MLP 2–8–3 (b): khá vừa.** Lỗi huấn luyện 11,8% (không cao hơn mức tham chiếu 13,1%, nên thiên lệch không đáng kể; lỗi huấn luyện có thể thấp hơn mức tham chiếu vì mạng khớp được một phần nhiễu của chính các mẫu huấn luyện) và lỗi kiểm định 15,3% (khoảng cách 3,5 điểm: phương sai vừa phải, kiểm định cách mức tham chiếu 2,2 điểm). Lỗi kiểm định gần như phẳng từ $n_s = 480$ (15,0% → 15,3%), nên thêm dữ liệu cho mô hình này ít lợi.

164. **MLP lớn (c): phương sai cao.** Với $n_s = 240$, lỗi huấn luyện chỉ 2,7% nhưng kiểm định 18,4% (khoảng cách 15,7 điểm). Khoảng cách này **thu hẹp** khi n_s tăng (33,7 → 6,6 điểm) vì mạng không còn đủ chỗ để "thuộc" cả 900 mẫu. Đây là tình huống mà thêm dữ liệu hoặc điều chuẩn hóa giúp được.

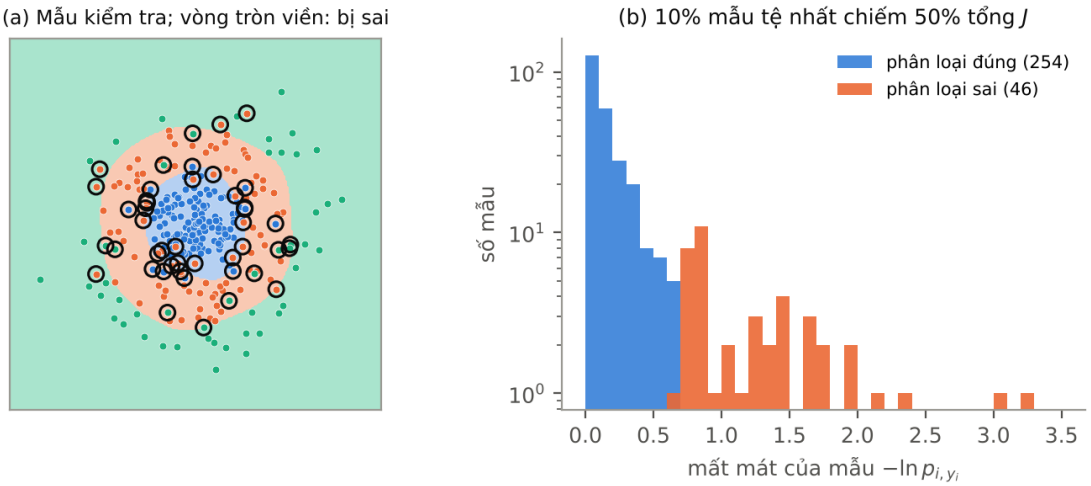
Lưu ý rằng lỗi kiểm định ở đây đo trên 300 mẫu nên cũng có sai số khoảng ± 2 điểm; các kết luận chỉ dựa vào xu hướng lớn.

Bảng chẩn đoán và hướng xử lý:

Lỗi huấn luyện	Lỗi kiểm định	Chẩn đoán	Cách xử lý
Cao (xa mức tham chiếu)	Cao, gần lỗi huấn luyện	Thiên lệch cao (thiếu khớp)	Mạng lớn hơn hoặc sâu hơn, đặc trưng tốt hơn, huấn luyện lâu hơn, giảm λ
Thấp	Cao hơn nhiều	Phương sai cao (quá khớp)	Thêm dữ liệu, L2, dropout, dừng sớm, mạng nhỏ hơn
Cao	Cao hơn nhiều	Cả hai	Xử lý thiên lệch trước, rồi phương sai
Thấp, gần mức tham chiếu	Gần lỗi huấn luyện	Tốt	Dừng, hoặc tinh chỉnh

Xem các mẫu bị sai

Các số tổng hợp không cho biết lỗi **nằm ở đâu**. Bước chẩn đoán quan trọng cuối cùng là mở các mẫu sai ra xem. Với mô hình cuối cùng (mục 5.3), có 46 mẫu sai trên 300. Mất mát trung bình của nhóm sai là 1,383, gấp khoảng 9 lần 0,149 của nhóm đúng. Nếu sắp các mẫu theo mất mát giảm dần, 10% mẫu tệ nhất (30 mẫu) đã chiếm 50% tổng J (Hình 28b). Cũng như ở mục 4.1, một số ít mẫu chi phối lỗi.



Hình 28. (a) 300 mẫu kiểm tra trên vùng quyết định của mô hình cuối cùng; các mẫu bị sai được khoanh tròn. (b) Phân bố mất mát $-\ln p_{i,y_i}$ của từng mẫu (thang logarit ở trục tung): mẫu sai nằm ở đuôi phải.

Hình 28a cho thấy các mẫu sai nằm gần như toàn bộ **trên biên giữa các vành**, đúng nơi hai phân phối chồng lên nhau (độ nhòe 0,7 so với khoảng cách giữa các bán kính 1,8 và 1,8). Đó là **nhiễu nội tại** của dữ liệu: ngay cả bộ phân loại tốt nhất cũng sai ở các điểm này, nên không nên cố ép mạng lớn hơn để "sửa" chúng (sẽ dẫn tới quá khớp như mục 4.4). Quy trình xem lỗi nên gồm:

165. **Sắp mẫu theo mất mát giảm dần** và xem kỹ vài chục mẫu đầu: chúng là nhãn sai thật sự, mẫu mơ hồ, hay mẫu mà mô hình thiếu đặc trưng?

166. **Xem lỗi theo từng lớp** (ma trận nhầm lẫn): lỗi dồn vào cặp lớp nào, có hệ thống không?

167. **So sánh với mức tham chiếu**: nếu lỗi còn lớn hơn nhiều mức này thì còn chỗ cải thiện; nếu đã sát thì phần còn lại phần lớn là nhiễu.

168. **Kiểm tra đầu vào**: giá trị thiếu, chưa chuẩn hóa, rò rỉ nhãn.

5.5. Huấn luyện và so sánh với *MLPClassifier* của scikit-learn và với Softmax Regression

Dùng *MLPClassifier*

Mạng mà ta cài đặt từ đầu có sẵn trong scikit-learn dưới tên *MLPClassifier*. Ta dùng nó để **kiểm tra cài đặt của mình** và làm đường cơ sở nhanh trong thực tế. Cấu hình tương ứng với mạng 2–16–3 (tanh, Adam, mini-batch 32, 200 epoch, L2 $\lambda = 10^{-3}$):

```
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import classification_report, confusion_matrix, log_loss

clf = MLPClassifier(hidden_layer_sizes=(16,), activation="tanh", solver="adam
",
                    alpha=1e-3 * 32, batch_size=32, learning_rate_init=0.01,
                    max_iter=200, tol=0, n_iter_no_change=1000, random_state=
0)
clf.fit(Xtr, ytr)
P = clf.predict_proba(Xkt)
print(confusion_matrix(ykt, P.argmax(axis=1)))
print(classification_report(ykt, P.argmax(axis=1), digits=3))
print("log-loss:", log_loss(ykt, P))
```

Các tham số tương ứng với những gì ta đã làm ở mục 3 và 4:

MLPClassifier	Trong mã của ta	Ghi chú
hidden_layer_sizes=(16,)	*kich_thuoc = [2, 16, 3]*	Lớp vào và ra suy ra từ dữ liệu; lớp ra tự dùng softmax khi có hơn hai lớp
activation="tanh"	*kich_hoat="tanh"*	Các lựa chọn khác: *"relu"*, *"logistic"*
solver="adam"	*thuat_toan="adam"*	Còn *"sgd"* (kèm *momentum*)
learning_rate_init	η	
batch_size	b	
max_iter	số epoch	Đếm theo epoch, không phải theo bước
alpha	λ	**Khác thang đo**, xem bên dưới
tol=0, *n_iter_no_change* lớn	Không dừng sớm	Mặc định thư viện dừng khi J huấn luyện không giảm quá *tol* trong 10 epoch liên tiếp

Hệ số *alpha*. Đọc mã nguồn của thư viện, ta thấy với mỗi mini-batch gồm b mẫu, hàm mất mát được cộng thêm số hạng $\frac{\alpha}{2b}\sum_l \|\mathbf{V}^{(l)}\|_F^2$ (hệ số chệch không bị phạt). So với công thức $\frac{\lambda}{2}\sum_l \|\mathbf{V}^{(l)}\|_F^2$ ở mục 4.4, suy ra $\lambda = \alpha/b$, tức là *alpha* $= \lambda b$. Với $\lambda = 10^{-3}$ và $b = 32$ ta đặt *alpha=1e-3 * 32*. Đây là chỗ rất dễ nhầm khi so sánh hai cài đặt. Khác biệt còn lại giữa hai cài

đặt: scikit-learn khởi tạo kiểu Xavier theo phân phối đều và cả hệ số chệch cũng được khởi tạo ngẫu nhiên (trong mã của ta hệ số chệch bắt đầu bằng 0); ngoài ra thứ tự xáo trộn mẫu khác.

Với một khởi tạo, kết quả báo cáo trên 300 mẫu kiểm tra (đầu ra rút gọn):

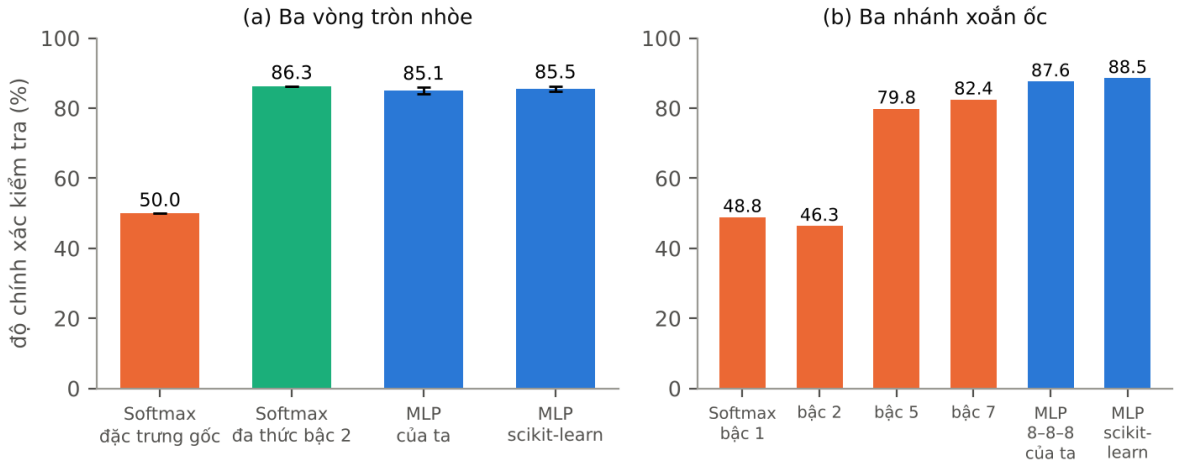
```
              precision    recall  f1-score   support
           0      0.885     0.927     0.906       150
           1      0.776     0.760     0.768       100
           2      0.867     0.780     0.821        50
    accuracy                         0.847       300
   macro avg      0.843     0.822     0.831       300
weighted avg      0.846     0.847     0.846       300
```

(Đây là kết quả của mô hình **của ta** ở mục 5.3, được đưa qua hàm `classification_report` của thư viện. Các số khớp tuyệt đối với bảng ở mục 5.3: accuracy 0,847, F1 macro 0,831, log-loss 0,338 cho cả hai cách tính. Nghĩa là các hàm `ma_tran_nham_lan` và `chi_so_theo_lop` của ta đúng.)

So sánh với Softmax Regression

Ta so sánh trên tập kiểm tra 300 mẫu (độ chính xác, F1 macro, log-loss; trung bình $\pm$ độ lệch chuẩn qua 5 khởi tạo cho hai mạng, mô hình Softmax là tất định). Mạng của ta và `MLPClassifier` dùng cùng siêu tham số ở trên và huấn luyện trên 900 mẫu.

Mô hình	Accuracy	F1 macro	Log-loss	Thời gian huấn luyện
Softmax Regression, đặc trưng gốc	50,0%	0,222	1,003	0,2 s
Softmax Regression, đặc trưng đa thức bậc 2	86,3%	0,849	0,345	0,2 s
MLP 2–16–3 của ta	85,1% $\pm$ 1,0%	0,832 $\pm$ 0,010	0,347 $\pm$ 0,004	0,6 s
`MLPClassifier`	85,5% $\pm$ 0,7%	0,836 $\pm$ 0,007	0,343 $\pm$ 0,004	1,2 s



Hình 29. Độ chính xác kiểm tra của các mô hình. (a) Ba vòng tròn nhòe (300 mẫu kiểm tra); thanh sai số là độ lệch chuẩn qua 5 khởi tạo. (b) Ba nhánh xoắn ốc (1200 mẫu kiểm tra): Softmax Regression với đặc trưng đa thức bậc 1, 2, 5, 7 và hai cài đặt mạng 8–8–8.

Có thể rút ra:

169. **Cài đặt của ta đúng.** Mạng từ đầu và `MLPClassifier` cho kết quả cùng cỡ (85,1% và 85,5%, chênh lệch nằm trong độ lệch chuẩn giữa các khởi tạo). Ngoài ra, mô hình Softmax Regression của ta (mục 4 của bài Softmax) trên đặc trưng đa thức bậc 2 cho đúng các số 0,863; 0,849; 0,345 với hồi quy logistic đa lớp của scikit-learn trên cùng đặc trưng.

170. **Softmax trên đặc trưng gốc thất bại** (50,0%, chính là mốc tầm thường; F1 macro 0,222 lộ ra điều đó).

171. **Softmax với đặc trưng đa thức bậc 2 lại ngang hoặc nhỉnh hơn mạng nơ-ron ở bài toán này** (86,3% so với 85,1% đến 85,5%). Lý do: ranh giới giữa các vòng tròn là đường tròn, có dạng $x_1^2 + x_2^2 = r^2$, nằm gọn trong lớp đặc trưng bậc hai. Khi ta **biết đúng** đặc trưng, mô hình tuyến tính đơn giản có thể ngang mạng nơ-ron, và nó huấn luyện nhanh hơn, ổn định hơn (lồi, không phụ thuộc khởi tạo).

Khi nào mạng nơ-ron thật sự có lợi?

Lợi thế của mạng là **không cần biết trước đặc trưng**. Để thấy rõ điều đó, ta thử bài toán mà đa thức bậc thấp không đủ: ba nhánh xoắn ốc (mục 4.5; 600 mẫu huấn luyện, 1200 mẫu kiểm tra). Softmax Regression với đặc trưng đa thức bậc d có $\binom{d+2}{2} - 1$ đặc trưng, nên số tham số tăng nhanh:

Mô hình	Số tham số	Accuracy kiểm tra
Softmax, bậc 1 (đặc trưng gốc)	9	48,8%
Softmax, bậc 2	18	46,3%

Mô hình	Số tham số	Accuracy kiểm tra
Softmax, bậc 3	30	35,6%
Softmax, bậc 5	63	79,8%
Softmax, bậc 7	108	82,4%
MLP 2–8–8–8–3 của ta	195	87,6%
`MLPClassifier` (8,8,8)	195	88,5%

Bậc 2 và 3 không giúp được (có khi còn kém hơn đặc trưng gốc, vì các đặc trưng đó không bắt được hình xoắn). Phải lên bậc 5 mới tốt lên, và bậc 7 đạt 82,4%; mạng 8–8–8 (Adam, $\eta = 0{,}01$, mini-batch 32, 1000 epoch, không điều chuẩn hóa) đạt 87,6% đến 88,5% mà không cần ta chọn bậc nào. (Số đo ở mạng là trung bình 3 khởi tạo.) Số tham số của mạng (195) lớn hơn Softmax bậc 7 (108), nên đây không phải là chuyện "ít tham số hơn" mà là **đặc trưng có cấu trúc đúng**. Chênh lệch giữa mạng của ta và `MLPClassifier` (0,9 điểm) nằm trong độ lệch chuẩn giữa các lần chạy (0,1 và 0,3 điểm) cộng sai số của 1200 mẫu kiểm tra (≈ 1 điểm); ta không kết luận cài đặt nào tốt hơn.

Khi nào dùng cái nào?

Nhu cầu	Gợi ý
Hiểu cơ chế, thử ý tưởng, dạy học	Cài đặt từ đầu (mục 3.7)
Đường cơ sở nhanh trên dữ liệu bảng cỡ vừa	`MLPClassifier` (và so với `LogisticRegression` có đặc trưng đa thức)
Dữ liệu lớn, mạng sâu, GPU, kiến trúc đặc thù (CNN, RNN, Transformer)	Các framework như PyTorch hoặc TensorFlow (mục 6.3)

Quy trình đánh giá tóm tắt

Gộp lại, một lần huấn luyện nghiêm túc nên theo thứ tự sau:

172. Chia phân tầng; chuẩn hóa bằng thống kê của tập huấn luyện.
173. Huấn luyện một mô hình đơn giản làm đường cơ sở (mốc tầm thường, Softmax Regression) để biết "tốt" là bao nhiêu.
174. Huấn luyện mạng, theo dõi đường cong học và chuẩn gradient (mục 4.3), kiểm tra vùng quyết định nếu vẽ được.
175. Chọn siêu tham số trên tập kiểm định hoặc kiểm định chéo; ưu tiên mô hình đơn giản.
176. Chẩn đoán thiên lệch và phương sai bằng đường cong học theo n_s; xem các mẫu sai.
177. Báo cáo **một lần** trên tập kiểm tra: ma trận nhầm lẫn, F1 từng lớp, log-loss, kèm khoảng tin cậy.

Mối liên hệ với mục trước và mục sau

Mục 4 giải thích cơ chế học; mục 5 cho các công cụ để **đo** và **chẩn đoán** cơ chế đó trên một bài toán cụ thể. Các phép đo ở mục 5 quay lại gần như mọi hiện tượng đã gặp: cao nguyên (mục 4.2) hiện ra ở đường cong accuracy; quá khớp (mục 4.4) hiện ra ở khoảng cách giữa J_{tr} và $J_{\text{kđ}}$ và ở log-loss; biểu diễn của lớp ẩn là hình ảnh cụ thể của ý tưởng ở mục 1.2.

Mục 6 tổng kết toàn chương: ưu và nhược điểm của mạng nơ-ron, so sánh bốn mô hình (Perceptron, Logistic Regression, Softmax Regression và mạng nơ-ron) trên cùng bộ tiêu chí, hướng mở rộng sang học sâu và các bài tập thực hành. Bảng và số liệu ở mục 5.5 chính là bước đầu của phần so sánh đó.

Tóm tắt mục 5

178. **Quan sát quá trình học.** Đường cong J và accuracy theo epoch cho biết mô hình học tốt, quá khớp, thiếu khớp hay không ổn định. Vùng quyết định theo epoch cho thấy mạng đi từ "luôn đoán lớp đông nhất" đến ba vành cong. Ở lớp ẩn cuối, các lớp tách bằng đường thẳng: mạng đã tự học đặc trưng để Softmax dùng.

179. **Chia dữ liệu.** Chia phân tầng thành huấn luyện, kiểm định, kiểm tra; chọn mọi thứ trên tập kiểm định, chỉ chạm tập kiểm tra một lần. Thống kê chuẩn hóa chỉ tính từ tập huấn luyện (tránh rò rỉ). Kiểm định chéo K lần dùng dữ liệu hiệu quả hơn khi dữ liệu ít.

180. **Chọn siêu tham số.** Tìm kiếm lưới trên tập kiểm định; vì $\text{SE} = \sqrt{p(1-p)/n}$ nên chênh lệch dưới vài điểm phần trăm trên vài trăm mẫu là nhiễu; dùng quy tắc một sai số chuẩn để chọn mô hình đơn giản nhất. Kết quả phụ thuộc cách chia: 20 cách chia cho độ chính xác từ 82,7% đến 89,3%.

181. **Chỉ số.** Ma trận nhầm lẫn $\mathbf{C} = \mathbf{Y}^T\widehat{\mathbf{Y}}$; precision theo cột, recall theo hàng, F1 là trung bình điều hòa; F1 macro cho lớp nhỏ trọng lượng ngang lớp lớn. Log-loss đo cả độ tự tin của xác suất và nhạy với quá khớp hơn accuracy. Mốc tầm thường: luôn đoán lớp đông nhất (50%, F1 macro 0,222).

182. **Chẩn đoán.** Đường cong học theo số mẫu: thiên lệch cao (hai đường gặp nhau ở mức cao: cần mô hình mạnh hơn) hay phương sai cao (khoảng cách lớn: cần dữ liệu hoặc điều chuẩn hóa). Xem các mẫu sai: một ít mẫu chiếm nửa J, và lỗi dồn vào biên giữa các lớp chồng lấn là nhiễu nội tại.

183. **So sánh.** Mạng của ta khớp với `MLPClassifier` (chú ý $\texttt{alpha} = \lambda b$) và Softmax của ta khớp với hồi quy logistic của scikit-learn. Khi biết đúng đặc trưng (đa thức bậc 2 cho các vòng tròn), Softmax ngang mạng; khi không biết (xoắn ốc), mạng vượt rõ rệt.

Câu hỏi kiểm tra nhanh

184. Một bài toán có 1000 mẫu, 950 mẫu thuộc lớp A và 50 mẫu thuộc lớp B. Mô hình luôn đoán A. Tính accuracy và F1 macro.

185. Cho ma trận nhầm lẫn $\mathbf{C} = \begin{bmatrix} 50 & 5 & 5 \\ 10 & 30 & 10 \\ 0 & 5 & 35 \end{bmatrix}$. Tính accuracy, precision, recall và F1 của từng lớp, và F1 macro.

186. Mô hình gán xác suất cho nhãn thật của 4 mẫu là $0{,}9;\ 0{,}8;\ 0{,}1;\ 0{,}6$. Tính log-loss. Nếu mẫu thứ ba được gán $0{,}01$ thay vì $0{,}1$ thì log-loss thay đổi thế nào?

187. Một mô hình đạt accuracy 80% trên tập kiểm tra 400 mẫu. Tính khoảng tin cậy 95%.

188. Lỗi huấn luyện 2%, lỗi kiểm định 15%, lỗi tham chiếu 3%. Chẩn đoán và đề xuất ít nhất hai cách xử lý.

189. Lỗi huấn luyện 18%, lỗi kiểm định 19%, lỗi tham chiếu 4%. Chẩn đoán. Thu thập thêm dữ liệu có giúp được không?

190. Bảng độ chính xác kiểm định của các cấu hình ở mục 5.2 có tốt nhất $86{,}0\%$ với $\mathrm{SE} = 2{,}0$ điểm. Dùng quy tắc một sai số chuẩn, cấu hình nào có m nhỏ nhất được chọn, biết các cấu hình có $m = 4$ đạt $85{,}6\%$ và $m = 2$ đạt $70{,}6\%$?

191. Kiểm định chéo 5 lần cho độ chính xác các phần là $0{,}85;\ 0{,}90;\ 0{,}88;\ 0{,}84;\ 0{,}86$. Tính trung bình và độ lệch chuẩn (chia cho $N = 5$). Với 1200 mẫu, mỗi lần huấn luyện dùng bao nhiêu mẫu và kiểm định trên bao nhiêu mẫu?

192. Muốn dùng $\lambda = 0{,}01$ trong `MLPClassifier` với `batch_size=64`, cần đặt `alpha` bằng bao nhiêu?

193. Tính số tham số của mạng 2–16–3.

194. Một bạn chuẩn hóa dữ liệu bằng trung bình và độ lệch chuẩn của **toàn bộ** 1500 mẫu rồi mới chia thành ba tập. Có vấn đề gì? Sửa thế nào?

195. Giải thích vì sao ở Hình 24b các lớp tách được bằng đường thẳng dù ở Hình 24a thì không.

Gợi ý đáp án

196. Accuracy $= 950/1000 = 95\%$. Với lớp A: Prec $= 950/1000 = 0{,}95$, Rec $= 1$, $\mathrm{F1}_A = 2 \cdot 0{,}95 \cdot 1/1{,}95 = 0{,}9744$. Với lớp B: không có mẫu nào được dự đoán là B và không mẫu B nào được tìm thấy nên Rec $= 0$ và $\mathrm{F1}_B = 0$. F1 macro $=$

$(0{,}9744 + 0)/2 = 0{,}487$: accuracy cao nhưng F1 macro cho thấy mô hình vô dụng với lớp B.

197. Tổng cột $(60,40,50)$, tổng hàng $(60,50,40)$, $n = 150$, đường chéo $50 + 30 + 35 = 115$. Accuracy $= 115/150 = 0{,}767$. Prec $= (0{,}833;\ 0{,}750;\ 0{,}700)$; Rec $= (0{,}833;\ 0{,}600;\ 0{,}875)$; F1 $= (0{,}833;\ 0{,}667;\ 0{,}778)$; F1 macro $= 0{,}759$.

198. $-\ln 0{,}9 = 0{,}105$, $-\ln 0{,}8 = 0{,}223$, $-\ln 0{,}1 = 2{,}303$, $-\ln 0{,}6 = 0{,}511$. Trung bình $= 3{,}142/4 = 0{,}785$. Nếu mẫu thứ ba là 0,01: $-\ln 0{,}01 = 4{,}605$ nên log-loss $= (0{,}105 + 0{,}223 + 4{,}605 + 0{,}511)/4 = 1{,}361$: chỉ một mẫu tự tin sai làm log-loss tăng gần gấp đôi.

199. SE $= \sqrt{0{,}8 \cdot 0{,}2/400} = 0{,}02$; khoảng tin cậy $0{,}80 \pm 1{,}96 \times 0{,}02 = 0{,}80 \pm 0{,}039$, tức khoảng 76,1% đến 83,9%.

200. Thiên lệch $= 2 - 3 < 0$ (không có), phương sai $= 15 - 2 = 13$ điểm: **phương sai cao (quá khớp)**. Cách xử lý: thêm dữ liệu, L2 hoặc dropout, dừng sớm, giảm kích thước mạng.

201. Thiên lệch $= 18 - 4 = 14$ điểm, phương sai $= 19 - 18 = 1$ điểm: **thiên lệch cao (thiếu khớp)**. Thêm dữ liệu gần như không giúp vì hai lỗi đã gần nhau; cần mô hình biểu diễn tốt hơn (lớn hơn hoặc sâu hơn, đặc trưng tốt hơn, huấn luyện lâu hơn, giảm điều chuẩn hóa).

202. Ngưỡng $= 86{,}0 - 2{,}0 = 84{,}0\%$. Cấu hình $m = 2$ (70,6%) dưới ngưỡng; $m = 4$ (85,6%) trên ngưỡng. Vậy chọn $m = 4$ (với $\lambda = 0$).

203. Trung bình $= (0{,}85 + 0{,}90 + 0{,}88 + 0{,}84 + 0{,}86)/5 = 0{,}866$. Độ lệch $= \sqrt{\frac{1}{5}\sum(a_f - 0{,}866)^2} = 0{,}0215$. Mỗi lần huấn luyện dùng 960 mẫu và kiểm định trên 240 mẫu $(1200/5)$.

204. $alpha = \lambda b = 0{,}01 \times 64 = 0{,}64$.

205. $(2 + 1) \times 16 + (16 + 1) \times 3 = 48 + 51 = 99$ tham số.

206. Rò rỉ dữ liệu: trung bình và độ lệch chuẩn của tập kiểm tra đã ảnh hưởng đến phép chuẩn hóa dùng khi huấn luyện, nên kết quả kiểm tra hơi lạc quan. Sửa: chia trước, tính $\boldsymbol{\mu}$, $\boldsymbol{\sigma}$ chỉ từ tập huấn luyện rồi dùng chung cho cả ba tập.

207. Điểm số lớp ra là $\mathbf{s} = \tilde{\mathbf{h}}\mathbf{W}^{(L)}$, tuyến tính theo $\mathbf{h}$, nên trong không gian (h_1, h_2) vùng thắng của từng lớp bị chặn bởi các đường thẳng. Các lớp ẩn trước đó biến đổi mặt phẳng đầu vào (cong, co giãn) sao cho các lớp trở nên tách được bằng đường thẳng ở không gian mới.

6. Tổng kết và mở rộng

Năm mục trước đã đi hết một vòng: từ giới hạn của mô hình tuyến tính (mục 1), cấu trúc của mạng truyền thẳng (mục 2), thuật toán lan truyền ngược (mục 3), cơ chế học (mục 4) đến cách đo và chẩn đoán (mục 5). Mục cuối này làm ba việc:

208. Nhìn lại toàn bộ để nói thẳng mạng nơ-ron **được gì và mất gì** (mục 6.1).
209. Đặt mạng nơ-ron cạnh ba mô hình quen thuộc, Perceptron, Logistic Regression và Softmax Regression, trên cùng bộ tiêu chí và cùng bộ dữ liệu (mục 6.2).
210. Chỉ ra mạng truyền thẳng mở ra con đường nào: các kiến trúc học sâu (CNN, RNN, Transformer) và các framework giúp ta không phải tự viết lan truyền ngược nữa (mục 6.3).

Cuối cùng, mục 6.4 là các bài tập thực hành để người đọc tự kiểm tra những gì đã học.

Trong mục này, quy ước ký hiệu vẫn như các mục trước: ma trận là chữ hoa in đậm (**X**, **W**), vector là chữ thường in đậm (**x**, **w**).

Ký hiệu bổ sung cho mục 6

Đối tượng	Ký hiệu	Ý nghĩa
Hàm sigmoid, softmax	$\sigma(z)$, $\mathrm{softmax}(\mathbf{z})$	Như mục 1 và mục 2
Bộ lọc tích chập	$\mathbf{F}$	Ma trận nhỏ (ví dụ 3×3), tham số cần học của lớp tích chập
Trạng thái ẩn tại bước t	$\mathbf{h}_t$	Vector "bộ nhớ" của mạng hồi quy
Truy vấn, khóa, giá trị	$\mathbf{Q}$, $\mathbf{K}$, $\mathbf{R}$	Ba ma trận trong cơ chế chú ý (chữ **R** thay cho "value" để không trùng **V** của mục 3)
Số tham số	$N_{\text{tham số}}$	Tổng số phần tử của mọi ma trận trọng số và độ lệch

6.1. Ưu điểm và hạn chế của mạng nơ-ron

Ưu điểm

1. Biểu diễn được các ranh giới phi tuyến mà không cần biết trước dạng của chúng. Đây là lý do mạng nơ-ron ra đời (mục 1). Ở mục 5.5, Softmax Regression chỉ vượt qua được ba vòng tròn khi ta *biết* phải thêm đặc trưng đa thức bậc 2; còn với ba nhánh xoắn ốc, đặc trưng đa thức bậc 7 chỉ đạt 82,4% (bậc 3 chỉ 35,6%) trong khi mạng 8-8-8 đạt 87,6% đến 88,5% mà không cần ai thiết kế đặc trưng. Mạng nơ-ron *tự học đặc trưng*: các lớp ẩn biến đổi dữ liệu cho đến khi lớp cuối, vốn chính là một Softmax Regression, có thể tách các lớp bằng đường thẳng (mục 5.1).

2. Có cơ sở lý thuyết cho khả năng biểu diễn. Định lý xấp xỉ phổ quát (mục 2) cho biết một lớp ẩn đủ rộng xấp xỉ được mọi hàm liên tục trên tập compact. Mục 4.5 bổ sung rằng *độ sâu* làm số

vùng tuyến tính tăng theo hàm mũ ($2k$ nơ-ron ReLU xếp thành k lớp có thể tạo 2^k đoạn), trong khi độ rộng chỉ tăng tuyến tính.

3. Một khung chung cho nhiều bài toán. Chỉ cần thay lớp ra và hàm mất mát là cùng một mạng giải được phân loại nhị phân (sigmoid và cross-entropy nhị phân), phân loại nhiều lớp (softmax và cross-entropy) hoặc hồi quy (đầu ra tuyến tính và sai số bình phương). Lan truyền ngược không đổi: luôn là $\mathbf{D}^{(L)}$ rồi đi lùi qua từng lớp (mục 3).

4. Huấn luyện hiệu quả nhờ lan truyền ngược. Chi phí tính toàn bộ gradient chỉ cỡ chi phí của một lần lan truyền thuận (hai đến ba lần), dù có hàng trăm nghìn tham số. Nhờ vậy mạng lớn mới huấn luyện được. Gradient còn kiểm tra được bằng sai phân hữu hạn (mục 3.4).

5. Mở rộng được theo hướng kiến trúc. Vì lan truyền ngược chỉ đòi hỏi mỗi khối là một hàm khả vi, ta có thể thay phép nhân ma trận đầy đủ bằng phép tích chập, bằng đệ quy theo thời gian hoặc bằng cơ chế chú ý mà không phải đổi thuật toán học (mục 6.3).

Hạn chế

1. Hàm mất mát không lồi. Softmax Regression có một cực tiểu toàn cục duy nhất nên kết quả không phụ thuộc điểm khởi tạo. Mạng nơ-ron thì có nhiều điểm yên ngựa và nhiều cực tiểu tương đương do đối xứng hoán vị (mục 4.2). Hệ quả cụ thể đã gặp: mạng XOR 2-2-1 chỉ đạt $J < 0{,}05$ ở 34 trong 40 khởi tạo, còn $m = 3$ là $39/40$ và $m = 4$, $m = 8$ là $40/40$. Kết quả của một lần chạy luôn đi kèm độ lệch chuẩn qua nhiều hạt giống ngẫu nhiên, đúng như ở các bảng mục 5.

2. Gradient biến mất hoặc bùng nổ ở mạng sâu. Chuẩn của tín hiệu lỗi bị chặn bởi tích các chuẩn $\| \mathbf{V} \|_2$ qua từng lớp (mục 4.3). Thí nghiệm mạng sigmoid 8 lớp thất bại (43% khi dùng Adam, 33,3% khi dùng SGD) còn ReLU với khởi tạo He đạt 99% đến 100%. Mạng sâu đòi hỏi những lựa chọn có chủ đích: hàm kích hoạt, khởi tạo, thuật toán tối ưu.

3. Dễ quá khớp. Số tham số lớn so với số mẫu. Ở thí nghiệm 60 mẫu của mục 4.4, không điều chuẩn hóa chỉ đạt 66,3%, thêm L2 lên 72,1%, còn mốc tham chiếu là 75,3%. Ở bài toán chữ số viết tay mục 6.4, với 100 mẫu, mạng 64-64-10 đạt đúng 100% trên tập huấn luyện nhưng chỉ 88,2% trên tập kiểm tra. Cần các công cụ ở mục 4.4 và mục 5.4.

4. Nhiều siêu tham số. Số lớp, số nơ-ron, hàm kích hoạt, tốc độ học, kích thước mini-batch, hệ số L2, tỉ lệ dropout, số epoch… Mỗi siêu tham số cần được chọn bằng tập kiểm định (mục 5.2), và vì sai số chuẩn của độ chính xác lớn ($\mathrm{SE} = \sqrt{p(1-p)/n}$) nên chênh lệch nhỏ giữa các cấu hình thường chỉ là nhiễu.

5. Chi phí tính toán và dữ liệu. Mạng của ta chạy 0,6 giây mỗi lần huấn luyện so với 0,2 giây của Softmax Regression trên cùng bài toán (và khoảng cách này tăng nhanh theo kích thước). Mạng lớn đòi hỏi nhiều dữ liệu và phần cứng chuyên dụng.

6. Khó diễn giải. Hệ số của Softmax Regression nói được "đặc trưng j làm tăng điểm số lớp k thêm bao nhiêu". Với mạng nhiều lớp, ý nghĩa của từng trọng số không còn đọc trực tiếp được. Ta chỉ có các công cụ gián tiếp như vùng quyết định và biểu diễn lớp ẩn (mục 5.1) hay xem các mẫu bị sai (mục 5.4).

7. Không phải lúc nào cũng thắng. Khi dữ liệu ít, khi dạng của quan hệ đã biết hoặc khi bài toán đơn giản, mô hình tuyến tính có đặc trưng tốt thường ngang mạng nơ-ron mà rẻ hơn, ổn định hơn và dễ diễn giải hơn (kết quả trên ba vòng tròn ở mục 5.5: Softmax với đa thức bậc 2 đạt 86,3%, mạng đạt 85,1% đến 85,5%). Theo nhiều so sánh được công bố trên dữ liệu bảng (mỗi cột là một đặc trưng có ý nghĩa riêng, như hồ sơ khách hàng), các mô hình dựa trên cây quyết định như rừng ngẫu nhiên hay tăng cường gradient thường ngang hoặc hơn mạng nơ-ron; mình nêu điều này từ hiểu biết chung chứ không kiểm chứng trong chương này.

Bảng tóm tắt

Tiêu chí	Ưu điểm	Hạn chế
Biểu diễn	Học được ranh giới phi tuyến và đặc trưng	Không rõ trước cần bao nhiêu nơ-ron, bao nhiêu lớp
Tối ưu	Lan truyền ngược rẻ; gradient kiểm tra được	Không lồi: nhiều cực tiểu, điểm yên ngựa, phụ thuộc khởi tạo
Tổng quát hóa	Điều chuẩn hóa mạnh (L2, dropout, dừng sớm)	Dễ quá khớp khi dữ liệu ít
Vận hành	Một khung cho nhiều bài toán, mở rộng được	Nhiều siêu tham số, tốn tính toán, khó diễn giải

Quy tắc thực hành. Luôn bắt đầu bằng mô hình đơn giản nhất (mục 5.5: Softmax Regression, có thể với đặc trưng đa thức) làm mốc so sánh. Chỉ chuyển sang mạng nơ-ron khi mô hình đơn giản đã được chứng tỏ là thiếu khớp (thiên lệch cao theo mục 5.4), và mạng phải *thắng mốc đó rõ ràng* theo tiêu chuẩn thống kê của mục 5.2.

6.2. So sánh Perceptron, Logistic Regression, Softmax Regression và mạng nơ-ron

Bốn mô hình là những bước kế tiếp nhau của cùng một câu chuyện. Để so sánh có hệ thống, ta viết cả bốn dưới cùng một dạng: **một phép biến đổi tuyến tính rồi một hàm kích hoạt**, và xem khác nhau ở đâu.

Bốn mô hình viết chung một dạng

Với mẫu $\mathbf{x}$ và $\tilde{\mathbf{x}} = [1, \mathbf{x}^T]^T$:

211. **Perceptron** (hai lớp, nhãn $y \in \{-1, +1\}$): $\hat{y} = \text{sign}(\mathbf{w}^T\tilde{\mathbf{x}})$. Khi một mẫu bị sai, cập nhật $\mathbf{w} \leftarrow \mathbf{w} + \eta\, y\, \tilde{\mathbf{x}}$.

212. **Logistic Regression** (hai lớp, $y \in \{0,1\}$): $\hat{p} = \sigma(\mathbf{w}^T\tilde{\mathbf{x}})$, mất mát cross-entropy nhị phân

$$J(\mathbf{w}) = -\frac{1}{n}\sum_{i=1}^{n}[y_i \ln\hat{p}_i + (1-y_i)\ln(1-\hat{p}_i)].$$

213. **Softmax Regression** (K lớp): $\mathbf{p} = \text{softmax}(\mathbf{W}^T\tilde{\mathbf{x}})$ với $\mathbf{W} \in \mathbb{R}^{(d+1)\times K}$, mất mát cross-entropy $J(\mathbf{W}) = -\frac{1}{n}\sum_i \sum_k y_{ik} \ln p_{ik}$ và gradient $\frac{1}{n}\tilde{\mathbf{X}}^T(\mathbf{P} - \mathbf{Y})$ (mục 1).

214. **Mạng nơ-ron** (MLP, L lớp): lặp lại phép biến đổi kèm hàm kích hoạt nhiều lần,

$$\mathbf{Z}^{(l)} = \tilde{\mathbf{A}}^{(l-1)}\mathbf{W}^{(l)}, \qquad \mathbf{A}^{(l)} = \varphi\left(\mathbf{Z}^{(l)}\right) \ (l < L), \qquad \mathbf{P} = \text{softmax}\left(\mathbf{Z}^{(L)}\right),$$

với $\mathbf{A}^{(0)} = \mathbf{X}$ (mục 2), và gradient tính bằng lan truyền ngược (mục 3).

Các quan hệ giữa chúng, như đã thấy dọc chương:

215. Logistic Regression là Softmax Regression với $K = 2$ (hai điểm số chỉ khác nhau một hằng số, nên chỉ cần một vector trọng số).

216. Softmax Regression là mạng nơ-ron **không có lớp ẩn** ($L = 1$).

217. Mạng nơ-ron là Softmax Regression áp dụng lên biểu diễn $\mathbf{A}^{(L-1)}$ do các lớp ẩn **tự học** (mục 5.1).

218. Perceptron khác Logistic Regression ở hai điểm: hàm kích hoạt là hàm bước thay vì sigmoid, và thuật toán chỉ cập nhật khi mẫu bị sai. Hàm bước có đạo hàm bằng 0 gần như ở mọi nơi, nên không thể áp dụng gradient descent trên một hàm mất mát trơn; vì vậy Perceptron không cho xác suất, và đây cũng là lý do mạng nơ-ron hiện đại dùng các hàm kích hoạt khả vi (mục 2).

Bảng so sánh theo tiêu chí

Tiêu chí	Perceptron	Logistic Regression	Softmax Regression	Mạng nơ-ron (MLP)
Số lớp	Hai	Hai	Nhiều	Nhiều
Đầu ra	Nhãn ± 1	Xác suất $\hat{p}$	Vector xác suất $\mathbf{p}$	Vector xác suất $\mathbf{p}$
Hàm kích hoạt ra	Hàm bước	Sigmoid	Softmax	Softmax
Hàm mất mát	Không có (cập nhật theo mẫu sai)	Cross-entropy nhị phân	Cross-entropy	Cross-entropy
Ranh giới quyết định	Tuyến tính	Tuyến tính	Tuyến tính (từng đoạn thẳng giữa các lớp)	Phi tuyến
Hội tụ	Chỉ khi dữ liệu tách được tuyến tính	Hàm mất mát lồi: cực tiểu toàn cục duy nhất	Hàm mất mát lồi: cực tiểu toàn cục duy nhất	Không lồi: phụ thuộc khởi tạo
Số tham số	$d + 1$	$d + 1$	$(d + 1)K$	$\sum_l (m_{l-1} + 1)\, m_l$
Đặc trưng	Phải tự thiết kế	Phải tự thiết kế	Phải tự thiết kế	Tự học ở lớp ẩn

Tiêu chí	Perceptron	Logistic Regression	Softmax Regression	Mạng nơ-ron (MLP)
Diễn giải	Dễ	Dễ (hệ số, tỉ số odds)	Dễ	Khó
Chi phí huấn luyện	Rất thấp	Thấp	Thấp	Cao hơn

(Với mạng nơ-ron, $m_0 = d$ và $m_L = K$; công thức tham số này đã dùng ở mục 2 và mục 5.5, chẳng hạn mạng 2-16-3 có $(2+1) \times 16 + (16+1) \times 3 = 99$ tham số.)

Thí nghiệm: cùng bốn bộ dữ liệu, cùng bốn mô hình

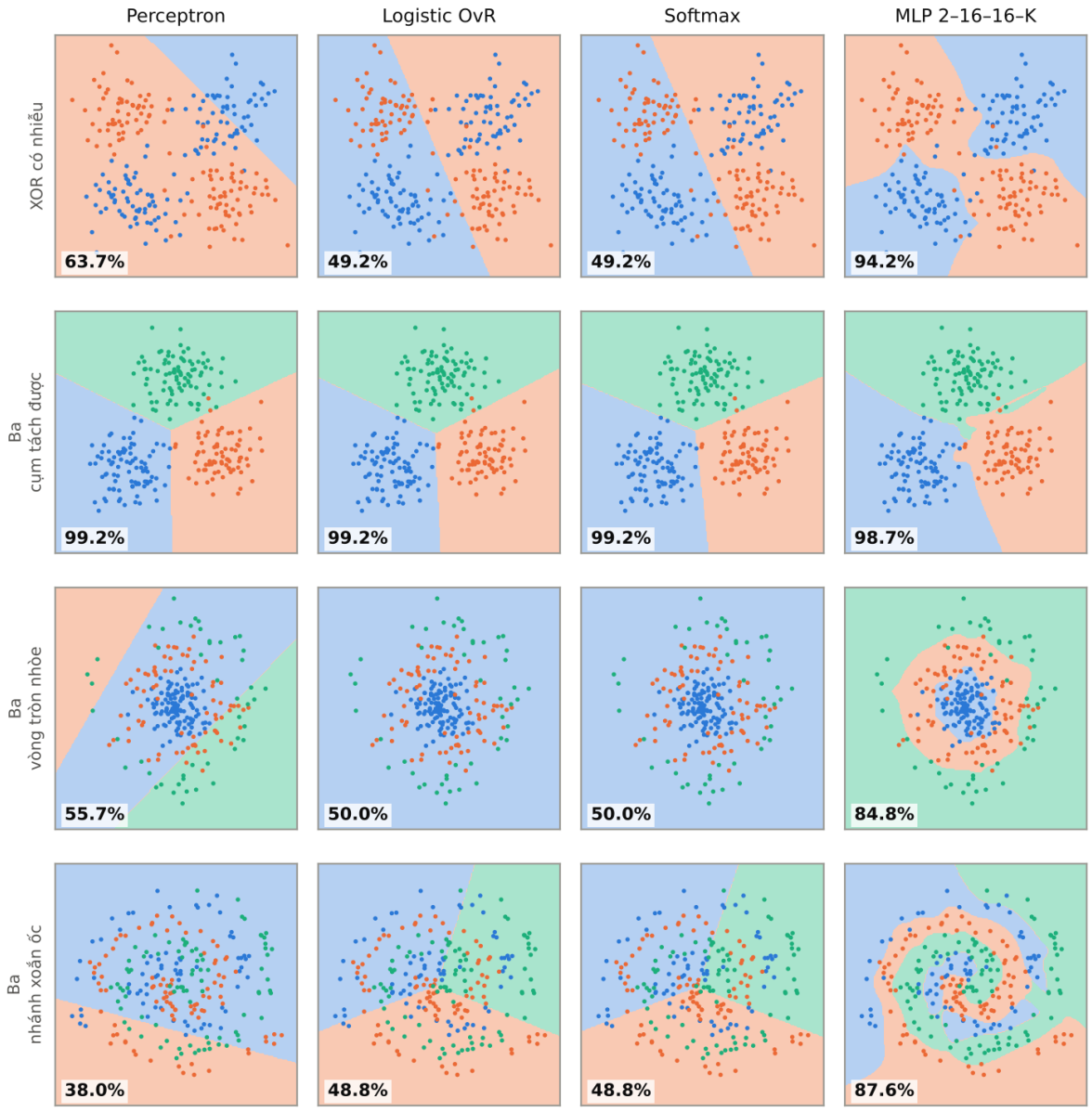
Để so sánh bằng số liệu, ta chạy cả bốn mô hình trên bốn bộ dữ liệu hai chiều có độ khó tăng dần:

219. **Ba cụm tách được:** ba đám mây tách tuyến tính.
220. **XOR có nhiễu:** hai lớp theo kiểu XOR (mục 1), đây là ví dụ kinh điển mà mô hình tuyến tính không thể giải.
221. **Ba vòng tròn nhòe:** bộ dữ liệu của mục 5 (các vòng chồng lấn).
222. **Ba nhánh xoắn ốc:** bộ dữ liệu của mục 4.

Cài đặt:

223. **Perceptron** và **Logistic Regression** vốn là mô hình hai lớp; ở bài toán K lớp ta dùng chiến lược **một-với-tất-cả** (one-vs-rest, OvR): huấn luyện K bộ phân loại, bộ thứ k phân biệt lớp k với phần còn lại, và dự đoán lớp có điểm số lớn nhất.
224. **Softmax Regression** dùng cài đặt của mục 1 (gradient descent, đủ nhiều epoch).
225. **Mạng nơ-ron** là mạng 2-16-16-K với tanh, huấn luyện bằng Adam như mục 3 và mục 4.
226. Mọi mô hình huấn luyện trên tập huấn luyện và đo độ chính xác trên tập kiểm tra riêng. Với Perceptron và mạng nơ-ron (vốn phụ thuộc khởi tạo hoặc thứ tự mẫu) ta chạy 5 hạt giống và báo cáo trung bình $\pm$ độ lệch chuẩn; Logistic OvR và Softmax là tất định (hàm mất mát lồi, bắt đầu từ $\mathbf{0}$) nên chỉ có một con số.

Kích thước: XOR 400 mẫu huấn luyện và 400 kiểm tra; ba cụm 600 và 600; ba vòng tròn 1500 và 1500; xoắn ốc 600 và 1200.



Hình 30. Vùng quyết định của bốn mô hình (cột) trên bốn bộ dữ liệu (hàng), hạt giống 0. Số ở góc là độ chính xác kiểm tra (%). Chỉ MLP cho vùng quyết định phù hợp ở ba bộ dữ liệu phi tuyến.

Độ chính xác kiểm tra (trung bình trên 5 hạt giống đối với Perceptron và MLP):

Dữ liệu	Perceptron	Logistic OvR	Softmax	MLP 2-16-16-K
Ba cụm tách được	$0{,}983 \pm 0{,}012$	0,992	0,992	$0{,}983 \pm 0{,}003$
XOR có nhiễu	$0{,}550 \pm 0{,}050$	0,492	0,492	$0{,}944 \pm 0{,}004$
Ba vòng tròn nhòe	$0{,}466 \pm 0{,}130$	0,500	0,500	$0{,}847 \pm 0{,}006$
Ba nhánh xoắn ốc	$0{,}380 \pm 0{,}059$	0,488	0,488	$0{,}884 \pm 0{,}006$

(Hình 30 vẽ hạt giống 0 nên các con số ở góc của Perceptron và MLP, như 63,7% cho Perceptron trên XOR hay 94,2% cho MLP, chỉ là một lần chạy trong 5 lần ở bảng.)

Nhận xét.

227. **Dữ liệu tách được tuyến tính:** cả bốn mô hình ngang nhau (98% đến 99%). Softmax và Logistic OvR nhỉnh hơn một chút (0,992 so với 0,983), nhưng chênh lệch 0,009 chỉ khoảng hai lần sai số chuẩn $\sqrt{0{,}99 \times 0{,}01/600} \approx 0{,}004$, còn độ lệch chuẩn qua hạt giống của Perceptron là 0,012; nên không đủ cơ sở kết luận mô hình nào "tốt hơn". Khi bài toán tuyến tính thì **không có lý do dùng mô hình phức tạp hơn**.

228. **XOR, vòng tròn, xoắn ốc:** Logistic OvR và Softmax chỉ đạt 0,49 đến 0,50, tức xấp xỉ đoán ngẫu nhiên hoặc đoán lớp đông nhất (với vòng tròn, đoán luôn lớp 0 đã được 0,50, đúng như mốc tầm thường ở mục 5.1). Đây là kết quả của thiên lệch cao: không phải do huấn luyện kém mà do một vùng tuyến tính **không thể** biểu diễn các ranh giới này. MLP đạt 0,847 đến 0,944.

229. **Perceptron** không ổn định trên dữ liệu không tách được và ở vòng tròn, xoắn ốc còn kém cả mô hình logistic ($0{,}466 \pm 0{,}130$ và $0{,}380 \pm 0{,}059$): vì thuật toán chỉ cập nhật theo mẫu sai nên nó không bao giờ dừng khi dữ liệu không tách được, và kết quả cuối phụ thuộc mẫu cuối cùng gặp phải (độ lệch chuẩn lớn). Đây là lý do ta thay nó bằng hàm mất mát trơn của Logistic Regression.

230. **MLP không phải lúc nào cũng hoàn hảo:** trên ba vòng tròn chồng lấn nó chỉ đạt 0,847 vì các vòng thực sự chồng lên nhau (nhiễu nội tại, mục 5.4); không mô hình nào vượt được ngưỡng này.

Hạn chế của thí nghiệm. Đây là bốn bộ dữ liệu hai chiều, kích thước từ vài trăm đến vài nghìn mẫu, một cấu hình mạng cố định và không tinh chỉnh riêng cho từng bài toán. Ta kết luận về *kiểu* hành vi (mô hình tuyến tính thiếu khớp trên dữ liệu phi tuyến) chứ không kết luận về độ chính xác tuyệt đối.

Chọn mô hình nào?

Từ bảng và thí nghiệm, ta rút ra một quy trình chọn mô hình:

1. Bắt đầu với **Logistic hoặc Softmax Regression** làm mốc: nhanh, ổn định, có cực tiểu toàn cục, dễ diễn giải.
2. Dùng đường cong học và chẩn đoán thiên lệch (mục 5.4). Nếu lỗi huấn luyện **cũng cao**, mô hình thiếu khớp: thử thêm đặc trưng (ví dụ đa thức) hoặc chuyển sang mạng nơ-ron.

3. Với mạng nơ-ron, tăng dần kích thước, điều chuẩn hóa vừa đủ (mục 4.4), chọn siêu tham số bằng tập kiểm định (mục 5.2) và so với mốc ở bước 1 bằng phép so sánh có tính đến sai số chuẩn.

4. Perceptron chủ yếu có giá trị lịch sử và sư phạm: nó là mô hình đơn giản nhất cho thấy ý tưởng "cập nhật trọng số theo lỗi".

6.3. Từ mạng nơ-ron truyền thẳng đến học sâu (CNN, RNN, Transformer) và các framework

Mọi thứ trong chương này, mất mát, lan truyền thuận, lan truyền ngược, mini-batch, điều chuẩn hóa, đánh giá, vẫn nguyên vẹn khi ta chuyển sang mạng sâu hiện đại. Phần khác biệt nằm ở **cấu trúc của mỗi lớp**: ta thay phép nhân ma trận đầy đủ $\mathbf{Z} = \tilde{\mathbf{A}}\mathbf{W}$ bằng một phép biến đổi khai thác cấu trúc của dữ liệu. Mục này giới thiệu ba hướng chính ở mức khái niệm.

Vì sao MLP chưa đủ?

Hãy xem một ảnh xám 28×28 (như chữ số viết tay trong tập MNIST): duỗi thành vector 784 chiều rồi đưa vào MLP. Lớp ẩn đầu tiên với 128 nơ-ron có $(784 + 1) \times 128 = 100\,480$ tham số, mạng 784-128-64-10 có tổng cộng

$$(784 + 1) \times 128 + (128 + 1) \times 64 + (64 + 1) \times 10 = 100\,480 + 8\,256 + 650 = 109\,386$$

tham số. Với ảnh 224×224 ba kênh màu, riêng lớp đầu đã có hàng chục triệu tham số. Có hai vấn đề:

231. **Quá nhiều tham số**, dễ quá khớp và tốn tính toán.

232. **Mất cấu trúc không gian.** Khi duỗi ảnh, hai điểm ảnh kề nhau và hai điểm ảnh cách xa nhau được đối xử như nhau; một chữ số dịch một điểm ảnh sang phải thành một vector gần như "xa lạ" đối với mạng. Tương tự, văn bản là chuỗi có thứ tự mà MLP với số đầu vào cố định không xử lý trực tiếp được.

Ba kiến trúc dưới đây giải quyết các vấn đề đó bằng cách đưa **giả định về cấu trúc dữ liệu** vào mạng.

Mạng tích chập (CNN)

Ý tưởng: dùng cùng một bộ lọc nhỏ **F** (ví dụ 3×3) trượt khắp ảnh để phát hiện một mẫu cục bộ (cạnh, góc…) ở mọi vị trí. Với ảnh **X** và bộ lọc **F** kích thước $k \times k$, đầu ra tại vị trí (i, j) là

$$z_{ij} = \sum_{u=1}^{k} \sum_{v=1}^{k} f_{uv}\, x_{i+u-1,\, j+v-1} + b,$$

rồi qua hàm kích hoạt (thường là ReLU) để được bản đồ đặc trưng. Hai tính chất then chốt:

233. **Chia sẻ trọng số.** Cùng $k^2 + 1$ tham số được dùng ở mọi vị trí. Một lớp gồm 32 bộ lọc 3×3 trên ảnh một kênh chỉ có $(9 + 1) \times 32 = 320$ tham số, so với hơn 100 000 của lớp đầy đủ ở trên, *bất kể kích thước ảnh*.

234. **Tính tương đương với phép dịch.** Dịch ảnh thì bản đồ đặc trưng dịch theo, nên mẫu được phát hiện ở bất kỳ vị trí nào.

Sau phép tích chập, thường dùng **gộp** (pooling), ví dụ lấy giá trị lớn nhất trong mỗi ô 2×2 để thu nhỏ bản đồ và tăng tính bất biến với dịch chuyển nhỏ. Cuối cùng, các bản đồ đặc trưng được duỗi ra và đưa vào một lớp đầy đủ kết thúc bằng softmax, tức là **một Softmax Regression trên đặc trưng tự học**, đúng tinh thần mục 5.1.

Điều quan trọng cho người học: một phép tích chập là một phép **tuyến tính** theo **F** giống như $\widetilde{\mathbf{A}}\mathbf{W}$ (chỉ là ma trận **W** có cấu trúc thưa và lặp). Do đó lan truyền ngược vẫn là quy tắc chuỗi; gradient theo **F** là tổng các đóng góp từ mọi vị trí mà bộ lọc đã trượt qua.

Mạng hồi quy (RNN)

Ý tưởng: xử lý chuỗi $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T$ (từ trong câu, giá trị theo thời gian) bằng một **trạng thái ẩn** $\mathbf{h}_t$ được cập nhật tại từng bước và mang thông tin của quá khứ:

$$\mathbf{h}_t = \tanh(\mathbf{W}_x^T \mathbf{x}_t + \mathbf{W}_h^T \mathbf{h}_{t-1} + \mathbf{b}).$$

Cùng bộ trọng số $\mathbf{W}_x$, $\mathbf{W}_h$ được dùng ở mọi bước (lại là chia sẻ trọng số, lần này theo thời gian). Đầu ra (ví dụ phân loại cả chuỗi) lấy từ $\mathbf{h}_T$ qua một lớp softmax.

Nếu "trải" mạng theo thời gian, ta được một mạng truyền thẳng có T lớp dùng chung trọng số, và lan truyền ngược ("lan truyền ngược qua thời gian") chính là thuật toán của mục 3 áp dụng lên mạng trải đó. Hệ quả: ta gặp lại **vấn đề gradient biến mất hoặc bùng nổ** của mục 4.3, bây giờ với T lớp tương ứng T bước thời gian: tín hiệu lỗi truyền ngược qua T bước bị nhân liên tiếp với cùng ma trận $\mathbf{W}_h$ nên hoặc co về 0 hoặc phình ra, mạng khó học các phụ thuộc xa. Các biến thể có **cổng** như LSTM và GRU được thiết kế để giữ tín hiệu ổn định hơn qua nhiều bước, và cắt gradient (mục 4.4) là biện pháp thông dụng để tránh bùng nổ.

Transformer và cơ chế chú ý

Ý tưởng: thay vì đọc chuỗi tuần tự, cho **mỗi vị trí nhìn trực tiếp vào mọi vị trí khác** và quyết định nên "chú ý" tới đâu. Với chuỗi T phần tử, xếp chúng thành ma trận $\mathbf{X} \in \mathbb{R}^{T \times d}$ rồi tạo ba ma trận bằng ba phép biến đổi tuyến tính (mỗi cái là một $\widetilde{\mathbf{A}}\mathbf{W}$ quen thuộc):

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \qquad \mathbf{K} = \mathbf{X}\mathbf{W}_K, \qquad \mathbf{R} = \mathbf{X}\mathbf{W}_R.$$

Đầu ra của cơ chế chú ý là

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{R}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{R},$$

trong đó softmax áp dụng theo từng hàng của ma trận $T \times T$ các điểm tương đồng. Hàng i của ma trận softmax là một **phân phối xác suất** cho biết vị trí i lấy thông tin từ các vị trí khác với tỉ

lệ nào; đầu ra tại vị trí i là trung bình có trọng số của các hàng của **R**. Hàm softmax mà ta đã học ở mục 1 xuất hiện lại ở đây, không phải ở lớp ra mà ở *bên trong* mạng, như một cơ chế chọn lọc mềm. Chia cho $\sqrt{d_k}$ để các điểm số không quá lớn làm softmax bão hòa (cùng hiện tượng với mục 4.3).

Một khối Transformer ghép cơ chế chú ý với một MLP nhỏ áp dụng riêng cho từng vị trí và các kết nối tắt (cộng đầu vào vào đầu ra), rồi xếp chồng nhiều khối. Vì các vị trí được xử lý song song, huấn luyện trên phần cứng hiện đại hiệu quả hơn RNN; đây là kiến trúc nền tảng của các mô hình ngôn ngữ lớn.

Tóm tắt ba kiến trúc

Kiến trúc	Giả định về dữ liệu	Thành phần mới thay cho $\widetilde{\mathbf{A}}\mathbf{W}$	Ví dụ ứng dụng
MLP	Không có cấu trúc đặc biệt	Phép nhân ma trận đầy đủ	Dữ liệu bảng, phần phân loại cuối của các mạng khác
CNN	Cấu trúc lưới cục bộ (ảnh)	Tích chập với bộ lọc dùng chung	Nhận dạng ảnh
RNN	Chuỗi có thứ tự	Cập nhật trạng thái ẩn dùng chung theo thời gian	Chuỗi thời gian, văn bản (kiểu cũ)
Transformer	Chuỗi, mọi vị trí tương tác trực tiếp	Cơ chế chú ý (softmax trên điểm tương đồng)	Mô hình ngôn ngữ, ảnh, âm thanh

Về mặt lịch sử, mạng tích chập kiểu LeNet xuất hiện từ thập niên 1980 đến 1990, mạng LSTM vào cuối thập niên 1990, bước ngoặt học sâu trên ảnh vào khoảng năm 2012 và Transformer vào khoảng năm 2017; các mốc năm này mình ghi theo trí nhớ nên người đọc nên đối chiếu với tài liệu gốc trước khi trích dẫn.

Các framework: ngừng tự viết lan truyền ngược

Trong chương này, ta tự viết lan truyền ngược bằng NumPy để hiểu thuật toán. Trong thực tế, người ta dùng các framework như PyTorch, TensorFlow hoặc JAX, với ba lợi ích chính:

5. **Vi phân tự động (automatic differentiation).** Ta chỉ viết lan truyền thuận; framework ghi lại đồ thị các phép tính và tự áp dụng quy tắc chuỗi để cho gradient. Về bản chất, đó chính là lan truyền ngược của mục 3, được tổng quát cho mọi phép tính khả vi. Vì thế việc đổi kiến trúc (CNN, RNN, Transformer) không đòi hỏi tự suy ra công thức gradient mới.
6. **Tăng tốc phần cứng.** Cùng một đoạn mã chạy trên GPU hoặc bộ tăng tốc chuyên dụng mà ít phải đổi.
7. **Thư viện dựng sẵn.** Các lớp, hàm kích hoạt, thuật toán tối ưu (SGD, momentum, Adam như mục 3.5), điều chuẩn hóa (L2, dropout như mục 4.4) và công cụ đánh giá.

Dưới đây là mạng 2-16-3 (hai đầu vào, 16 nơ-ron ẩn tanh, ba lớp) của mục 5 viết bằng PyTorch. So với phiên bản NumPy, các việc "tính $\mathbf{D}^{(l)}$ rồi nhân ngược" nằm gọn trong dòng *loss.backward()*:

```
import torch, torch.nn as nn, torch.nn.functional as F

mang = nn.Sequential(
    nn.Linear(2, 16),   # (2 + 1) * 16 = 48 tham số
    nn.Tanh(),
    nn.Linear(16, 3),   # (16 + 1) * 3 = 51 tham số
)
toi_uu = torch.optim.Adam(mang.parameters(), lr=0.01)

# X: tensor (n, 2) kiểu float; y: tensor (n,) chứa nhãn nguyên 0, 1, 2
for epoch in range(200):
    chi_so = torch.randperm(len(X))
    for k in range(0, len(X), 32):
        lo = chi_so[k:k + 32]
        toi_uu.zero_grad()                          # xóa gradient cũ
        loss = F.cross_entropy(mang(X[lo]), y[lo])  # softmax + cross-entrop
y
        loss.backward()                             # lan truyền ngược tự độ
ng
        toi_uu.step()                               # cập nhật bằng Adam
```

Hai chi tiết cần nhớ: *F.cross_entropy* nhận trực tiếp **điểm số z** (chưa qua softmax) và nhãn nguyên, rồi tính softmax và cross-entropy bên trong theo cách ổn định số học (mục 2.4); và *mang.parameters()* gom mọi $\mathbf{W}^{(l)}$ kèm độ lệch (mỗi lớp *nn.Linear* lưu ma trận trọng số và vector độ lệch riêng, tương đương hàng đầu của $\mathbf{W}^{(l)}$ ở cách viết của ta).

Lưu ý trung thực: trong môi trường mình dùng để chạy các thí nghiệm của chương này, việc cài đặt PyTorch không hoàn thành được, nên đoạn mã trên **chưa được mình chạy**; nó là cách viết chuẩn mà mình tin là đúng, nhưng người đọc nên chạy thử và đối chiếu. Phần được kiểm chứng bằng thực nghiệm là cài đặt NumPy của chương: gradient của lan truyền ngược trùng với sai phân hữu hạn (mục 3.4). Cũng vì chưa cài được framework, mình không chạy so sánh CNN với MLP; các khẳng định ở mục này về CNN, RNN, Transformer là kiến thức chung, không phải kết quả thí nghiệm của chương.

Từ cài đặt của ta đến thực tế. Biết lan truyền ngược "từ đầu" vẫn quan trọng vì: (i) giúp *gỡ lỗi* (gradient biến mất, mất mát không giảm, quá khớp, đều đã được phân tích ở mục 4 và mục 5); (ii) giúp hiểu các tùy chọn của framework (khởi tạo, bộ tối ưu, điều chuẩn hóa); (iii) giúp đọc các kiến trúc mới như tổ hợp của những khối quen thuộc.

6.4. Bài tập thực hành

Các bài dưới đây gồm hai nhóm: bài tính tay hoặc suy luận (bài 1 đến 6) và bài lập trình trên bộ dữ liệu chữ số viết tay *load_digits* của scikit-learn (bài 7 đến 10). Bộ *load_digits* gồm 1797 ảnh xám 8×8 (mỗi ảnh là vector 64 chiều, giá trị 0 đến 16) của 10 chữ số. Cách chuẩn bị dữ liệu dùng chung (theo mục 5.2):

235. Chia phân tầng $60/20/20$ thành tập huấn luyện (1074 ảnh), kiểm định (359 ảnh) và kiểm tra (364 ảnh).

236. Chia các giá trị cho 16, sau đó chuẩn hóa bằng trung bình và độ lệch chuẩn của riêng tập huấn luyện.

237. Các mạng dùng $\tanh$ ở lớp ẩn, softmax ở lớp ra, cross-entropy.

Bài tập

Bài 1 (tham số). Tính số tham số của mạng 64-32-10 và của mạng 64-64-10. So sánh với số mẫu huấn luyện (1074) và với Softmax Regression trên cùng dữ liệu (không lớp ẩn).

Bài 2 (so sánh mô hình). Những mô hình nào trong bảng mục 6.2 không thể giải bài toán XOR? Giải thích bằng ranh giới quyết định. Vì sao ba vòng tròn đồng tâm cũng không giải được bằng Softmax Regression trên đặc trưng gốc, nhưng giải được nếu thêm đặc trưng x_1^2, x_2^2?

Bài 3 (chọn mô hình). Một bài toán có lỗi huấn luyện 12%, lỗi kiểm định 13%, lỗi tham chiếu 3% với Softmax Regression. Chẩn đoán và đề xuất bước tiếp theo. Sau khi chuyển sang mạng nơ-ron lớn, lỗi huấn luyện 0%, lỗi kiểm định 9%. Chẩn đoán lại và đề xuất hai cách xử lý.

Bài 4 (CNN và tham số). Một lớp tích chập có 16 bộ lọc 5×5 trên ảnh một kênh. Tính số tham số. Một lớp đầy đủ nối ảnh 28×28 với $16 \times 24 \times 24$ nơ-ron có bao nhiêu tham số? (Ảnh 28×28 với bộ lọc 5×5 và không đệm cho bản đồ đặc trưng 24×24.)

Bài 5 (chú ý). Cho điểm tương đồng của một vị trí với ba vị trí trong chuỗi là $\mathbf{s} = [2,\ 1,\ 0]$ (đã chia cho $\sqrt{d_k}$). Tính các trọng số chú ý $\mathrm{softmax}(\mathbf{s})$. Nếu nhân đôi các điểm số thành $[4,2,0]$ thì các trọng số thay đổi thế nào, và điều đó nói gì về vai trò của hệ số $1/\sqrt{d_k}$?

Bài 6 (RNN). Với RNN một chiều ($\mathbf{h}_t$ là số thực) $h_t = \tanh(w\, h_{t-1})$ (bỏ đầu vào và độ lệch), $|w| < 1$, hãy chỉ ra vì sao đạo hàm $\partial h_T / \partial h_0$ tiến về 0 khi T lớn. (Gợi ý: dùng quy tắc chuỗi và $|\tanh'| \le 1$.)

Bài 7 (lập trình, mốc cơ bản). Huấn luyện Softmax Regression và mạng 64-32-10 trên bộ chữ số, báo cáo độ chính xác kiểm tra. Chạy mạng với 5 hạt giống và báo cáo trung bình và độ lệch chuẩn.

Bài 8 (lập trình, ma trận nhầm lẫn). Tính ma trận nhầm lẫn $\mathbf{C} = \mathbf{Y}^T \widehat{\mathbf{Y}}$ trên tập kiểm tra cho mạng ở bài 7 (hạt giống 0). Hai chữ số nào bị nhầm với nhau nhiều nhất? Xem các ảnh bị sai bằng mắt và nhận xét.

Bài 9 (lập trình, bộ tối ưu). Với mạng 64-32-10, huấn luyện 50 epoch bằng SGD ($\eta = 0{,}1$ và $\eta = 0{,}5$), momentum ($\eta = 0{,}05$) và Adam ($\eta = 0{,}001$ và $\eta = 0{,}01$), mini-batch 32, 3 hạt giống. So sánh độ chính xác kiểm định.

Bài 10 (lập trình, điều chuẩn hóa). Lấy ngẫu nhiên chỉ 100 ảnh huấn luyện. Với mạng 64-64-10 huấn luyện 500 epoch, mini-batch 20, 5 hạt giống, so sánh bốn cấu hình: không điều chuẩn hóa, L2 ($\lambda = 0{,}03$), dropout ($p = 0{,}3$) và dừng sớm. Báo cáo độ chính xác huấn luyện và kiểm tra, và chẩn đoán theo mục 5.4.

Mã khung cho các bài lập trình (dùng các hàm của chương: *huan_luyen*, *huan_luyen_dc*, *lan_truyen_thuan*, *du_doan*):

```
import numpy as np
from sklearn.datasets import load_digits

d = load_digits()
X, y = d.data / 16.0, d.target

# chia phân tầng 60/20/20 (mục 5.2)
rng = np.random.default_rng(0)
tr, va, te = [], [], []
for k in range(10):
    idx = rng.permutation(np.where(y == k)[0])
    a, b = int(0.6 * len(idx)), int(0.8 * len(idx))
    tr += list(idx[:a]); va += list(idx[a:b]); te += list(idx[b:])
tr, va, te = [rng.permutation(i) for i in (tr, va, te)]

# chuẩn hóa chỉ bằng thống kê của tập huấn luyện (tránh rò rỉ)
mu, sd = X[tr].mean(0), X[tr].std(0) + 1e-8
Xtr, Xva, Xte = [(X[i] - mu) / sd for i in (tr, va, te)]
ytr, yva, yte = y[tr], y[va], y[te]
Ytr, Yva = np.eye(10)[ytr], np.eye(10)[yva]

# bài 7: mạng 64-32-10, tanh, Adam eta = 0.01, mini-batch 32, 100 epoch, lamb
da = 1e-3
Ws, ltr, lva, _ = huan_luyen_dc(Xtr, Ytr, [64, 32, 10], Xva, Yva,
                                so_epoch=100, b=32, seed=0, lam=1e-3)
acc = np.mean(du_doan(Xte, Ws, "tanh") == yte)
```

Gợi ý đáp án

Các con số của bài lập trình là kết quả mình đã chạy với đúng quy trình trên; kết quả của bạn có thể lệch chút ít nếu chia dữ liệu hoặc hạt giống khác.

238. **Bài 1.** Mạng 64-32-10: $(64+1)\times 32+(32+1)\times 10=2080+330=2410$ tham số. Mạng 64-64-10: $(64+1)\times 64+(64+1)\times 10=4160+650=4810$ tham số. Softmax Regression: $(64+1)\times 10=650$ tham số. Số tham số của hai mạng bằng khoảng 2,2 và 4,5 lần số mẫu huấn luyện (1074), nên cần chú ý điều chuẩn hóa; ở bài 10 mạng 64-64-10 có 4810 tham số nhưng chỉ có 100 mẫu, tức nhiều gấp khoảng 48 lần số mẫu.

239. **Bài 2.** Perceptron, Logistic Regression và Softmax Regression đều có ranh giới tuyến tính (từng đoạn thẳng) nên không thể phân tách XOR: không có đường thẳng nào tách $\{(0,0),(1,1)\}$ khỏi $\{(0,1),(1,0)\}$. Chỉ mạng nơ-ron có lớp ẩn giải được (mục 1; mục 2.5 và mục 3.6 cho mạng 2-2-1). Với ba vòng tròn đồng tâm, ranh giới giữa các lớp là các đường tròn $x_1^2+x_2^2=r^2$; đây là phương trình tuyến

tính theo các đặc trưng mới (x_1^2, x_2^2) (thậm chí chỉ cần $x_1^2 + x_2^2$) nên Softmax trên đặc trưng bậc 2 biểu diễn được, trong khi trên (x_1, x_2) gốc thì không. Đây chính là điều ta thấy ở mục 5.5 (86,3% cho Softmax bậc 2).

240. **Bài 3.** Lỗi huấn luyện 12% cao hơn nhiều so với tham chiếu 3% (thiên lệch $= 12 - 3 = 9$ điểm), còn khoảng cách giữa kiểm định và huấn luyện chỉ 1 điểm: **thiên lệch cao (thiếu khớp)**. Thêm dữ liệu gần như không giúp; cần mô hình mạnh hơn (đặc trưng đa thức hoặc mạng nơ-ron). Sau khi chuyển sang mạng lớn: lỗi huấn luyện 0% so với kiểm định 9%, khoảng cách 9 điểm, và thiên lệch gần như bằng 0: **phương sai cao (quá khớp)**. Hai cách xử lý: thêm điều chuẩn hóa (L2, dropout) hoặc dừng sớm; giảm kích thước mạng hoặc thêm dữ liệu (mục 5.4).

241. **Bài 4.** Lớp tích chập: $(5 \times 5 + 1) \times 16 = 416$ tham số. Lớp đầy đủ: đầu vào $28 \times 28 = 784$, đầu ra $16 \times 24 \times 24 = 9216$ nơ-ron, nên $(784 + 1) \times 9216 = 7\,234\,560$ tham số, gấp khoảng $17\,000$ lần lớp tích chập, vì lớp tích chập dùng chung 416 tham số ở mọi vị trí.

242. **Bài 5.** $e^2 = 7{,}389$, $e^1 = 2{,}718$, $e^0 = 1$, tổng $= 11{,}107$. Các trọng số: $(0{,}665;\ 0{,}245;\ 0{,}090)$. Với $[4,2,0]$: $e^4 = 54{,}598$, $e^2 = 7{,}389$, $e^0 = 1$, tổng $= 62{,}987$; các trọng số $(0{,}867;\ 0{,}117;\ 0{,}016)$. Nhân đôi điểm số làm phân phối **nhọn hơn**: vị trí đầu chiếm 86,7% thay vì 66,5%. Nếu điểm số quá lớn (do chiều d_k lớn làm tích vô hướng lớn), softmax gần như thành one-hot, bão hòa và gradient tiến về 0 (cùng hiện tượng bão hòa ở mục 4.3); hệ số $1/\sqrt{d_k}$ giữ độ lớn của điểm số ở mức hợp lý.

243. **Bài 6.** $h_t = \tanh(wh_{t-1})$ nên $\partial h_t / \partial h_{t-1} = w\,(1 - h_t^2)$, có giá trị tuyệt đối $\leq |w| < 1$ (vì $|\tanh'| \leq 1$). Theo quy tắc chuỗi, $\partial h_T / \partial h_0 = \prod_{t=1}^{T} w\,(1 - h_t^2)$ có giá trị tuyệt đối $\leq |w|^T \to 0$ khi $T \to \infty$. Vậy tín hiệu từ bước đầu hầu như không ảnh hưởng đến h_T và gradient theo các bước xa biến mất (cùng cơ chế với mục 4.3, chuỗi tích các hệ số nhỏ hơn 1).

244. **Bài 7.** Mạng 64-32-10 (tanh, Adam $\eta = 0{,}01$, mini-batch 32, 100 epoch, $\lambda = 10^{-3}$), 5 hạt giống: độ chính xác huấn luyện 1,00, kiểm định $0{,}970 \pm 0{,}008$, kiểm tra $0{,}979 \pm 0{,}004$ (mỗi lần chạy khoảng 0,5 giây). Softmax Regression (gradient descent, $\eta = 0{,}5$, 1500 epoch): độ chính xác kiểm tra 0,959. Chênh lệch khoảng 2 điểm phần trăm nhưng sai số chuẩn trên 364 mẫu cỡ $\sqrt{0{,}97 \times 0{,}03/364} \approx 0{,}009$, nên chênh lệch này khoảng hai lần sai số chuẩn: đáng chú ý nhưng không áp đảo, khác hẳn khoảng cách hàng chục điểm trên các bộ dữ liệu phi tuyến ở mục 6.2. Điều này hợp lý vì chữ số 8×8 khá gần tách được tuyến tính.

245. **Bài 8.** Với hạt giống 0, mạng chỉ sai vài mẫu trên 364 ảnh kiểm tra. Nhầm lẫn nhiều nhất: chữ số 5 bị đoán là 3 (2 mẫu); các lỗi còn lại mỗi loại chỉ 1 mẫu (ví dụ

$8 \to 1$, $6 \to 4$, $6 \to 1$, $5 \to 9$, $2 \to 8$). Số mẫu sai quá ít để rút ra quy luật chắc chắn; khi xem các ảnh bị sai, nhiều ảnh 8×8 khó đọc kể cả với người, một dạng nhiễu nội tại (mục 5.4).

246. **Bài 9.** Độ chính xác kiểm định sau 50 epoch (trung bình 3 hạt giống): SGD $\eta = 0{,}1$: 0,960; SGD $\eta = 0{,}5$: 0,961; momentum $\eta = 0{,}05$: 0,960; Adam $\eta = 0{,}001$: 0,964; Adam $\eta = 0{,}01$: 0,961. Mọi cấu hình đều trong khoảng 96,0% đến 96,4%, chênh lệch nhỏ hơn sai số chuẩn trên 359 mẫu (khoảng $\sqrt{0{,}96 \times 0{,}04/359} \approx 0{,}010$), nên **không thể** kết luận bộ tối ưu nào tốt hơn trên bài toán này; điều quan trọng hơn là chọn tốc độ học hợp lý (mục 3.5).

247. **Bài 10.** Mạng 64-64-10 với 100 mẫu huấn luyện, 500 epoch, 5 hạt giống (độ chính xác huấn luyện / kiểm tra): không điều chuẩn hóa 1,00/0,882; L2 ($\lambda = 0{,}03$) 1,00/0,891; dropout ($p = 0{,}3$) 1,00/0,880; dừng sớm 1,00/0,882. Cả bốn đều ghi nhớ hoàn toàn tập huấn luyện (100%) và chỉ khoảng 88% trên tập kiểm tra: **phương sai cao** (khoảng cách xấp xỉ 12 điểm). Chỉ L2 cho cải thiện nhẹ (0,891 so với 0,882, tức 0,9 điểm), nhỏ hơn sai số chuẩn trên 364 mẫu (cỡ 0,017); nên trong thí nghiệm này, các biện pháp điều chuẩn hóa **không** cho khác biệt đáng tin cậy. Điều chuẩn hóa không phải phép màu: với 100 mẫu, cách chắc chắn hơn là thu thập thêm dữ liệu (với toàn bộ 1074 mẫu, mạng 64-32-10 ở bài 7 đạt 97,9% trên tập kiểm tra; kiến trúc khác nhau nên đây chỉ là so sánh thô, đúng tinh thần đường cong học ở mục 5.4).

Mối liên hệ với toàn chương

Mục 6 khép lại chương theo đúng thứ tự dẫn dắt của các mục trước:

248. Mục 1 đặt câu hỏi: mô hình tuyến tính **không giải được** XOR, vòng tròn, xoắn ốc. Mục 6.2 trả lời bằng số liệu: Softmax Regression chỉ đạt 0,49 đến 0,50 còn MLP đạt 0,847 đến 0,944.

249. Mục 2 và mục 3 xây dựng mạng và thuật toán học. Mục 6.3 cho thấy các kiến trúc hiện đại là cùng ý tưởng "biến đổi tuyến tính rồi kích hoạt phi tuyến, huấn luyện bằng lan truyền ngược" với phần biến đổi tuyến tính được cấu trúc lại, và framework tự động hóa lan truyền ngược.

250. Mục 4 giải thích những khó khăn của mạng nơ-ron (không lồi, gradient biến mất, quá khớp), và mục 6.1 gom chúng thành phần hạn chế.

251. Mục 5 cho công cụ đánh giá, và mục 6.4 dùng chính các công cụ đó trong các bài tập.

Tóm tắt mục 6

252. **Ưu điểm.** Học được ranh giới phi tuyến và đặc trưng; có định lý xấp xỉ phổ quát; một khung chung cho nhiều bài toán; huấn luyện được nhờ lan truyền ngược; mở rộng được theo kiến trúc.

253. **Hạn chế.** Hàm mất mát không lồi nên kết quả phụ thuộc khởi tạo; gradient biến mất hoặc bùng nổ khi mạng sâu; dễ quá khớp; nhiều siêu tham số; tốn tính toán; khó diễn giải; không phải lúc nào cũng thắng mô hình đơn giản.

254. **Bốn mô hình là một chuỗi kế tiếp.** Perceptron (hàm bước) → Logistic Regression (sigmoid, hàm mất mát trơn) → Softmax Regression (nhiều lớp) → mạng nơ-ron (thêm các lớp ẩn học đặc trưng). Thí nghiệm: trên dữ liệu phi tuyến, ba mô hình tuyến tính đạt 0,38 đến 0,55 còn MLP đạt 0,847 đến 0,944; trên dữ liệu tách được, cả bốn ngang nhau (98% đến 99%).

255. **Học sâu.** CNN dùng bộ lọc dùng chung để khai thác cấu trúc lưới; RNN dùng trạng thái ẩn dùng chung theo thời gian cho chuỗi; Transformer dùng cơ chế chú ý với softmax. Tất cả vẫn huấn luyện bằng lan truyền ngược, và các framework như PyTorch tự động hóa bước đó bằng vi phân tự động.

256. **Quy tắc thực hành.** Bắt đầu từ mô hình đơn giản làm mốc; dùng chẩn đoán thiên lệch và phương sai để quyết định có cần mô hình phức tạp hơn không; so sánh có tính đến sai số chuẩn.

Câu hỏi kiểm tra nhanh

257. Trình bày mối quan hệ giữa Logistic Regression, Softmax Regression và mạng nơ-ron trong một câu.

258. Vì sao Perceptron không thể huấn luyện bằng gradient descent trên một hàm mất mát trơn?

259. Kể ba hạn chế của mạng nơ-ron và công cụ trong chương này để xử lý từng hạn chế.

260. Một lớp tích chập với 8 bộ lọc 3×3 trên ảnh một kênh có bao nhiêu tham số?

261. Trong cơ chế chú ý, softmax được áp dụng theo hàng hay theo cột của ma trận điểm tương đồng, và hàng của ma trận đó có tính chất gì?

262. Vì sao biết lan truyền ngược "từ đầu" vẫn có ích khi đã có framework?

Gợi ý đáp án

263. Softmax Regression là mạng nơ-ron không có lớp ẩn; Logistic Regression là Softmax Regression với hai lớp; mạng nơ-ron thêm các lớp ẩn để tự học đặc trưng cho một Softmax Regression ở lớp cuối.

264. Hàm bước có đạo hàm bằng 0 ở hầu hết mọi nơi (và không xác định tại điểm nhảy), nên gradient không cho thông tin về hướng cải thiện.

265. Ví dụ: không lồi (chạy nhiều hạt giống, báo cáo trung bình và độ lệch chuẩn); gradient biến mất (ReLU, khởi tạo He, kiểm tra gradient); quá khớp (L2, dropout, dừng sớm, đường cong học).

266. $(3 \times 3 + 1) \times 8 = 80$ tham số.

267. Theo hàng: mỗi hàng là một phân phối xác suất (các phần tử không âm, tổng bằng 1) cho biết một vị trí lấy thông tin từ các vị trí khác theo tỉ lệ nào.

268. Vì giúp gỡ lỗi (xác định gradient biến mất, mất mát không giảm, quá khớp), hiểu các tùy chọn của framework và đọc được các kiến trúc mới như tổ hợp của các khối quen thuộc.